AI for Business: A Hands-On Guide

Karthik Srinivasan

Contents

1. Foundations **1**
1.1 LLM Foundations 1
1.2 Calling LLMs: Providers and Fallback 19

2. Retrieval-Augmented Generation (RAG) **23**
2.1 RAG from First Principles 23
2.2 RAG with Vector Databases 33
2.3 RAG with Structured Data 52
2.4 RAG with Images and Video 72

3. RAG Frameworks **97**
3.1 LlamaIndex 97
3.2 LangChain 113
3.3 LangGraph 132

4. Fine-Tuning **153**
4.1 Fine-Tuning with QLoRA 153

5. Agents **163**
5.1 Single-Agent Systems 163
5.2 Multi-Agent Systems 171
5.3 Autonomous Agents 182
5.4 Tool-Using Agents and MCP 191

6. Deployment and No-Code **214**
6.1 Deployment and No-Code 214

1. Foundations

1.1 LLM Foundations

Large Language Models have rapidly become one of the most consequential technologies in business. From generating marketing copy to summarizing legal contracts, these systems demonstrate a remarkable ability to process and produce human language. Yet for business professionals who will deploy, evaluate, and manage these systems, a surface-level understanding is not enough. To make sound decisions about when and how to use LLMs, you need to understand what is happening beneath the surface. This chapter introduces the four foundational building blocks that make modern language models work: tokenization, embeddings, TF-IDF, and attention.

1.1.1 How Large Language Models Work

At their core, LLMs are statistical models trained to predict the next word (or, more precisely, the next token) in a sequence. The process of creating a working LLM involves three distinct phases. During **pre-training**, the model is exposed to vast quantities of text from books, websites, code repositories, and other sources. Through this exposure, the model learns the statistical patterns of language: grammar, facts, reasoning patterns, and even stylistic conventions. Pre-training is computationally expensive, often requiring thousands of GPUs running for weeks or months, and it produces a base model that can complete text but is not yet tuned for conversation or specific tasks.

The second phase, **post-training**, refines the base model for practical use. This includes techniques like instruction tuning, where the model learns to follow directions, and reinforcement learning from human feedback (RLHF), where human evaluators help the model distinguish helpful responses from harmful or unhelpful ones. Post-training is what transforms a raw text-completion engine into the conversational assistants and task-oriented tools that businesses actually deploy.

Finally, **inference** is the phase where the trained model is put to work. When you type a prompt into ChatGPT or call an API from your application, the model processes your input through its learned parameters and generates a response token by token. Understanding these three phases helps explain both the capabilities and the limitations of LLMs: the model can only draw on patterns it encountered during training, it behaves according to the guardrails established during post-training, and the quality of its output at inference time depends heavily on how well the input is structured.

1.1.2 Tokenization

Before an LLM can process any text, that text must be converted into a numerical format the model can work with. This is where tokenization comes in. A tokenizer breaks input text into discrete units called tokens, which might be whole words, parts of words, or even individual characters, depending on the tokenization scheme. Each token is then mapped to a unique integer ID from the model's vocabulary.

Consider the sentence "Is the earth spherical?" A tokenizer based on the LLaMA model might break this into seven tokens, while GPT-2′s tokenizer produces five. The differences arise because each model uses a different vocabulary and a different algorithm for deciding where to split text. Modern LLMs typically use subword tokenization methods like Byte Pair Encoding (BPE), which strike a balance between keeping common words intact and breaking rare words into recognizable pieces.

Tokenization has practical consequences that business users encounter regularly. The now-famous "strawberry problem" illustrates this well: when asked "How many r's are in the word strawberry?", many LLMs answer incorrectly because they never see the individual letters. The tokenizer might encode "strawberry" as a single token or as subword pieces like "straw" and "berry," neither of which preserves the letter-level information needed to count individual characters. This is not a failure of reasoning but a direct consequence of how the model perceives text at the token level. Understanding tokenization helps explain why LLMs

struggle with spelling tasks, character counting, and certain kinds of text manipulation, even when they excel at higher-level language understanding.

1.1.3 Embeddings

Once text has been tokenized into integer IDs, those IDs must be transformed into a representation the model can actually reason over. This is the role of embeddings. An embedding maps each token ID to a dense vector, a list of numbers (typically hundreds or thousands of dimensions) that positions the token in a high-dimensional space. The key insight is that these vectors are not arbitrary. Through training, tokens that appear in similar contexts end up with similar vectors, meaning the geometric relationships between vectors capture semantic relationships between words.

For example, in a well-trained embedding space, the vector for "computer" will be closer to the vector for "business" than to the vector for "cooking." Cosine similarity, a standard measure of how closely two vectors point in the same direction, quantifies these relationships numerically. The accompanying notebook demonstrates this using both a small custom-trained Word2Vec model and a pre-trained GloVe model trained on billions of words from Wikipedia. The pre-trained model produces far more meaningful similarity scores because it has seen enough text to learn robust semantic relationships.

Embeddings are fundamental not only to LLMs but to virtually every modern NLP application. They are the mechanism through which language, an inherently symbolic and discrete system, gets translated into the continuous mathematical space where neural networks operate. When you hear about "vector databases" or "semantic search" in later chapters, those systems are built directly on the embedding representations introduced here.

1.1.4 TF-IDF: A Classical Baseline

Before learned embeddings became dominant, the standard approach to representing text numerically was TF-IDF, which stands for Term Frequency-Inverse Document Frequency. TF-IDF assigns a weight to each word in a document based on two factors: how frequently the word appears in that specific document (term frequency) and how rare the word is across the entire collection of documents (inverse document frequency). Words that appear frequently in one document but rarely in others receive high weights, making them useful for distinguishing that document from the rest.

TF-IDF is worth understanding for two reasons. First, it remains a practical and effective tool for many text analysis tasks in business, from document classification to search ranking, and you may encounter it in production systems. Second, contrasting TF-IDF with learned embeddings clarifies what makes modern approaches more powerful. TF-IDF treats each word independently: it has no notion that "good" and "excellent" are related, and it cannot capture word order or context. Embeddings, by contrast, encode semantic similarity directly in the vector space, and when combined with attention mechanisms, they can represent the meaning of words in context. The notebook for this section walks through a complete TF-IDF calculation so you can see exactly how the weights are derived and appreciate both the elegance and the limitations of this classical method.

1.1.5 Attention and the Transformer Architecture

The final building block, and arguably the most important one, is the attention mechanism. Introduced in the landmark 2017 paper "Attention Is All You Need" by Vaswani and colleagues at Google, the attention mechanism is the core innovation behind the transformer architecture that powers every modern LLM.

The fundamental problem attention solves is context. In the sentence "The bank was steep and covered in wildflowers," the word "bank" refers to a riverbank. In "She walked into the bank to deposit a check," it refers to a financial institution. The same token must take on different meanings depending on the surrounding words. Self-attention achieves this by allowing each token in a sequence to look at every other token and compute a relevance score. Tokens that are contextually important to each other receive higher attention weights, and this information is used to produce context-aware representations.

The accompanying notebook demonstrates this concretely with a sentiment analysis task. When the model processes the sentence "the movie was boring," the attention mechanism learns to assign the highest weight to the word "boring," which carries the most sentiment information. Without the attention layer, the model must treat every word's embedding equally, losing the ability to focus on what matters most. This capacity to dynamically weigh the importance of different parts of the input is what allows transformers to handle long-range dependencies and nuanced meaning in ways that earlier architectures could not.

The transformer architecture assembles these building blocks into a coherent system. Text enters as raw characters, gets broken into tokens, gets mapped to embedding vectors, and then passes through multiple layers of self-attention and feed-forward networks. Each layer refines the representations, building increasingly abstract and context-sensitive features. The result is a model that can generate coherent paragraphs, translate between languages, summarize documents, and answer complex questions, all from the same underlying architecture.

1.1.6 From Theory to Practice

The four notebooks that accompany this chapter let you work with each of these concepts directly. The tokenization notebook compares legacy NLP tokenization with modern subword tokenizers from LLaMA and GPT-2. The embedding notebook trains a Word2Vec model from scratch and then explores a pre-trained GloVe model to see how semantic similarity works at scale. The TF-IDF notebook walks through the mathematics of term weighting on a small corpus. And the attention notebook builds a simple sentiment classifier with and without an attention layer so you can see, quantitatively, the difference attention makes.

Each notebook is self-contained and runs on Google Colab preferably with T4 GPU. Notebooks also run on CPU mode in Google Colab but may take a little longer to complete.

1.1.7 References

- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems*, 30.
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *Proceedings of NAACL-HLT 2019*.
- Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient Estimation of Word Representations in Vector Space. *Proceedings of ICLR 2013*.
- Sennrich, R., Haddow, B., & Birch, A. (2016). Neural Machine Translation of Rare Words with Subword Units. *Proceedings of the 54th Annual Meeting of the ACL*.
- Pennington, J., Socher, R., & Manning, C. D. (2014). GloVe: Global Vectors for Word Representation. *Proceedings of EMNLP 2014*.

Open in Colab

```
from tensorflow.keras.preprocessing.text import Tokenizer
from transformers import LlamaTokenizer, AutoTokenizer

# 1. Define the input text
data = [
  "The earth is spherical.",
  "The earth is a planet."
]

# 2. Initialize Tokenizer (simulating the vocabulary build)
tokenizer = Tokenizer(num_words=15, lower=True, split=' ')
tokenizer.fit_on_texts(data)

# 3. Convert text to Sequence of Integers (Token IDs)
ID_sequences = tokenizer.texts_to_sequences(data)

# Output the dictionary (Vocabulary) and the IDs
print("ID dictionary:", tokenizer.word_index)
print("ID sequences:", ID_sequences)
```

```
# 1. Load a pre-trained tokenizer
tokenizer = LlamaTokenizer.from_pretrained('openlm-research/open_llama_3b_v2')

# 2. Define the input string
prompt = 'Is the earth spherical?'

# 3. Convert text to tokens (input_ids)
input_ids = tokenizer(prompt, return_tensors="pt").input_ids

print(input_ids)
```

```
# 1. Load a larger pre-trained tokenizer
tokenizer_larger = AutoTokenizer.from_pretrained('gpt2')

# 2. Define the input string
prompt = 'Is the earth spherical?'

# 3. Convert text to tokens (input_ids)
input_ids = tokenizer_larger(prompt, return_tensors="pt").input_ids

print(input_ids)
```

1.1.7.0.1 *Observations:*

· *Case Sensitivity:* “Hello” vs “hello” results in different tokens. · *Space Handling:* “hello world” (joined) vs “hello world” results in different token sequences. For instance, “hello world” might split into “hello” and “ world” (with a leading space included in the token). · *Vocabulary Size:* The compression trades off sequence length for vocabulary size. A larger vocabulary (e.g., 100k symbols) results in shorter sequences of integers

1.1.7.0.2 For comparison, let us do legacy tokenization

```
import nltk
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import word_tokenize
import string

# Download necessary NLTK data (run once)
try:
```

```
    nltk.data.find('tokenizers/punkt')
except LookupError:
    nltk.download('punkt')
# Add explicit download for 'punkt_tab' as suggested by the error traceback
try:
    # NLTK's find typically looks for specific files, but downloading
'punkt_tab' as a collection
    # is suggested by the error. We will attempt to find a component of it,
and if not found,
    # proceed with the download.
    nltk.data.find('tokenizers/punkt_tab/english.pickle') # Check for a known
file within punkt_tab
except LookupError:
    nltk.download('punkt_tab') # Explicitly download 'punkt_tab' collection

try:
    nltk.data.find('corpora/stopwords')
except LookupError:
    nltk.download('stopwords')
try:
    nltk.data.find('corpora/wordnet')
except LookupError:
    nltk.download('wordnet')

# 1. Define the input text
text = 'Is the earth spherical?'

print(f"Original text: '{text}'")

# 2. Tokenization (splitting into words)
tokens = word_tokenize(text)
print(f"\nTokens: {tokens}")

# 3. Lowercasing
lower_tokens = [word.lower() for word in tokens]
print(f"Lowercase tokens: {lower_tokens}")

# 4. Remove punctuation
punctuation_free_tokens = [word for word in lower_tokens if word not in
string.punctuation]
print(f"Punctuation-free tokens: {punctuation_free_tokens}")

# 5. Remove stop words
stop_words = set(stopwords.words('english'))
filtered_tokens = [word for word in punctuation_free_tokens if word not in
stop_words]
print(f"Stop-word free tokens: {filtered_tokens}")

# 6. Lemmatization
lemmatizer = WordNetLemmatizer()
lemmatized_tokens = [lemmatizer.lemmatize(word) for word in filtered_tokens]
print(f"Lemmatized tokens: {lemmatized_tokens}")
```

1.1.7.0.3 *Observations:*

Legacy Tokenization code is long. Also, the outcome is not suitable for GenAI

1.1.8 Before we conclude:

1.1.8.1 Just for context, how do computers read alphabets in bits:

```
# 1. Define the input text
character = "a"

# 2. Encode to UTF-8
```

```python
byte_data = character.encode('utf-8')

# 3. Retrieve the Integer Value (The "Emoji" or Symbol)
integer_value = byte_data[0] # Fix: Access the first byte to get its integer
value

# 4. Convert to Binary string (The "1 0" representation)
bit_representation = format(integer_value, '08b')

print(f"Character: {character}")
print(f"Integer (0-255): {integer_value}")
print(f"UTF-8 Bits: {bit_representation}")
```

Open in Colab

1.1.9 Word Embeddings: Teaching Machines the Meaning of Words

How does a computer understand that "king" and "queen" are related, or that "Paris" is to "France" what "Tokyo" is to "Japan"? The answer lies in **word embeddings** – a technique that converts words into numerical vectors (lists of numbers) that capture their meaning. Words that appear in similar contexts end up with similar vectors, which means we can actually *measure* how related two concepts are. In this notebook, we will train our own simple embeddings and then explore a powerful pre-trained model to see this in action.

```
!pip install gensim
```

1.1.9.1 Setting Up: Word2Vec with Gensim

We are using the `gensim` library, one of the most popular Python packages for working with word embeddings. Gensim gives us access to the **Word2Vec** algorithm (developed by Google in 2013) as well as a collection of pre-trained embedding models. Think of it as your toolkit for turning words into numbers that actually mean something.

```
from gensim.models import Word2Vec
import gensim.downloader as api
```

1.1.9.2 Step 1: Creating a Toy Corpus

Word2Vec learns by reading lots of text and figuring out which words tend to appear near each other. Below, we create a very small "corpus" (collection of sentences) for demonstration purposes. In the real world, you would feed Word2Vec millions of sentences – think all of Wikipedia or years of news articles. With only five short sentences, our model will not learn much, but it is enough to see *how* the process works.

```
# 1. Prepare your data (a list of lists of words)
# This small corpus is used for demonstration; a real corpus would be much
larger.
sentences = [
    ['AI', 'for', 'business', 'course', 'is', 'fun'],
    ['AI', 'is', 'cool'],
    ['I', 'am', 'cool'],
    ['AI','uses','computers'],
    ['I','like','to','cook']

]
```

1.1.9.3 Step 2: Training the Word2Vec Model

Here we train Word2Vec on our tiny corpus. The key parameters control the learning process: `vector_size=100` means each word will be represented as a list of 100 numbers, `window=5` tells the model to look at words within 5 positions of each other, and `min_count=1` ensures we keep every word even if it only appears once. The model slides a window across each sentence and learns to predict nearby words – that is how it discovers meaning.

```
# 2. Train the Word2Vec model
# vector_size: Dimensionality of the word vectors (e.g., 100)
# window: Maximum distance between the current and predicted word within a
sentence
# min_count: Ignores all words with total frequency lower than this
# workers: Number of CPU cores to use for training
model = Word2Vec(sentences, vector_size=100, window=5, min_count=1, workers=4)
```

1.1.9.4 Step 3: Examining a Word Vector

Once trained, every word in our vocabulary has its own vector – a list of 100 numbers (since we set `vector_size=100`). These numbers do not have an obvious human interpretation on their own, but their *relative positions* encode meaning. We are printing just the first 5 elements of the vector for "AI" to get a feel for what these look like. In a well-trained model, words with similar meanings would have vectors pointing in similar directions.

```
# 3. Access the word vectors
# The trained vectors are accessible via the 'wv' attribute
word_vector = model.wv['AI']
print(f"Vector for 'AI' (first 5 elements): {word_vector[:5]}")
```

1.1.9.5 Step 4: Measuring Similarity Between Words

This is where embeddings get exciting. **Cosine similarity** measures how closely two word vectors point in the same direction, on a scale from −1 (opposite) to +1 (identical). If two words frequently appear in similar contexts, their vectors will be close and the cosine similarity will be high. Notice that with our tiny corpus, the similarity scores are not very meaningful – "AI" and "business" show roughly the same similarity as "AI" and "computers." This is because Word2Vec needs *much* more data to learn real-world relationships.

```
# 4. Perform similarity tasks
# Find words similar to a given word
similar_words = model.wv.most_similar('AI', topn=3)
print(f"Words similar to 'AI': {similar_words}")

# Calculate cosine similarity between two words
print(f"Similarity between 'AI' and 'business': {model.wv.similarity('AI',
'business')}")
print(f"Similarity between 'AI' and 'computers': {model.wv.similarity('AI',
'computers')}")
print(f"Similarity between 'AI' and 'cook': {model.wv.similarity('AI',
'cook')}")
```

1.1.9.6 Using Pre-Trained Embeddings: GloVe

Instead of training from scratch on limited data, we can use **GloVe** (Global Vectors for Word Representation), a model pre-trained on billions of words from Wikipedia and news articles. Pre-trained embeddings are like hiring an expert who has already read the entire internet – they come with rich, nuanced understanding of word relationships baked in. The download may take a moment since we are pulling down 66 MB of learned word vectors.

```
# Load a pre-trained model (e.g., GloVe with 50 dimensions trained on
Wikipedia)
# This download may take a few moments
wv2 = api.load('glove-wiki-gigaword-50')
```

1.1.9.7 Exploring GloVe Vectors

Now let us look at the vector for "computer" from the GloVe model. Notice that the numbers are much larger and more varied compared to our toy model – that is because GloVe was trained on vastly more data and has had the chance to learn meaningful patterns. These 50-dimensional vectors pack a surprising amount of semantic information into a compact numerical representation.

```
# Access the vector for a word
vector = wv2['computer']
print(f"Vector for 'computer' (first 5 elements): {vector[:5]}")
```

1.1.9.8 Real-World Similarity Scores

Now we see cosine similarity with a properly trained model, and the results make much more intuitive sense. “Computer” and “business” score about 0.70 – a strong relationship, since computers are widely used in business. “Cooking” and “computers” score only about 0.36 – related in the sense that they are both common English words, but semantically quite distant. These scores are the foundation of **semantic search**, recommendation systems, and many other AI applications you encounter daily.

```
# Calculate cosine similarity between two words
print(f"Similarity between 'computer' and 'business':
{wv2.similarity('computer', 'business')}")
print(f"Similarity between 'cooking' and 'computers':
{wv2.similarity('cooking', 'computers')}")
```

1.1.9.9 A Gotcha: Vocabulary Limitations

The cell below will throw a `KeyError` – and that is intentional. The GloVe model we loaded was trained on lowercase text, so it knows “ai” but not “AI” (uppercase). This is a common real-world issue: pre-trained models have a fixed vocabulary, and anything not in that vocabulary simply cannot be looked up. In practice, you would lowercase your input or choose a model with a larger vocabulary. It is a good reminder that these tools, while powerful, have practical limitations you need to be aware of.

```
print(f"Similarity between 'AI' and 'cook': {wv2.similarity('AI', 'cook')}")
```

Open in Colab

1.1.10 TF-IDF: Finding the Words That Actually Matter

Before deep learning and fancy embeddings, there was **TF-IDF** (Term Frequency-Inverse Document Frequency) – and it is still remarkably useful today. Imagine you are searching through thousands of documents to find the most relevant ones. Common words like "the" and "is" appear everywhere and tell you nothing. TF-IDF solves this elegantly: it scores each word based on how important it is *to a specific document* relative to the entire collection. Words that are frequent in one document but rare across all documents get the highest scores. It remains a go-to baseline for text search, document classification, and keyword extraction.

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

1.1.10.1 Step 1: Creating Our Document Collection

We start with three tiny "documents" – just short phrases for clarity. In a real scenario, each document might be a product review, an email, or a news article. Notice that some words like "AI" and "cool" appear in multiple documents, while others like "am" appear in only one. TF-IDF will exploit exactly this pattern to determine which words are most distinctive for each document.

```
d0 = 'AI is cool'
d1 = 'I am cool'
d2 = 'AI'
string = [d0, d1, d2]
```

1.1.10.2 Step 2: Computing TF-IDF

Scikit-learn's `TfidfVectorizer` handles everything in two steps. First, `fit` learns the vocabulary and computes the IDF (Inverse Document Frequency) values from the entire collection. Then, `transform` converts each document into a vector of TF-IDF scores. The result is a matrix where each row is a document and each column is a word – and each cell contains the TF-IDF score for that word in that document.

```
tfidf = TfidfVectorizer()
result = tfidf.fit_transform(string)
```

1.1.10.3 Step 3: Understanding IDF Values

The **IDF** (Inverse Document Frequency) score tells us how "rare" or distinctive a word is across the entire collection. Words that appear in many documents get lower IDF values, while words that appear in only one document get higher values. Here, "ai" and "cool" both appear in 2 out of 3 documents, so they get an IDF of about 1.29. Meanwhile, "am" and "is" each appear in only 1 document, earning a higher IDF of about 1.69. The rarer the word across documents, the more information it carries.

```
print('\nidf values:')
for ele1, ele2 in zip(tfidf.get_feature_names_out(), tfidf.idf_):
    print(ele1, ':', ele2)
```

1.1.10.4 Step 4: The Full TF-IDF Matrix

Now we see the complete picture. The vocabulary index tells us which column corresponds to which word. The sparse matrix output shows only the non-zero entries (efficient for large datasets where most documents do not contain most words). The dense matrix view makes it easier to read: each row is a document, each column is a word, and the values are the final TF-IDF scores. Notice that document 3 (which contains only "AI") gives "ai" a perfect score of 1.0 – it is the only word there, so it carries all the weight. The scores are also L2-

normalized per row, meaning each document vector has a length of 1, which makes them ready for comparison.

```
print('\nWord indexes:')
print(tfidf.vocabulary_)
print('\ntf-idf value:')
print(result)
print('\ntf-idf values in matrix form:')
print(result.toarray())
```

1.1.10.5 Bonus: Handling Single-Character Words

If you noticed that the word "I" was missing from our results – good catch. By default, `TfidfVectorizer` uses a tokenizer that skips single-character words, since they are often just noise (punctuation, stray letters). But sometimes single-character words matter – like the pronoun "I" in English. The code below overrides the default by providing a custom `token_pattern` that includes single-character words. This is a practical lesson: always check what your preprocessing tools are doing behind the scenes, because defaults can quietly throw away information you care about.

```
### IF you made an astute observation that the word 'I' is not shown as a
term. This is because TfidfVectorizer by default uses a tokenizer that ignores
single-character words.
### This is often done to remove very short words or punctuation that might
not carry significant meaning.
## Below code explicitly requires TfidfVectorizer() to include single
character word in its bag of words.

tfidf_with_single_char = TfidfVectorizer(token_pattern=r'(?u)\b\w+\b')
result_with_single_char = tfidf_with_single_char.fit_transform(string)

print('\nidf values (that also include single-character words):')
for ele1, ele2 in zip(tfidf_with_single_char.get_feature_names_out(),
tfidf_with_single_char.idf_):
    print(ele1, ':', ele2)

print('\nWord indexes:')
print(tfidf_with_single_char.vocabulary_)

print('\ntf-idf values in matrix form:')
print(result_with_single_char.toarray())
```

Open in Colab

1.1.11 The Attention Mechanism: How AI Learns to Focus

The **attention mechanism** is the single most important innovation behind modern AI language models like ChatGPT, Gemini, and Claude. Before attention, neural networks processed text like a conveyor belt – one word at a time, left to right, hoping to remember everything along the way. Attention changed the game by letting the model look at *all* words simultaneously and decide which ones matter most for the task at hand. In this notebook, we build a simple sentiment classifier and use attention weights to see *exactly* which words the model focuses on when making its prediction.

1.1.11.1 Building a Sentiment Classifier with Attention

The code below does quite a lot in one cell, so let us walk through the big picture. We start with a small labeled dataset of movie reviews (positive = 1, negative = 0), duplicated 10 times to give the model more examples. The text goes through three stages: first, a `TextVectorization` layer converts words to numbers; then, an `Embedding` layer turns those numbers into meaningful vectors; and finally, an `Attention` layer lets the model weigh which words matter most. The output is a single number between 0 and 1 representing sentiment. We also capture the attention weights so we can visualize them later.

```
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, models

# 1. Dataset: 100 samples of real text

### Try improving data and see how the test prediction changes.
data = [
    ("the movie was great", 1), ("i loved the acting", 1), ("simply amazing",
1),
    ("a masterpiece of cinema", 1), ("really fun and exciting", 1),
    ("the plot was boring", 0), ("terrible acting script", 0), ("i hated it",
0),
    ("waste of time", 0), ("it was a bad experience", 0)
] * 10

texts = [item[0] for item in data]
labels = np.array([item[1] for item in data])

# 2. Text Preprocessing
max_tokens = 100
sequence_length = 5
vectorize_layer = layers.TextVectorization(max_tokens=max_tokens,
output_sequence_length=sequence_length)
vectorize_layer.adapt(texts)
X_train = vectorize_layer(texts)

# 3. Build Model
inputs = layers.Input(shape=(sequence_length,), name="input_layer")
embedding = layers.Embedding(max_tokens, 16, name="embedding_layer")(inputs)

# Built-in Keras Attention
# We set return_attention_scores=True to get the weights for visualization
attention_output, weights = layers.Attention(name="attention_layer")(
    [embedding, embedding], return_attention_scores=True
)

# Pool the results and classify
```

```
flat = layers.GlobalAveragePooling1D()(attention_output)
outputs = layers.Dense(1, activation='sigmoid', name="sentiment_output")(flat)

model = models.Model(inputs=inputs, outputs=[outputs, weights])

# 4. Use a list for loss. 'None' for the attention weights output.
model.compile(
    optimizer='adam',
    loss=['binary_crossentropy', None],
    metrics=['accuracy', None]
)

# 5. Train
print("Training model...")
### Try changing the no of epochs and see how the attention in the test data
changes.
model.fit(X_train, labels, epochs=100, verbose=0)
print("Training complete.\n")
```

1.1.11.2 Interpreting the Attention Weights

Now for the payoff – we feed a test sentence ("the movie was boring") into our trained model and look at what the attention mechanism learned. The model correctly leans toward a negative sentiment. More interestingly, the attention weights reveal *why*: the word "boring" gets the highest weight (about 0.32), meaning the model focused most heavily on that word when making its decision. Words like "the" and "was" get much lower weights because they do not carry sentiment information. This interpretability is one of attention's superpowers – unlike a black-box model, we can peek inside and see the reasoning.

```
# 6. Predict and Interpret
test_sentence = ["the movie was boring"]
X_test = vectorize_layer(test_sentence)
prediction, attention_weights = model.predict(X_test)

# Map numbers back to words
vocab = vectorize_layer.get_vocabulary()
words = [vocab[idx] for idx in X_test[0].numpy() if idx != 0]

print(f"Sentence: '{test_sentence[0]}'")
print(f"Sentiment: {'Positive' if prediction[0] > 0.5 else 'Negative'}
({prediction[0][0]:.4f})")
print("-" * 40)

# Average weights across the query dimension to see overall word importance
avg_weights = np.mean(attention_weights[0], axis=0)

for word, weight in zip(words, avg_weights):
    bar = "▇" * int(weight * 40)
    print(f"{word:<10} | {weight:.4f} {bar}")
```

1.1.11.3 Does More Data Help? Training with a Richer Dataset

Our first model used only 10 unique reviews (duplicated). Now we scale up to 100 unique reviews – 50 positive and 50 negative – with much more varied language. This is closer to a real-world scenario where the model encounters diverse phrasing. After training on this richer dataset, notice how the sentiment prediction becomes more confident (further from 0.5) and the attention weights shift. The word "boring" still dominates, but the model now has a more nuanced understanding because it has seen many more examples of positive and negative language.

```
## Now try it with a slightly better data:

# 50 Positive Samples
pos_reviews = [
    ("the movie was great", 1), ("i loved the acting", 1), ("simply amazing",
```

```
1),
    ("a masterpiece of cinema", 1), ("really fun and exciting", 1),
("incredible story line", 1),
    ("absolutely wonderful experience", 1), ("the best film ever", 1),
("highly recommended movie", 1),
    ("superb performance by all", 1), ("i enjoyed every minute", 1),
("brilliant directing", 1),
    ("a true classic film", 1), ("spectacular visuals", 1), ("i was
impressed", 1),
    ("pure joy to watch", 1), ("it was very touching", 1), ("a beautiful
story", 1),
    ("top notch acting", 1), ("very entertaining film", 1), ("the
cinematography was stunning", 1),
    ("captivating from start to finish", 1), ("refreshing and original", 1),
("greatest movie of year", 1),
    ("it made me happy", 1), ("an emotional journey", 1), ("wonderful cast",
1),
    ("powerful and moving", 1), ("delightful cinema", 1), ("i really liked
it", 1),
    ("fantastic plot", 1), ("breathtaking scenes", 1), ("perfectly executed",
1),
    ("charming movie", 1), ("excellent work", 1), ("it was so good", 1),
    ("lovely film", 1), ("bold and brave story", 1), ("sweet and funny", 1),
    ("a must watch", 1), ("i felt inspired", 1), ("outstanding production",
1),
    ("very well made", 1), ("smart and witty", 1), ("uplifting ending", 1),
    ("cool characters", 1), ("impressive quality", 1), ("honest and raw", 1),
    ("magical feeling", 1), ("i loved it", 1)
]

# 50 Negative Samples
neg_reviews = [
    ("the plot was boring", 0), ("terrible acting script", 0), ("i hated it",
0),
    ("waste of time", 0), ("it was a bad experience", 0), ("very dull and
slow", 0),
    ("worst movie ever", 0), ("extremely disappointed", 0), ("poorly
directed", 0),
    ("the script was weak", 0), ("did not like it", 0), ("annoying
characters", 0),
    ("boring story", 0), ("pathetic attempt at drama", 0), ("it was so noisy",
0),
    ("waste of money", 0), ("terrible directing", 0), ("ugly visuals", 0),
    ("i fell asleep", 0), ("the ending was bad", 0), ("not worth watching",
0),
    ("failed to impress", 0), ("silly plot", 0), ("boring and predictable",
0),
    ("i was very frustrated", 0), ("bad script writing", 0), ("horrible
acting", 0),
    ("nothing makes sense", 0), ("clunky dialogue", 0), ("very mediocre", 0),
    ("disaster of a film", 0), ("low quality production", 0), ("uninspired
film", 0),
    ("cheesy and cheap", 0), ("too long and boring", 0), ("i regret watching",
0),
    ("terrible movie", 0), ("complete failure", 0), ("badly paced", 0),
    ("offensive and loud", 0), ("pointless story", 0), ("awful experience",
0),
    ("it was painful", 0), ("messy plot", 0), ("boring and dry", 0),
    ("zero stars", 0), ("worst film of year", 0), ("really bad", 0),
    ("uninteresting and plain", 0), ("it was terrible", 0)
]

data = pos_reviews + neg_reviews
np.random.shuffle(data) # Good practice to shuffle!

texts = [item[0] for item in data]
```

```
labels = np.array([item[1] for item in data])

# 2. Text Preprocessing
max_tokens = 100
sequence_length = 5
vectorize_layer = layers.TextVectorization(max_tokens=max_tokens,
output_sequence_length=sequence_length)
vectorize_layer.adapt(texts)
X_train = vectorize_layer(texts)

# 3. Build Model
inputs = layers.Input(shape=(sequence_length,), name="input_layer")
embedding = layers.Embedding(max_tokens, 16, name="embedding_layer")(inputs)

# Built-in Keras Attention
# We set return_attention_scores=True to get the weights for visualization
attention_output, weights = layers.Attention(name="attention_layer")(
    [embedding, embedding], return_attention_scores=True
)

# Pool the results and classify
flat = layers.GlobalAveragePooling1D()(attention_output)
outputs = layers.Dense(1, activation='sigmoid', name="sentiment_output")(flat)

model = models.Model(inputs=inputs, outputs=[outputs, weights])

# 4. Use a list for loss. 'None' for the attention weights output.
model.compile(
    optimizer='adam',
    loss=['binary_crossentropy', None],
    metrics=['accuracy', None]
)

# 5. Train
print("Training model...")
### Try changing the no of epochs and see how the attention in the test data
changes.
model.fit(X_train, labels, epochs=100, verbose=0)
print("Training complete.\n")

# 6. Predict and Interpret
test_sentence = ["the movie was boring"]
X_test = vectorize_layer(test_sentence)
prediction, attention_weights = model.predict(X_test)

# Map numbers back to words
vocab = vectorize_layer.get_vocabulary()
words = [vocab[idx] for idx in X_test[0].numpy() if idx != 0]

print(f"Sentence: '{test_sentence[0]}'")
print(f"Sentiment: {'Positive' if prediction[0] > 0.5 else 'Negative'}
({prediction[0][0]:.4f})")
print("-" * 40)

# Average weights across the query dimension to see overall word importance
avg_weights = np.mean(attention_weights[0], axis=0)

for word, weight in zip(words, avg_weights):
    bar = "▇" * int(weight * 40)
    print(f"{word:<10} | {weight:.4f} {bar}")
```

1.1.11.4 What Happens Without Attention?

To truly appreciate what attention does, let us remove it entirely. The cell below builds the same model architecture but skips the attention layer – the embeddings go straight to pooling and classification. Without attention, the model treats every word as equally

important. It can still learn sentiment to some degree (the prediction is still somewhat negative), but it loses the ability to *focus* on the words that matter most. Notice that we can no longer visualize per-word importance because there are no attention weights to inspect. This is why attention was such a breakthrough: it gives models both better performance *and* interpretability.

```
### Let us try without the attention layer:

# 50 Positive Samples
pos_reviews = [
    ("the movie was great", 1), ("i loved the acting", 1), ("simply amazing",
1),
    ("a masterpiece of cinema", 1), ("really fun and exciting", 1),
("incredible story line", 1),
    ("absolutely wonderful experience", 1), ("the best film ever", 1),
("highly recommended movie", 1),
    ("superb performance by all", 1), ("i enjoyed every minute", 1),
("brilliant directing", 1),
    ("a true classic film", 1), ("spectacular visuals", 1), ("i was
impressed", 1),
    ("pure joy to watch", 1), ("it was very touching", 1), ("a beautiful
story", 1),
    ("top notch acting", 1), ("very entertaining film", 1), ("the
cinematography was stunning", 1),
    ("captivating from start to finish", 1), ("refreshing and original", 1),
("greatest movie of year", 1),
    ("it made me happy", 1), ("an emotional journey", 1), ("wonderful cast",
1),
    ("powerful and moving", 1), ("delightful cinema", 1), ("i really liked
it", 1),
    ("fantastic plot", 1), ("breathtaking scenes", 1), ("perfectly executed",
1),
    ("charming movie", 1), ("excellent work", 1), ("it was so good", 1),
    ("lovely film", 1), ("bold and brave story", 1), ("sweet and funny", 1),
    ("a must watch", 1), ("i felt inspired", 1), ("outstanding production",
1),
    ("very well made", 1), ("smart and witty", 1), ("uplifting ending", 1),
    ("cool characters", 1), ("impressive quality", 1), ("honest and raw", 1),
    ("magical feeling", 1), ("i loved it", 1)
]

# 50 Negative Samples
neg_reviews = [
    ("the plot was boring", 0), ("terrible acting script", 0), ("i hated it",
0),
    ("waste of time", 0), ("it was a bad experience", 0), ("very dull and
slow", 0),
    ("worst movie ever", 0), ("extremely disappointed", 0), ("poorly
directed", 0),
    ("the script was weak", 0), ("did not like it", 0), ("annoying
characters", 0),
    ("boring story", 0), ("pathetic attempt at drama", 0), ("it was so noisy",
0),
    ("waste of money", 0), ("terrible directing", 0), ("ugly visuals", 0),
    ("i fell asleep", 0), ("the ending was bad", 0), ("not worth watching",
0),
    ("failed to impress", 0), ("silly plot", 0), ("boring and predictable",
0),
    ("i was very frustrated", 0), ("bad script writing", 0), ("horrible
acting", 0),
    ("nothing makes sense", 0), ("clunky dialogue", 0), ("very mediocre", 0),
    ("disaster of a film", 0), ("low quality production", 0), ("uninspired
film", 0),
    ("cheesy and cheap", 0), ("too long and boring", 0), ("i regret watching",
0),
```

```
    ("terrible movie", 0), ("complete failure", 0), ("badly paced", 0),
    ("offensive and loud", 0), ("pointless story", 0), ("awful experience",
0),
    ("it was painful", 0), ("messy plot", 0), ("boring and dry", 0),
    ("zero stars", 0), ("worst film of year", 0), ("really bad", 0),
    ("uninteresting and plain", 0), ("it was terrible", 0)
]

data = pos_reviews + neg_reviews
np.random.shuffle(data) # Good practice to shuffle!

texts = [item[0] for item in data]
labels = np.array([item[1] for item in data])

# 2. Text Preprocessing
max_tokens = 100
sequence_length = 5
vectorize_layer = layers.TextVectorization(max_tokens=max_tokens,
output_sequence_length=sequence_length)
vectorize_layer.adapt(texts)
X_train = vectorize_layer(texts)

# 3. Build Model
inputs = layers.Input(shape=(sequence_length,), name="input_layer")
embedding = layers.Embedding(max_tokens, 16, name="embedding_layer")(inputs)

# Built-in Keras Attention

### COMMENTED
# attention_output, weights = layers.Attention(name="attention_layer")(
#     [embedding, embedding], return_attention_scores=True
# )

# Pool the results and classify

## REPLACED this line to use embedding directly
# flat = layers.GlobalAveragePooling1D()(attention_output)
flat = layers.GlobalAveragePooling1D()(embedding)

outputs = layers.Dense(1, activation='sigmoid', name="sentiment_output")(flat)

# 5. Train
model.fit(X_train, labels, epochs=100, verbose=0)

## Weights not required
# model = models.Model(inputs=inputs, outputs=[outputs, weights])
model = models.Model(inputs, outputs)

### model.compile is simpler without attention layer
# 4. Use a list for loss. 'None' for the attention weights output.
#model.compile(
#    optimizer='adam',
#    loss=['binary_crossentropy', None],
#    metrics=['accuracy', None]
#)
model.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])


# 5. Train
print("Training model...")
### Try changing the no of epochs and see how the attention in the test data
changes.
model.fit(X_train, labels, epochs=100, verbose=0)
print("Training complete.\n")
```

```python
# 6. Predict and Interpret
test_sentence = ["the movie was boring"]
X_test = vectorize_layer(test_sentence)

## Since we do not have attention_weights, this line also changes
## prediction, attention_weights = model.predict(X_test)
prediction = model.predict(X_test)

# Map numbers back to words
vocab = vectorize_layer.get_vocabulary()
words = [vocab[idx] for idx in X_test[0].numpy() if idx != 0]

print(f"Sentence: '{test_sentence[0]}'")
print(f"Sentiment: {'Positive' if prediction[0] > 0.5 else 'Negative'}
({prediction[0][0]:.4f})")
print("-" * 40)

### ATTENTION WEIGHTS NOT AVAILABLE FOR THIS MODEL
# Average weights across the query dimension to see overall word importance
#avg_weights = np.mean(attention_weights[0], axis=0)

#for word, weight in zip(words, avg_weights):
#    bar = "█" * int(weight * 40)
#    print(f"{word:<10} | {weight:.4f} {bar}")
```

1.2 Calling LLMs: Providers and Fallback

Large language models are not monolithic. They are offered by a growing number of providers, each with its own models, pricing tiers, rate limits, and areas of strength. Understanding this landscape is essential for any business professional who wants to build reliable AI-powered applications rather than fragile prototypes that break the moment a single API goes down.

1.2.1 The Provider Landscape

The most recognizable name in the space is OpenAI, whose GPT family of models set the standard for conversational AI and general-purpose text generation. OpenAI offers powerful models such as GPT-4o through a pay-per-token API, making it straightforward to integrate into applications but potentially expensive at scale. Google's Gemini platform represents a strong alternative, with models that excel at multimodal tasks involving text, images, and code. For students and early-stage projects, Gemini is particularly attractive because it offers a generous free tier of 500 requests per day, enough for coursework and experimentation without spending a dollar.

Beyond these two commercial giants, a vibrant ecosystem of open-source and inference-optimized providers has emerged. Groq provides lightning-fast inference on open-source models like LLaMA and Mixtral using custom hardware, offering 30 requests per minute at no cost. HuggingFace hosts thousands of open-source models and provides free inference endpoints for many of them. Cohere offers 20 requests per minute on its free plan and specializes in enterprise retrieval and generation tasks. Grok, offered by xAI, provides $25 per month in free credits. OpenRouter acts as a unified gateway to dozens of models from multiple providers, including several free-tier options. Finally, Ollama allows you to run open-source models entirely on your own machine, which eliminates API costs altogether and keeps your data private.

Why does this diversity matter? In a business context, relying on a single LLM provider creates risk. Providers experience outages, impose rate limits that can throttle your application during peak usage, and adjust pricing without warning. Different models also have different strengths: one model may produce more creative marketing copy while another is better at structured data extraction. Having access to multiple providers gives you flexibility to optimize for cost, speed, capability, and availability simultaneously.

1.2.2 API-Based Access vs. Local Models

There are two fundamentally different ways to use an LLM. The first is through a cloud API, where you send a prompt over the internet to a provider's servers and receive a response. This is the approach taken by OpenAI, Gemini, Groq, and most other commercial services. API-based access requires no special hardware, starts working in minutes, and gives you access to the largest and most capable models available. The tradeoff is that you are sending your data to a third party, you are subject to rate limits and usage costs, and your application depends on network connectivity.

The second approach is running a model locally using tools like Ollama. Local models run on your own hardware, which means your data never leaves your machine. This is critical for applications involving sensitive business data, patient records, or proprietary information. Local models also have no rate limits and no per-request costs. The tradeoff is that you need a reasonably powerful machine (a modern laptop with 16 GB of RAM can run smaller models), and locally hosted models are generally less capable than the largest cloud-hosted ones. For the exercises in this book, cloud APIs with free tiers will be the primary path, but we will note where local models via Ollama are a viable alternative.

1.2.3 Automatic Fallback for Production Applications

Consider a scenario: you have built a customer-facing application that uses Gemini's API for generating responses. At 10 AM on a Tuesday, your application suddenly starts returning errors because you have exceeded Gemini's daily request quota. Your users see failures, your stakeholders lose confidence, and you scramble to rewrite code to point at a different

provider. This is exactly the kind of brittleness that separates a classroom demo from a production-grade application.

The solution is automatic fallback. Rather than hardcoding a single provider into your application, you configure a prioritized list of providers. When the first provider returns an error, whether due to rate limiting, authentication failure, or a server outage, your application automatically tries the next provider on the list. The user never notices the switch. This pattern is common in production systems across the technology industry, from content delivery networks to payment processing, and it applies equally well to LLM-powered applications.

1.2.4 The `llm_cascade` Package

To make fallback seamless throughout this book, every notebook uses the `llm_cascade` package, a lightweight Python library designed specifically for this course. You install it once, set your API keys as environment variables (or Colab Secrets), and call `get_cascade()` to get an LLM client that automatically detects which providers you have configured. When you call `llm.generate()`, the package tries your providers in order and falls back transparently if one fails. The response object tells you not only the generated text but also which provider and model actually handled the request.

This means you can follow along with every exercise in this book regardless of which API keys you have. If you only have a Gemini key, the cascade uses Gemini. If you have both Gemini and Groq keys, it tries Gemini first and falls back to Groq when needed. If you prefer to run everything locally, you can configure Ollama as your sole provider. The code in every subsequent chapter stays the same either way.

For students on a budget, the recommended starting point is to sign up for free-tier access to Gemini (500 requests per day) and Groq (30 requests per minute). Together, these two providers will handle virtually all of the exercises in this book at zero cost. Instructions for obtaining API keys and configuring them are provided in the `llm_cascade` reference page that follows this introduction.

The next page provides a complete technical reference for `llm_cascade`, including installation instructions, supported providers, model overrides, and embedding generation.

1.2.5 References

1. OpenAI. “API Reference.” OpenAI Platform Documentation, 2025. https://platform.openai.com/docs/api-reference
2. Google. “Gemini API Overview.” Google AI for Developers, 2025. https://ai.google.dev/gemini-api/docs
3. Groq. “Groq API Documentation.” GroqCloud Developer Hub, 2025. https://console.groq.com/docs

1.2.6 The llm_cascade Package

Every notebook in this book uses `llm_cascade` for automatic LLM provider fallback. Here's a quick reference.

1.2.6.1 Install

```
pip install git+https://github.com/KarAnalytics/llm_cascade.git
```

1.2.6.2 Quick start

```
from llm_cascade import get_cascade

llm = get_cascade()                          # auto-detects your API keys
response = llm.generate("What is Python?")   # falls back if one provider is
down
print(response.text)                         # the answer
print(response.provider)                     # which provider answered (e.g.,
"Gemini")
print(response.model)                        # which model was used
```

1.2.6.3 Supported providers

Provider	Env Variable	Free Tier
OpenAI	`OPENAI_API_KEY`	Limited free credits
Gemini	`GEMINI_API_KEY`	500 req/day
Ollama Cloud	`OLLAMA_API_KEY`	Free tier
Grok (xAI)	`XAI_API_KEY`	$25/month free
Groq	`GROQ_API_KEY`	30 req/min
HuggingFace	`HF_TOKEN`	Free inference
Cohere	`COHERE_API_KEY`	20 req/min
OpenRouter	`OPENROUTER_API_KEY`	Free models

Set any key in your `.env` file or Colab Secrets. The cascade tries them in order and falls back automatically on quota errors, auth failures, or server outages.

1.2.6.4 Override models per notebook

```
llm = get_cascade(models={"Gemini": "gemini-2.5-pro", "OpenAI": "gpt-4o"})
# or after creation:
llm.set_model("Gemini", "gemini-2.5-pro")
```

1.2.6.5 Embeddings (local, no API key needed)

```
embedding = llm.get_embedding("some text")  # uses all-MiniLM-L6-v2 locally
```

Source: github.com/KarAnalytics/llm_cascade

2. Retrieval-Augmented Generation (RAG)

2.1 RAG from First Principles

Large language models are remarkably capable, but they share three fundamental limitations that create serious problems for business applications. First, LLMs hallucinate: they generate confident, fluent answers that are factually wrong. A model asked about your company's Q3 revenue might invent a plausible-sounding number rather than admitting it does not know. Second, LLMs have a knowledge cutoff. Every model was trained on a snapshot of the world that ends on a specific date, meaning it cannot answer questions about events, policies, or data that emerged after training. Third, and most critically for enterprises, LLMs have no access to private data. Your internal HR policies, proprietary research reports, customer records, and strategic documents were never part of any model's training set, so the model simply cannot reason about them.

Retrieval-Augmented Generation, or RAG, solves all three problems with a single architectural pattern. Instead of asking the LLM to recall facts from memory, you first retrieve relevant documents from your own data sources, then augment the prompt with that retrieved context, and finally let the model generate an answer grounded in the evidence you provided. The model is no longer guessing. It is reading your documents and synthesizing an answer from them, much like a well-prepared analyst who reviews the relevant files before writing a memo.

2.1.1 The Retrieve-Augment-Generate Pipeline

A RAG system follows a straightforward three-stage pipeline. In the retrieval stage, the user's question is used to search a collection of documents and find the passages most relevant to the query. In the augmentation stage, those retrieved passages are assembled into a prompt alongside the user's question and any system-level instructions. In the generation stage, the LLM reads the augmented prompt and produces an answer that draws on the retrieved evidence.

What makes this elegant is that each stage is modular. You can swap out the retrieval mechanism (keyword search, semantic search, hybrid approaches) without changing the generation step. You can switch LLM providers without touching the retrieval logic. And you can update your document collection at any time without retraining anything. This modularity is what makes RAG the dominant approach for knowledge-grounded AI applications in production today.

2.1.2 Chunking Documents

Before documents can be searched, they must be broken into smaller pieces called chunks. A 50-page PDF cannot be fed into a model's context window in its entirety, nor would doing so help with precise retrieval. Instead, the document is split into passages, typically ranging from 200 to 1,000 tokens each, that are individually indexed and searchable.

Chunk size is a critical design decision that involves a genuine tradeoff. Small chunks (say, 200 tokens) provide high precision: when a chunk matches a query, the retrieved text is tightly focused on the relevant topic, and less irrelevant context is injected into the prompt. However, small chunks risk losing important surrounding context. A sentence that says "the company exceeded its targets" is meaningless without knowing which company and which targets. Larger chunks (800 to 1,000 tokens) preserve more context but may include irrelevant material that dilutes the signal or wastes precious tokens in the model's context window.

In practice, most RAG systems experiment with chunk sizes and use overlapping windows, where each chunk shares some text with the preceding and following chunks, to ensure that important information at chunk boundaries is not lost. There is no universally optimal chunk size; the right choice depends on the nature of your documents, the types of questions users ask, and the context window of your chosen LLM.

2.1.3 Embeddings and Semantic Search

The magic of modern RAG lies in semantic search powered by embeddings. As you explored in Chapter 1, an embedding is a dense numerical vector that captures the meaning of a piece of text. When two passages discuss similar concepts, their embedding vectors point in similar directions in high-dimensional space, even if they use completely different words. This is a dramatic improvement over traditional keyword search, which would fail to connect a query about "employee attrition" with a document discussing "staff turnover."

To perform semantic search, every chunk in your document collection is converted into an embedding vector and stored alongside its text. When a user asks a question, that question is also embedded into a vector using the same model. The system then computes the cosine similarity between the question vector and every chunk vector. Cosine similarity measures the angle between two vectors: a value of 1.0 means the vectors are identical in direction (maximally similar), while a value of 0.0 means they are orthogonal (unrelated). The chunks with the highest cosine similarity scores are returned as the most relevant results.

This entire process – embedding documents, embedding the query, computing similarities, and ranking results – can be implemented in a few dozen lines of Python using nothing more than an embedding API and NumPy. Building it from scratch, as you will do in the hands-on notebook that follows this introduction, is the best way to internalize how semantic search actually works before relying on frameworks that abstract it away.

2.1.4 Prompt Construction: Stuffing Context

Once the top-K most relevant chunks have been retrieved, they are "stuffed" into the LLM prompt. A typical prompt template looks something like: "You are a helpful assistant. Use the following context to answer the user's question. If the answer is not in the context, say you don't know." This is followed by the concatenated chunk texts and then the user's question.

This approach is called context stuffing, and it is deliberately simple. The LLM does not need any special fine-tuning or modification to work with retrieved context. You are simply providing it with reference material in the prompt, the same way you might hand a colleague a stack of documents before asking them a question. The quality of the answer depends heavily on the quality of the retrieved chunks: if the retrieval stage surfaces the right passages, the LLM will generally produce an accurate, well-grounded answer. If retrieval fails, no amount of prompt engineering will compensate.

2.1.5 RAG vs. Fine-Tuning

A common question is when to use RAG versus fine-tuning. Fine-tuning permanently modifies a model's weights by training it on additional data, effectively teaching the model new knowledge or behaviors. RAG leaves the model's weights untouched and instead provides external knowledge at inference time through the prompt.

RAG is the right choice when your knowledge base changes frequently (product catalogs, policy documents, news), when you need to cite sources and provide provenance for answers, when you want to maintain a single general-purpose model across multiple use cases, and when you need to get started quickly without the expense and complexity of training. Fine-tuning is more appropriate when you need to change the model's style or behavior (such as adopting a specific brand voice), when the task requires specialized reasoning patterns that prompting alone cannot achieve, or when you need the knowledge baked into the model for latency-critical applications where retrieval would add unacceptable delay.

In practice, many production systems combine both approaches: a fine-tuned model that has learned the organization's tone and reasoning style, augmented with RAG to provide up-to-date factual grounding. But for the vast majority of business use cases, RAG alone is sufficient and dramatically easier to implement and maintain.

2.1.6 What Comes Next

The hands-on notebook that follows this introduction walks you through building a complete RAG pipeline from scratch. You will chunk real documents, generate embeddings using an API, compute cosine similarity with NumPy, retrieve relevant passages, construct augmented prompts, and generate grounded answers – all without any frameworks or abstractions. This from-scratch implementation gives you the foundational understanding that will make the framework-based approaches in later chapters feel intuitive rather than magical.

2.1.7 References

1. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Kuttler, H., Lewis, M., Yih, W., Rocktaschel, T., Riedel, S., & Kiela, D. (2020). "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." *Advances in Neural Information Processing Systems*, 33, 9459-9474. https://arxiv.org/abs/2005.11401
2. Gao, Y., Xiong, Y., Dibia, V., Zhang, L., & Han, J. (2024). "Retrieval-Augmented Generation for Large Language Models: A Survey." *arXiv preprint arXiv:2312.10997*. https://arxiv.org/abs/2312.10997
3. OpenAI. "Embeddings Guide." OpenAI Platform Documentation, 2025. https://platform.openai.com/docs/guides/embeddings
4. Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., & Yih, W. (2020). "Dense Passage Retrieval for Open-Domain Question Answering." *Proceedings of EMNLP 2020*. https://arxiv.org/abs/2004.04906

Open in Colab

Code courtesy: This code has been adapted from towardsai.net/book code materials accompanying this book: Building LLMs for production

2.1.8 RAG from First Principles — No Framework, No Magic

In the previous chapter you may have seen RAG through a framework like LangChain, where a single function call hides all the plumbing. This notebook strips away every abstraction and builds the entire RAG pipeline by hand: chunk the text, embed the chunks, compute cosine similarity, retrieve the best matches, stuff them into a prompt, and call an LLM. By the end, you will understand every moving part well enough to build your own RAG system from scratch — or to debug one that someone else built with a framework.

2.1.8.1 Install and Import

We install only two things beyond standard Python: `openai` and `google-genai` for LLM access, `scikit-learn` for cosine similarity math, and `llm_cascade` — a lightweight wrapper that auto-detects whichever API key you have configured and routes requests to the cheapest available provider. There is no LangChain, no vector database library, no orchestration framework. Everything else we build ourselves.

```
!pip install -qU git+https://github.com/KarAnalytics/llm_cascade.git openai
google-genai scikit-learn
```

2.1.9 Step 1: Load and Chunk the Dataset

Every RAG pipeline starts with raw text. Ours comes from a CSV of blog articles about the LLaMA2 model. The challenge is that these articles are too long to embed as single units — embedding models work best on passages of a few hundred to a thousand characters. So we need to **chunk** them: break each article into fixed-size pieces that are small enough to embed meaningfully, yet large enough to carry coherent information.

2.1.9.1 Download the Data

We download two CSV files: one with the raw article text, and one with pre-computed embeddings (in case you want to skip the embedding step later). Both are hosted on GitHub so this notebook runs without any local file dependencies.

The dataset includes several articles from the TowardsAI blog, which provide an in-depth explanation of the LLaMA2 model. These articles will serve as our "knowledge base" — the documents that RAG will search through when answering questions.

2.1.9.2 Chunk the Articles

The `split_into_chunks` function below is intentionally simple: it slices text into fixed 1,024-character windows with no overlap. Each article gets split independently, and all chunks are collected into a single list. We then load them into a Pandas DataFrame so we can attach embedding vectors as a new column later. This is the "R" foundation of RAG — without good chunks, retrieval cannot find the right context.

```
!wget https://raw.githubusercontent.com/AlaFalaki/tutorial_notebooks/main/
data/mini-llama-articles.csv
!wget https://raw.githubusercontent.com/AlaFalaki/tutorial_notebooks/main/
data/mini-llama-articles-with_embeddings.csv
```

2.1.10 Step 2: Generate Embeddings

Now we convert each text chunk into a numerical vector. An **embedding** is a list of numbers (in our case, 384 numbers) that captures the *meaning* of a piece of text. Two chunks about similar topics will have vectors that point in roughly the same direction; two chunks

about unrelated topics will point in very different directions. This is the mathematical trick that makes semantic search possible — instead of matching keywords, we match meaning.

The embedding model we use (`all-MiniLM-L6-v2`) runs entirely on your machine. No API call, no cost, no data leaving your laptop. It is small and fast, but surprisingly effective for retrieval tasks.

```
chunks = []

# Load the file as a CSV
with open("./mini-llama-articles.csv", mode="r", encoding="utf-8") as file:
    csv_reader = csv.reader(file)

    for idx, row in enumerate(csv_reader):
        if idx == 0:
            continue
            # Skip header row
        chunks.extend(split_into_chunks(row[1]))
```

```
# Convert the JSON list to a Pandas Dataframe
df = pd.DataFrame(chunks, columns=["chunk"])

df.keys()
```

After embedding, every row in our DataFrame now has two columns: the original text chunk and its 384-dimensional vector. This DataFrame *is* our vector database — just a table in memory. Production systems use dedicated vector stores (Pinecone, Weaviate, ChromaDB) for speed and persistence, but the underlying data structure is the same: text paired with its embedding.

2.1.11 Step 3: Embed the User's Question

To find which chunks are relevant to a question, we need to put the question into the same vector space as the chunks. We embed the question using the exact same model (`all-MiniLM-L6-v2`), producing another 384-dimensional vector. Now we can compare apples to apples: the question vector lives in the same space as all the chunk vectors, and we can measure how "close" each chunk is to the question.

```
# Embeddings use a local HuggingFace model (all-MiniLM-L6-v2)
# No API key needed — runs locally via llm.get_embedding()
```

2.1.12 Step 4: Understand Cosine Similarity (A Quick Detour)

Before we search the full dataset, let us build intuition for how cosine similarity works. We will embed two sentences — one that clearly answers our question and one that is completely irrelevant — and compare their similarity scores to the question vector. If the math is working correctly, the relevant sentence should score close to 1.0 and the irrelevant one should score near 0.0.

Notice the scores: the irrelevant sentence ("The sky is blue") scores near zero, while the relevant sentence ("LLaMA2 model has a total of 2B parameters") scores close to 0.90. Cosine similarity ranges from −1 to 1, where 1 means identical direction in vector space. A score of 0.90 is very high — it tells us the embedding model "understands" that both the question and the good source are talking about the same topic. This is semantic search in action: no keyword matching, just meaning.

```
# Define the user question, and convert it to embedding.
QUESTION = "How many parameters LLaMA2 model has?"
QUESTION_emb = llm.get_embedding(QUESTION)

len(QUESTION_emb)
```

2.1.13 Step 5: Retrieve the Best Chunks

Now we apply cosine similarity at scale. We compare the question embedding against *every* chunk embedding in our DataFrame, producing a similarity score for each one. The chunks

with the highest scores are the ones whose meaning is closest to the question — these are our retrieved context. We pick the top 3 and will feed them into the LLM prompt in the next step.

Calculating the similarity of embedding representations can help us to find pieces of text that are close to each other. In the following sample you see how the Cosine Similarity metric can identify which sentence could be a possible answer for the given user question. Obviously, the unrelated answer will score lower.

```
BAD_SOURCE_emb = llm.get_embedding("The sky is blue.")
GOOD_SOURCE_emb = llm.get_embedding("LLaMA2 model has a total of 2B
parameters.")
```

The indices `[52, 89, 114]` point to the three chunks whose embeddings are closest to our question. Let us inspect them to see if the retrieval makes sense — do they actually talk about LLaMA2 parameters? If they do, our embedding-based search is working. If they do not, we would need to revisit our chunking strategy or try a different embedding model.

2.1.13.1 LLM Response Function

With the relevant chunks retrieved, we need a way to call an LLM. The `gemini_response` function below uses `llm_cascade` to send a prompt to whichever provider is available — Gemini, Ollama, Groq, OpenAI, or others. It accepts a system prompt (to set the LLM's role) and a user prompt (containing the question and context). This is a thin wrapper; the real work happens in how we *construct* the prompt in the next step.

2.1.14 Step 6: Augment the Prompt and Generate

This is the "A" and "G" in RAG — **Augment** and **Generate**. We take the three retrieved chunks and insert them directly into the prompt between `<START_OF_CONTEXT>` and `<END_OF_CONTEXT>` tags. The system prompt tells the LLM to act as a context-grounded assistant that only answers AI-related questions. By embedding the retrieved text right in the prompt, we give the LLM specific evidence to draw from instead of relying on whatever it memorized during pre-training. The result should be a factual, grounded answer that cites information from the actual articles.

```
number_of_chunks_to_retrieve = 3

# Sort the scores
highest_index = np.argmax(cosine_similarities)

# Pick the N highest scored chunks
indices = np.argsort(cosine_similarities[0])[::-1]
[:number_of_chunks_to_retrieve]
print(indices)
```

2.1.14.1 Bonus: How RAG Defeats Knowledge Cutoff and Hallucination

Every LLM has a **knowledge cutoff** — the date when its training data ended. Ask about events or models released after that date, and the LLM will either refuse to answer or, worse, confidently make something up (hallucinate). RAG solves this elegantly: by stuffing fresh, factual context into the prompt, we override the model's stale parametric memory with up-to-date evidence.

The next two cells demonstrate this with a concrete example. We provide a chunk about LLaMA 4 (released after most models' training cutoff) and ask the same question with and without that context. Watch how the augmented version gives a precise, correct answer while the non-augmented version either admits ignorance or hallucinates outdated information.

2.1.14.2 LLM Response Function (via llm_cascade)

Instead of a hardcoded Gemini or OpenAI client, we use `llm.generate()` from `llm_cascade`. It automatically picks the first available provider and falls back if quota is exhausted. Each call prints which provider/model was used.

```
# Using llm_cascade instead of hardcoded Gemini client

def gemini_response(system_prompt, prompt):
    """Generate a response using llm_cascade (auto-fallback across
providers)."""
    try:
        response = llm.generate(prompt, system_prompt=system_prompt)
        return response.text
    except Exception as e:
        return f'Error: {e}'
```

2.1.15 Without Augmentation — the Control Experiment

Now we ask the exact same question but *without* providing any context. The LLM must rely entirely on its parametric knowledge — whatever it memorized during training. If its training data predates the LLaMA 4 release, it will either say "I don't know" (the honest answer) or hallucinate incorrect details (the dangerous answer). Compare this output to the augmented version above to see exactly how much value the retrieval step adds.

2.1.15.1 What We Built — and What Comes Next

Let us recap the full pipeline we built from scratch:

1. **Chunk** — Split raw articles into 1,024-character pieces.
2. **Embed** — Convert each chunk into a 384-dimensional vector using a local sentence-transformer.
3. **Store** — Keep the chunks and their embeddings in a simple Pandas DataFrame (our DIY vector database).
4. **Query** — Embed the user's question with the same model.
5. **Retrieve** — Use cosine similarity to find the top-k most relevant chunks.
6. **Augment** — Stuff those chunks into the LLM prompt as context.
7. **Generate** — Let the LLM produce a grounded answer.

Every RAG framework — LangChain, LlamaIndex, Haystack — automates these exact seven steps. Now that you have built them by hand, you understand what those frameworks are doing under the hood. In the next chapters, we will use LangChain to do this more concisely, and we will explore RAG beyond documents: with SQL databases and graph databases as knowledge sources.

```
from google import genai
from google.genai import types

# Use the Gemini API to answer the questions based on the retrieved pieces of
text.
try:
    # Formulating the system prompt and condition the model to answer only AI-
related questions.
    system_prompt = """You are an assistant and expert in answering questions
from a chunks of content.
                       Only answer AI-related question, else say that you
cannot answer this question."""

    # User prompt with the user's question
    prompt = """
        Read the following informations that might contain the context you
require to answer the question.
        You can use the informations starting from the <START_OF_CONTEXT> tag
and end with the <END_OF_CONTEXT> tag.
        Here is the content:\n\n<START_OF_CONTEXT>\n{}\n<END_OF_CONTEXT>\n\n"
```

```
        Please provide an informative and accurate answer to the following
question based on the avaiable context.
        Be concise and take your time. \nQuestion: {}\nAnswer:"
      """

    # Add the retrieved pieces of text to the prompt.
    prompt = prompt.format("".join(df.chunk[indices]), QUESTION)


    response = gemini_response(system_prompt,prompt)
    print(response)

except Exception as e:
    print(f"An error occurred: {e}")

# Consider this as a retrieved chunk
# https://ai.meta.com/blog/llama-4-multimodal-intelligence/

Example_chunk = """
# Llama 4 Technical Overview

Meta has introduced the Llama 4 model family, marking a significant
advancement in open-weight multimodal AI with three distinct variants designed
for different use cases. The Llama 4 Scout represents the more compact option
with 17 billion active parameters utilizing 16 experts within a total
parameter count of 109 billion, designed to fit on a single H100 GPU with Int4
quantization. The Llama 4 Maverick serves as the flagship model with 17
billion active parameters but employs 128 experts across 400 billion total
parameters, fitting on a single H100 host and designed as the primary
workhorse for general assistant and chat applications. The preview model Llama
4 Behemoth functions as a teacher model with 288 billion active parameters, 16
experts, and nearly 2 trillion total parameters, currently still in training
but demonstrating state-of-the-art performance on mathematical and reasoning
benchmarks.
The architecture represents Meta's first implementation of mixture-of-experts
(MoE) design in the Llama series, fundamentally changing how the models
process information by activating only a fraction of total parameters for each
token. The MoE architecture alternates between dense and mixture-of-experts
layers, with the Maverick model specifically using 128 routed experts
alongside a shared expert, where each token is processed by the shared expert
and routed to one of the 128 specialized experts. This design significantly
improves compute efficiency during both training and inference while
delivering higher quality outputs compared to dense models with equivalent
computational budgets.
Native multimodality represents another breakthrough, achieved through early
fusion architecture that seamlessly integrates text and vision tokens into a
unified model backbone during pre-training. This approach enables joint
training with massive amounts of unlabeled text, image, and video data, with
the vision encoder based on MetaCLIP but trained separately in conjunction
with a frozen Llama model to optimize adaptation to the language model. The
models support up to 48 images during pre-training and demonstrate strong
performance with up to 8 images in post-training scenarios, enabling
sophisticated visual reasoning and understanding tasks across multiple input
modalities.
The training infrastructure incorporates several technical innovations,
including the MetaP technique for reliable hyperparameter optimization that
transfers well across different model configurations and training scenarios.
The models were trained on over 30 trillion tokens, representing more than
double the training data of Llama 3, with support for 200 languages including
over 100 languages with more than 1 billion tokens each. Training efficiency
was maximized through FP8 precision without quality sacrifice, achieving 390
TFLOPs per GPU utilization on 32,000 GPUs during Behemoth pre-training, while
a specialized mid-training phase enhanced core capabilities and enabled the
industry-leading 10 million token context length for Scout.
Context length capabilities represent a dramatic advancement, with Llama 4
Scout supporting 10 million tokens compared to Llama 3's 128K limit through
```

```
the innovative iRoPE architecture that employs interleaved attention layers
without positional embeddings and inference-time temperature scaling of
attention. This architecture enables superior length generalization and opens
possibilities for multi-document summarization, extensive user activity
parsing for personalization, and reasoning over vast codebases, with
compelling performance demonstrated in retrieval tasks and cumulative negative
log-likelihood evaluations over 10 million tokens of code.
The post-training pipeline underwent significant revision with a three-stage
approach consisting of lightweight supervised fine-tuning, online
reinforcement learning, and lightweight direct preference optimization. A
critical insight involved removing over 50% of data tagged as easy using Llama
models as judges, focusing training on harder examples to prevent over-
constraining that could restrict exploration during online RL and lead to
suboptimal performance in reasoning, coding, and mathematics. The continuous
online RL strategy alternated between model training and adaptive data
filtering to retain only medium-to-hard difficulty prompts, proving highly
beneficial for compute and accuracy trade-offs.
Safety and bias mitigation received comprehensive attention through multi-
layered approaches spanning pre-training data filtering, post-training policy
conformance, and system-level safeguards. Meta introduced Generative Offensive
Agent Testing (GOAT) to address traditional red-teaming limitations by
simulating multi-turn interactions of medium-skilled adversarial actors,
enabling more efficient vulnerability detection while allowing human experts
to focus on novel adversarial areas. Particular emphasis was placed on
addressing political and social bias inherent in internet training data, with
Llama 4 demonstrating significant improvements over Llama 3 in presenting
balanced perspectives on contentious issues without favoring particular
viewpoints.
The distillation process for creating smaller models from Behemoth required
novel approaches, including a dynamic loss function that weighted soft and
hard targets throughout training, with codistillation during pre-training
amortizing computational costs for the majority of training data. Post-
training the 2 trillion parameter Behemoth model necessitated pruning 95% of
supervised fine-tuning data compared to 50% for smaller models, followed by
large-scale reinforcement learning focused on sampling hard prompts through
pass@k analysis and constructing training curricula of increasing difficulty.
The unprecedented scale required complete infrastructure overhaul, including
optimized MoE parallelization and fully asynchronous online RL training
framework that improved training efficiency by approximately 10x over previous
generations through flexible GPU allocation and resource balancing across
multiple models.
"""

QUESTION = "How many parameters LLaMA 4 model has?"

# Formulating the system prompt
system_prompt = """ You are an assistant and expert in answering questions
from a chunks of content.
                    Only answer AI-related question, else say that you cannot
answer this question."""

# Combining the system prompt with the user's question
prompt = """Read the following informations that might contain the context you
require to answer the question.
            You can use the informations starting from the <START_OF_CONTEXT>
tag and end with the <END_OF_CONTEXT> tag.
            Here is the content:
\n\n<START_OF_CONTEXT>\n{}\n<END_OF_CONTEXT>\n\n".
            Please provide an informative and accurate answer to the following
question based on the avaiable context.
            Be concise and take your time. \nQuestion: {}\nAnswer:
            """

prompt = prompt.format(Example_chunk, QUESTION)
```

```
response = gemini_response(system_prompt,prompt)
print(response)

QUESTION = "How many parameters LLaMA 4 model has?"

# Formulating the system prompt
system_prompt = "You are an assistant and expert in answering questions."

# Combining the system prompt with the user's question
prompt = "Be concise and take your time to answer the following question.
\nQuestion: {}\nAnswer:"

prompt = prompt.format(QUESTION)

response = gemini_response(system_prompt,prompt)
print(response)
```

2.2 RAG with Vector Databases

In the previous chapter, you built a RAG pipeline from scratch using NumPy arrays to store embeddings and brute-force cosine similarity to find relevant passages. That approach is excellent for learning, but it does not scale. When your document collection grows from a handful of passages to thousands or millions of chunks, storing all embeddings in a Python list and comparing every one of them against every query becomes prohibitively slow and memory-intensive. This is the problem that vector databases were designed to solve.

2.2.1 What Vector Databases Do

A vector database is a specialized data store optimized for three operations: storing high-dimensional embedding vectors alongside their metadata, indexing those vectors using data structures that enable fast approximate search, and retrieving the most similar vectors to a given query vector without scanning the entire collection. Traditional relational databases are built around exact matches on structured fields – find all customers where `state = 'Kansas'`. Vector databases are built around similarity – find the ten document chunks whose meaning is closest to this question.

Under the hood, vector databases use approximate nearest neighbor (ANN) algorithms such as HNSW (Hierarchical Navigable Small World) graphs or IVF (Inverted File Index) structures. These algorithms trade a small amount of accuracy for massive speed improvements. Instead of computing similarity against every vector in the collection, they navigate an index structure that quickly narrows the search to a small neighborhood of likely candidates. The result is that a query against a million vectors can return results in milliseconds rather than seconds.

2.2.2 ChromaDB: A Practical Starting Point

This book uses ChromaDB as its primary vector database because it strikes an ideal balance between simplicity and capability. ChromaDB is open-source, installs with a single `pip install` command, runs entirely in-process without requiring a separate server, and provides a clean Python API that closely mirrors the conceptual steps of RAG. For learning purposes, ChromaDB removes all infrastructure friction so you can focus on understanding the patterns rather than wrestling with deployment configurations.

ChromaDB also handles embedding generation automatically if you want it to, though in our notebooks we generate embeddings explicitly using the `llm_cascade` package so that you see every step. Collections in ChromaDB function like tables in a relational database: each collection holds a set of documents, their embedding vectors, and optional metadata that can be used for filtering during search.

2.2.3 The Indexing Step

Building a searchable knowledge base with a vector database follows the same conceptual pipeline you implemented manually in the previous chapter, but with the storage and indexing handled for you. First, you load your source documents and split them into chunks of appropriate size. Second, you generate an embedding vector for each chunk using an embedding model. Third, you store each chunk's text, embedding, and metadata in a ChromaDB collection. ChromaDB automatically builds an index over the stored vectors so that subsequent searches are fast.

This indexing step is typically performed once (or periodically as documents are updated) and represents the "offline" phase of a RAG system. The computational cost is dominated by the embedding generation, since every chunk must pass through the embedding model. Once indexed, the collection is ready to serve queries.

2.2.4 The Query Step

When a user asks a question, the query step mirrors retrieval in reverse. The question is first converted into an embedding vector using the same embedding model that was used during indexing – this consistency is essential, since vectors from different models are not comparable. ChromaDB then performs a similarity search against its indexed collection and

returns the top-K most similar chunks, along with their text, metadata, and similarity scores. These retrieved chunks are assembled into a prompt, and the LLM generates a grounded answer.

The entire query path – embed, search, retrieve, generate – typically completes in one to two seconds, with the LLM generation step accounting for the majority of that latency. The vector search itself, even over large collections, is nearly instantaneous thanks to the ANN index.

2.2.5 The Broader Vector Database Landscape

ChromaDB is an excellent choice for learning and prototyping, but production applications often require databases with different characteristics. Pinecone is a fully managed cloud service that eliminates all operational overhead and scales automatically, making it popular with teams that want to avoid managing infrastructure. Weaviate is an open-source vector database with a rich feature set including hybrid search that combines semantic and keyword matching. Milvus, originally developed at Zilliz, is designed for very large-scale deployments handling billions of vectors. FAISS, developed by Meta, is a library rather than a database – it provides highly optimized similarity search algorithms but requires you to manage storage and persistence yourself.

The choice among these options depends on your requirements. For a classroom exercise or an internal prototype, ChromaDB is ideal. For a production application with moderate scale and a preference for managed services, Pinecone is a strong choice. For large-scale enterprise deployments with complex filtering and hybrid search needs, Weaviate or Milvus offer the necessary capabilities. FAISS is best suited for research environments or applications where you need maximum control over the search algorithms.

2.2.6 What Comes Next

The hands-on notebooks that follow this introduction apply the vector database approach to real-world datasets. You will see how to load documents into ChromaDB, build searchable collections, perform similarity queries, and generate grounded answers – all following the same retrieve-augment-generate pattern from the previous chapter, but now with infrastructure that scales gracefully beyond toy examples.

2.2.7 References

1. Chroma. "ChromaDB Documentation." Chroma, 2025. https://docs.trychroma.com/
2. Malkov, Y. A., & Yashunin, D. A. (2020). "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs." *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4), 824-836. https://arxiv.org/abs/1603.09320
3. Johnson, J., Douze, M., & Jegou, H. (2021). "Billion-Scale Similarity Search with GPUs." *IEEE Transactions on Big Data*, 7(3), 535-547. https://arxiv.org/abs/1702.08734
4. Pinecone. "What is a Vector Database?" Pinecone Learning Center, 2025. https://www.pinecone.io/learn/vector-database/

Open in Colab

2.2.8 RAG Example (North America): ChromaDB + LLM Cascade

Every large language model has a knowledge cutoff – a date beyond which it simply has not seen the world. Ask it about last quarter's GDP figures or your company's internal policies, and it will either confess ignorance or, worse, confidently hallucinate an answer. **Retrieval-Augmented Generation (RAG)** solves this by giving the model a research assistant: before answering, the system retrieves relevant documents from a knowledge base and injects them into the prompt as context. The model then generates its response grounded in real evidence rather than memory alone.

In this notebook we build a compact RAG pipeline over the **CIA World Factbook** – a public-domain dataset that contains structured intelligence about every country on earth, from population statistics to trade balances to energy production. We scope the demo to **North America only** so that indexing runs in under a minute on free-tier hardware, but the architecture generalizes to any document corpus.

What you will learn:

- How to ingest structured JSON and flatten it into retrieval-ready text documents
- How recursive chunking with overlap preserves context continuity at chunk boundaries
- How ChromaDB handles embedding and vector storage automatically, with no external API key required
- How grounding an LLM on retrieved context changes its answers – and where it still falls short

Why ChromaDB? Unlike cloud vector databases that require API keys and billing, ChromaDB runs entirely in-process. It pairs with a local sentence-transformer model (`all-MiniLM-L6-v2`) to embed your text, so the entire pipeline – from ingestion to retrieval to generation – works offline and for free. That makes it ideal for classroom demos where you want students focused on the concepts, not on credential management.

```
%pip install -q -U chromadb langchain-community langchain-text-splitters
pandas requests git+https://github.com/KarAnalytics/llm_cascade.git sentence-
transformers
```

2.2.8.1 1) Imports and Environment Setup

Before we build anything, we need our toolkit. The libraries below each play a distinct role in the RAG pipeline: `pandas` for tabular inspection, `requests` for fetching raw data from GitHub, `chromadb` for vector storage and retrieval, and `RecursiveCharacterTextSplitter` from LangChain for intelligent chunking. Notice that we do not import any embedding library explicitly – ChromaDB will handle that internally when we configure it later.

API keys are resolved automatically. In Google Colab, you can store them in Secrets (e.g., `GEMINI_API_KEY`, `OPENAI_API_KEY`, `GROQ_API_KEY`). Outside Colab, the code falls back to environment variables. The `llm_cascade` library tries each configured provider in order, so you only need one working key to proceed.

```
import os
import pandas as pd
import requests
import chromadb
from langchain_text_splitters import RecursiveCharacterTextSplitter

from llm_cascade import get_cascade
```

```
llm = get_cascade()


def generate_text(prompt, system_prompt=None, **kwargs):
    response = llm.generate(prompt, system_prompt=system_prompt)
    return response.text, response.provider


def has_llm_provider():
    return True  # llm_cascade auto-detects available providers
```

2.2.8.2 2) Build a Minimal RAG Pipeline

A RAG pipeline has four distinct phases, each solving a different problem. First, **data ingestion** fetches raw documents and converts them into plain text that an embedding model can understand. Second, **chunking** splits long documents into smaller, overlapping pieces so that retrieval can zero in on the most relevant passage rather than returning an entire country profile. Third, **vector storage** embeds each chunk into a high-dimensional vector and indexes it for fast similarity search. Fourth, **grounded generation** retrieves the top-K most relevant chunks for a user question and feeds them to the LLM as context, constraining its answer to what the evidence actually says.

We will build each phase as a standalone function, then wire them together at the end. This modular design makes it easy to swap components – for example, replacing ChromaDB with Pinecone, or changing the chunking strategy – without rewriting the rest of the pipeline.

2.2.8.2.1 Function: `_collect_text_lines`

The CIA World Factbook stores each country as deeply nested JSON – dictionaries inside dictionaries inside lists. A vector database, however, needs flat text strings to embed. This recursive helper walks the entire JSON tree, collects every leaf value, and pairs it with its full key path (e.g., `Economy.GDP.real growth rate.text: 2.1%`). The result is a list of human-readable lines that preserve the hierarchical context of each fact.

Why does this matter? If we simply dumped the raw JSON into ChromaDB, the embedding model would waste capacity on braces, brackets, and structural tokens that carry no semantic meaning. By flattening first, every token in the embedded text contributes to the meaning of the passage, which directly improves retrieval quality when a student later asks "What is Mexico's GDP growth rate?"

```
# --- PHASE 1: DATA INGESTION (Pandas + GitHub) ---
def _collect_text_lines(obj, parent_key=""):
    """Recursively flatten nested JSON into readable key-value text lines."""
    lines = []

    if isinstance(obj, dict):
        for k, v in obj.items():
            next_key = f"{parent_key}.{k}" if parent_key else str(k)
            lines.extend(_collect_text_lines(v, next_key))
    elif isinstance(obj, list):
        for i, item in enumerate(obj):
            next_key = f"{parent_key}[{i}]" if parent_key else f"[{i}]"
            lines.extend(_collect_text_lines(item, next_key))
    else:
        if obj is None:
            return []
        text = str(obj).strip()
        if text:
            lines.append(f"{parent_key}: {text}")
    return lines
```

2.2.8.2.2 Function: `_get_country_json_paths`

The full CIA World Factbook covers over 250 countries and territories, which would take several minutes to download and index on free-tier hardware. For a classroom demo, speed

matters more than completeness, so this function filters the GitHub repository tree to keep only files under `north-america/`. That gives us seven countries – enough to demonstrate every RAG concept while keeping the total pipeline runtime under 60 seconds.

The function uses the GitHub Trees API to list all files in the repository without downloading them, then applies the region filter. The sorted, deduplicated output ensures that every student in the room gets the same document order, which makes it easier to compare results during class discussion.

```
def _get_country_json_paths():
    """Discover country JSON files, restricted to North America."""
    tree_url = "https://api.github.com/repos/factbook/factbook.json/git/trees/
master?recursive=1"
    r = requests.get(tree_url, timeout=30)
    r.raise_for_status()
    tree = r.json().get("tree", [])

    allowed_prefixes = ("north-america/",)
    country_paths = []
    for item in tree:
        path = item.get("path", "")
        if item.get("type") == "blob" and path.endswith(".json") and "/" in
path:
            if path.lower().startswith(allowed_prefixes):
                country_paths.append(path)

    # Deduplicate and sort for stable classroom demos.
    return sorted(set(country_paths))
```

2.2.8.2.3 Function: `get_world_data`

This is the orchestrator for the entire ingestion phase. It discovers country files, downloads each one, flattens the nested JSON into readable text using `_collect_text_lines`, and packages the result as a list of document dictionaries – each containing a `content` string and a `metadata` dict with the country code and source path. The metadata travels alongside the content through chunking and indexing, so that when we retrieve a chunk later, we always know which country it came from.

Think of this function as the "ETL" step of the RAG pipeline: it extracts raw data from an external source, transforms it into a format suitable for embedding, and loads it into memory ready for the next phase.

```
def get_world_data():
    """Fetch all country records and convert each into one retrieval-ready
full-text document."""
    print("Step 1: Discovering country files from GitHub...")
    base_url = "https://raw.githubusercontent.com/factbook/factbook.json/
master"
    country_paths = _get_country_json_paths()
    print(f"Discovered {len(country_paths)} country files.")

    docs = []
    for path in country_paths:
        try:
            r = requests.get(f"{base_url}/{path}", timeout=20)
            r.raise_for_status()
            payload = r.json()
        except Exception as exc:
            print(f"Skipping {path}: {exc}")
            continue

        lines = _collect_text_lines(payload)
        if not lines:
            continue
```

```
        country_code = path.split("/")[-1].replace(".json", "").upper()
        combined_text = (
            f"COUNTRY_CODE: {country_code}\n"
            f"SOURCE_PATH: {path}\n\n"
            + "\n".join(lines)
        )

        docs.append(
            {
                "content": combined_text,
                "metadata": {
                    "country": country_code,
                    "path": path,
                },
            }
        )

    print(f"Loaded {len(docs)} country documents.")
    return docs
```

2.2.8.2.4 Function: `chunk_documents`

A single country document can run to thousands of lines – far too long for an embedding model to capture in one vector. Chunking solves this by splitting each document into smaller, digestible pieces. We use LangChain's `RecursiveCharacterTextSplitter`, which tries to break text at natural boundaries (paragraph breaks, then sentence boundaries, then word boundaries) before falling back to a hard character limit.

The two key parameters are `chunk_size` (800 characters by default) and `chunk_overlap` (100 characters). The overlap is crucial: without it, a fact that straddles two chunks would be split in half, and neither chunk would contain the complete information. Experimenting with these values is one of the most impactful tuning knobs in any RAG system.

```
## --- PHASE 2: RECURSIVE CHUNKING ---
def chunk_documents(docs, chunk_size=800, chunk_overlap=100):
    """Chunk long records so semantic retrieval can match smaller relevant
spans."""
    print("Step 2: Chunking with overlap...")
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=chunk_size,
        chunk_overlap=chunk_overlap,
    )

    chunks, metadatas, ids = [], [], []
    for doc in docs:
        split_texts = splitter.split_text(doc["content"])
        for i, text in enumerate(split_texts):
            chunks.append(text)
            metadatas.append(doc["metadata"])
            ids.append(f"{doc['metadata']['country']}_{i}")
    return chunks, metadatas, ids


# Small inspection check for students before moving on.
docs_preview = get_world_data()
print(f"Loaded {len(docs_preview)} source documents.")
print("Example metadata:", docs_preview[0]["metadata"] if docs_preview else
"No documents loaded")
```

2.2.8.2.5 Checkpoint 1: Data and Chunking Reflection

Pause here and consider these questions before moving on. They highlight the design decisions that most affect RAG quality downstream.

1. Why do we transform structured JSON fields into one text document per country before retrieval? What would go wrong if we indexed the raw JSON directly?

2. How might changing `chunk_size` from 800 to 300 affect precision and recall? Would the answer to "What is Canada's population?" improve or degrade?
3. What is the purpose of overlap, and what failure mode does it reduce?

2.2.8.2.6 Function: `build_vector_store`

This function wires together the embedding model selection, collection management, and batch upsert logic. We explicitly specify `all-MiniLM-L6-v2` as the embedding model – a compact, fast sentence-transformer that runs locally without any API key. It produces 384-dimensional vectors and handles English text well enough for our factbook data.

The function deletes any existing collection before creating a fresh one. This is a classroom convenience: it prevents the confusing situation where a student runs the notebook twice and ends up with duplicate chunks that skew retrieval results. In production, you would instead use versioned collections or incremental upserts with deduplication logic.

```
def build_vector_store(docs, db_path="./factbook_db",
collection_name="world_facts", batch_size=5000):
    """Create (or reset) a Chroma collection and upsert chunked records in
batches."""
    chunks, metadatas, ids = chunk_documents(docs)

    chroma_client = chromadb.PersistentClient(path=db_path)

    # Explicitly specify the embedding model that converts text chunks into
vectors
    from chromadb.utils.embedding_functions import
SentenceTransformerEmbeddingFunction
    embedding_fn = SentenceTransformerEmbeddingFunction(model_name="all-
MiniLM-L6-v2")
    print("  Embedding model: all-MiniLM-L6-v2 (local, no API key needed)")

    # Delete any existing collection to avoid embedding function conflicts,
then create fresh
    try:
        chroma_client.delete_collection(name=collection_name)
    except Exception:
        pass
    collection = chroma_client.create_collection(
        name=collection_name,
        embedding_function=embedding_fn,
    )

    print("Step 3: Storing vectors in ChromaDB...")
    for start in range(0, len(ids), batch_size):
        stop = start + batch_size
        collection.upsert(
            documents=chunks[start:stop],
            metadatas=metadatas[start:stop],
            ids=ids[start:stop],
        )

    print(f"Stored {len(ids)} chunks in collection '{collection_name}'.")
    return collection

collection = build_vector_store(docs_preview)
```

2.2.8.2.7 2B) Retrieve Context and Generate a Grounded Answer

This is where the "retrieval" meets the "generation" in RAG. When a user asks a question, we first embed the question using the same sentence-transformer model and find the top-K nearest chunks in the vector store. Those chunks become the context paragraph in the LLM prompt, and a carefully worded system instruction tells the model to answer **only** from the provided context – if the answer is not there, it should say so rather than guessing.

This retrieve-then-generate pattern is the beating heart of every RAG system. The retrieval step acts as an evidence filter, dramatically narrowing the LLM's attention from its entire training corpus down to a handful of relevant passages. The prompting constraint – "use ONLY the provided context" – further reduces hallucination by giving the model explicit permission to say "I don't know." Together, these two mechanisms produce answers that are both relevant and verifiable: a human can inspect the retrieved chunks to confirm whether the model's claims are actually supported by the evidence.

2.2.8.2.8 Functions: `answer_with_rag` and `answer_without_rag`

These two functions are the core of our teaching comparison. `answer_with_rag` performs the full retrieve-then-generate pipeline: it queries ChromaDB for the top-K chunks, assembles them into a context block, and prompts the LLM to answer only from that evidence. `answer_without_rag` skips retrieval entirely and sends the bare question to the LLM, relying solely on whatever the model memorized during pre-training.

Running the same question through both functions reveals exactly what RAG adds – and what it costs. The RAG answer will be grounded in specific, verifiable data from the Factbook (exchange rates, population figures, trade balances), while the non-RAG answer will draw on the model's general knowledge, which may be outdated, vague, or simply wrong. Comparing them side by side is the single most effective way to build intuition for when RAG is worth the engineering overhead and when a bare LLM call suffices.

```
def answer_with_rag(
    question,
    collection,
    n_results=4,
    **kwargs,
):
    """Retrieve nearest chunks, then answer using the LLM cascade."""
    print(f"Step 4: Retrieving context for: '{question}'")
    results = collection.query(query_texts=[question], n_results=n_results)
    context = "\n\n".join(results["documents"][0])

    print("Step 5: Generating grounded answer...")
    system_prompt = "You are a careful analyst. Use ONLY the provided context
to answer. If the answer is not in the context, say you don't know."
    user_prompt = f"CONTEXT:\n{context}\n\nQUESTION:\n{question}"
    answer_text, provider = generate_text(
        prompt=user_prompt,
        system_prompt=system_prompt,
    )
    print(f"Provider used: {provider}")
    return answer_text, context


def answer_without_rag(question, **kwargs):
    """Direct LLM answer without retrieval."""
    answer_text, provider = generate_text(prompt=question)
    print(f"Provider used: {provider}")
    return answer_text
```

2.2.8.2.9 2C) Run an End-to-End Query

Now we put it all together. The cell below runs one or more comparison questions through both pathways – with and without RAG – and prints the results side by side. Pay close attention to the `[RETRIEVED CONTEXT PREVIEW]` section: it shows you exactly which chunks the vector store returned. This is your audit trail. If the RAG answer is wrong, the first thing to check is whether the retrieved context actually contained the relevant information. A bad answer from good context is a generation problem (fix the prompt); a bad answer from irrelevant context is a retrieval problem (fix the chunking or the query).

Tip: try uncommenting the other comparison questions to see how the system handles hallucination tests (asking about fictional "Atlantis"), cross-country comparisons, and questions that blend factbook data with general world knowledge.

2.2.8.2.10 Function: `preview`

This small helper trims long model outputs and retrieved context so the classroom output stays readable. Without it, a single country's flattened text can fill several screens, burying the pedagogical signal in noise. The `max_len` parameter defaults to 900 characters for answers and 1,200 for context, which is enough to see the key facts without scrolling endlessly.

```
comparison_questions = [
    #{
    #    "type": "Hallucination test (out-of-scope)",
    #    "question": "What is the current prime minister of Atlantis, and what
is Atlantis GDP growth?",
    #},
    {
        "type": "Local info extraction",
        "question": "Summarize Mexico's economy overview in 3 bullet points.",
    },
    #{
    #    "type": "Local info extraction",
    #    "question": "What is the population and GDP of Canada?",
    #},
    #{
    #    "type": "Local + public knowledge blend",
    #    "question": "Explain how the United States trade policies may
influence its North American neighbors.",
    #},
]


def preview(text, max_len=900):
    text = text or ""
    return text[:max_len] + ("..." if len(text) > max_len else "")


if "collection" not in globals():
    print("Error: Build the Chroma vector store first before running this
comparison.")
elif not has_llm_provider():
    print("Error: No LLM API key configured. Set at least one API key in Colab
Secrets.")
else:
    try:
        for i, item in enumerate(comparison_questions, start=1):
            q_type = item["type"]
            q = item["question"]
            print("=" * 90)
            print(f"Q{i}. {q_type}")
            print(f"QUESTION: {q}")
            print()

            direct_answer = answer_without_rag(q)
            print("[WITHOUT RAG]")
            print(preview(direct_answer))

            rag_answer, rag_context = answer_with_rag(q, collection,
n_results=4)
            print("[WITH RAG]")
            print(preview(rag_answer))

            print("[RETRIEVED CONTEXT PREVIEW]")
```

```
        print(preview(rag_context, max_len=1200))
        print()
except Exception as e:
    print(f"Error: {e}")
```

2.2.8.2.11 Checkpoint 4: Evaluation Reflection

Now that you have seen the with-RAG and without-RAG answers side by side, reflect on what the comparison reveals about each approach's strengths and weaknesses.

1. Compare the RAG answer against the retrieved context preview. Which specific claims in the answer are directly supported by the retrieved chunks, and which (if any) appear to be the model filling in gaps?
2. What additional evidence would improve confidence in the answer? Would retrieving more chunks (`n_results=8` instead of 4) help, or would it introduce noise?
3. Design one out-of-scope query (about a country not in our index, or a topic the Factbook does not cover) and predict how each approach – with RAG and without RAG – should respond.

2.2.8.2.12 Function: `preview`

This small helper trims long model outputs and retrieved context so the classroom output stays readable.

Why it matters: it keeps the demo focused on interpretation instead of overwhelming students with raw text.

Open in Colab

Learning goals:

- Understand data ingestion and document construction
- Learn recursive chunking for retrieval quality
- Build a vector index with ChromaDB
- Ground LLM generation using retrieved context

```
# Install modern package names used by current LangChain ecosystem.
# Note: text splitters are now in a dedicated package: langchain-text-
splitters
%pip install -q -U google-genai chromadb langchain-community langchain-text-
splitters pandas requests git+https://github.com/KarAnalytics/llm_cascade.git
sentence-transformers
```

2.2.9 1) Imports and Environment Setup

This section imports core libraries and handles API key lookup.

In Google Colab, you can store your API keys in Secrets (e.g., `GEMINI_API_KEY`, `OPENAI_API_KEY`, `GROQ_API_KEY`, etc.). Outside Colab, the code falls back to environment variables.

```
import os
import pandas as pd
import requests
import chromadb
from langchain_text_splitters import RecursiveCharacterTextSplitter


from llm_cascade import get_cascade

llm = get_cascade()
```

2.2.10 2) Build a Minimal RAG Pipeline

Pipeline phases:

1. Data ingestion from GitHub JSON documents
2. Chunking with overlap to preserve context continuity
3. Vector storage and semantic retrieval in ChromaDB
4. Grounded generation with Gemini

2.2.10.1 Function: `_collect_text_lines`

This recursive helper converts nested JSON into flat text lines that can be embedded and retrieved.

Why it matters: vector databases work on text, so structured JSON must be turned into readable, searchable content first.

```
# --- PHASE 1: DATA INGESTION (Pandas + GitHub) ---
def _collect_text_lines(obj, parent_key=""):
    """Recursively flatten nested JSON into readable key-value text lines."""
    lines = []

    if isinstance(obj, dict):
        for k, v in obj.items():
            next_key = f"{parent_key}.{k}" if parent_key else str(k)
            lines.extend(_collect_text_lines(v, next_key))
    elif isinstance(obj, list):
```

```
        for i, item in enumerate(obj):
            next_key = f"{parent_key}[{i}]" if parent_key else f"[{i}]"
            lines.extend(_collect_text_lines(item, next_key))
    else:
        if obj is None:
            return []
        text = str(obj).strip()
        if text:
            lines.append(f"{parent_key}: {text}")
    return lines
```

2.2.10.2 Function: `_get_country_json_paths`

This function discovers which country JSON files exist in the GitHub repository.

Why it matters: the demo stays flexible because it does not rely on a hard-coded country list.

```
def _get_country_json_paths():
    """Discover all country JSON files in the factbook repo (no hard-coded
country list)."""
    tree_url = "https://api.github.com/repos/factbook/factbook.json/git/trees/
master?recursive=1"
    r = requests.get(tree_url, timeout=30)
    r.raise_for_status()
    tree = r.json().get("tree", [])

    country_paths = []
    for item in tree:
        path = item.get("path", "")
        if item.get("type") == "blob" and path.endswith(".json") and "/" in
path:
            if not path.lower().startswith(("source/", "tools/", "scripts/")):
                country_paths.append(path)

    # Deduplicate and sort for stable classroom demos.
    return sorted(set(country_paths))
```

2.2.10.3 Function: `get_world_data`

This function fetches each country record, flattens it into text, and packages it with metadata.

Why it matters: it creates the retrieval-ready documents that the rest of the RAG pipeline depends on.

```
def get_world_data():
    """Fetch all country records and convert each into one retrieval-ready
full-text document."""
    print("Step 1: Discovering country files from GitHub...")
    base_url = "https://raw.githubusercontent.com/factbook/factbook.json/
master"
    country_paths = _get_country_json_paths()
    print(f"Discovered {len(country_paths)} country files.")

    docs = []
    for path in country_paths:
        try:
            r = requests.get(f"{base_url}/{path}", timeout=20)
            r.raise_for_status()
            payload = r.json()
        except Exception as exc:
            print(f"Skipping {path}: {exc}")
            continue
```

```
        lines = _collect_text_lines(payload)
        if not lines:
            continue

        country_code = path.split("/")[-1].replace(".json", "").upper()
        combined_text = (
            f"COUNTRY_CODE: {country_code}\n"
            f"SOURCE_PATH: {path}\n\n"
            + "\n".join(lines)
        )

        docs.append(
            {
                "content": combined_text,
                "metadata": {
                    "country": country_code,
                    "path": path,
                },
            }
        )

    print(f"Loaded {len(docs)} country documents.")
    return docs
```

2.2.10.4 Function: `chunk_documents`

This function splits long country documents into overlapping chunks before indexing.

Why it matters: smaller chunks improve retrieval precision, while overlap reduces the chance of losing context at chunk boundaries.

```
## --- PHASE 2: RECURSIVE CHUNKING ---
def chunk_documents(docs, chunk_size=800, chunk_overlap=100):
    """Chunk long records so semantic retrieval can match smaller relevant
spans."""
    print("Step 2: Chunking with overlap...")
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=chunk_size,
        chunk_overlap=chunk_overlap,
    )

    chunks, metadatas, ids = [], [], []
    for doc in docs:
        split_texts = splitter.split_text(doc["content"])
        for i, text in enumerate(split_texts):
            chunks.append(text)
            metadatas.append(doc["metadata"])
            ids.append(f"{doc['metadata']['country']}_{i}")
    return chunks, metadatas, ids


# Small inspection check for students before moving on.
docs_preview = get_world_data()
print(f"Loaded {len(docs_preview)} source documents.")
print("Example metadata:", docs_preview[0]["metadata"] if docs_preview else
"No documents loaded")
```

2.2.10.5 Checkpoint 1: Data and Chunking Reflection

1. Why do we transform structured JSON fields into one text document per country before retrieval?
2. How might changing `chunk_size` from 800 to 300 affect precision and recall?
3. What is the purpose of overlap, and what failure mode does it reduce?

2.2.10.6 2A) Build and Populate the Vector Store

In this phase, we transform chunks into retrievable vector records inside ChromaDB.

Teaching note: for reproducible classroom runs, we clear the collection before re-inserting documents.

2.2.10.7 Function: `build_vector_store`

This function creates the Chroma collection, clears older demo records, and writes new chunk batches into the vector store.

Why it matters: it is the bridge between text preprocessing and semantic retrieval.

This code ran for 30 minutes for me, so be warned. Try GPU/TPU in runtime to speedup.

```
def build_vector_store(docs, db_path="./factbook_db",
collection_name="world_facts", batch_size=5000):
    """Create (or reset) a Chroma collection and upsert chunked records in
batches."""
    chunks, metadatas, ids = chunk_documents(docs)

    chroma_client = chromadb.PersistentClient(path=db_path)
    collection = chroma_client.get_or_create_collection(name=collection_name)

    # Clear older records so each class run starts cleanly.
    existing = collection.get(include=[])
    if existing.get("ids"):
        collection.delete(ids=existing["ids"])

    print("Step 3: Storing vectors in ChromaDB...")
    for start in range(0, len(ids), batch_size):
        stop = start + batch_size
        collection.upsert(
            documents=chunks[start:stop],
            metadatas=metadatas[start:stop],
            ids=ids[start:stop],
        )

    print(f"Stored {len(ids)} chunks in collection '{collection_name}'.")
    return collection

collection = build_vector_store(docs_preview)
```

2.2.10.8 Checkpoint 2: Vector Store Reflection

1. Why do we clear old records before upserting during a classroom demo?
2. What reproducibility issue appears if we repeatedly insert without cleanup?
3. If this were production, when would you keep historical vectors instead?

2.2.10.9 2B) Retrieve Context and Generate a Grounded Answer

This phase demonstrates the core RAG idea: retrieve relevant evidence first, then condition the LLM on that evidence.

Prompting rule used here: answer only from retrieved context; otherwise say you do not know.

2.2.10.10 Functions: `answer_with_rag` and `answer_without_rag`

These two functions create the core teaching comparison in the notebook.

`answer_with_rag` retrieves evidence before generation. `answer_without_rag` sends the question directly to Gemini with no retrieved context.

```
def answer_with_rag(question, collection, n_results=4):
    '''Retrieve nearest chunks, then ask LLM to answer using only retrieved
```

```
context.'''
    print(f"Step 4: Retrieving context for: '{question}'")
    results = collection.query(query_texts=[question], n_results=n_results)
    context = (chr(10) + chr(10)).join(results['documents'][0])

    print('Step 5: Generating grounded answer with LLM...')
    system_prompt = 'You are a careful analyst. Use ONLY the provided context
to answer. If the answer is not in the context, say you don' + chr(39) + 't
know.'
    prompt = 'CONTEXT:' + chr(10) + context + chr(10) + chr(10) + 'QUESTION:'
+ chr(10) + question
    response = llm.generate(prompt, system_prompt=system_prompt)
    return response.text, context


def answer_without_rag(question):
    '''Direct LLM answer without retrieval for comparison against RAG.'''
    response = llm.generate(question)
    return response.text
```

2.2.10.11 Checkpoint 3: Retrieval and Prompting Reflection

1. Why do we retrieve first and generate second in a RAG system?
2. What is the effect of changing `n_results` from 2 to 5?
3. How does the prompt reduce hallucinations, and where can it still fail?

2.2.10.12 2C) Run an End-to-End Query

You can now test questions and inspect both the answer and retrieved evidence.

Tip: print retrieved context in class to discuss why the model answered the way it did.

2.2.10.13 Function: `preview`

This small helper trims long model outputs and retrieved context so the classroom output stays readable.

Why it matters: it keeps the demo focused on interpretation instead of overwhelming students with raw text.

```
comparison_questions = [
    {
        "type": "Hallucination test (out-of-scope)",
        "question": "What is the current prime minister of Atlantis, and what
is Atlantis GDP growth?",
    },
    {
        "type": "Local info extraction",
        "question": "Summarize Japan's economy overview in 3 bullet points.",
    },
    {
        "type": "Local info extraction",
        "question": "Compare inflation-related details for Brazil and
Canada.",
    },
    {
        "type": "Local + public knowledge blend",
        "question": "Explain how Japan's economic profile may influence its
role in global supply chains today.",
    },
]


def preview(text, max_len=900):
    text = text or ""
    return text[:max_len] + ("..." if len(text) > max_len else "")
```

```
if "collection" not in globals():
    print("Error: Build the Chroma vector store in Cell 9 before running this
comparison.")
else:
    try:
        for i, item in enumerate(comparison_questions, start=1):
            q_type = item["type"]
            q = item["question"]
            print("\n" + "=" * 90)
            print(f"Q{i}. {q_type}")
            print(f"QUESTION: {q}\n")

            direct_answer = answer_without_rag(q)
            print("[WITHOUT RAG]")
            print(preview(direct_answer))

            rag_answer, rag_context = answer_with_rag(q, collection,
n_results=4)
            print("\n[WITH RAG]")
            print(preview(rag_answer))

            print("\n[RETRIEVED CONTEXT PREVIEW]")
            print(preview(rag_context, max_len=1200))
    except Exception as e:
        print(f"Error: {e}")
```

2.2.10.14 Checkpoint 4: Evaluation Reflection

1. Compare the final answer against the retrieved context. Which statements are directly supported?
2. What additional evidence would improve confidence in the answer?
3. Design one out-of-scope query and explain how the system should respond.

2.2.11 3) Teaching Notes and Suggested Exercises

Exercises for class discussion:

- Change `chunk_size` and `chunk_overlap`; evaluate answer quality
- Increase `n_results` and observe grounding changes
- Add more countries and test comparative questions
- Intentionally ask out-of-scope questions to inspect hallucination control

Open in Colab

2.2.12 RAG with a Featherweight LLM

Can Retrieval-Augmented Generation work with a model so small it barely qualifies as a "large" language model? In this notebook we find out. We wire up **SmolLM2-135M** — a HuggingFace model with only 135 million parameters — into a full LangChain RAG pipeline backed by ChromaDB. The point is not to get GPT-4-quality answers; it is to prove that the *pattern* of RAG (retrieve relevant context, stuff it into a prompt, generate) works at any scale. If you understand this notebook, you understand the skeleton that every production RAG system is built on — the only difference is swapping in a bigger brain.

2.2.12.1 Install Dependencies

The cell below installs everything we need in one shot. **LangChain** provides the orchestration glue — it connects the language model, the vector store, and the retrieval logic into a single pipeline. **ChromaDB** (`langchain-chroma`) is an open-source vector database that will store our document embeddings. **HuggingFace Transformers** gives us access to the SmolLM2 model itself, and **sentence-transformers** provides the embedding model that converts text into vectors. Think of this as assembling a toolkit: LangChain is the workbench, ChromaDB is the filing cabinet, and the HuggingFace libraries are the engine.

2.2.12.2 1. Load the "Micro" Model

Here is where the experiment gets interesting. **SmolLM2-135M-Instruct** has only 135 million parameters — roughly 500 times smaller than GPT-3 and over a thousand times smaller than GPT-4. We chose it deliberately: it is small enough to run on a free Colab GPU in seconds, yet it has been instruction-tuned, so it understands the format of a question-and-answer prompt. We load it in half-precision (`float16`) and force it onto the GPU with `device_map="cuda"` to keep inference fast.

The `max_new_tokens=50` cap is another deliberate choice. A 135M-parameter model tends to ramble or repeat itself if you let it generate too much text. Capping the output at 50 tokens forces it to be concise — and lets us see quickly whether it actually absorbed the retrieved context or just hallucinated. Finally, we wrap the HuggingFace pipeline in `HuggingFacePipeline` so that LangChain can call it like any other LLM.

2.2.12.3 2. Upload Your Document

RAG needs a knowledge source. Here we ask you to upload a plain-text file — this could be a Wikipedia article, a company memo, or any prose document. The file is read into a single Python string that we will chunk and embed in the next step. In a production system this upload step would be replaced by a document ingestion pipeline, but for a classroom demo, drag-and-drop keeps things simple.

2.2.12.4 3. Chunk, Embed, and Build the RAG Pipeline

This single cell does three things that form the backbone of any RAG system.

Chunking. The `CharacterTextSplitter` breaks the uploaded text into 1,000-character pieces with no overlap. Chunking matters because embedding models and LLMs have limited context windows — feeding them the entire document at once would either truncate it or dilute the signal. Smaller chunks mean each vector represents a focused topic, which improves retrieval precision.

Embedding and storing. Each chunk is converted into a 384-dimensional vector by `all-MiniLM-L6-v2`, a lightweight sentence-transformer. These vectors are stored in **ChromaDB**, an in-memory vector database. When a question arrives, ChromaDB will compare the

question's vector against every chunk vector using cosine similarity and return the closest matches.

Pipeline assembly. `RetrievalQA.from_chain_type` ties the retriever to the LLM. We set `k=1` — retrieve only the single best-matching chunk — to keep things fast and to make it crystal clear which piece of context the model is working from. The `"stuff"` chain type simply concatenates the retrieved chunk into the prompt; other chain types exist for multi-document summarization, but "stuff" is the simplest and most transparent.

2.2.12.5 4. Query the Pipeline

Now we put the pipeline to the test. When you call `rag.invoke(query)`, LangChain performs three steps behind the scenes: (1) embed the question, (2) retrieve the best-matching chunk from ChromaDB, and (3) pass both the chunk and the question to SmolLM2 for generation. Look at the output carefully — you will likely see the model echo the LangChain prompt template and the retrieved context before its answer. This is a quirk of very small models: they do not always know where the "instructions" end and the "answer" begins. A larger model would suppress the template and give you a clean response.

2.2.12.6 5. A Second Query

We run a second question to see how the model handles a different type of information need. Notice that the retrieval step may or may not pull a different chunk this time — it depends on which chunk's embedding is closest to this new question. The quality of the answer will vary: sometimes the tiny model nails it, sometimes it trails off or repeats itself. That inconsistency is the price of using a 135M-parameter model — and it is exactly why production systems use larger models for generation while keeping the same retrieval architecture you see here.

2.2.12.7 Takeaways: What Did We Learn?

The most important lesson from this notebook is that **RAG is an architecture, not a model**. The retrieve-then-generate pattern works whether the generator has 135 million parameters or 1.8 trillion. The retrieval side (chunking, embedding, vector search) was actually quite good here — cosine similarity found the right context every time. It was the *generation* side that struggled, because a tiny model lacks the reasoning capacity to synthesize a clean, well-structured answer.

In a real deployment, you would keep ChromaDB (or a production vector store like Pinecone or Weaviate) and swap SmolLM2 for a capable model like GPT-4o or Gemini. The retrieval pipeline stays the same; only the last mile — the LLM call — changes. That modularity is the whole point of the LangChain abstraction, and it is why understanding this featherweight demo transfers directly to building enterprise-grade RAG systems.

```
## Large chunks mean less work for the vector DB
text_splitter = CharacterTextSplitter(chunk_size=1000, chunk_overlap=0)
docs = text_splitter.create_documents([raw_text])

embedding_model = HuggingFaceEmbeddings(model_name='sentence-transformers/all-
MiniLM-L6-v2')
db = Chroma.from_documents(docs, embedding_model)

# --- 3. The RAG Pipeline ---
# Use k=1 to make it even faster (only looks at the single best match)
rag = RetrievalQA.from_chain_type(
    llm=llm,
    chain_type='stuff',
    retriever=db.as_retriever(search_kwargs={"k": 1})
)


## --- 4. Run the Query ---
query = "What is the main topic of the document?"
response = rag.invoke(query)
```

```python
print(f"\n--- Answer ---\n{response['result']}")

# --- 5. Run the Query ---
query = "When did the event begin, when will it end?"
response = rag.invoke(query)

print(f"\n--- Answer ---\n{response['result']}")
```

2.3 RAG with Structured Data

The previous chapters built RAG pipelines that retrieve text chunks from documents and vector databases. That approach works well when your knowledge base consists of unstructured text, but most enterprise data does not live in PDFs or markdown files. It lives in relational databases, data warehouses, and increasingly in graph databases that capture complex relationships between entities. The good news is that RAG is not limited to text documents. Any data source that can provide evidence for an LLM's answer can serve as the retrieval backend. This chapter explores two structured-data variants of RAG that follow the same retrieve-augment-generate pattern you already know, but replace vector similarity search with query execution against a database.

2.3.1 DBMS RAG: Grounding Answers on SQL Query Results

When your data is structured and tabular, the most natural retrieval mechanism is a SQL query. DBMS RAG implements this idea through a two-phase retrieval process that is conceptually different from document RAG.

In the first phase, the system retrieves the database schema rather than any actual data. The schema, consisting of CREATE TABLE statements, column names, data types, foreign key constraints, and a handful of sample rows, is injected into the prompt as context. This schema context tells the LLM what tables exist, how they relate to each other, and what the data looks like. The LLM then uses this context to generate a SQL query that answers the user's natural-language question. This text-to-SQL step is the first LLM call in the pipeline.

In the second phase, the generated SQL query is executed against the database, and the actual query results are fed back to the LLM in a second prompt. The LLM now composes a natural-language answer grounded entirely on the retrieved rows. This two-call architecture is what distinguishes DBMS RAG from document RAG, where retrieval and grounding happen in a single step. In document RAG, the chunks you retrieve are the evidence. In DBMS RAG, the schema is retrieved first to enable query generation, and then the query results become the evidence.

This distinction has profound practical implications. Because the database engine handles aggregations, joins, and filtering, DBMS RAG produces exact numerical answers. When a user asks "What is the total quantity of all shipments?", the LLM generates `SELECT SUM(Quantity) FROM SHIPMENTS`, the database returns the precise total, and the LLM reports it. Document RAG would have to retrieve individual shipment records as text chunks, hope that all relevant chunks were found, and then rely on the LLM to add them up correctly in its head. For structured, numeric, analytical questions, DBMS RAG is simply the right tool.

2.3.2 Graph RAG: Traversing Relationships with Cypher

Some data is inherently relational in a way that goes beyond what tables and foreign keys express conveniently. International trade networks, social graphs, supply chains, and organizational hierarchies are all examples where the relationships between entities are as important as the entities themselves. Graph databases model this data as nodes and edges, and query languages like Cypher allow you to express complex traversal patterns that would require deeply nested joins in SQL.

Graph RAG follows exactly the same pattern as DBMS RAG but substitutes a graph database for the relational database and Cypher queries for SQL. The system extracts the graph schema, which describes node labels, relationship types, and their properties, and provides it to the LLM as context. The LLM generates a Cypher query, the query is executed against the graph database, and the results ground the final answer. The accompanying notebook uses Kuzu, an embedded graph database, loaded with international trade data where countries are nodes and trade flows are directed edges with export values and year attributes.

The power of Graph RAG becomes apparent with multi-hop questions. A query like "Find countries in Africa that exported to both the USA and China in 2017" requires matching a pattern across multiple relationships simultaneously. In Cypher, this is a natural pattern match. In SQL, it would require multiple self-joins, subqueries, and careful filtering. Graph

RAG makes these relationship-heavy questions accessible through natural language by leveraging the LLM's ability to generate appropriate graph traversal patterns.

It is worth noting that Graph RAG is not infallible. The LLM must correctly map entity names in the user's question to the identifiers stored in the graph. If the user says "USA" but the database stores "United States," the generated Cypher query may return empty results even though the data exists. The accompanying notebook demonstrates this pitfall explicitly and shows how manually correcting the query produces the expected answer. This is an important lesson: the retrieval mechanism is only as good as the LLM's understanding of the schema and data conventions.

2.3.3 The Shared Pattern: Retrieve, Augment, Generate

Despite their differences in retrieval mechanism, both DBMS RAG and Graph RAG follow the same fundamental pattern that underlies all RAG systems. First, context is retrieved from an external data source. Second, that context is augmented into a prompt alongside the user's question. Third, the LLM generates an answer grounded on the retrieved evidence. The only thing that changes across RAG variants is how the retrieval step works: vector similarity for documents, SQL execution for relational data, and Cypher execution for graph data.

This modularity is what makes RAG such a flexible architecture. You are not locked into a single retrieval mechanism. In a real enterprise application, you might combine multiple RAG backends: a vector database for searching policy documents, a SQL backend for querying sales figures, and a graph database for exploring supplier relationships, all feeding context into the same LLM.

2.3.4 When to Choose Which Approach

The choice between document RAG, DBMS RAG, and Graph RAG depends on the nature of your data and the types of questions you need to answer. Document RAG is the right choice when your knowledge base consists of unstructured text such as articles, reports, contracts, and manuals. It excels at finding semantically relevant passages even when the user's phrasing differs from the source text.

DBMS RAG is the right choice when your data is structured, tabular, and numeric. Questions involving aggregations, filtering, sorting, and joins over well-defined schemas are best answered by letting a database engine do what it was designed to do. The LLM's role is to translate natural language into SQL and then narrate the results.

Graph RAG is the right choice when the relationships in your data are first-class concerns. If users ask questions about paths, connections, influence, or multi-hop relationships, a graph database expresses these patterns far more naturally than relational tables. Supply chain analysis, fraud detection in transaction networks, and organizational network analysis are all strong candidates for Graph RAG.

2.3.5 The Notebooks

The two notebooks that follow put these ideas into practice. The first notebook, **DBMS_RAG_SQLite**, builds a SQLite database from the classic Supplier-Parts schema, extracts schema context, generates SQL from natural-language questions, and compares grounded answers against ungrounded LLM responses. The second notebook, **GRAPH_RAG_Trade**, constructs a Kuzu graph database from international trade data, generates Cypher queries, and visualizes the retrieved subgraphs to help you verify the LLM's answers. Together, they demonstrate that RAG is a general-purpose architecture whose retrieval mechanism can be adapted to whatever data source holds the evidence your application needs.

2.3.6 References

- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Kuttler, H., Lewis, M., Yih, W., Rocktaschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-Augmented

Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*, 33.
- Rajkumar, N., Li, R., & Baber, D. (2022). Evaluating the Text-to-SQL Capabilities of Large Language Models. *arXiv preprint arXiv:2204.00498*.
- Jin, Z., Guo, R., & Shi, S. (2023). A Survey on Text-to-SQL Parsing: Concepts, Methods, and Future Directions. *arXiv preprint arXiv:2208.13629*.
- Feng, J., Chen, Z., Li, D., & Cai, J. (2023). Knowledge Graph-Augmented Language Models for Knowledge-Grounded Dialogue Generation. *arXiv preprint arXiv:2305.18846*.
- Kuzu Documentation. (2024). Kuzu: An Embeddable Property Graph Database Management System. https://kuzudb.com/

Open in Colab

2.3.7 RAG with a Local DBMS: Text-to-SQL using SQLite

What happens when the knowledge you need is not buried in documents, but neatly organized in database tables? In previous chapters we retrieved text chunks using vector similarity. Here we take a fundamentally different approach: instead of searching for *similar text*, we ask the LLM to write a *SQL query* that fetches the exact data we need. The database does the heavy lifting — joins, filters, aggregations — and the LLM simply translates English into SQL, then translates the results back into English.

How it works:

1. Build a SQLite database from the classic **Supplier-Parts** schema (DDL from a `.sql` file, data from an `.xlsx` file).
2. Extract the database **schema** as structured context — this is the "retrieval" step.
3. Given a natural-language question, the LLM generates a **SQL query**.
4. Execute the SQL and feed the **results back** to the LLM for a natural-language answer.

Learning goals:

- Understand how RAG applies beyond vector databases
- See how schema metadata serves as retrieval context
- Practice grounding LLM output on real query results
- Compare answers with and without database context

Provider setup: Uses `llm_cascade` for automatic LLM provider selection.
Store your LLM API keys in Colab Secrets (or a local `.env` file). Supported keys: `GEMINI_API_KEY`, `OPENAI_API_KEY`, `GROQ_API_KEY`, `HF_TOKEN`, `COHERE_API_KEY`, `XAI_API_KEY`, `OLLAMA_API_KEY`, `OPENROUTER_API_KEY`.

```
%pip install -q -U openpyxl git+https://github.com/KarAnalytics/llm_cascade.
git sentence-transformers
```

2.3.7.1 1) Imports and Provider Helpers

SQLite is built into Python — no extra database driver needed.
We use **openpyxl** to load data from the Excel file.

The provider helpers below are identical to the pattern used in the RAG countries notebook: Gemini first, Ollama Cloud fallback, keys from Colab Secrets or `.env`.

```
import os
import sqlite3
import re
from pathlib import Path

import openpyxl

from llm_cascade import get_cascade

llm = get_cascade()


def generate_text(prompt, system_prompt=None, **kwargs):
    response = llm.generate(prompt, system_prompt=system_prompt)
    return response.text, response.provider


def has_llm_provider():
```

```
    return True  # llm_cascade auto-detects available providers

print("Provider ready (llm_cascade)")
```

2.3.7.2 2) Build the SQLite Database from External Files

We load the **Supplier-Parts** database from two companion files hosted in a public GitHub repo:

File	Source	Purpose
`SUPPLIER_PARTS_DDL.sql`	GitHub	CREATE TABLE statements (schema definition)
`SUPPLIER_PARTS.xlsx`	GitHub	Data for each table (one sheet per table)

The next cell downloads both files automatically. If you want to try your own database, uncomment **Option B** in the cell to upload local `.sql` and `.xlsx` files instead.

The demo database has three tables:

Table	Purpose
`SUPPLIERS`	Supplier ID, name, status, and city
`PARTS`	Part ID, name, color, weight, and city
`SHIPMENTS`	Which supplier ships which part, with quantity

This is the classic **Supplier-Parts** schema used in database textbooks — small enough to fit in a single prompt, but rich enough to demonstrate joins, aggregations, and foreign-key relationships.

```
import urllib.request

DB_PATH = 'supplier_parts.db'
DDL_FILE = 'SUPPLIER_PARTS_DDL.sql'
XLSX_FILE = 'SUPPLIER_PARTS.xlsx'

#
==============================================================================
# Option A (default): download the demo files from GitHub
#
==============================================================================
# This is the default so the notebook runs on a fresh Colab without needing
# any file uploads. Students can swap in their own DDL and data files by
# using Option B below instead.

DDL_URL = 'https://raw.githubusercontent.com/KarAnalytics/datasets/master/
SUPPLIER_PARTS_DDL.sql'
XLSX_URL = 'https://github.com/KarAnalytics/datasets/raw/master/SUPPLIER_
PARTS.xlsx'

print(f'Downloading {DDL_FILE}...')
urllib.request.urlretrieve(DDL_URL, DDL_FILE)
print(f'Downloading {XLSX_FILE}...')
urllib.request.urlretrieve(XLSX_URL, XLSX_FILE)
print('Downloaded both files.')
print()

#
==============================================================================
# Option B: upload your own files instead
#
==============================================================================
# Uncomment the lines below if you want to try a different database. Upload
# your .sql (DDL) file and .xlsx (data) file to the Colab workspace first,
```

```
# then update DDL_FILE / XLSX_FILE above to match your filenames.
#
# from google.colab import files
# uploaded = files.upload()  # a file picker will appear
# DDL_FILE = list(uploaded.keys())[0]       # e.g., 'my_schema.sql'
# XLSX_FILE = list(uploaded.keys())[1]      # e.g., 'my_data.xlsx'


# --- Build the SQLite database from the files ---
if Path(DB_PATH).exists():
    Path(DB_PATH).unlink()  # remove stale DB from previous runs

conn = sqlite3.connect(DB_PATH)
cur = conn.cursor()

# 1) Execute DDL from the .sql file
ddl_text = Path(DDL_FILE).read_text(encoding='utf-8')
cur.executescript(ddl_text)
print(f'Schema loaded from {DDL_FILE}')

# 2) Load data from Excel (one sheet per table)
wb = openpyxl.load_workbook(XLSX_FILE, read_only=True, data_only=True)

# Insertion order matters because of foreign-key constraints:
# SUPPLIERS and PARTS first, then SHIPMENTS.
table_order = ['SUPPLIERS', 'PARTS', 'SHIPMENTS']

for table_name in table_order:
    ws = wb[table_name]
    rows = list(ws.iter_rows(values_only=True))
    headers = rows[0]       # first row is column names
    data_rows = rows[1:]    # remaining rows are data
    placeholders = ', '.join('?' * len(headers))
    cur.executemany(
        f'INSERT INTO {table_name} VALUES ({placeholders})',
        data_rows,
    )
    print(f'  {table_name}: {len(data_rows)} rows inserted')

wb.close()
conn.commit()
print(f'\nDatabase "{DB_PATH}" ready.')
```

2.3.7.3 3) Schema Extraction — the "Retrieval" in DBMS RAG

This is the conceptual heart of the notebook, so pause and think about it carefully. In document RAG, the retrieval step searches a vector store for text chunks similar to the question. In DBMS RAG, we do something very different: **we retrieve the database schema itself** — the `CREATE TABLE` statements that describe what tables exist, what columns they have, and how they relate to each other.

Why the schema and not the data? Because the schema is small and fixed-size regardless of how many rows the database has. A database with 10 rows and a database with 10 billion rows have the same schema. By giving the LLM the schema (plus a few sample rows so it understands data formats), we equip it to write valid SQL queries that will fetch the exact data needed — without ever stuffing millions of rows into a prompt.

Think of it this way: in document RAG, retrieval *is* the answer (the chunks contain the evidence). In DBMS RAG, retrieval gives the LLM a *map* of the data, and the LLM uses that map to write a SQL query that goes and gets the actual evidence.

```
def get_schema_context(db_path, sample_rows=3):
    """Extract CREATE TABLE statements and sample rows as a text block."""
    con = sqlite3.connect(db_path)
    cur = con.cursor()
```

```
    # Get all CREATE TABLE DDL
    cur.execute("SELECT name, sql FROM sqlite_master WHERE type='table' ORDER
BY name")
    tables = cur.fetchall()

    parts = []
    for table_name, ddl in tables:
        parts.append(f"-- Table: {table_name}")
        parts.append(ddl + ";")

        # Sample rows
        cur.execute(f"SELECT * FROM [{table_name}] LIMIT {int(sample_rows)}")
        rows = cur.fetchall()
        col_names = [desc[0] for desc in cur.description]
        parts.append(f"-- Sample rows ({table_name}): columns = {col_names}")
        for row in rows:
            parts.append(f"--   {row}")
        parts.append("")

    con.close()
    return "\n".join(parts)


schema_context = get_schema_context(DB_PATH)
print(schema_context)
```

2.3.7.4 4) Text-to-SQL Generation

This is the core RAG step: the LLM receives the schema context and the user's natural-language question, then produces a SQL query. Notice the two-phase design: the first LLM call *generates* SQL (it does not answer the question directly), and we will *execute* that SQL against the real database before asking the LLM to compose a final answer. This separation is key — the LLM never sees the raw data during SQL generation, only the schema. It is essentially writing a recipe for how to find the answer.

Safety note: We only allow `SELECT` statements. The `extract_sql` function strips markdown fences and validates that the query starts with `SELECT` before execution, preventing any accidental `DROP TABLE` or `UPDATE` commands. In a production system you would also run queries with a read-only database connection.

```
SQL_SYSTEM_PROMPT = (
    "You are an expert SQL analyst. Given the database schema below, write a "
    "single SQLite-compatible SELECT query that answers the user's question.
\n"
    "Return ONLY the SQL query — no explanation, no markdown fences, no
comments.\n"
    "If the question cannot be answered from the schema, reply with: SELECT
'NOT_ANSWERABLE' AS result;"
)


def extract_sql(raw_text):
    """Strip markdown fences or extra prose, returning just the SQL
statement."""
    match = re.search(r"```(?:sql)?\s*\n?(.*?)```", raw_text, re.DOTALL |
re.IGNORECASE)
    if match:
        return match.group(1).strip()
    return raw_text.strip().rstrip(";")


def generate_sql(question, schema, **kwargs):
    """Ask the LLM to produce a SELECT query for the given question."""
    prompt = f"DATABASE SCHEMA:\n{schema}\n\nQUESTION:\n{question}"
```

```
    raw_response, provider = generate_text(
        prompt=prompt,
        system_prompt=SQL_SYSTEM_PROMPT,
    )
    sql = extract_sql(raw_response)
    return sql, provider


def execute_sql(db_path, sql):
    """Run a read-only SQL query and return column names + rows."""
    normalized = sql.strip().upper()
    if not normalized.startswith("SELECT"):
        return [], [], "Blocked: only SELECT queries are allowed."

    con = sqlite3.connect(db_path)
    try:
        cur = con.execute(sql)
        columns = [desc[0] for desc in cur.description]
        rows = cur.fetchall()
        return columns, rows, None
    except Exception as exc:
        return [], [], str(exc)
    finally:
        con.close()


print("Text-to-SQL functions ready.")
```

2.3.7.5 5) Answer Generation — Grounding on Query Results

After executing the SQL, we feed the **actual query results** back to the LLM and ask it to compose a natural-language answer. This is the second LLM call, and it follows the exact same pattern as document RAG: retrieved evidence goes into the prompt, and the model must base its answer on that evidence alone. The difference is that in document RAG the evidence was text chunks found by vector search; here the evidence is database rows returned by a SQL query.

This two-call architecture is what makes DBMS RAG so powerful for numerical questions. When you ask "What is the total quantity of all shipments?", the LLM does not have to add numbers in its head (which it is bad at). Instead, it writes `SELECT SUM(Quantity) FROM SHIPMENTS`, the database computes the exact answer, and the LLM simply wraps that number in a sentence. The math is always right because SQL does the math.

```
ANSWER_SYSTEM_PROMPT = (
    "You are a helpful business analyst. Use ONLY the provided SQL results to
answer "
    "the user's question. If the results are empty or the query returned
NOT_ANSWERABLE, "
    "say you cannot determine the answer from the available data."
)


def answer_with_dbms_rag(question, db_path=DB_PATH, **kwargs):
    """Full DBMS-RAG pipeline: schema retrieval -> SQL generation -> execution
-> answer."""
    schema = get_schema_context(db_path)

    print(f"  [Step 1] Schema retrieved ({len(schema)} chars)")

    sql, sql_provider = generate_sql(question, schema)
    print(f"  [Step 2] SQL generated (provider: {sql_provider})")
    print(f"           {sql}")

    columns, rows, error = execute_sql(db_path, sql)
    if error:
```

```
        print(f"  [Step 3] SQL execution error: {error}")
        return f"SQL failed: {error}", sql, sql_provider

    print(f"  [Step 3] SQL executed — {len(rows)} row(s) returned")

    result_text = f"Columns: {columns}\n"
    for row in rows:
        result_text += f"  {row}\n"

    prompt = (
        f"QUESTION:\n{question}\n\n"
        f"SQL QUERY USED:\n{sql}\n\n"
        f"QUERY RESULTS:\n{result_text}"
    )
    answer, ans_provider = generate_text(
        prompt=prompt,
        system_prompt=ANSWER_SYSTEM_PROMPT,
    )
    print(f"  [Step 4] Answer generated (provider: {ans_provider})")

    return answer, sql, sql_provider


def answer_without_rag(question, **kwargs):
    """Direct LLM answer with no database context."""
    answer, provider = generate_text(prompt=question)
    return answer, provider


print("RAG answer pipeline ready.")
```

2.3.7.6 6) Run End-to-End Examples

Time to see the full pipeline in action. We test several questions and compare two approaches side by side:

- **With DBMS RAG:** schema retrieval, SQL generation, execution, and a grounded answer based on real query results.
- **Without RAG:** the LLM answers the question directly with no database access.

Pay close attention to the "without RAG" answers. The model has no way to know that total shipment quantity is 5,300 — it has never seen this particular database. So it either asks for data it does not have (the honest response) or invents a plausible-sounding but completely wrong number (hallucination). The RAG version, by contrast, always gives the exact answer because the SQL engine computed it.

```
questions = [
    "What is the total quantity of all shipments?",
    #"Which supplier has shipped the most parts? Show their name and total
quantity.",
    #"List all parts supplied by supplier S1, with quantities.",
    #"How many distinct parts are shipped from London?",
    #"What is the average shipment quantity per supplier located in Paris?",
]


def preview(text, max_len=800):
    text = text or ""
    return text[:max_len] + ("..." if len(text) > max_len else "")


if not has_llm_provider():
    print("Error: No LLM API key configured. Set at least one API key in Colab
Secrets.")
else:
    for i, q in enumerate(questions, start=1):
```

```
    print("\n" + "=" * 80)
    print(f"Q{i}. {q}\n")

    try:
        answer_rag, sql_used, prov = answer_with_dbms_rag(q)
        print("\n  [WITH DBMS RAG]")
        print(preview(answer_rag))
    except Exception as e:
        print(f"  RAG error: {e}")

    try:
        answer_direct, prov_direct = answer_without_rag(q)
        print(f"\n  [WITHOUT RAG] (provider: {prov_direct})")
        print(preview(answer_direct))
    except Exception as e:
        print(f"  Direct error: {e}")
```

2.3.7.6.1 Checkpoint: Reflection Questions

1. Compare the RAG and non-RAG answers. Which ones show hallucinated numbers?
2. What happens if you ask a question about data that is **not** in the database (e.g., supplier revenue or profit)?
3. What are the risks of letting an LLM generate SQL against a production database?
4. How does this DBMS-RAG pattern compare to the vector-store RAG we used with ChromaDB?

2.3.7.7 7) How DBMS RAG Works — and How It Compares to Document RAG

This notebook implements RAG using a **SQL database as the knowledge source** instead of chunked documents. The pipeline has **two retrieval phases**, both appended to prompts before going to the LLM — so it's structurally similar to document RAG, but with a twist.

2.3.7.7.1 The pipeline (per question)

```
Question
  |
  +-> Step 1: Retrieve SCHEMA (not data)
  |     get_schema_context() pulls CREATE TABLE statements
  |     + 3 sample rows per table
  |
  +-> Step 2: LLM generates SQL
  |     Prompt = schema + question -> LLM -> SELECT ...
  |
  +-> Step 3: Execute the SQL on the database
  |     Runs against SQLite, returns columns + rows
  |
  +-> Step 4: LLM writes the final answer
        Prompt = question + SQL + query results -> LLM -> natural language
answer
```

Two LLM calls per question:

- **Text-to-SQL** (`generate_sql`) — schema goes in, SQL query comes out
- **Answer generation** (inside `answer_with_dbms_rag`) — query results go in, natural-language answer comes out

2.3.7.7.2 Is it different from document RAG?

Same core principle, different retrieval mechanism. Both patterns follow: `retrieve -> stuff into prompt -> generate answer`. The differences:

	Document RAG	DBMS RAG (this notebook)
What's retrieved first?	Text chunks similar to the question	The database **schema** (DDL + sample rows)

	Document RAG	DBMS RAG (this notebook)
How is retrieval done?	Vector similarity (cosine) on embeddings	Just dump all CREATE TABLE statements
Is question content matched against stored data?	Yes, directly (embedding search)	**No** — retrieval is schema, not data
When does the actual data enter the prompt?	Immediately (chunks = data)	In a **second LLM call**, after SQL executes
Scales with data size?	Gets slower (more chunks to search)	**No** — schema is fixed size even if the DB has billions of rows
Handles aggregations?	Poorly (LLM must do math in its head)	**Perfectly** (SQL `SUM`, `COUNT`, `AVG` are exact)

2.3.7.7.3 The key insight

In document RAG, **retrieval and grounding happen in the same step** — the chunks you retrieve ARE the evidence. Whatever the vector search brings back is what the LLM sees.

In DBMS RAG, **retrieval happens twice**:

- **First retrieval = schema only** (what tables exist, what columns, what types)
- **Second "retrieval" = the SQL query results** (the actual evidence, fetched dynamically)

This is why it's so much better for structured data:

- Document RAG has to hope the right chunks got retrieved. If your question is "total sales in Q3," it retrieves chunks that mention Q3 sales — but the LLM has to add them up, and may miss rows or hallucinate.
- DBMS RAG doesn't hope. It asks the LLM to write `SELECT SUM(amount) WHERE quarter='Q3'`, runs it, and shows the exact answer.

2.3.7.7.4 What does `get_schema_context()` actually retrieve?

Two things:

- **`CREATE TABLE` statements** from SQLite's `sqlite_master` metadata table. This is cheap — it returns the same few thousand characters regardless of whether the tables have 100 rows or 100 million.
- **3 sample rows per table.** This gives the LLM a feel for the data format (e.g., "oh, dates are stored as strings like `2025-Q1`, supplier IDs are integers, colors are lowercase"). These sample rows are the closest thing to "chunks" in DBMS RAG.

2.3.7.7.5 The second LLM call — where "actual retrieval" happens

After `execute_sql()` runs and returns rows, the notebook stuffs those rows INTO the prompt for the second LLM call:

```
prompt = (
    f'QUESTION:\n{question}\n\n'
    f'SQL QUERY USED:\n{sql}\n\n'
    f'QUERY RESULTS:\n{result_text}'
)
```

This is exactly the same pattern as document RAG — retrieved evidence + question arrow.r LLM arrow.r answer. The only difference is that the "retrieval" was done via a SQL query instead of a vector search.

2.3.7.7.6 TL;DR

Is DBMS RAG different from document RAG? Yes in how retrieval is done, no in the overall pattern.

- Document RAG: `vector_search(question) -> chunks -> prompt -> LLM -> answer`
- DBMS RAG: `read_schema() -> prompt -> LLM -> SQL -> execute SQL -> results -> prompt -> LLM -> answer`

Both pipelines ground the LLM's final answer on retrieved evidence. DBMS RAG just retrieves via SQL execution instead of embedding similarity — which makes it perfect for structured/numeric data where aggregations and joins matter, and completely impractical for unstructured text where there's no schema to query.

2.3.7.7.7 Where this notebook sits in the RAG family

Each type of RAG uses a different retrieval mechanism but the same *pattern* (retrieve evidence arrow.r stuff into prompt arrow.r ground the answer):

Type	Best for	Retrieval mechanism	Example notebook
Document RAG	Unstructured text (articles, PDFs)	Vector similarity	`LlamaIndex_RAG`, `LangChain_demo`
DBMS RAG (this one)	Structured tabular data	Text-to-SQL + execution	`DBMS_RAG_SQLite`
Graph RAG	Relational / network data	Text-to-Cypher + execution	`GRAPH_RAG_Trade`
Image / Video RAG	Visual content	CLIP embedding similarity	`Image_RAG`, `Video_RAG`

2.3.7.7.8 Key takeaways

- RAG is **not limited to vector databases** — any structured data source can serve as retrieved context.
- In DBMS RAG, the **schema** is the retrieval context and **SQL execution** is the grounding mechanism.
- Grounding the final answer on actual query results prevents hallucination of specific numbers.
- Loading schema from `.sql` files and data from `.xlsx` files mirrors real-world enterprise workflows.

2.3.7.7.9 Exercises

- Add a new table (e.g., `PROJECTS` with a project-supplier-part relationship) and ask questions that require joins across it.
- Try asking ambiguous questions and observe how the LLM interprets them.
- Modify `SQL_SYSTEM_PROMPT` to request query explanations alongside the SQL.
- Compare the quality of SQL generated by different models in `llm_cascade` (Gemini vs. Groq vs. Ollama).
- Swap in a completely different database by uncommenting Option B in cell 5 and uploading your own `.sql` and `.xlsx` files.
- Discuss: when would you prefer DBMS RAG over vector-store RAG for a business application?

Open in Colab

2.3.8 Graph RAG: Retrieval-Augmented Generation with Graph Databases

Some data is inherently about *relationships*. Who trades with whom? Which countries are connected through supply chains? What is the shortest path between two trading partners? Relational databases can answer these questions, but the SQL gets ugly fast — you need self-joins, recursive CTEs, and complex subqueries. Graph databases are purpose-built for this kind of data: they store entities as **nodes** and connections as **edges**, and they answer relationship queries in a fraction of the code.

In this notebook we build a **Graph RAG** pipeline using **Kuzu**, a lightweight embedded graph database. The pattern mirrors what we did with SQL RAG — extract the schema, let the LLM write a query, execute it, ground the answer on results — but the query language is **Cypher** instead of SQL, and the data model is a graph instead of tables. The result is a system that excels at questions like "Which African countries export to both the USA and China?" — queries that would require multiple joins in SQL but are a single, readable pattern match in Cypher.

How it works:

1. Build a graph database from international trade data (countries as nodes, trade flows as edges).
2. Extract the **graph schema** (node labels, relationship types, properties) as LLM context.
3. The LLM generates a **Cypher query** from a natural-language question.
4. Execute the Cypher query and feed results back to the LLM for a natural-language answer.
5. **Visualize** the retrieved subgraph to see the data structure.

Learning goals:

- Understand why graph databases are better than SQL for relationship-heavy data
- Learn the basics of **Cypher**, the graph query language
- See how graph schema serves the same role as SQL DDL in DBMS RAG
- Experience the limitations of LLM-generated Cypher (and how to debug it)

Provider setup: Uses `llm_cascade` for automatic LLM provider selection.
Store your LLM API keys in Colab Secrets (or a local `.env` file). Supported keys: `GEMINI_API_KEY`, `OPENAI_API_KEY`, `GROQ_API_KEY`, `HF_TOKEN`, `COHERE_API_KEY`, `XAI_API_KEY`, `OLLAMA_API_KEY`, `OPENROUTER_API_KEY`.

```
!pip install -q -U kuzu networkx matplotlib pandas git+https://github.com/
KarAnalytics/llm_cascade.git sentence-transformers
```

2.3.8.1 1) Imports and Provider Helpers

We use **Kuzu** as our embedded graph database — it runs in-process (no server to install) and supports the **Cypher** query language, the same language used by Neo4j and other production graph databases. For visualization we use **NetworkX** and **Matplotlib** to render subgraphs as node-and-edge diagrams. The `llm_cascade` library handles LLM provider selection automatically, just as it did in the SQL RAG notebook.

```
import os
import re
from pathlib import Path

import kuzu
import pandas as pd
```

```
import networkx as nx
import matplotlib.pyplot as plt

from llm_cascade import get_cascade

llm = get_cascade()


def generate_text(prompt, system_prompt=None, **kwargs):
    response = llm.generate(prompt, system_prompt=system_prompt)
    return response.text, response.provider


def has_llm_provider():
    return True  # llm_cascade auto-detects available providers


print("Provider ready (llm_cascade)")
```

2.3.8.2 2) Build the Graph Database

International trade data is a natural fit for a graph. Each **country** is a node with properties like name, continent, and population. Each **trade flow** is a directed edge from exporter to importer, carrying properties like the year and dollar value of exports. This structure makes it trivial to ask questions like "Who does Germany export to?" (follow outgoing edges) or "Which countries are trade hubs connecting two continents?" (find nodes with many incoming and outgoing edges).

We load the data from two CSV files hosted on GitHub:

File	Role	Columns
`country_names_simple.csv`	Node Data	ID, Country, Continent, Population
`edges_trade_2014to17_simple.csv`	Relationship Data	Year, Source, Target, Type, export_val

The code below downloads both files, creates the Kuzu schema (one node table and one relationship table), and loads the data. Notice the parallel to how we built the SQLite database in the previous notebook — the steps are conceptually identical (define schema, load data), just expressed in graph terms instead of relational terms.

```
import shutil
import os
from pathlib import Path
import pandas as pd # Ensure pandas is imported

DB_FOLDER = "trade_db"
COUNTRY_CSV_URL = "https://raw.githubusercontent.com/KarAnalytics/Internationa
lTrade2017/main/country_names_simple.csv"
TRADE_CSV_URL = "https://raw.githubusercontent.com/KarAnalytics/InternationalT
rade2017/main/edges_trade_2014to17_simple.csv"

LOCAL_COUNTRY_CSV = "country_names_simple.csv"
LOCAL_TRADE_CSV = "edges_trade_2014to17_simple.csv"

# 1. Clean up existing DB for a fresh run
db_path = Path(DB_FOLDER)

# Close any existing connections before attempting to remove files
# This helps release file locks from previous runs
if 'conn' in globals() and conn is not None: # Use globals() to check for
existence across cell executions
    try:
```

```
        conn.close()
        print("Closed existing Kùzu connection.")
    except Exception as e:
        print(f"Error closing connection: {e}")
if 'db' in globals() and db is not None:
    try:
        db.close()
        print("Closed existing Kùzu database.")
    except Exception as e:
        print(f"Error closing database: {e}")

if db_path.exists():
    if db_path.is_dir():
        print(f"Removing existing database directory: {db_path}")
        shutil.rmtree(db_path)
    elif db_path.is_file():
        print(f"Removing existing database file: {db_path}")
        os.remove(db_path)

# Also explicitly clean up the .wal file if it exists, as Kùzu sometimes
leaves it outside the main folder.
wal_file_path = Path(str(db_path) + ".wal")
if wal_file_path.is_file():
    print(f"Removing existing WAL file: {wal_file_path}")
    os.remove(wal_file_path)

# 2. Initialize Kùzu
db = kuzu.Database(DB_FOLDER)
conn = kuzu.Connection(db)

# 3. Create Schema
print("Defining schema...")
conn.execute("CREATE NODE TABLE Country(id STRING, name STRING, continent
STRING, population INT64, PRIMARY KEY (id))")
conn.execute("CREATE REL TABLE TRADE(FROM Country TO Country, year INT64, type
STRING, export_val DOUBLE)")

# 4. Load Node Data
print(f"Downloading and loading nodes from {COUNTRY_CSV_URL}...")
# Download CSV locally first as Kùzu COPY does not support URLs directly
df_country = pd.read_csv(COUNTRY_CSV_URL)
df_country.to_csv(LOCAL_COUNTRY_CSV, index=False)
conn.execute(f'COPY Country FROM "{LOCAL_COUNTRY_CSV}" (HEADER=TRUE)')
Path(LOCAL_COUNTRY_CSV).unlink() # Clean up local file

# 5. Load Relationship Data
# Note: Our CSV column order is (Year, Source, Target, Type, export_val).
# Kùzu's COPY REL expects the FROM/TO columns to match the relationship
definition.
# We'll use pandas to rearrange the columns to (Source, Target, Year, Type,
export_val)
# so Kùzu can match 'Source' to 'FROM' and 'Target' to 'TO'.
print(f"Downloading and loading relationships from {TRADE_CSV_URL}...")
df_trade = pd.read_csv(TRADE_CSV_URL)
cols = ['Source', 'Target', 'Year', 'Type', 'export_val']
df_trade[cols].to_csv(LOCAL_TRADE_CSV, index=False)
conn.execute(f'COPY TRADE FROM "{LOCAL_TRADE_CSV}" (HEADER=TRUE)')
Path(LOCAL_TRADE_CSV).unlink() # Clean up local file

print("\nGraph Database ready.")
```

2.3.8.3 3) Graph Schema Extraction — the "Retrieval" Step

Just as SQL RAG retrieves `CREATE TABLE` statements, Graph RAG retrieves the **graph schema**: what node labels exist, what properties they have, and what relationship types connect them. This schema plays the exact same role as the DDL in DBMS RAG — it is a

compact description of the data's structure that the LLM needs in order to write valid queries.

The key difference is representational. A SQL schema says "there is a table called SHIPMENTS with columns SID, PID, and Quantity." A graph schema says "there is a relationship type TRADE from Country to Country with properties year, type, and export_val." Both convey the same kind of information — what entities exist and how they connect — but the graph schema makes the connectivity explicit. The LLM reads this schema and learns: "Ah, countries are connected by TRADE edges, each edge has a year and a dollar value, and I can traverse these edges with Cypher patterns."

```
def get_graph_schema(conn):
    """Extract graph schema for LLM context."""
    schema_parts = ["GRAPH SCHEMA:"]

    # Nodes
    schema_parts.append("Nodes:")
    nodes = conn.execute("CALL SHOW_TABLES() RETURN *").get_as_df()
    node_tables = nodes[nodes['type'] == 'NODE']['name'].tolist()
    for nt in node_tables:
        props = conn.execute(f"CALL TABLE_INFO('{nt}') RETURN *").get_as_df()
        prop_desc = ", ".join([f"{r['name']} ({r['type']})" for _, r in
props.iterrows()])
        schema_parts.append(f"  - Label: {nt} | Properties: {prop_desc}")

    # Relationships
    schema_parts.append("Relationships:")
    rels = nodes[nodes['type'] == 'REL']['name'].tolist()
    for rt in rels:
        props = conn.execute(f"CALL TABLE_INFO('{rt}') RETURN *").get_as_df()
        prop_desc = ", ".join([f"{r['name']} ({r['type']})" for _, r in
props.iterrows()])
        # Get connectivity
        # Kùzu SHOW_TABLES output for REL tables doesn't directly show FROM/TO
in some versions,
        # but we know it's Country to Country here. For a dynamic demo, we'd
query it.
        schema_parts.append(f"  - Type: {rt} | Properties: {prop_desc} |
Connections: (Country)-[:{rt}]->(Country)")

    return "\n".join(schema_parts)

schema_context = get_graph_schema(conn)
print(schema_context)
```

2.3.8.4 4) Text-to-Cypher RAG Pipeline

Now we build the heart of Graph RAG: translating natural-language questions into **Cypher** queries. Cypher is to graph databases what SQL is to relational databases — a declarative query language — but its syntax is designed around pattern matching. Instead of writing `SELECT ... FROM ... JOIN ... ON ...`, you write patterns like `(a:Country)-[:TRADE]->(b:Country)` that visually resemble the graph structure you are searching for. This makes Cypher particularly intuitive for relationship queries: "find all countries that Germany exports to" becomes `MATCH (deu:Country {id:'deu'})-[:TRADE]->(partner:Country) RETURN partner.name`.

The pipeline follows the same two-call pattern as SQL RAG: the first LLM call generates the Cypher query, we execute it against Kuzu, and the second LLM call turns the raw results into a natural-language answer. The `extract_cypher` helper strips any markdown formatting the LLM might add, and we cap results at 50 rows to keep the final prompt manageable.

```
CYPHER_SYSTEM_PROMPT = (
    "You are a Cypher query expert. Given the graph schema below, write a
```

```
read-only "
    "Cypher query to answer the user's question.\n"
    "- Output ONLY the raw Cypher query code.\n"
    "- Do not include markdown fences, triple backticks, or any explanation.
\n"
    "- Return a maximum of 50 relevant nodes or relationships to keep context
manageable.\n"
    "- If the question cannot be answered, return: MATCH (n) RETURN
'NOT_ANSWERABLE' AS result LIMIT 1;"
)

def extract_cypher(raw_text):
    # Strip markdown fences if any
    match = re.search(r"```(?:cypher)?\s*\n?(.*?)```", raw_text, re.DOTALL |
re.IGNORECASE)
    if match: return match.group(1).strip()
    return raw_text.strip().rstrip(";")

def execute_cypher(conn, query):
    try:
        # Execute and get results
        res = conn.execute(query)
        cols = res.get_column_names()
        rows = []
        while res.has_next():
            rows.append(res.get_next())
        return cols, rows, None
    except Exception as e:
        return [], [], str(e)

def answer_with_graph_rag(question, conn):
    schema = get_graph_schema(conn)
    prompt = f"SCHEMA:\n{schema}\n\nQUESTION:\n{question}"

    # 1. Generate Cypher
    raw_cypher, prov_c = generate_text(prompt,
system_prompt=CYPHER_SYSTEM_PROMPT)
    query = extract_cypher(raw_cypher)

    print(f"  [Cypher Generated]: {query}")

    # 2. Execute
    cols, rows, error = execute_cypher(conn, query)
    if error: return f"Query Error: {error}", query

    # 3. Grounded Answer
    results_text = f"Columns: {cols}\nRows: {rows[:10]} (truncated)"
    ans_prompt = f"QUESTION: {question}\n\nCYPHER QUERY: {query}\n\nRESULTS:
{results_text}"
    answer, prov_a = generate_text(ans_prompt, system_prompt="Answer concisely
based ONLY on the graph results.")
    return answer, query
```

2.3.8.5 5) Graph Visualization

One of the most compelling advantages of graph databases is that their data has a natural visual representation. Nodes are circles, edges are arrows, and you can literally *see* the structure of your data. The function below queries the immediate trade partners of a given country and renders the result as a directed graph using NetworkX. Edge labels show export values in dollars, making it easy to spot which trade relationships are the most significant. This kind of visualization is much harder to produce from relational data, where the connections are implicit in foreign keys rather than explicit in the data model.

```
def visualize_subgraph(conn, target_country_id, depth=1):
    """Visualize immediate trade partners of a country."""
```

```
    query = f"""
    MATCH (s:Country {{id: '{target_country_id}'}})-[r:TRADE]->(t:Country)
    RETURN s.name, t.name, r.export_val, r.year
    LIMIT 20
    """
    _, rows, _ = execute_cypher(conn, query)

    G = nx.DiGraph()
    for row in rows:
        s_name, t_name, val, year = row
        G.add_edge(s_name, t_name, weight=val, label=f"{year}")

    plt.figure(figsize=(10, 6))
    pos = nx.spring_layout(G, k=0.5)
    nx.draw(G, pos, with_labels=True, node_color='skyblue', node_size=2000,
font_size=10, arrowsize=20)

    # Edge labels showing a bit of info
    edge_labels = {(u, v): f'${d["weight"]:,.0f}' for u, v, d in
G.edges(data=True)}
    nx.draw_networkx_edge_labels(G, pos, edge_labels=edge_labels, font_size=8)

    plt.title(f"Trade Connections for {target_country_id} (Sample)")
    plt.show()
```

2.3.8.6 6) Run Examples

Let us put the pipeline to work. We ask the LLM a question that requires traversing the graph — finding Germany's top trade partners by export value in 2014. The pipeline will generate a Cypher query with `ORDER BY` and `LIMIT` clauses, execute it, and return a grounded answer. We also visualize Germany's trade subgraph so you can see the network of connections firsthand.

```
if not has_llm_provider():
    print("Configuration Error: Please set API keys.")
else:
    q1 = "Who were the top 3 trade partners of Germany (deu) in 2014 by export
value?"
    print(f"\nQuestion: {q1}")
    ans, cypher = answer_with_graph_rag(q1, conn)
    print(f"Answer: {ans}")

    print("\nVisualizing subgraph for Germany...")
    visualize_subgraph(conn, "deu")
```

2.3.8.7 When Graph RAG Gets It Wrong

Graph RAG is not foolproof — and this section demonstrates exactly where it breaks down. The LLM must generate syntactically valid Cypher *and* use the correct property values. If the LLM writes `{name: 'USA'}` when the database stores `'United States'`, the query returns zero results even though the data is there. This is a fundamental challenge of text-to-query RAG: the LLM is guessing at data values it has never seen. Including sample data in the schema context (as we did in SQL RAG) can help, but it does not guarantee correctness. The next two queries illustrate this problem.

```
q2 = "Find countries in Africa that exported to both the USA (usa) and China
(chn) in 2017."
print(f"\nQuestion: {q2}")
ans, cypher = answer_with_graph_rag(q2, conn)
print(f"Answer: {ans}")
```

The query above returned zero results, claiming no African countries export to both the USA and China. That is clearly wrong — we know from common sense that many African nations trade with both superpowers. The problem is that the LLM used `{name: 'USA'}` and

`{name: 'China'}` in the Cypher query, but the database stores these countries as `'United States'` and `'China'` (or uses ISO codes). A simple name mismatch caused the entire query to silently return nothing. This is a critical lesson: **zero results does not mean the answer is zero — it may mean the query is wrong.**

```
q3 = "Which countries in Africa imported from USA (usa) in 2014."
print(f"\nQuestion: {q3}")
ans, cypher = answer_with_graph_rag(q3, conn)
print(f"Answer: {ans}")
```

2.3.8.8 Debugging: Writing Cypher by Hand

The results above looked suspicious, so let us do what any good analyst would do: write the queries ourselves and see what the data actually says. The cells below use the correct country name (`'United States'` instead of `'USA'`) and confirm that the data is there — 49 African countries imported from the US in 2014, and 48 African countries exported to both the US and China in 2017. The LLM's Cypher was structurally correct but used the wrong property values, a common and instructive failure mode.

```
### Query: Which countries in Africa imported from USA (usa) in 2014.

# The Cypher query adjusted for your database schema:
# - Node labels are 'Country'
# - The property for the country name is 'name'
# - The relationship is 'TRADE'
query = """
MATCH (usa:Country)-[t:TRADE]->(af:Country)
WHERE usa.name = 'United States'
  AND af.continent = 'Africa'
  AND t.year = 2014
  AND t.export_val > 0
RETURN af.name AS countryName ;
"""

# Execute the query and return the results as a Pandas DataFrame
# This will display as a formatted table in the notebook output
results_df = conn.execute(query).get_as_df()
print(results_df.head())
print(results_df.shape)

### USA exports to almost all African countries each year, this is something I
knew in advance.

### QUERY: Find countries in Africa that exported to both the USA (usa) and
China (chn) in 2017.

# The Cypher query adjusted for your database schema:
# - Node labels are 'Country'
# - The property for the country name is 'name'
# - The relationship is 'TRADE'
query = """
MATCH (usa:Country {name: 'United States'}), (chn:Country {name: 'China'})
MATCH (af:Country {continent: 'Africa'})-[t1:TRADE {year: 2017}]->(usa)
MATCH (af)-[t2:TRADE {year: 2017}]->(chn)
WHERE t1.export_val > 0 AND t2.export_val > 0
RETURN af.name AS countryName
"""

# Execute the query and return the results as a Pandas DataFrame
# This will display as a formatted table in the notebook output
results_df = conn.execute(query).get_as_df()
print(results_df.head())
print(results_df.shape)

### Almost every African country exports to USA as well as China each year,
this is something I knew in advance.
```

2.3.8.9 7) Reflection and Key Takeaways

Why graphs? Graph databases shine when your questions are about *connections* — who trades with whom, what paths exist between entities, which nodes are central to a network. The Cypher query language makes these patterns readable and concise. In SQL, a two-hop traversal requires nested joins; in Cypher, it is a single line: `(a)-[:TRADE]->(b)-[:TRADE]->(c)`.

Graph schema = SQL DDL. Just as DBMS RAG retrieves `CREATE TABLE` statements, Graph RAG retrieves node labels and relationship types. Both serve the same purpose: giving the LLM enough structural knowledge to write valid queries. The schema is compact and fixed-size regardless of how much data the graph contains.

The value mismatch problem. As we saw, the LLM may generate syntactically correct Cypher that uses wrong property values (e.g., `'USA'` vs. `'United States'`). This is an active area of research. Solutions include: adding sample node data to the schema context, using fuzzy matching on property values, or adding a validation step that checks query results before presenting them to the user.

Exercises to try:

1. Ask the LLM about multi-hop trade paths (e.g., "Which countries connect Germany to Brazil through a single intermediary?").
2. Modify the schema context to include sample node values and see if the LLM produces better queries.
3. Compare the same question answered by SQL RAG (if you had the trade data in a relational table) vs. Graph RAG. Which query is more readable?
4. Discuss: for what kinds of business data would you choose a graph database over a relational one?

2.4 RAG with Images and Video

The previous chapters applied RAG to text documents and structured databases. In each case, the core pattern remained the same: retrieve relevant evidence, augment the prompt with that evidence, and generate a grounded answer. This chapter extends that pattern to visual content. Instead of embedding text or executing database queries, we embed images and video frames into a vector space and retrieve them using natural-language queries. The enabling technology is CLIP, a model that bridges the gap between language and vision by placing both modalities into a shared embedding space.

2.4.1 CLIP: Bridging Text and Images

Contrastive Language-Image Pre-training, or CLIP, is a model developed by OpenAI that was trained on hundreds of millions of image-text pairs collected from the internet. During training, CLIP learns to produce embeddings where an image and its corresponding textual description end up close together in the same high-dimensional vector space. An image of a red sports car and the text "a red sports car" will have high cosine similarity, even though they are entirely different types of input.

CLIP achieves this through a dual-encoder architecture. One encoder processes images and produces a fixed-dimensional vector. A separate encoder processes text and produces a vector of the same dimensionality. The training objective pushes matching image-text pairs closer together while pushing non-matching pairs apart. The result is a shared embedding space where you can meaningfully compare a text vector against an image vector using the same cosine similarity metric you already know from document RAG.

This cross-modal capability is what makes visual RAG possible. In document RAG, both the query and the documents are text, so a single text embedding model suffices. In image RAG, the query is text but the knowledge base consists of images. Without a model like CLIP that can embed both modalities into a common space, there would be no way to compute similarity between a user's typed question and a collection of images.

2.4.2 Image RAG: From Text Queries to Visual Answers

The Image RAG pipeline follows a workflow that will feel familiar from earlier chapters. First, each image in the knowledge base is passed through CLIP's image encoder to produce an embedding vector. These vectors form a searchable index, analogous to the vector store in document RAG. When a user submits a natural-language query such as "red circular product," the query is passed through CLIP's text encoder to produce a query vector. Cosine similarity is computed between the query vector and every image vector in the index, and the top-k most similar images are retrieved.

The retrieved images then need to be interpreted and described in natural language. This is where multimodal large language models enter the pipeline. A vision-capable model such as Google's Gemini receives the retrieved images along with the user's question and any available metadata, then generates a grounded answer that references specific visual details from the retrieved images. The LLM is not hallucinating product descriptions from its training data; it is describing exactly what it sees in the images that were retrieved as evidence.

The accompanying Image RAG notebook demonstrates this pipeline using a synthetic product catalog of colored geometric shapes. While the images are simple, the pipeline is identical to what you would deploy in production with real product photographs. The key components, CLIP embedding, cosine similarity retrieval, and multimodal answer generation, remain exactly the same regardless of image complexity.

2.4.3 Video RAG: Adding the Temporal Dimension

Video RAG extends the image approach by adding a preprocessing step: keyframe extraction. A video is a sequence of frames over time, and processing every frame would be computationally wasteful since consecutive frames are nearly identical. Instead, the pipeline

samples keyframes at regular intervals, typically one frame per second, creating a manageable set of representative images from the video.

Each extracted keyframe is then embedded with CLIP exactly as in Image RAG. The critical addition is that each frame's embedding is paired with its timestamp, preserving the temporal context that makes video data unique. When a user asks "When does the green product appear?", the retrieval step identifies the most visually relevant frames and reports not just what was found but when it occurs in the video. The multimodal LLM then synthesizes an answer that references both the visual content and the timing.

The accompanying Video RAG notebook generates a synthetic product showcase video where different products appear in sequence, extracts keyframes, builds a CLIP index, and demonstrates temporal retrieval. The same architecture applies to real-world video at any scale. The only changes for production deployment would be more sophisticated keyframe selection, such as detecting scene changes rather than sampling at fixed intervals, and using a vector database to handle larger frame collections efficiently.

2.4.4 The Role of Multimodal LLMs

A distinctive feature of visual RAG compared to text-based RAG is the role the LLM plays in the generation step. In document RAG, the LLM receives text chunks and produces a text answer, operating entirely within a single modality. In visual RAG, the LLM must cross modalities: it receives images as input and produces text as output. This requires a model with vision capabilities, such as Google's Gemini, OpenAI's GPT-4o, or similar multimodal architectures that can process both pixel data and language.

These multimodal LLMs do not merely caption images. They can reason about visual content, compare multiple retrieved images, identify patterns, and synthesize observations into coherent answers. When a user asks "What are the cheapest circular products in the catalog?", the model examines the retrieved images, reads the price labels visible in the images, identifies the circular shapes, and composes a comparative answer. The grounding is visual rather than textual, but the principle is identical: the model bases its answer on retrieved evidence rather than parametric knowledge.

2.4.5 Business Applications

Visual RAG opens up application areas that text-based RAG simply cannot address. In e-commerce, customers can describe what they are looking for in natural language and retrieve matching product images from a catalog of millions without relying on manual tagging or keyword metadata. In media and entertainment, production teams can search vast video archives with queries like "scenes with outdoor lighting and two people talking" to find relevant footage in seconds. In manufacturing and quality control, visual RAG can match inspection photos against a database of known defect patterns. In healthcare, medical imaging pipelines can retrieve similar diagnostic images to support clinical decision-making. In each case, the same retrieve-augment-generate pattern applies, with CLIP providing the cross-modal bridge between human language and visual data.

2.4.6 The Notebooks

The two notebooks that follow implement these ideas end to end. The first notebook, **Image RAG**, builds a synthetic product catalog, creates a CLIP embedding index, retrieves images by text query, and generates multimodal answers with Gemini. The second notebook, **Video RAG**, generates a synthetic product showcase video, extracts keyframes, indexes them with CLIP, and demonstrates temporal retrieval with timestamped answers. Together, they show that the RAG pattern generalizes naturally from text to pixels, with CLIP and multimodal LLMs providing the machinery to bridge the modality gap.

2.4.7 References

- Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., & Sutskever, I. (2021). Learning Transferable

Visual Models From Natural Language Supervision. *Proceedings of the 38th International Conference on Machine Learning (ICML).*

- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Kuttler, H., Lewis, M., Yih, W., Rocktaschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*, 33.
- Team Gemini, Anil, R., Borgeaud, S., et al. (2024). Gemini: A Family of Highly Capable Multimodal Models. *arXiv preprint arXiv:2312.11805.*
- Luo, H., Ji, L., Zhong, M., Chen, Y., Lei, W., Duan, N., & Li, T. (2022). CLIP4Clip: An Empirical Study of CLIP for End to End Video Clip Retrieval and Captioning. *Neurocomputing*, 508, 293-304.
- Jia, C., Yang, Y., Xia, Y., Chen, Y., Parekh, Z., Pham, H., Le, Q., Sung, Y., Li, Z., & Duerig, T. (2021). Scaling Up Visual and Vision-Language Representation Learning With Noisy Text Supervision. *Proceedings of the 38th International Conference on Machine Learning (ICML).*

Open in Colab

2.4.8 RAG with Images: Image Retrieval and Multimodal Q&A

This notebook demonstrates **Retrieval-Augmented Generation (RAG)** where the knowledge source is a collection of **images** rather than text documents or a database.

How it works:

1. Build a small image dataset (synthetic product catalog images generated in-notebook — no external data needed).
2. Compute **CLIP embeddings** for each image to create a searchable vector index.
3. Given a natural-language query, compute a **text embedding** and retrieve the most relevant images.
4. Send the retrieved images to a **multimodal LLM** which generates a grounded, natural-language answer.

Learning goals:

- Understand how RAG extends to visual/multimodal data
- See how CLIP jointly embeds text and images into a shared vector space
- Practice grounding LLM output on retrieved image evidence
- Compare answers with and without image context

Provider setup: Gemini is tried first; if quota is exhausted, Ollama Cloud is used automatically.
Store your LLM API keys in Colab Secrets (or a local `.env` file). Supported keys: `GEMINI_API_KEY`, `OPENAI_API_KEY`, `GROQ_API_KEY`, `HF_TOKEN`, `COHERE_API_KEY`, `XAI_API_KEY`, `OLLAMA_API_KEY`, `OPENROUTER_API_KEY`.

```
!pip install -q google-genai Pillow==11.0.0 transformers==4.46.3 git+https://
github.com/KarAnalytics/llm_cascade.git
```

2.4.8.1 1) Imports and Provider Helpers

We use **CLIP** (from Hugging Face `transformers`) to embed images and text into the same vector space, and **PIL/Pillow** for image handling.

The provider helpers below follow the same pattern used in the DBMS RAG notebook: Gemini first, Ollama Cloud fallback, keys from Colab Secrets or `.env`.

```
import os
import base64
import io
import json
import numpy as np
from pathlib import Path

from PIL import Image, ImageDraw, ImageFont
from google import genai
from google.genai import types as genai_types

from llm_cascade import get_cascade
from llm_cascade.providers import _get_key

llm = get_cascade()


def generate_text(prompt, system_prompt=None, **kwargs):
    response = llm.generate(prompt, system_prompt=system_prompt)
```

```
    return response.text, response.provider


def has_llm_provider():
    return True  # llm_cascade auto-detects available providers


def get_gemini_api_key():
    return _get_key("GEMINI_API_KEY")


def is_gemini_quota_error(exc):
    msg = str(exc).lower()
    return any(s in msg for s in ("quota", "resource_exhausted", "429", "rate
limit", "exceeded your current quota"))


print("Provider ready (llm_cascade + Gemini multimodal)")
```

2.4.8.2 2) Generate a Synthetic Image Dataset

Since we do not have external image data, we create a small **synthetic product catalog** consisting of simple colored shapes with labels. Each image represents a product with specific visual attributes (shape, color, size, pattern).

This keeps the demo self-contained while illustrating the full Image RAG pipeline.

Product	Shape	Color	Pattern
Widget A	Circle	Red	Solid
Widget B	Square	Blue	Striped
Gadget X	Triangle	Green	Solid
Gadget Y	Circle	Yellow	Dotted
Tool M	Rectangle	Purple	Solid
Tool N	Diamond	Orange	Striped

```
import math

# --- Product catalog definition ---
PRODUCTS = [
    {"name": "Widget A", "shape": "circle",    "color": "#E74C3C",
"color_name": "red",    "pattern": "solid",   "price": 12.99, "category":
"widgets"},
    {"name": "Widget B", "shape": "square",    "color": "#3498DB",
"color_name": "blue",   "pattern": "striped", "price": 24.50, "category":
"widgets"},
    {"name": "Gadget X", "shape": "triangle",  "color": "#2ECC71",
"color_name": "green",  "pattern": "solid",   "price": 8.75,  "category":
"gadgets"},
    {"name": "Gadget Y", "shape": "circle",    "color": "#F1C40F",
"color_name": "yellow", "pattern": "dotted",  "price": 15.00, "category":
"gadgets"},
    {"name": "Tool M",   "shape": "rectangle", "color": "#9B59B6",
"color_name": "purple", "pattern": "solid",   "price": 34.99, "category":
"tools"},
    {"name": "Tool N",   "shape": "diamond",   "color": "#E67E22",
"color_name": "orange", "pattern": "striped", "price": 19.25, "category":
"tools"},
    {"name": "Widget C", "shape": "circle",    "color": "#1ABC9C",
"color_name": "teal",   "pattern": "solid",   "price": 11.50, "category":
"widgets"},
    {"name": "Gadget Z", "shape": "square",    "color": "#E91E63",
"color_name": "pink",   "pattern": "dotted",  "price": 29.99, "category":
```

```
"gadgets"},
]


def draw_product_image(product, size=256):
    """Generate a synthetic product image with shape, color, pattern, and
label."""
    img = Image.new("RGB", (size, size), "white")
    draw = ImageDraw.Draw(img)
    margin = 40
    cx, cy = size // 2, size // 2
    r = size // 2 - margin  # radius or half-size

    color = product["color"]
    shape = product["shape"]

    # Draw the shape
    if shape == "circle":
        draw.ellipse([cx - r, cy - r, cx + r, cy + r], fill=color,
outline="black", width=2)
    elif shape == "square":
        draw.rectangle([cx - r, cy - r, cx + r, cy + r], fill=color,
outline="black", width=2)
    elif shape == "rectangle":
        draw.rectangle([cx - r, cy - r // 2, cx + r, cy + r // 2], fill=color,
outline="black", width=2)
    elif shape == "triangle":
        points = [(cx, cy - r), (cx - r, cy + r), (cx + r, cy + r)]
        draw.polygon(points, fill=color, outline="black", width=2)
    elif shape == "diamond":
        points = [(cx, cy - r), (cx + r, cy), (cx, cy + r), (cx - r, cy)]
        draw.polygon(points, fill=color, outline="black", width=2)

    # Add pattern overlay
    if product["pattern"] == "striped":
        for y in range(0, size, 12):
            draw.line([(0, y), (size, y)], fill="white", width=2)
    elif product["pattern"] == "dotted":
        for dx in range(margin, size - margin, 20):
            for dy in range(margin, size - margin, 20):
                draw.ellipse([dx - 3, dy - 3, dx + 3, dy + 3], fill="white")

    # Add product name label at top
    try:
        font = ImageFont.truetype("arial.ttf", 16)
    except (OSError, IOError):
        font = ImageFont.load_default()
    bbox = draw.textbbox((0, 0), product["name"], font=font)
    tw = bbox[2] - bbox[0]
    draw.text(((size - tw) // 2, 5), product["name"], fill="black", font=font)

    # Add price label at bottom
    price_text = f"${product['price']:.2f}"
    bbox = draw.textbbox((0, 0), price_text, font=font)
    tw = bbox[2] - bbox[0]
    draw.text(((size - tw) // 2, size - 25), price_text, fill="black",
font=font)

    return img


# Generate and save all product images
IMAGE_DIR = Path("product_images")
IMAGE_DIR.mkdir(exist_ok=True)

image_paths = []
```

```
for product in PRODUCTS:
    img = draw_product_image(product)
    filename = product["name"].lower().replace(" ", "_") + ".png"
    path = IMAGE_DIR / filename
    img.save(path)
    image_paths.append(path)

print(f"Generated {len(image_paths)} product images in '{IMAGE_DIR}/'")
print("Files:", [p.name for p in image_paths])
```

2.4.8.2.1 Preview the Generated Images

Before we index anything, it is worth taking a moment to look at our dataset. The images below are intentionally simple – colored geometric shapes with text labels – but they carry enough visual variety (shape, color, pattern, price) to make retrieval interesting. In a production system these would be real product photographs, but synthetic images let us control every attribute and verify that CLIP picks up on the right visual cues.

```
import matplotlib.pyplot as plt

fig, axes = plt.subplots(2, 4, figsize=(14, 7))
for ax, product, path in zip(axes.flat, PRODUCTS, image_paths):
    img = Image.open(path)
    ax.imshow(img)
    ax.set_title(f"{product['name']} ({product['color_name']})", fontsize=10)
    ax.axis("off")
plt.suptitle("Synthetic Product Catalog", fontsize=14, fontweight="bold")
plt.tight_layout()
plt.show()
```

2.4.8.3 3) Build the Image Index with CLIP Embeddings

CLIP (Contrastive Language-Image Pre-training) is the key technology that makes Image RAG possible. Unlike traditional image classifiers that output a fixed set of labels, CLIP maps both text and images into the **same 512-dimensional vector space**. It was trained on hundreds of millions of image-caption pairs from the internet, learning to push matching image-text pairs close together and non-matching pairs apart.

Why does this matter for RAG? Because once images and text live in a shared space, we can measure the similarity between a user's natural-language query and every image in our catalog using ordinary cosine similarity – no manual tagging, no predefined categories. The model "understands" that the text "red circular product" should be close to an image of a red circle, even though those are entirely different data modalities. This is analogous to how ChromaDB uses text embeddings in document RAG, but here the bridge spans from language to vision.

We compute an embedding for each product image below, normalize them to unit vectors, and store the result as our searchable image index.

```
from transformers import CLIPProcessor, CLIPModel
import torch

# Load CLIP model (downloads ~600 MB on first run)
CLIP_MODEL_NAME = "openai/clip-vit-base-patch32"
clip_model = CLIPModel.from_pretrained(CLIP_MODEL_NAME)
clip_processor = CLIPProcessor.from_pretrained(CLIP_MODEL_NAME)

print(f"CLIP model loaded: {CLIP_MODEL_NAME}")
print(f"Embedding dimension: {clip_model.config.projection_dim}")

def embed_images(image_paths, model, processor):
    """Compute CLIP embeddings for a list of image files."""
    images = [Image.open(p).convert("RGB") for p in image_paths]
    inputs = processor(images=images, return_tensors="pt", padding=True)
    with torch.no_grad():
        embeddings = model.get_image_features(**inputs)
```

```
    # Normalize to unit vectors for cosine similarity
    embeddings = embeddings / embeddings.norm(dim=-1, keepdim=True)
    return embeddings.numpy()


def embed_text(query, model, processor):
    """Compute CLIP embedding for a text query."""
    inputs = processor(text=[query], return_tensors="pt", padding=True,
truncation=True)
    with torch.no_grad():
        embeddings = model.get_text_features(**inputs)
    embeddings = embeddings / embeddings.norm(dim=-1, keepdim=True)
    return embeddings.numpy()


# Build the image index
image_embeddings = embed_images(image_paths, clip_model, clip_processor)
print(f"Image index built: {image_embeddings.shape[0]} images,
{image_embeddings.shape[1]}-dim embeddings")
```

With our index built – 8 images, each represented as a 512-dimensional unit vector – we are ready to perform cross-modal search. Notice that the embedding functions above handle text and images through different CLIP encoders (`get_image_features` vs. `get_text_features`), but both produce vectors in the same space. That shared geometry is what lets us compare apples (text) to oranges (images) using a single dot product.

2.4.8.4 4) Image Retrieval – the "Retrieval" in Image RAG

Now that every product image has been mapped to a 512-dimensional vector, we can perform retrieval. Given a text query, we embed it with CLIP's text encoder and compute **cosine similarity** between the query vector and every image vector in our index. Because we normalized all embeddings to unit length earlier, this cosine similarity reduces to a simple dot product – a single matrix multiplication gives us a ranked list of all images.

A word about the similarity scores you will see below: CLIP similarities tend to be modest in absolute terms (often 0.2-0.4 for good matches) because the 512-dimensional space is vast and the model must accommodate enormous variety. What matters is the **relative ordering** – the top-ranked image should be the most visually relevant to the query, even if its raw score looks low. Think of the score not as a confidence percentage but as a ranking signal.

This is the image equivalent of vector similarity search in document RAG.

```
def retrieve_images(query, image_embeddings, image_paths, products, model,
processor, top_k=3):
    """Retrieve the top-k most relevant images for a text query."""
    query_emb = embed_text(query, model, processor)
    # Cosine similarity (embeddings are already normalized)
    similarities = (query_emb @ image_embeddings.T).flatten()
    # Get top-k indices
    top_indices = similarities.argsort()[::-1][:top_k]

    results = []
    for idx in top_indices:
        results.append({
            "rank": len(results) + 1,
            "product": products[idx],
            "path": image_paths[idx],
            "similarity": float(similarities[idx]),
        })
    return results


# Test retrieval
test_query = "red circular product"
results = retrieve_images(test_query, image_embeddings, image_paths, PRODUCTS,
```

```
clip_model, clip_processor)

print(f"Query: '{test_query}'\n")
for r in results:
    print(f"  #{r['rank']} {r['product']['name']} "
          f"({r['product']['color_name']} {r['product']['shape']}) "
          f"— similarity: {r['similarity']:.4f}")

# Visualize retrieval results
def show_retrieval_results(query, results):
    """Display retrieved images side by side."""
    n = len(results)
    fig, axes = plt.subplots(1, n, figsize=(5 * n, 5))
    if n == 1:
        axes = [axes]
    for ax, r in zip(axes, results):
        img = Image.open(r["path"])
        ax.imshow(img)
        ax.set_title(f"#{r['rank']} {r['product']
['name']}\nsim={r['similarity']:.3f}", fontsize=11)
        ax.axis("off")
    fig.suptitle(f"Query: \"{query}\"", fontsize=13, fontweight="bold")
    plt.tight_layout()
    plt.show()


show_retrieval_results(test_query, results)
```

2.4.8.5 5) Multimodal Answer Generation

This is the "Generation" half of Image RAG – the step where a multimodal LLM actually looks at the retrieved images and produces a grounded answer. Unlike document RAG where the LLM receives text chunks, here the model receives **actual image pixels** alongside product metadata and the user's question.

We send images as base64-encoded data to the Gemini API, which can natively process both text and images in a single request. This is what makes multimodal LLMs so powerful for Image RAG: they do not just rely on metadata tags – they can "see" the product's color, shape, and visual details directly. If the LLM provider does not support vision (e.g., a text-only fallback model), we gracefully degrade to sending the product metadata as text context, which still produces reasonable answers but loses the visual grounding.

```
def image_to_base64(image_path):
    """Convert an image file to a base64 string."""
    with open(image_path, "rb") as f:
        return base64.b64encode(f.read()).decode("utf-8")


def build_metadata_context(results):
    """Build a text description of retrieved products for text-only
fallback."""
    lines = ["Retrieved products (ranked by visual similarity):"]
    for r in results:
        p = r["product"]
        lines.append(
            f"  #{r['rank']} {p['name']}: {p['color_name']} {p['shape']}, "
            f"{p['pattern']} pattern, ${p['price']:.2f},
category={p['category']} "
            f"(similarity={r['similarity']:.3f})"
        )
    return "\n".join(lines)


ANSWER_SYSTEM_PROMPT = (
    "You are a helpful product catalog assistant. Use ONLY the provided images
and "
```

```
    "product metadata to answer the user's question. If the retrieved products
do not "
    "contain enough information, say so. Do not invent products that are not
shown."
)


def generate_multimodal_answer(question, results, gemini_model="gemini-2.5-
flash", **kwargs):
    """Send retrieved images + metadata to a multimodal LLM for a grounded
answer."""
    metadata_text = build_metadata_context(results)
    gemini_key = get_gemini_api_key()

    # --- Try Gemini (native multimodal) ---
    if gemini_key:
        try:
            client = genai.Client(api_key=gemini_key)
            parts = []
            parts.append(genai_types.Part.from_text(
                text=f"{ANSWER_SYSTEM_PROMPT}\n\n{metadata_text}\n\nQUESTION:
{question}"
            ))
            for r in results:
                img_bytes = Path(r["path"]).read_bytes()
                parts.append(genai_types.Part.from_bytes(
                    data=img_bytes,
                    mime_type="image/png",
                ))

            resp = client.models.generate_content(
                model=gemini_model,
                contents=parts,
            )
            return resp.text, "gemini"
        except Exception as exc:
            if not is_gemini_quota_error(exc):
                raise
            print("Gemini quota exhausted. Falling back to text-only LLM
cascade ...")

    # --- Fallback: text-only via llm_cascade ---
    prompt = f"{metadata_text}\n\nQUESTION: {question}"
    response = llm.generate(prompt, system_prompt=ANSWER_SYSTEM_PROMPT)
    return response.text, response.provider


print("Multimodal answer generation ready.")
```

2.4.8.6 6) Full Image RAG Pipeline

Now we connect the two halves of the system – retrieval and generation – into a single end-to-end function. Given a natural-language question, the pipeline embeds the query with CLIP, retrieves the most relevant product images, and then hands those images (along with their metadata) to a multimodal LLM for a grounded answer.

We also define a "without RAG" baseline that sends the same question directly to the LLM with no image context at all. Comparing the two reveals a striking pattern: without retrieval, the model has no knowledge of our product catalog and either refuses to answer or invents products that do not exist. This is the same hallucination problem we saw in document RAG, but it becomes even more vivid when the "ground truth" is visual.

```
def answer_with_image_rag(question, top_k=3, gemini_model="gemini-2.5-flash",
**kwargs):
    """Full Image RAG pipeline: embed query -> retrieve images -> multimodal
```

```
answer."""
    print(f"  [Step 1] Embedding query with CLIP ...")
    results = retrieve_images(
        question, image_embeddings, image_paths, PRODUCTS,
        clip_model, clip_processor, top_k=top_k,
    )
    print(f"  [Step 2] Retrieved {len(results)} images:")
    for r in results:
        print(f"           #{r['rank']} {r['product']['name']}
(sim={r['similarity']:.3f})")

    print(f"  [Step 3] Generating multimodal answer ...")
    answer, provider = generate_multimodal_answer(
        question, results,
        gemini_model=gemini_model,
    )
    print(f"  [Step 4] Answer generated (provider: {provider})")

    return answer, results, provider


def answer_without_rag(question, **kwargs):
    """Direct LLM answer with no image context."""
    answer, provider = generate_text(prompt=question)
    return answer, provider


print("Full Image RAG pipeline ready.")
```

2.4.8.7 7) Run End-to-End Examples

We test several questions and compare:

- **With Image RAG:** CLIP retrieval + multimodal grounded answer
- **Without RAG:** direct LLM answer (no product images or metadata)

Watch for hallucinations in the "without RAG" answers — the model has no access to our product catalog.

```
questions = [
    "What red products do you have?",
    # "Show me the cheapest circular product.",
   # "Which products have a striped pattern?",
    # "I need a green product. What are my options?",
    "What is the most expensive product in the catalog?",
]


def preview(text, max_len=800):
    text = text or ""
    return text[:max_len] + ("..." if len(text) > max_len else "")


if not has_llm_provider():
    print("Error: No LLM API key configured. Set at least one API key in Colab
Secrets.")
else:
    for i, q in enumerate(questions, start=1):
        print("\n" + "=" * 80)
        print(f"Q{i}. {q}\n")

        try:
            answer_rag, retrieved, prov = answer_with_image_rag(q)
            show_retrieval_results(q, retrieved)
            print("\n  [WITH IMAGE RAG]")
            print(preview(answer_rag))
```

```
    except Exception as e:
        print(f"  RAG error: {e}")

    try:
        answer_direct, prov_direct = answer_without_rag(q)
        print(f"\n  [WITHOUT RAG] (provider: {prov_direct})")
        print(preview(answer_direct))
    except Exception as e:
        print(f"  Direct error: {e}")
```

2.4.8.7.1 Checkpoint: Reflection Questions

1. Compare the RAG and non-RAG answers. Which ones hallucinate products that don't exist?
2. How does CLIP's cross-modal embedding enable text-to-image search without manual tagging?
3. What happens when you query for something not in the catalog (e.g., "black hexagonal product")?
4. How does this Image RAG pattern compare to the DBMS RAG and vector-store RAG approaches?

2.4.8.8 8) Interactive Query (Optional)

Now it is your turn. The cell below lets you ask your own questions against the product image catalog. Try queries that test different aspects of CLIP's cross-modal understanding: ask about colors, shapes, patterns, price ranges, or combinations of attributes. You might also try querying for something that does not exist in the catalog (e.g., "black hexagonal product") to see how the system handles out-of-distribution queries – it will still return the "closest" matches, but with lower similarity scores.

```
# Change this question to anything you want to ask about the product catalog.
my_question = "What purple or blue products do you have, and which is
cheaper?"

if has_llm_provider():
    print(f"Question: {my_question}\n")
    try:
        answer, retrieved, provider = answer_with_image_rag(my_question)
        show_retrieval_results(my_question, retrieved)
        print(f"\nAnswer ({provider}):\n{answer}")
    except Exception as e:
        print(f"Error: {e}")
else:
    print("No LLM API key configured. Set at least one API key in Colab
Secrets.")
```

2.4.8.9 9) Teaching Notes and Exercises

Key takeaways:

- RAG extends naturally to images and other modalities via **cross-modal embeddings** like CLIP.
- CLIP embeds text and images into a **shared vector space**, enabling text-to-image retrieval without manual tagging.
- The retrieval step uses the same cosine similarity as document RAG, but operates on image embeddings.
- Multimodal LLMs can directly "see" retrieved images, producing answers grounded on visual evidence.
- Even with synthetic data, the pipeline demonstrates real-world patterns used in e-commerce, medical imaging, and visual search.

Exercises:

- Replace the synthetic images with real product photos (e.g., from a Kaggle dataset) and observe how retrieval quality changes.

- Experiment with different CLIP model sizes (`clip-vit-large-patch14`) and compare retrieval accuracy.
- Add more products with similar colors/shapes and test whether the system can still distinguish them.
- Implement a **re-ranking** step where the LLM re-orders retrieved images by relevance before answering.
- Discuss: when would you prefer Image RAG over text-based RAG for a business application?

Open in Colab

2.4.9 RAG with Video: Visual Question-Answering over Video Frames

This notebook demonstrates **Retrieval-Augmented Generation (RAG)** where the knowledge source is a **video**. Instead of searching documents or databases, we extract frames from a video, index them with CLIP embeddings, and retrieve the most relevant frames to answer natural-language questions.

How it works:

1. Generate a short **synthetic video** (animated product showcase — no external data needed).
2. Extract **keyframes** at regular intervals.
3. Compute **CLIP embeddings** for each frame to create a searchable index.
4. Given a natural-language question, retrieve the most relevant frame(s) via cosine similarity.
5. Send retrieved frames to a **multimodal LLM** for a grounded, natural-language answer.

Learning goals:

- Understand how RAG applies to temporal/video data
- See how frame extraction converts video into an image retrieval problem
- Practice grounding LLM output on visual evidence from specific moments in a video
- Compare answers with and without video context

Store whichever API keys you have in Colab Secrets (or a local `.env` file): `GEMINI_API_KEY`, `OLLAMA_API_KEY`, `XAI_API_KEY`, `GROQ_API_KEY`, `HF_TOKEN`, `COHERE_API_KEY`, `OPENROUTER_API_KEY`

```
!pip install -q google-genai Pillow==11.0.0 transformers==4.46.3 matplotlib
opencv-python-headless git+https://github.com/KarAnalytics/llm_cascade.git
```

2.4.9.1 1) Imports and Provider Helpers (7-Vendor Cascade)

We use **OpenCV** to create and read video, **CLIP** to embed frames and text queries into the same vector space, and **PIL/Pillow** for image display. If you have already worked through the Image RAG notebook, you will recognize most of these imports – and that is the point. Video RAG is fundamentally the same pattern as Image RAG, applied to a different modality. The main new ingredient is OpenCV, which handles the mechanics of extracting individual frames from a video file.

```
#
==============================================================================
# Imports + LLM Cascade
#
==============================================================================

import os
import base64
import io
import math
import numpy as np
from pathlib import Path

import cv2
from PIL import Image, ImageDraw, ImageFont
from google import genai
from google.genai import types as genai_types
```

```python
from llm_cascade import get_cascade
from llm_cascade.providers import _get_key

llm = get_cascade()


def has_llm_provider():
    """True if llm_cascade has at least one provider configured."""
    return True  # get_cascade() already validates


def generate_text(prompt, system_prompt=None):
    """
    Send a text prompt through llm_cascade.
    Returns (response_text, provider_name).
    """
    resp = llm.generate(prompt, system_prompt=system_prompt)
    return resp.text, f"{resp.provider}/{resp.model}"


print(f"LLM cascade ready.")
```

2.4.9.2 2) Generate a Synthetic Video

We create a short animated video that simulates a **product showcase reel**. The video has distinct scenes, each showing a different product with visual attributes that change over time.

This keeps the demo self-contained (no external video files needed) while providing enough visual variety to test retrieval.

Time (sec)	Scene	Content
0–3	Title card	"Acme Corp Product Showcase" on blue background
3–7	Product 1	Red circle — "Widget Alpha" with price
7–11	Product 2	Green triangle — "Gadget Beta" with price
11–15	Product 3	Blue square — "Tool Gamma" with price
15–19	Product 4	Orange diamond — "Device Delta" with price
19–22	Closing card	"Thank you! Visit acme-corp.example.com"

```python
VIDEO_PATH = "product_showcase.mp4"
FPS = 10
WIDTH, HEIGHT = 640, 480

# Scene definitions
SCENES = [
    {
        "name": "title",
        "start": 0, "end": 3,
        "label": "Acme Corp\nProduct Showcase 2026",
        "bg_color": (40, 60, 120),
        "shape": None,
        "product": None,
    },
    {
        "name": "widget_alpha",
        "start": 3, "end": 7,
        "label": "Widget Alpha – $12.99",
        "bg_color": (255, 245, 238),
        "shape": "circle",
        "shape_color": (231, 76, 60),
        "product": {"name": "Widget Alpha", "price": 12.99, "color": "red",
```

```
"shape": "circle"},
    },
    {
        "name": "gadget_beta",
        "start": 7, "end": 11,
        "label": "Gadget Beta — $8.75",
        "bg_color": (240, 255, 240),
        "shape": "triangle",
        "shape_color": (46, 204, 113),
        "product": {"name": "Gadget Beta", "price": 8.75, "color": "green",
"shape": "triangle"},
    },
    {
        "name": "tool_gamma",
        "start": 11, "end": 15,
        "label": "Tool Gamma — $34.99",
        "bg_color": (235, 245, 255),
        "shape": "square",
        "shape_color": (52, 152, 219),
        "product": {"name": "Tool Gamma", "price": 34.99, "color": "blue",
"shape": "square"},
    },
    {
        "name": "device_delta",
        "start": 15, "end": 19,
        "label": "Device Delta — $19.25",
        "bg_color": (255, 248, 230),
        "shape": "diamond",
        "shape_color": (230, 126, 34),
        "product": {"name": "Device Delta", "price": 19.25, "color": "orange",
"shape": "diamond"},
    },
    {
        "name": "closing",
        "start": 19, "end": 22,
        "label": "Thank you!\nVisit acme-corp.example.com",
        "bg_color": (40, 60, 120),
        "shape": None,
        "product": None,
    },
]


def draw_frame_pil(scene, t, width=WIDTH, height=HEIGHT):
    """Draw a single video frame for a given scene and time offset using
PIL."""
    bg = scene["bg_color"]
    img = Image.new("RGB", (width, height), bg)
    draw = ImageDraw.Draw(img)

    # Load font
    try:
        font_large = ImageFont.truetype("/usr/share/fonts/truetype/dejavu/
DejaVuSans-Bold.ttf", 28)
        font_small = ImageFont.truetype("/usr/share/fonts/truetype/dejavu/
DejaVuSans.ttf", 22)
    except (OSError, IOError):
        try:
            font_large = ImageFont.truetype("arial.ttf", 28)
            font_small = ImageFont.truetype("arial.ttf", 22)
        except (OSError, IOError):
            font_large = ImageFont.load_default()
            font_small = font_large

    cx, cy = width // 2, height // 2
```

```
    if scene["shape"] is None:
        # Title or closing card — centered text
        text_color = (255, 255, 255)
        lines = scene["label"].split("\n")
        total_h = len(lines) * 40
        y_start = cy - total_h // 2
        for j, line in enumerate(lines):
            bbox = draw.textbbox((0, 0), line, font=font_large)
            tw = bbox[2] - bbox[0]
            draw.text(((width - tw) // 2, y_start + j * 40), line,
fill=text_color, font=font_large)
    else:
        # Product scene — animated shape (gentle bounce)
        bounce = int(15 * math.sin(t * 3))
        shape_cy = cy - 30 + bounce
        r = 80
        color = scene["shape_color"]

        if scene["shape"] == "circle":
            draw.ellipse([cx - r, shape_cy - r, cx + r, shape_cy + r],
fill=color, outline=(0, 0, 0), width=3)
        elif scene["shape"] == "square":
            draw.rectangle([cx - r, shape_cy - r, cx + r, shape_cy + r],
fill=color, outline=(0, 0, 0), width=3)
        elif scene["shape"] == "triangle":
            pts = [(cx, shape_cy - r), (cx - r, shape_cy + r), (cx + r,
shape_cy + r)]
            draw.polygon(pts, fill=color, outline=(0, 0, 0), width=3)
        elif scene["shape"] == "diamond":
            pts = [(cx, shape_cy - r), (cx + r, shape_cy), (cx, shape_cy + r),
(cx - r, shape_cy)]
            draw.polygon(pts, fill=color, outline=(0, 0, 0), width=3)

        # Product label at top
        bbox = draw.textbbox((0, 0), scene["label"], font=font_small)
        tw = bbox[2] - bbox[0]
        draw.text(((width - tw) // 2, 20), scene["label"], fill=(30, 30, 30),
font=font_small)

        # Timestamp at bottom-right
        ts_text = f"t={t:.1f}s"
        draw.text((width - 90, height - 30), ts_text, fill=(120, 120, 120),
font=font_small)

    return img


def generate_video(path, scenes, fps, width=WIDTH, height=HEIGHT):
    """Generate an MP4 video from scene definitions."""
    fourcc = cv2.VideoWriter_fourcc(*"mp4v")
    writer = cv2.VideoWriter(path, fourcc, fps, (width, height))

    for scene in scenes:
        duration = scene["end"] - scene["start"]
        n_frames = int(duration * fps)
        for i in range(n_frames):
            t = scene["start"] + i / fps
            pil_img = draw_frame_pil(scene, t, width, height)
            # Convert PIL (RGB) -> OpenCV (BGR)
            frame = cv2.cvtColor(np.array(pil_img), cv2.COLOR_RGB2BGR)
            writer.write(frame)

    writer.release()


generate_video(VIDEO_PATH, SCENES, FPS)
```

```
total_duration = SCENES[-1]["end"]
file_size_kb = Path(VIDEO_PATH).stat().st_size / 1024
print(f"Video generated: '{VIDEO_PATH}'")
print(f"  Duration: {total_duration}s | FPS: {FPS} | Resolution: {WIDTH}
x{HEIGHT} | Size: {file_size_kb:.0f} KB")
```

2.4.9.3 3) Extract Keyframes from the Video

Here is the crucial insight behind Video RAG: we do not try to process the video as a continuous stream. Instead, we **sample keyframes** at regular intervals and treat each one as an independent image. This converts the temporal video retrieval problem into the familiar image retrieval problem we already solved in the Image RAG notebook.

Why not process the full video? A 22-second video at 10 FPS contains 220 frames, most of which are nearly identical to their neighbors. Processing every frame would waste compute and storage without improving retrieval quality. By extracting one frame per second, we capture each scene while keeping our index compact. For longer or faster-paced videos you would adjust this sampling rate – the tradeoff is always between coverage (catching brief but important moments) and efficiency (keeping the index manageable).

```
def extract_keyframes(video_path, interval_sec=1.0):
    """Extract frames from a video at a fixed time interval.

    Returns a list of dicts: {"timestamp": float, "image": PIL.Image}
    """
    cap = cv2.VideoCapture(video_path)
    fps = cap.get(cv2.CAP_PROP_FPS)
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    duration = total_frames / fps
    frame_interval = int(fps * interval_sec)

    frames = []
    frame_idx = 0
    while True:
        cap.set(cv2.CAP_PROP_POS_FRAMES, frame_idx)
        ret, frame = cap.read()
        if not ret:
            break
        # Convert BGR -> RGB -> PIL
        rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        pil_img = Image.fromarray(rgb)
        timestamp = frame_idx / fps
        frames.append({"timestamp": timestamp, "image": pil_img})
        frame_idx += frame_interval

    cap.release()
    return frames, duration


keyframes, video_duration = extract_keyframes(VIDEO_PATH, interval_sec=1.0)
print(f"Extracted {len(keyframes)} keyframes from {video_duration:.1f}s video
(1 frame/sec)")
print(f"Timestamps: {[f'{kf["timestamp"]:.1f}s' for kf in keyframes]}")
```

2.4.9.3.1 Preview Keyframes

The grid below shows every other keyframe from our 22-second video. You can see the distinct scenes: a title card, four product showcases (each with a different colored shape), and a closing card. Each of these frames will become an entry in our CLIP index, tagged with its timestamp so we can tell the user not just *what* was found but *when* in the video it appeared.

```
import matplotlib.pyplot as plt

# Show a subset of keyframes (every other frame to save space)
```

```
display_frames = keyframes[::2]
n = len(display_frames)
cols = min(n, 6)
rows = math.ceil(n / cols)

fig, axes = plt.subplots(rows, cols, figsize=(4 * cols, 3.5 * rows))
axes_flat = axes.flat if hasattr(axes, 'flat') else [axes]
for i, ax in enumerate(axes_flat):
    if i < n:
        ax.imshow(display_frames[i]["image"])
        ax.set_title(f"t = {display_frames[i]['timestamp']:.1f}s",
fontsize=10)
    ax.axis("off")
fig.suptitle("Keyframes from Product Showcase Video (every 2s)", fontsize=13,
fontweight="bold")
plt.tight_layout()
plt.show()
```

2.4.9.4 4) Build the Frame Index with CLIP Embeddings

Just like in Image RAG, we use **CLIP** to embed each keyframe into a 512-dimensional vector. At query time we embed the user's text query with the same model and find the closest frame(s) by cosine similarity. The embedding and retrieval code is nearly identical to what we wrote for static images – because once you extract keyframes, a video *is* just a collection of images.

The key difference from Image RAG is that each frame also carries a **timestamp**. This temporal metadata transforms a simple "which image matches?" answer into a richer "at what point in the video can you find this?" answer. Timestamps are what make Video RAG genuinely useful for applications like meeting recording search, surveillance review, or training video navigation.

```
from transformers import CLIPProcessor, CLIPModel
import torch

CLIP_MODEL_NAME = "openai/clip-vit-base-patch32"
clip_model = CLIPModel.from_pretrained(CLIP_MODEL_NAME)
clip_processor = CLIPProcessor.from_pretrained(CLIP_MODEL_NAME)

print(f"CLIP model loaded: {CLIP_MODEL_NAME}")
print(f"Embedding dimension: {clip_model.config.projection_dim}")

def embed_frames(keyframes, model, processor):
    """Compute CLIP embeddings for a list of keyframe images."""
    images = [kf["image"].convert("RGB") for kf in keyframes]
    inputs = processor(images=images, return_tensors="pt", padding=True)
    with torch.no_grad():
        embeddings = model.get_image_features(**inputs)
    embeddings = embeddings / embeddings.norm(dim=-1, keepdim=True)
    return embeddings.numpy()


def embed_text(query, model, processor):
    """Compute CLIP embedding for a text query."""
    inputs = processor(text=[query], return_tensors="pt", padding=True,
truncation=True)
    with torch.no_grad():
        embeddings = model.get_text_features(**inputs)
    embeddings = embeddings / embeddings.norm(dim=-1, keepdim=True)
    return embeddings.numpy()


# Build the frame index
frame_embeddings = embed_frames(keyframes, clip_model, clip_processor)
print(f"Frame index built: {frame_embeddings.shape[0]} frames,
{frame_embeddings.shape[1]}-dim embeddings")
```

Our frame index is now ready: 22 keyframes, each represented as a 512-dimensional CLIP vector. Notice how the code above is almost identical to the image embedding code from the Image RAG notebook – the only difference is that our "images" come from video frames rather than from files on disk. This reuse of the same CLIP infrastructure across modalities is a powerful design pattern: once you can convert any data source into images, you get cross-modal search for free.

2.4.9.5 5) Frame Retrieval — the "Retrieval" in Video RAG

Given a text query, we:

1. Embed the query with CLIP's text encoder.
2. Compute **cosine similarity** against all frame embeddings.
3. Return the top-k most similar frames with their timestamps.

The timestamp tells us *when* in the video the answer can be found.

```
## 6) Answer Generation -- Multimodal LLM with Retrieved Frames

The multimodal LLM receives the **retrieved frames** as actual images, along
with **metadata** about each frame (timestamp, scene name, product info) and
the user's **question**. It generates an answer grounded on what it can see in
the frames, referencing specific timestamps to tell the user exactly where in
the video the relevant content appears.

This is where Video RAG delivers something that pure text RAG never could:
temporal grounding. Instead of just saying "the red product costs $12.99," the
system can say "at t=4.0s in the video, you can see Widget Alpha, a red circle
priced at $12.99." That temporal precision is what makes Video RAG valuable
for applications like searching through hours of meeting recordings or
surveillance footage.

We also provide a "without RAG" function that answers with no video context,
so we can compare the grounded answer against a baseline that has never seen
our video.
```

2.4.9.6 6) Answer Generation — Multimodal LLM with Retrieved Frames

The multimodal LLM receives:

- The **retrieved frames** (as images)
- **Metadata** about each frame (timestamp, scene, product info)
- The user's **question**

It generates an answer grounded on what it can see in the frames and the associated timestamps.

We also provide a "without RAG" function that answers with no video context.

```
def build_frame_context(results):
    """Build a text description of retrieved frames."""
    lines = ["Retrieved video frames (ranked by visual similarity):"]
    for r in results:
        prod = r["product"]
        prod_str = (
            f"{prod['name']}: {prod['color']} {prod['shape']},
${prod['price']:.2f}"
            if prod else "(title/closing card)"
        )
        lines.append(
            f"  #{r['rank']} timestamp={r['timestamp']:.1f}s,
scene={r['scene_name']}, "
            f"product={prod_str}, similarity={r['similarity']:.3f}"
        )
    return "\n".join(lines)
```

```
VIDEO_RAG_SYSTEM_PROMPT = (
    "You are a helpful video analyst. You are given keyframes extracted from a
product "
    "showcase video, along with their timestamps and metadata. Answer the
user's question "
    "based on what you can SEE in the frames and the provided metadata.
Reference specific "
    "timestamps when relevant (e.g., 'at t=5.0s'). If the frames do not
contain enough "
    "information, say so. Do not invent products or details not shown."
)


def generate_answer_with_frames(question, results):
    """Send retrieved frames + metadata to an LLM.

    Gemini gets native multimodal (images + text) if GEMINI_API_KEY is set.
    Otherwise falls back to llm_cascade with text-only metadata context.
    """
    context_text = build_frame_context(results)

    # Try Gemini multimodal first (needs actual images)
    gemini_key = _get_key("GEMINI_API_KEY")
    if gemini_key:
        try:
            client = genai.Client(api_key=gemini_key)
            parts = []
            parts.append(genai_types.Part.from_text(
text=f"{VIDEO_RAG_SYSTEM_PROMPT}\n\n{context_text}\n\nQUESTION: {question}"
            ))
            for r in results:
                buf = io.BytesIO()
                r["image"].save(buf, format="PNG")
                parts.append(genai_types.Part.from_bytes(data=buf.getvalue(),
mime_type="image/png"))
            resp = client.models.generate_content(model="gemini-2.5-flash",
contents=parts)
            return resp.text, "Gemini (multimodal)"
        except Exception as exc:
            print(f"  [Gemini multimodal] failed: {exc} -- falling back to
text-only cascade")

    # Fallback: text-only via llm_cascade
    prompt = f"{context_text}\n\nQUESTION: {question}"
    resp = llm.generate(prompt, system_prompt=VIDEO_RAG_SYSTEM_PROMPT)
    return resp.text, f"{resp.provider}/{resp.model}"


def generate_answer_without_rag(question):
    """Direct LLM answer with NO video frames."""
    return generate_text(question)


print("Answer generation functions ready (llm_cascade).")
```

2.4.9.7 7) Full Video RAG Pipeline

Putting it all together into a single callable function. The pipeline mirrors what we built for Image RAG – embed the query, retrieve relevant frames, generate an answer – but now each result includes a timestamp that grounds the answer in the video's timeline. This is the complete Video RAG loop: from a natural-language question to a temporally grounded, visually informed answer.

```
def answer_with_video_rag(question, top_k=3):
    """Full Video RAG: extract frames -> CLIP retrieval -> multimodal LLM
answer."""
    print(f"  [Step 1] Embedding query with CLIP ...")
    results = retrieve_frames(
        question, frame_embeddings, keyframes, clip_model, clip_processor,
top_k=top_k,
    )
    print(f"  [Step 2] Retrieved {len(results)} frame(s):")
    for r in results:
        prod_name = r['product']['name'] if r['product'] else '(card)'
        print(f"           #{r['rank']} t={r['timestamp']:.1f}s — {prod_name}
(sim={r['similarity']:.3f})")

    print(f"  [Step 3] Generating multimodal answer ...")
    answer, provider = generate_answer_with_frames(question, results)
    print(f"  [Step 4] Answer generated (provider: {provider})")
    return answer, results, provider


print("Video RAG pipeline ready.")
```

2.4.9.8 8) Run End-to-End Examples: With RAG vs. Without RAG

We test questions about the product showcase video and compare:

- **With Video RAG:** CLIP retrieves the relevant frame(s), multimodal LLM answers with visual + temporal grounding
- **Without RAG:** LLM answers with no video context — it knows nothing about our product showcase

Watch how the without-RAG answers either refuse to answer (no context) or hallucinate product details.

```
questions = [
    "What is the red product shown in the video and how much does it cost?",
    #"At what point in the video does the green triangle appear?",
    #"What is the most expensive product in this showcase video?",
    #"What shape is the orange product?",
    "What text appears at the beginning of the video?",
]


def preview(text, max_len=800):
    text = text or ""
    return text[:max_len] + ("..." if len(text) > max_len else "")


if not has_llm_provider():
    print("Error: No LLM API key configured. Set at least one API key in Colab
Secrets.")
else:
    for i, q in enumerate(questions, start=1):
        print("\n" + "=" * 80)
        print(f"Q{i}. {q}")
        print("=" * 80)

        # --- WITH VIDEO RAG ---
        print("\n--- WITH VIDEO RAG ---")
        try:
            answer_rag, retrieved, prov = answer_with_video_rag(q)
            show_retrieved_frames(q, retrieved)
            print(f"\n  Answer (provider: {prov}):")
            print(preview(answer_rag))
        except Exception as e:
            print(f"  RAG error: {e}")
```

```
    # --- WITHOUT RAG ---
    print("\n--- WITHOUT RAG (LLM knowledge only) ---")
    try:
        answer_direct, prov_direct = generate_answer_without_rag(q)
        print(f"  Answer (provider: {prov_direct}):")
        print(preview(answer_direct))
    except Exception as e:
        print(f"  Direct error: {e}")

    print()
```

2.4.9.8.1 Checkpoint: Reflection Questions

1. **Temporal grounding:** Did the RAG answers include timestamps? How useful is knowing *when* in the video something appears?
2. **Without-RAG failure:** The LLM has never seen our synthetic video — what kind of responses did it give? Refusal? Hallucination?
3. **Frame sampling:** We sampled 1 frame/second. What would happen with 1 frame every 5 seconds? Every 0.5 seconds?
4. **Comparison with Image RAG:** What extra complexity does the temporal dimension add over static Image RAG?

2.4.9.9 9) Interactive Query (Optional)

Ask your own questions about the product showcase video.

```
# Change this to any question about the product showcase video
my_question = "How many products are shown in total, and what are their
prices?"

if has_llm_provider():
    print(f"Question: {my_question}\n")

    print("=" * 60)
    print("WITH VIDEO RAG:")
    print("=" * 60)
    try:
        answer, retrieved, provider = answer_with_video_rag(my_question,
top_k=4)
        show_retrieved_frames(my_question, retrieved)
        print(f"\nAnswer ({provider}):\n{answer}")
    except Exception as e:
        print(f"Error: {e}")

    print("\n" + "=" * 60)
    print("WITHOUT RAG:")
    print("=" * 60)
    try:
        answer_direct, prov_direct = generate_answer_without_rag(my_question)
        print(f"\nAnswer ({prov_direct}):\n{answer_direct}")
    except Exception as e:
        print(f"Error: {e}")
else:
    print("No LLM API key configured. Set at least one API key in Colab
Secrets.")
```

2.4.9.10 10) Teaching Notes and Exercises

Key takeaways:

- **Video RAG = frame extraction + Image RAG + temporal metadata.** The video is decomposed into keyframes that are indexed and retrieved just like static images.
- **Timestamps add a temporal dimension** that Image RAG lacks — answers can reference *when* in the video something occurs.

- **Frame sampling rate** is a key design choice: too sparse and you miss content; too dense and you waste compute and storage.
- The same **CLIP + multimodal LLM** stack from Image RAG works here — the main new component is the frame extraction step.
- **Without-RAG comparison** is especially stark for video: the LLM has never seen the video, so it either refuses or hallucinates.

Exercises:

- Download a short YouTube clip (e.g., a product review) and replace the synthetic video. How does CLIP handle real-world video frames?
- Experiment with different frame sampling rates (0.5s, 2s, 5s) and measure the impact on retrieval accuracy.
- Add **scene change detection** instead of fixed-interval sampling — extract frames only when the visual content changes significantly.
- Combine video frames with **audio transcription** (e.g., Whisper) for a multimodal RAG that uses both visual and spoken content.
- Discuss: In what business scenarios is Video RAG most valuable? (e.g., surveillance review, training video search, meeting recording Q&A, sports analytics)

3. RAG Frameworks

3.1 LlamaIndex

In the preceding chapters, you built RAG pipelines from scratch: chunking documents, generating embeddings, computing cosine similarities, constructing prompts, and calling an LLM. That hands-on experience was essential for understanding what RAG actually does under the hood. But in practice, writing all of that plumbing for every new project is tedious and error-prone. LlamaIndex exists to eliminate that repetitive work. It is a Python framework that abstracts the entire RAG pipeline into a handful of lines of code, letting you go from a folder of documents to a working question-answering system in minutes rather than hours.

3.1.1 Philosophy: Opinionated and Batteries-Included

LlamaIndex takes an opinionated, batteries-included approach to RAG. Where other frameworks give you a toolbox of components and ask you to wire them together, LlamaIndex makes strong default choices on your behalf. It decides how to chunk your documents, which embedding model to use, how to build the vector index, and how to construct the retrieval prompt. You can override any of these decisions, but the framework is designed so that the defaults work well for the most common use case: document question-answering. This philosophy makes LlamaIndex particularly attractive for rapid prototyping and for teams that want a working RAG system without spending days tuning pipeline parameters. Point it at your documents, ask a question, and get a grounded answer.

3.1.2 Key Components

The LlamaIndex workflow revolves around four core components. The first is `SimpleDirectoryReader`, a data loader that reads files from a local directory and converts them into LlamaIndex `Document` objects. It handles plain text, PDFs, and dozens of other formats automatically. The second is `VectorStoreIndex`, which takes those documents, chunks them, generates embeddings, and builds an in-memory vector index in a single call. The third is the `query_engine`, created by calling `.as_query_engine()` on an index. The query engine accepts a natural-language question, retrieves the most relevant chunks, constructs an augmented prompt, sends it to the LLM, and returns the answer. The entire retrieve-augment-generate pipeline is encapsulated in that one method call. The fourth component is `Settings`, a global configuration object where you specify which LLM and which embedding model to use. By setting `Settings.llm` and `Settings.embed_model` once at the top of your script, every downstream component automatically picks up those choices without additional configuration.

3.1.3 Data Loaders and LlamaHub

One of LlamaIndex's most powerful features is its ecosystem of data loaders, available through LlamaHub. While `SimpleDirectoryReader` handles local files, LlamaHub offers connectors to over 300 external data sources. You can pull documents directly from Google Drive, Notion, Confluence, Slack, GitHub repositories, AWS S3 buckets, SQL databases, and many more. Each connector is a drop-in replacement: swap `SimpleDirectoryReader` for `GithubRepositoryReader` or `GoogleDriveReader`, and the rest of your pipeline remains unchanged. This means the same three-line RAG pattern works whether your documents live in a local folder, a cloud storage bucket, or a team wiki. For enterprise applications, where data is scattered across dozens of systems, this connector ecosystem can save weeks of integration work.

3.1.4 What Gets Abstracted Away

If you compare a LlamaIndex pipeline to the from-scratch RAG you built in earlier chapters, the contrast is striking. In the manual approach, you wrote explicit code for every step: splitting text into chunks with careful attention to overlap, calling an embedding API for each chunk, storing vectors in a list or array, computing cosine similarities, ranking results, assembling a prompt template, and parsing the LLM response. With LlamaIndex, all of those steps still happen, but they happen inside the framework.

`VectorStoreIndex.from_documents()` handles chunking, embedding, and indexing in one call. The query engine handles retrieval, prompt construction, and generation in another. The tradeoff is visibility: when something goes wrong in a from-scratch pipeline, you can inspect every intermediate result. When something goes wrong inside LlamaIndex, you need to understand the framework's internals to debug it. This is why building RAG from first principles before adopting a framework is so valuable. You know what the framework is doing because you have done it yourself.

3.1.5 The Hands-On Notebook

The notebook that follows this introduction demonstrates LlamaIndex using three sets of fictional company documents: Acme Analytics, Globex Cybersecurity, and Nextera Green Solutions. These companies are entirely synthetic, meaning the LLM has never encountered them in training. Any correct answer must come from RAG retrieval, not from the model's parametric memory. You will load all three document sets using `SimpleDirectoryReader`, build a unified vector index, and query across companies to see how LlamaIndex automatically surfaces the right documents regardless of which company the question concerns. The notebook also includes a commented-out example using the GitHub data loader from LlamaHub, so you can see how easily one data source can be swapped for another.

3.1.6 References

1. LlamaIndex Documentation. "Getting Started." LlamaIndex, 2025. https://docs.llamaindex.ai/
2. LlamaHub. "LlamaIndex Data Connectors." LlamaHub, 2025. https://llamahub.ai/
3. Liu, J. "LlamaIndex: A Data Framework for LLM Applications." GitHub, 2025. https://github.com/run-llama/llama_index
4. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., et al. (2020). "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." *Advances in Neural Information Processing Systems*, 33, 9459-9474. https://arxiv.org/abs/2005.11401

3.1.7 LlamaIndex RAG Demo: Document Q&A with a Framework

This notebook demonstrates how **LlamaIndex** simplifies building RAG pipelines. Instead of manually chunking documents, computing embeddings, and querying vector stores, LlamaIndex handles the plumbing so you can focus on the application logic.

How it works:

1. Create a small set of **synthetic company documents** (no external data needed).
2. Use LlamaIndex to **ingest, chunk, embed, and index** the documents in one step.
3. Given a natural-language question, LlamaIndex **retrieves** relevant chunks and sends them to an LLM.
4. Compare answers **with RAG** (LlamaIndex pipeline) vs. **without RAG** (direct LLM call).

Learning goals:

- Understand what a RAG framework (LlamaIndex) does under the hood
- See how LlamaIndex abstracts document loading, chunking, embedding, and retrieval
- Compare the framework approach to the manual RAG we built in earlier notebooks
- Observe how RAG grounding eliminates hallucination on private/synthetic data

Provider setup: This notebook uses the `llm_cascade` package, which auto-detects your API keys and falls back to the next provider if one is unavailable. Supported providers: OpenAI, Gemini, Ollama, Grok (xAI), Groq, HuggingFace, Cohere, OpenRouter.

Store any of these API keys in Colab Secrets (or a local `.env` file): `OPENAI_API_KEY`, `GEMINI_API_KEY`, `OLLAMA_API_KEY`, `XAI_API_KEY`, `GROQ_API_KEY`, `HF_TOKEN`, `COHERE_API_KEY`, `OPENROUTER_API_KEY`

```
!pip install -q -U llama-index llama-index-llms-openai-like llama-index-llms-
gemini llama-index-embeddings-huggingface llama-index-readers-github google-
genai openai git+https://github.com/KarAnalytics/llm_cascade.git sentence-
transformers
```

3.1.7.1 1) Imports and Provider Helpers (7-Vendor Cascade)

We configure **LlamaIndex** to use whichever LLM provider you have available. The cascade tries each provider in order and falls through on quota errors, ensuring the notebook works regardless of which API keys you have set up.

For **embeddings**, we use a local HuggingFace model (`all-MiniLM-L6-v2`) so that embedding always works regardless of which LLM API key you have. This is a deliberate design choice: by keeping the embedding model local, we avoid the situation where a rate-limited API blocks the entire pipeline. The LLM is the only component that needs an external API call.

```
from pathlib import Path
from llm_cascade.providers import PROVIDERS, _load_env, _get_key,
_is_retriable_error

_load_env()

def get_available_providers():
    return [p for p in PROVIDERS if _get_key(p['key_env'])]

def has_llm_provider():
    return len(get_available_providers()) > 0
```

```
# Print status
available = get_available_providers()
if available:
    print('Providers configured (in fallback order):')
    for p in available:
        print(f"  + {p['name']:<16} model = {p['default_model']}")
else:
    print('WARNING: No API keys found.')
```

3.1.7.2 2) Configure LlamaIndex LLM and Embeddings

LlamaIndex needs two things configured before it can build a RAG pipeline:

- An **LLM** for generating answers (we pick the first available provider from our cascade)
- An **embedding model** for converting text to vectors (we use a local HuggingFace model – no API key needed)

This cell configures LlamaIndex's global `Settings` object so all downstream components use our chosen models. The `CascadeLLM` wrapper you see below is necessary because LlamaIndex expects an LLM object that conforms to its own interface, but our `llm_cascade` package has its own API. The wrapper bridges the two, translating LlamaIndex's `complete()` calls into `llm_cascade`'s `generate()` calls. Without this adapter, we would be locked into a single provider and lose the automatic fallback behavior that makes these notebooks robust across different API key configurations.

```
from llama_index.core import Settings
from llama_index.core.llms import CustomLLM, LLMMetadata, CompletionResponse,
CompletionResponseGen
from llama_index.embeddings.huggingface import HuggingFaceEmbedding
from llm_cascade import get_cascade
from typing import Any

# ---- Embedding model (local, no API key needed)
-----------------------------
Settings.embed_model = HuggingFaceEmbedding(model_name="sentence-transformers/
all-MiniLM-L6-v2")
print("Embedding model: sentence-transformers/all-MiniLM-L6-v2 (local)")


# ---- Custom LlamaIndex LLM that wraps llm_cascade for automatic fallback
----
class CascadeLLM(CustomLLM):
    """LlamaIndex LLM that delegates to llm_cascade (8-provider fallback)."""
    context_window: int = 8000
    num_output: int = 1024
    model_name: str = "llm_cascade"

    @property
    def metadata(self) -> LLMMetadata:
        return LLMMetadata(
            context_window=self.context_window,
            num_output=self.num_output,
            model_name=self.model_name,
        )

    def complete(self, prompt: str, **kwargs: Any) -> CompletionResponse:
        cascade = get_cascade(verbose=False)
        response = cascade.generate(prompt)
        return CompletionResponse(text=response.text,
additional_kwargs={"provider": response.provider, "model": response.model})

    def stream_complete(self, prompt: str, **kwargs: Any) ->
CompletionResponseGen:
        # Non-streaming fallback
```

```
        resp = self.complete(prompt, **kwargs)
        def gen():
            yield resp
        return gen()


Settings.llm = CascadeLLM()
print("LLM: CascadeLLM (auto-fallback across available providers)")
```

3.1.7.3 3) Create Synthetic Company Documents

We generate **fictional documents for three companies** that the LLM has never seen in training. This ensures that correct answers can **only** come from RAG retrieval, not from parametric memory. Each company's documents live in a **separate directory**, simulating real-world data silos.

Directory	Company	Documents
`acme_docs/`	Acme Analytics Inc. (BI tools, Lawrence KS)	Overview, Products, Financials, Handbook, Case Study
`globex_docs/`	Globex Cybersecurity Corp. (cybersecurity, Boston)	Overview, Products, Incident Report
`nextera_docs/`	Nextera Green Solutions (sustainability consulting, Denver)	Overview, Services, Impact Report

Later, we will use LlamaIndex **Data Loaders** (`SimpleDirectoryReader`) to ingest all three directories — demonstrating how LlamaIndex can pull information from multiple, separate data sources and unify them in a single searchable index.

```
DOCUMENTS = {
    "company_overview.txt": """ACME ANALYTICS INC. — COMPANY OVERVIEW

Acme Analytics Inc. was founded in 2019 by Dr. Sarah Chen and Marcus Rivera
in Lawrence, Kansas. The company specializes in AI-powered business
intelligence
tools for mid-market companies (100–5,000 employees).

Headquarters: 1420 Jayhawk Boulevard, Lawrence, KS 66045
Employees: 287 (as of January 2026)
Annual Revenue (2025): $42.3 million
Funding: Series B ($18M raised in March 2023 from Midwest Ventures)

Mission: "To democratize data analytics so every business decision is informed
by evidence, not intuition."

The company operates three offices: Lawrence (HQ), Chicago (sales), and
Austin (engineering). CEO: Dr. Sarah Chen. CTO: Marcus Rivera. CFO: Linda
Park.""",

    "products_and_services.txt": """ACME ANALYTICS — PRODUCTS AND SERVICES

1. InsightBoard Pro (Flagship Product)
   - Real-time dashboard platform with natural language query interface
   - Pricing: $49/user/month (Standard), $89/user/month (Enterprise)
   - Supports PostgreSQL, MySQL, Snowflake, BigQuery, and Redshift
   - 1,847 active enterprise customers as of Q4 2025

2. DataPipe ETL
   - Automated data pipeline builder with 200+ pre-built connectors
   - Pricing: starts at $500/month for up to 10 million records/day
   - Launched in September 2024

3. PredictIQ
   - ML-powered forecasting add-on for InsightBoard Pro
```

```
   - Pricing: $29/user/month (requires InsightBoard Pro subscription)
   - Uses proprietary time-series model trained on retail and logistics data
   - Beta launched March 2025, GA release planned for June 2026

All products include 24/7 email support. Enterprise plans include dedicated
account manager and 99.9% SLA.""",

    "q4_2025_financials.txt": """ACME ANALYTICS — Q4 2025 FINANCIAL RESULTS
(CONFIDENTIAL)

Period: October 1 — December 31, 2025

Revenue:          $12.8 million (Q4) / $42.3 million (FY 2025)
Gross Margin:     78.2%
Operating Profit: $1.9 million (Q4) / $5.1 million (FY 2025)
Net Income:       $1.4 million (Q4)
Cash on Hand:     $14.7 million
Burn Rate:        Company is cash-flow positive since Q2 2025

Key Metrics:
- ARR (Annual Recurring Revenue): $48.6 million (up 34% YoY)
- Net Revenue Retention: 118%
- Customer Acquisition Cost (CAC): $8,200
- Customer Lifetime Value (LTV): $67,400
- LTV/CAC Ratio: 8.2x

Headcount grew from 241 to 287 employees during 2025.
R&D spending: 31% of revenue. Sales & Marketing: 28% of revenue.""",

    "employee_handbook_excerpt.txt": """ACME ANALYTICS — EMPLOYEE HANDBOOK
(EXCERPT)

PAID TIME OFF (PTO):
- All full-time employees receive 22 days of PTO per year (accrued monthly).
- PTO increases to 27 days after 3 years of service.
- Unused PTO can be carried over (max 5 days) or paid out at year-end.
- Sick leave: 10 days per year (separate from PTO).

REMOTE WORK POLICY:
- Engineering and Data Science teams: fully remote eligible.
- Sales and Customer Success: hybrid (minimum 2 days/week in office).
- All employees may work remotely up to 4 weeks/year from any US location.
- International remote work requires VP approval and tax review.

PROFESSIONAL DEVELOPMENT:
- Annual learning budget: $2,500 per employee.
- Conference attendance: up to 2 conferences per year with manager approval.
- Tuition reimbursement: up to $5,250/year for degree programs.""",

    "customer_case_study.txt": """CUSTOMER CASE STUDY: BIGRETAIL CORP

Company: BigRetail Corp (1,200 retail stores across 38 states)
Challenge: Siloed data across POS, inventory, and CRM systems made it
impossible for regional managers to get timely insights.

Solution: Deployed InsightBoard Pro Enterprise + DataPipe ETL
- Connected 14 data sources in 3 weeks using DataPipe
- 340 regional managers now use InsightBoard daily
- Natural language queries replaced manual SQL report requests

Results (after 6 months):
- 62% reduction in time-to-insight (from 4 days to 1.5 days average)
- $3.2 million saved in inventory carrying costs
- 23% increase in regional manager satisfaction scores
- SQL report request backlog eliminated entirely
```

```
Quote from BigRetail CIO Janet Torres:
\"InsightBoard Pro transformed how our managers interact with data.
They went from waiting days for a report to asking questions in plain
English and getting answers in seconds.\""""
}

# Write documents to disk
DOC_DIR = Path("acme_docs")
DOC_DIR.mkdir(exist_ok=True)

for filename, content in DOCUMENTS.items():
    (DOC_DIR / filename).write_text(content, encoding="utf-8")

print(f"Created {len(DOCUMENTS)} documents in '{DOC_DIR}/':")
for f in sorted(DOC_DIR.iterdir()):
    print(f"  {f.name} ({f.stat().st_size} bytes)")

# --- Company 2: Globex Cybersecurity Corp. → globex_docs/ ---

GLOBEX_DOCUMENTS = {
    "globex_overview.txt": """GLOBEX CYBERSECURITY CORP. — COMPANY OVERVIEW

Globex Cybersecurity Corp. was founded in 2017 by former NSA analyst
James Whitfield and AI researcher Dr. Priya Narayanan in Boston,
Massachusetts.
The company provides AI-driven cybersecurity solutions for financial services
and healthcare organizations.

Headquarters: 88 Federal Street, Suite 1200, Boston, MA 02110
Employees: 412 (as of January 2026)
Annual Revenue (2025): $78.5 million
Funding: Series C ($45M raised in June 2024 from CyberVentures Capital
and Goldman Sachs Growth Equity)

Mission: "To make enterprise-grade cybersecurity autonomous, adaptive,
and accessible to every regulated industry."

The company operates four offices: Boston (HQ), Washington D.C. (government
relations), Tel Aviv (R&D), and Singapore (APAC sales).
CEO: James Whitfield. CTO: Dr. Priya Narayanan. CISO: Robert Tanaka.""",

    "globex_products.txt": """GLOBEX CYBERSECURITY — PRODUCTS AND SERVICES

1. ShieldAI Platform (Flagship Product)
   - AI-powered threat detection and automated incident response
   - Uses behavioral analysis to detect zero-day attacks in real time
   - Pricing: $120/endpoint/year (Standard), $195/endpoint/year (Premium)
   - 623 enterprise customers across 18 countries as of Q4 2025
   - Average detection time: 2.3 minutes (industry average: 197 days)

2. VaultGuard
   - Data encryption and access management platform for regulated industries
   - HIPAA, SOX, and PCI-DSS compliant out of the box
   - Pricing: $35,000/year (up to 500 users), $60,000/year (unlimited users)
   - Launched January 2023

3. ThreatIntel Live
   - Real-time threat intelligence feed with dark web monitoring
   - Pricing: $8,500/month (includes API access and analyst dashboard)
   - Aggregates data from 2,400+ sources updated every 15 minutes
   - Integrates with Splunk, CrowdStrike, Palo Alto, and Microsoft Sentinel

All products include a dedicated Security Operations Center (SOC) team for
Premium customers. Globex holds ISO 27001 and SOC 2 Type II
certifications.""",
```

```
    "globex_incident_report.txt": """GLOBEX CYBERSECURITY — 2025 THREAT
LANDSCAPE REPORT (PUBLIC)

Published: February 2026

KEY FINDINGS FROM GLOBEX'S CUSTOMER BASE (2025):

Total incidents analyzed: 14,237 across 623 customers
- Ransomware attacks: up 42% YoY (3,891 incidents)
- Phishing campaigns: up 28% YoY (5,102 incidents)
- Supply chain attacks: up 67% YoY (1,245 incidents — fastest-growing
category)
- Insider threats: relatively stable at 1,834 incidents

ShieldAI Platform Performance:
- Blocked 97.3% of attacks before any data exfiltration
- Mean time to detect (MTTD): 2.3 minutes
- Mean time to respond (MTTR): 8.7 minutes (automated response)
- False positive rate: 0.4% (down from 1.2% in 2024)

NOTABLE CASE: MERIDIAN HEALTH SYSTEM
- 340-bed hospital network targeted by Medusa ransomware variant
- ShieldAI detected anomalous lateral movement within 47 seconds
- Automated containment isolated 3 affected servers in under 2 minutes
- Zero patient data exfiltrated; zero downtime for clinical systems
- Estimated savings: $12M+ (average healthcare ransomware cost per Ponemon)

INDUSTRY PREDICTIONS FOR 2026:
- AI-generated phishing emails will become indistinguishable from human-
written
- Quantum-resistant encryption adoption will begin in financial services
- Average cost of a data breach expected to exceed $5.2 million globally""",
}

GLOBEX_DIR = Path("globex_docs")
GLOBEX_DIR.mkdir(exist_ok=True)

for filename, content in GLOBEX_DOCUMENTS.items():
    (GLOBEX_DIR / filename).write_text(content, encoding="utf-8")

print(f"Created {len(GLOBEX_DOCUMENTS)} documents in '{GLOBEX_DIR}/':")
for f in sorted(GLOBEX_DIR.iterdir()):
    print(f"  {f.name} ({f.stat().st_size} bytes)")

# --- Company 3: Nextera Green Solutions → nextera_docs/ ---

NEXTERA_DOCUMENTS = {
    "nextera_overview.txt": """NEXTERA GREEN SOLUTIONS — COMPANY OVERVIEW

Nextera Green Solutions was founded in 2020 by climate scientist Dr. Elena
Vasquez and former McKinsey partner David Okafor in Denver, Colorado.
The company provides sustainability consulting and carbon accounting software
to help companies meet ESG (Environmental, Social, Governance) targets.

Headquarters: 1700 Lincoln Street, Suite 3400, Denver, CO 80203
Employees: 156 (as of January 2026)
Annual Revenue (2025): $23.8 million
Funding: Series A ($12M raised in November 2023 from Climate Capital Partners
and Patagonia Ventures)

Mission: "To give every company a clear, data-driven path to net zero —
turning climate commitments into measurable action."

The company operates two offices: Denver (HQ) and London (European
operations).
CEO: Dr. Elena Vasquez. COO: David Okafor. VP Engineering: Kenji Tanaka.
```

```
Nextera is a certified B Corporation and a member of the UN Global
Compact.""",

    "nextera_services.txt": """NEXTERA GREEN SOLUTIONS — SERVICES AND PLATFORM

1. CarbonLens Platform (Flagship Product)
   - Automated Scope 1, 2, and 3 carbon emissions tracking
   - Integrates with ERP systems (SAP, Oracle, NetSuite) to pull real data
   - AI-powered recommendations for emission reduction strategies
   - Pricing: $2,500/month (Standard, up to $500M revenue companies)
              $6,500/month (Enterprise, unlimited + custom reporting)
   - 312 active customers as of Q4 2025

2. ESG Reporting Suite
   - Automated generation of sustainability reports (GRI, SASB, TCFD, CDP)
   - Audit-ready documentation with full data lineage
   - Pricing: $18,000/year (bundled with CarbonLens Enterprise)
   - Standalone: $24,000/year
   - Launched April 2024

3. Sustainability Strategy Consulting
   - Science-Based Targets initiative (SBTi) alignment services
   - Supply chain decarbonization roadmaps
   - Pricing: Project-based ($75,000 — $250,000 per engagement)
   - Team of 28 consultants with backgrounds in climate science and industry

4. Carbon Offset Marketplace
   - Verified carbon credits from 45 projects across 12 countries
   - Blockchain-verified chain of custody for each credit
   - Transaction fee: 4.5% per credit purchase
   - Beta launched August 2025

All platform customers receive quarterly sustainability benchmarking reports
comparing their performance against industry peers.""",

    "nextera_impact_report.txt": """NEXTERA GREEN SOLUTIONS — 2025 IMPACT
REPORT

Published: January 2026

COMPANY IMPACT METRICS:
- Total customer carbon emissions tracked: 47.3 million metric tons CO2e
- Verified emission reductions enabled: 6.8 million metric tons CO2e (14.4%)
- Customers who improved their CDP score after using Nextera: 89%
- Average time to produce ESG report reduced from 14 weeks to 3.5 weeks

NOTABLE CLIENT RESULTS:

1. SummitBrew Coffee (specialty coffee chain, 380 locations)
   - Reduced Scope 3 supply chain emissions by 31% in 18 months
   - Switched 62% of logistics to electric vehicles using Nextera roadmap
   - First coffee chain to achieve SBTi-validated near-term target

2. Pacific Northwest Lumber Co. (timber and building materials)
   - Used CarbonLens to discover 40% of emissions came from transportation
   - Optimized shipping routes: saved 8,200 metric tons CO2e and $1.4M/year
   - Achieved carbon-neutral certification for 2025 operations

3. Meridian Financial Group (regional bank, $8B assets)
   - Implemented financed emissions tracking (PCAF methodology)
   - Identified $2.1B in loan portfolio exposed to high transition risk
   - Developed green lending program: $340M in sustainability-linked loans

AWARDS AND RECOGNITION (2025):
- Fast Company "Most Innovative Companies" — Energy category (#7)
- B Corp "Best for the World" — Environment category
```

```
- Colorado Governor's Award for Clean Energy Innovation""",
}

NEXTERA_DIR = Path("nextera_docs")
NEXTERA_DIR.mkdir(exist_ok=True)

for filename, content in NEXTERA_DOCUMENTS.items():
    (NEXTERA_DIR / filename).write_text(content, encoding="utf-8")

print(f"Created {len(NEXTERA_DOCUMENTS)} documents in '{NEXTERA_DIR}/':")
for f in sorted(NEXTERA_DIR.iterdir()):
    print(f"  {f.name} ({f.stat().st_size} bytes)")
```

3.1.7.4 4) Ingest Documents Using LlamaIndex Data Loaders (via LlamaHub)

A key feature of LlamaIndex is its **Data Loaders** available via LlamaHub. Data Loaders can pull information from **hundreds of sources** – Google Docs, Notion, **GitHub**, Slack, databases, PDFs, or simple `.txt` files. Think of LlamaHub as a plugin ecosystem: instead of writing custom ingestion code for each data source, you install a loader and point it at your data.

Here we use `SimpleDirectoryReader` (the built-in Data Loader for local files) to load documents from **three separate company directories** into a single searchable index. Under the hood, `SimpleDirectoryReader` reads each file, wraps it in a LlamaIndex `Document` object with metadata (file name, path), and passes it to `VectorStoreIndex`. The index then automatically chunks each document, computes embeddings using our configured HuggingFace model, and stores everything in an in-memory vector store. All of that – chunking, embedding, indexing – happens in a single `VectorStoreIndex.from_documents()` call.

Want to try the GitHub Data Loader? LlamaHub also offers `GithubRepositoryReader`, which reads files directly from a GitHub repository – no `git clone` needed. See the commented-out code below to try it yourself. You just need a free GitHub token (any GitHub account, no special permissions needed for public repos): https://github.com/settings/tokens?type=beta

Loader	Source	Package	Used here?
`SimpleDirectoryReader`	Local `.txt` files	`llama-index` (built-in)	**Yes (default)**
`GithubRepositoryReader`	GitHub repo	`llama-index-readers-github` (LlamaHub)	Commented-out example
`GoogleDriveReader`	Google Drive	`llama-index-readers-google`	Available via LlamaHub
`NotionPageReader`	Notion pages	`llama-index-readers-notion`	Available via LlamaHub
`ConfluenceReader`	Atlassian Confluence	`llama-index-readers-confluence`	Available via LlamaHub
`S3Reader`	AWS S3 buckets	`llama-index-readers-s3`	Available via LlamaHub
...and 300+ more			

```
from llama_index.core import SimpleDirectoryReader, VectorStoreIndex

# ============================================================================
# DEFAULT: Load documents from local directories using SimpleDirectoryReader
# ============================================================================
```

```python
LOCAL_DIRS = {
    "acme_docs":    "Acme Analytics (BI tools)",
    "globex_docs":  "Globex Cybersecurity (security)",
    "nextera_docs": "Nextera Green Solutions (sustainability)",
}

all_documents = []
for dir_name, label in LOCAL_DIRS.items():
    loader = SimpleDirectoryReader(dir_name)
    docs = loader.load_data()
    print(f"  Loaded: '{dir_name}/' → {len(docs)} docs  [{label}]")
    all_documents.extend(docs)

print(f"Total documents loaded from {len(LOCAL_DIRS)} directories:
{len(all_documents)}")

# ============================================================================
# OPTIONAL: Try the GitHub Data Loader instead (LlamaHub)
#
# This reads the SAME documents directly from a GitHub repo — no git clone
# needed. To try it:
#   1. Generate a free GitHub token: https://github.com/settings/tokens?type=
beta
#      (any GitHub account works; no special permissions needed for public
repos)
#   2. Add it to your .env file:  GITHUB_TOKEN=github_pat_xxxx
#      Or in Colab: add a Secret named GITHUB_TOKEN
#   3. Uncomment the code below and comment out the SimpleDirectoryReader
above
# ============================================================================

# from llama_index.readers.github import GithubRepositoryReader, GithubClient
#
# github_client = GithubClient(github_token=_get_key("GITHUB_TOKEN"),
verbose=False)
#
# GITHUB_DIRS = {
#     "acme_docs":    "Acme Analytics (BI tools)",
#     "globex_docs":  "Globex Cybersecurity (security)",
#     "nextera_docs": "Nextera Green Solutions (sustainability)",
# }
#
# all_documents = []
# for dir_path, label in GITHUB_DIRS.items():
#     reader = GithubRepositoryReader(
#         github_client=github_client,
#         owner="KarAnalytics",                    # GitHub username
#         repo="llamaindexdata",        # repo name
#         filter_directories=([dir_path],
GithubRepositoryReader.FilterType.INCLUDE),
#         verbose=False,
#     )
#     docs = reader.load_data(branch="main")
#     print(f"  GitHub Loader: '{dir_path}/' → {len(docs)} docs  [{label}]")
#     all_documents.extend(docs)

# ----------------------------------------------------------------------------
# Build a SINGLE vector index across all three companies
# ----------------------------------------------------------------------------
index = VectorStoreIndex.from_documents(all_documents, show_progress=True)
print(f"Vector index built with {len(all_documents)} chunks")

# Create a query engine
query_engine = index.as_query_engine(similarity_top_k=3)
print("Query engine ready — ask questions about ANY of the three companies!")
```

Notice how few lines of code that took. Two calls – `SimpleDirectoryReader` to load, `VectorStoreIndex.from_documents()` to index – replaced all the manual chunking, embedding, and vector store insertion we did in earlier notebooks. This is the core value proposition of LlamaIndex: it abstracts the RAG plumbing so you can focus on choosing the right data sources and tuning retrieval quality.

3.1.7.5 5) What Just Happened? Peeking Under the Hood

The two lines of code above – `SimpleDirectoryReader` and `VectorStoreIndex.from_documents()` – did a remarkable amount of work. They loaded 11 text files, split them into chunks, computed a 384-dimensional embedding for each chunk using our local HuggingFace model, and organized those embeddings into a searchable index. If you built RAG manually in the earlier notebooks, you know that each of those steps required explicit code: reading files, calling a text splitter, batching through an embedding API, and inserting into a vector store.

Let us inspect what LlamaIndex created so we can verify that the abstraction is doing sensible things. The cell below runs a test retrieval query and shows which chunks came back, including their similarity scores and source files. This transparency is important: frameworks are powerful, but you should always be able to look inside and confirm that the right documents are being retrieved for the right reasons.

```
# Inspect the chunks LlamaIndex created
retriever = index.as_retriever(similarity_top_k=3)

# Test retrieval without LLM — just see what chunks come back
test_query = "How much revenue did Acme make in Q4 2025?"
retrieved_nodes = retriever.retrieve(test_query)

print(f"Query: '{test_query}'")
print(f"Retrieved {len(retrieved_nodes)} chunks:\n")
for i, node in enumerate(retrieved_nodes, 1):
    source = node.metadata.get('file_name', 'unknown')
    score = node.score
    text_preview = node.text[:200].replace('\n', ' ')
    print(f"  Chunk #{i} (score={score:.4f}, source={source})")
    print(f"    {text_preview}...\n")
```

3.1.7.6 6) Query with RAG vs. Without RAG

Now we compare:

- **With RAG (LlamaIndex):** The query engine retrieves relevant chunks, sends them + the question to the LLM, and returns a grounded answer.
- **Without RAG (Direct LLM):** We ask the same question directly to the LLM with no context. Since the data is fictional, the LLM either refuses or hallucinates.

For the "without RAG" call we use `generate_text()` from our 7-vendor cascade.

```
from llm_cascade import get_cascade

_llm_raw = get_cascade(verbose=False)


def generate_text_no_rag(prompt, system_prompt=None):
    """Direct LLM call via llm_cascade (no RAG context)."""
    response = _llm_raw.generate(prompt, system_prompt=system_prompt)
    return response.text, response.provider


print("Direct LLM function ready (for without-RAG comparison).")
```

3.1.7.7 7) Run End-to-End Examples: With RAG vs. Without RAG

We test questions about **three fictional companies** that no LLM has seen in training. Notice how LlamaIndex **routes each question to the correct company's documents** — even though all three are in the same index. The source attribution shows which file answered each question.

- With RAG: correct, specific answers grounded on the right company's documents
- Without RAG: refusals, hedging, or hallucinated numbers

```
questions = [
    # Acme Analytics questions
    "What was Acme Analytics' revenue in Q4 2025?",
    #"How many days of PTO do new employees get at Acme?",
    # Globex Cybersecurity questions
    #"What is Globex Cybersecurity's flagship product and how fast does it
detect threats?",
    #"How did Globex help Meridian Health System during a ransomware attack?",
    # Nextera Green Solutions questions
    #"Who founded Nextera Green Solutions and where is it headquartered?",
    #"How much carbon emissions reduction did Nextera enable for its customers
in 2025?",
]


def preview(text, max_len=800):
    text = str(text) if text else ""
    return text[:max_len] + ("..." if len(text) > max_len else "")


if not has_llm_provider():
    print("Error: No LLM API key configured. Set at least one API key in Colab
Secrets.")
else:
    for i, q in enumerate(questions, start=1):
        print("=" * 80)
        print(f"Q{i}. {q}")
        print("=" * 80)

        # --- WITH RAG (LlamaIndex) ---
        print("--- WITH RAG (LlamaIndex) ---")
        try:
            response = query_engine.query(q)
            print("  Answer:")
            print(preview(response.response))
            sources = set()
            for node in response.source_nodes:
                sources.add(node.metadata.get("file_name", "unknown"))
            print(f"  Sources: {', '.join(sources)}")
        except Exception as e:
            print(f"  RAG error: {e}")

        # --- WITHOUT RAG ---
        print("--- WITHOUT RAG (LLM knowledge only) ---")
        try:
            answer_direct, prov_direct = generate_text_no_rag(q)
            print(f"  Answer (provider: {prov_direct}):")
            print(preview(answer_direct))
        except Exception as e:
            print(f"  Direct error: {e}")

        print()
```

3.1.7.7.1 Checkpoint: Reflection Questions

1. **Hallucination check:** Did the without-RAG answers invent specific revenue numbers or employee counts for a fictional company?
2. **Source attribution:** LlamaIndex tells us *which document* the answer came from. Why is this important for trust?
3. **Chunking:** How does the chunk size affect retrieval quality? What if a key fact spans two chunks?
4. **Framework vs. manual:** Compare this LlamaIndex approach to the manual RAG we built in the VectorDB and DBMS_RAG notebooks. What did the framework handle for us?

3.1.7.8 8) Interactive Query (Optional)

Now it is your turn to explore. Change the question in the cell below to anything you want to ask about the three fictional companies. Try questions that span multiple documents (e.g., "Compare Acme's revenue to Globex's products") or questions about specific details buried deep in a single document. Pay attention to the source attributions – they tell you which file LlamaIndex retrieved each answer from, which is a useful trust signal in any RAG system.

```
# Change this to any question about Acme Analytics
my_question = "What is Acme's LTV/CAC ratio and what does it mean?"

if has_llm_provider():
    print(f"Question: {my_question}\n")

    print("=" * 60)
    print("WITH RAG (LlamaIndex):")
    print("=" * 60)
    try:
        response = query_engine.query(my_question)
        print(f"\n{response.response}")
        sources = set()
        for node in response.source_nodes:
            sources.add(node.metadata.get('file_name', 'unknown'))
        print(f"\nSources: {', '.join(sources)}")
    except Exception as e:
        print(f"Error: {e}")

    print("\n" + "=" * 60)
    print("WITHOUT RAG:")
    print("=" * 60)
    try:
        answer_direct, prov = generate_text_no_rag(my_question)
        print(f"\n[{prov}] {answer_direct}")
    except Exception as e:
        print(f"Error: {e}")
else:
    print("Set at least one API key first.")
```

3.1.7.9 9) Bonus: Customizing the RAG Pipeline

LlamaIndex is highly configurable, and understanding the knobs you can turn is essential for moving from a demo to a production system. Here we show how to tweak three important parameters:

- **Chunk size** – smaller chunks yield more precise retrieval (the system finds the exact paragraph), while larger chunks give the LLM more surrounding context per chunk. The tradeoff is granularity vs. context.
- **Top-k** – how many chunks to retrieve. More chunks provide broader coverage but increase the prompt length (and cost). Too few chunks risk missing relevant information.
- **System prompt** – customize the LLM's behavior and persona for your specific use case.

The cell below rebuilds the index with a different chunk size and top-k, then runs the same question to show how these parameters affect the answer.

```
from llama_index.core.node_parser import SentenceSplitter

# Re-index with smaller chunks and more overlap
splitter = SentenceSplitter(chunk_size=256, chunk_overlap=50)
small_chunk_index = VectorStoreIndex.from_documents(
    all_documents,
    transformations=[splitter],
    show_progress=True,
)

# Query with more chunks retrieved
custom_engine = small_chunk_index.as_query_engine(similarity_top_k=5)

test_q = "What is the pricing for InsightBoard Pro?"
print(f"Question: {test_q}\n")

if has_llm_provider():
    try:
        response = custom_engine.query(test_q)
        print(f"Answer (small chunks, top_k=5):")
        print(response.response)
        print(f"\nChunks used: {len(response.source_nodes)}")
        for i, node in enumerate(response.source_nodes, 1):
            print(f"  #{i} score={node.score:.4f} |
{node.metadata.get('file_name', '?')}")
            print(f"     {node.text[:100].replace(chr(10), ' ')}...")
    except Exception as e:
        print(f"Error: {e}")
```

3.1.7.10 10) Teaching Notes and Exercises

Key takeaways:

- **LlamaIndex** abstracts the RAG pipeline: document loading, chunking, embedding, indexing, retrieval, and LLM prompting — all in a few lines of code.
- Under the hood, it does the **same thing** as our manual RAG notebooks (embed arrow.r retrieve arrow.r generate), but with less boilerplate.
- **Source attribution** (which document chunk answered the question) comes for free.
- The framework is **highly configurable**: chunk size, overlap, top-k, embedding model, LLM, and prompt templates can all be swapped.
- Using **synthetic/private data** makes the with-vs-without-RAG comparison very clear — the LLM literally cannot answer without retrieval.

LlamaIndex vs. Manual RAG — When to use which:

Aspect	Manual RAG	LlamaIndex
Learning	Better for understanding fundamentals	Hides implementation details
Speed to prototype	Slower	Much faster
Customization	Full control	Configurable, but within framework
Production readiness	Build it yourself	Batteries included

Exercises:

- Add more documents (e.g., a product roadmap, a competitor analysis) and see how retrieval adapts.
- Experiment with different chunk sizes (128, 512, 1024) and compare answer quality.

- Try using a different embedding model (e.g., `BAAI/bge-small-en-v1.5`) and compare retrieval accuracy.
- Add a **metadata filter** so queries only search specific document types (e.g., only financials).

3.2 LangChain

LangChain is a modular, open-source toolkit for building applications powered by large language models. Where LlamaIndex offers an opinionated, batteries-included approach that gets you from documents to answers in three lines of code, LangChain takes the opposite philosophy: it gives you a collection of composable components and lets you wire them together explicitly. You choose the document loader, the text splitter, the embedding model, the vector store, and the chain architecture. Nothing is hidden and nothing is assumed. This makes LangChain more verbose for simple use cases but far more flexible when your application outgrows a standard RAG template. If LlamaIndex is a point-and-shoot camera, LangChain is a camera with interchangeable lenses, manual focus, and full exposure control.

3.2.1 Core Components

LangChain organizes its functionality into a set of well-defined component categories. Document Loaders handle ingestion from various sources, much like LlamaIndex's data loaders. The `DirectoryLoader` and `TextLoader` classes read local files, while community-contributed loaders handle PDFs, web pages, databases, and cloud services. Text Splitters break documents into chunks, with `RecursiveCharacterTextSplitter` being the most commonly used option. It splits on natural boundaries (paragraphs, sentences, words) and supports configurable chunk sizes and overlap, giving you fine-grained control over the chunking strategy. Embeddings convert text into vectors using models from HuggingFace, OpenAI, or other providers. Vector Stores handle storage and similarity search, with LangChain supporting FAISS, ChromaDB, Pinecone, Weaviate, and many others through a unified interface. Finally, Chains and Runnables compose these components into end-to-end pipelines.

3.2.2 LCEL: The Pipe Syntax for Composing Chains

The modern way to build pipelines in LangChain is through LCEL, the LangChain Expression Language. LCEL uses Python's pipe operator (`|`) to chain processing steps together, creating a syntax that reads like a data flow diagram. A typical RAG chain in LCEL looks like this:

```
chain = retriever | format_docs | prompt | llm | parser
```

Reading left to right: the retriever fetches relevant document chunks, `format_docs` joins them into a single context string, the prompt template inserts that context alongside the user's question, the LLM generates a response, and the output parser extracts the final text. Each component in the pipe is a Runnable, an object with an `.invoke()` method that accepts input and returns output. The pipe operator connects the output of one Runnable to the input of the next, much like Unix pipes connect shell commands.

This syntax is more than cosmetic convenience. LCEL chains automatically support streaming (`.stream()` for token-by-token output), batch processing (`.batch()` for parallel execution across multiple inputs), async execution (`.ainvoke()` for non-blocking calls), and integration with LangSmith for tracing and debugging. A hand-written Python class that does the same RAG logic would need to implement all of these features manually.

3.2.3 RunnableLambda: Bridging Plain Functions into LCEL Pipes

There is one subtlety that trips up newcomers. LCEL's pipe operator only works between Runnables, objects that implement the `.invoke()` interface. A plain Python function does not have this interface, so writing `my_function | another_function` would fail with a `TypeError`. The solution is `RunnableLambda(func)`, which wraps any plain function into a Runnable. You only need to wrap the first function in a pipe explicitly; once the left side of a pipe is a Runnable, LangChain automatically coerces any plain functions on the right into Runnables. This small utility is what makes it possible to inject arbitrary custom logic into an LCEL pipeline without writing a full Runnable subclass.

3.2.4 Agents with Tools

RAG is just one pattern that LangChain supports. Its real power emerges with Agents, LLMs that can decide which functions to call based on the user's question. An agent is given a set of tools, plain Python functions with descriptions, and a system prompt that instructs it to reason about which tool to invoke. When the user asks "What was Acme's revenue last quarter?", the agent might call a `company_lookup` tool. When asked "What is 15% of 2.4 million?", it calls a `calculator` tool. When no tool is needed, it answers directly.

This decision-making process follows the ReAct pattern: Reason about the question, Act by calling a tool, Observe the result, and then either act again or produce a final answer. In the accompanying notebook, the agent loop is implemented as an LCEL pipe where each iteration sends the conversation to the LLM, parses the response for a tool-call JSON, executes the tool if one is requested, and appends the result to the conversation history. A Python loop drives the iteration because LCEL chains are inherently linear and cannot express "repeat until a condition is met." This limitation is precisely what motivates LangGraph, which you will encounter in the next chapter.

3.2.5 When to Use LangChain vs. LlamaIndex

The choice between the two frameworks is not about which is better in the abstract but about which fits your use case. LlamaIndex excels at document question-answering scenarios where you want a working prototype quickly and the default chunking, indexing, and retrieval behavior is good enough. Its high-level abstractions minimize boilerplate and let you focus on the data rather than the plumbing. LangChain excels when you need fine-grained control over the pipeline, when your application involves agents and tool use, when you want to compose non-standard workflows, or when you need to integrate with a broad ecosystem of vector stores, LLM providers, and community extensions. Many production teams use both: LlamaIndex for straightforward document ingestion and indexing, and LangChain for the orchestration and agent logic that sits on top.

3.2.6 The Hands-On Notebook

The notebook that follows this introduction builds a complete RAG pipeline with LangChain using the same three fictional companies (Acme Analytics, Globex Cybersecurity, and Nextera Green Solutions) that you encountered in the LlamaIndex chapter. You will load documents with `DirectoryLoader`, split them with `RecursiveCharacterTextSplitter`, embed and index them in a FAISS vector store, and build a retrieval chain using LCEL pipe syntax. The notebook then goes beyond RAG to demonstrate a tool-calling agent that can look up company information, perform calculations, and report the current date, all orchestrated through an LCEL-based ReAct loop. This progression from retrieval to agency illustrates why LangChain has become the default framework for LLM application development.

3.2.7 References

1. LangChain Documentation. "Introduction." LangChain, 2025. https://python.langchain.com/docs/introduction/
2. LangChain Documentation. "LangChain Expression Language (LCEL)." LangChain, 2025. https://python.langchain.com/docs/concepts/lcel/
3. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). "ReAct: Synergizing Reasoning and Acting in Language Models." *International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/2210.03629
4. Johnson, J., Douze, M., & Jegou, H. (2021). "Billion-Scale Similarity Search with GPUs." *IEEE Transactions on Big Data*, 7(3), 535-547. https://github.com/facebookresearch/faiss
5. Chase, H. "LangChain: Building Applications with LLMs through Composability." GitHub, 2025. https://github.com/langchain-ai/langchain

Open in Colab

3.2.8 LangChain RAG Demo: Document Q&A with a Framework

This notebook demonstrates how **LangChain** simplifies building RAG pipelines. LangChain is one of the most popular frameworks for building LLM-powered applications, providing modular components for document loading, text splitting, embeddings, vector stores, and retrieval chains.

How it works:

1. Load **synthetic company documents** from three separate directories (no external data needed).
2. Use LangChain to **split, embed, and store** the documents in a FAISS vector store. NOTE that FAISS is not a vector database. It is an open-source library for efficient, high-speed similarity search and clustering of dense vectors. Unlike ChromaDB or Pinecone, which are managed or structured vector databases, FAISS is a low-level, high-performance library that stores vectors primarily in-memory.
3. Build a **retrieval chain** that fetches relevant chunks and sends them to an LLM.
4. Compare answers **with RAG** (LangChain pipeline) vs. **without RAG** (direct LLM call).

Learning goals:

- Understand LangChain's modular architecture: Loaders arrow.r Splitters arrow.r Embeddings arrow.r Vector Stores arrow.r Chains
- See how LangChain compares to LlamaIndex for building RAG pipelines
- Observe how RAG grounding eliminates hallucination on private/synthetic data

Provider setup: This notebook uses the `llm_cascade` package, which auto-detects your API keys and falls back to the next provider if one is unavailable. Supported providers: OpenAI, Gemini, Ollama, Grok (xAI), Groq, HuggingFace, Cohere, OpenRouter.

Store any of these API keys in Colab Secrets (or a local `.env` file): `OPENAI_API_KEY`, `GEMINI_API_KEY`, `OLLAMA_API_KEY`, `XAI_API_KEY`, `GROQ_API_KEY`, `HF_TOKEN`, `COHERE_API_KEY`, `OPENROUTER_API_KEY`

```
!pip install -q -U langchain langgraph langchain-community langchain-
huggingface langchain-text-splitters faiss-cpu google-genai openai langchain-
google-genai git+https://github.com/KarAnalytics/llm_cascade.git sentence-
transformers
```

3.2.8.1 Imports and Provider Helpers (7-Vendor Cascade)

We configure the same 7-provider cascade used across all our RAG notebooks. LangChain can work with any OpenAI-compatible API, so our cascade integrates naturally – we point LangChain's `ChatOpenAI` class at whichever provider is available, using its OpenAI-compatible endpoint. This means the same LangChain code works whether you are running against OpenAI, Gemini, Groq, or any other provider in the cascade.

```
from pathlib import Path
from llm_cascade.providers import PROVIDERS, _load_env, _get_key,
_is_retriable_error

_load_env()

def get_available_providers():
    return [p for p in PROVIDERS if _get_key(p['key_env'])]
```

```
def has_llm_provider():
    return len(get_available_providers()) > 0

# Print status
available = get_available_providers()
if available:
    print('Providers configured (in fallback order):')
    for p in available:
        print(f"  + {p['name']:<16} model = {p['default_model']}")
else:
    print('WARNING: No API keys found.')
```

3.2.8.2 Configure LangChain LLM and Embeddings

LangChain needs two components before we can build a retrieval chain:

- An **LLM** for generating answers (we use `ChatOpenAI` pointed at the first available provider from our cascade)
- An **embedding model** for converting text to vectors (local HuggingFace model – no API key needed)

One thing to notice here: LangChain uses `ChatOpenAI` as its LLM wrapper even when talking to non-OpenAI providers. This works because most LLM providers now expose OpenAI-compatible API endpoints. We simply swap the `base_url` and `api_key` to point at Gemini, Groq, or whatever provider is available. The rest of LangChain's pipeline does not need to know or care which model is behind the API.

```
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_community.chat_models import ChatOpenAI

# ---- Embedding model (local, no API key needed)
-----------------------------
embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/all-
MiniLM-L6-v2")
print("Embedding model: sentence-transformers/all-MiniLM-L6-v2 (local)")

# ---- LLM with runtime fallback (cascades if quota is exhausted)
--------------
llm_candidates = []
for provider in get_available_providers():
    api_key = _get_key(provider["key_env"])
    try:
        if provider["style"] == "gemini":
            from langchain_google_genai import ChatGoogleGenerativeAI
            candidate = ChatGoogleGenerativeAI(
                google_api_key=api_key,
                model=provider["default_model"],
            )
        else:
            candidate = ChatOpenAI(
                api_key=api_key,
                base_url=provider["base_url"],
                model=provider["default_model"],
            )
        llm_candidates.append((provider, candidate))
        print(f"  + {provider['name']:<16} ({provider['default_model']})")
    except Exception as e:
        print(f"  - {provider['name']:<16} skipped: {e}")

if llm_candidates:
    primary_provider, primary_llm = llm_candidates[0]
    fallback_llms = [c for _, c in llm_candidates[1:]]

    if fallback_llms:
        llm = primary_llm.with_fallbacks(fallback_llms)
        print(f"\nPrimary LLM: {primary_provider['name']}
```

```
({primary_provider['default_model']})")
        print(f"Fallbacks:   {', '.join(p['name'] for p, _ in
llm_candidates[1:])}")
        print("(If primary hits quota, automatically tries the next
provider)")
    else:
        llm = primary_llm
        print(f"\nLLM provider: {primary_provider['name']}
({primary_provider['default_model']})")
        print("(No fallback providers configured)")

    llm_provider = primary_provider
else:
    llm = None
    primary_llm = None
    llm_provider = None
    print("ERROR: Could not configure any LLM provider. Check your API keys.")
```

3.2.8.3 Create Synthetic Company Documents

We generate **fictional documents for three companies** that the LLM has never seen in training. This ensures that correct answers can **only** come from RAG retrieval, not from parametric memory. Each company's documents live in a **separate directory**, simulating real-world data silos.

Directory	Company	Documents
`acme_docs/`	Acme Analytics Inc. (BI tools, Lawrence KS)	Overview, Products, Financials, Handbook, Case Study
`globex_docs/`	Globex Cybersecurity Corp. (cybersecurity, Boston)	Overview, Products, Incident Report
`nextera_docs/`	Nextera Green Solutions (sustainability consulting, Denver)	Overview, Services, Impact Report

Later, we will use LlamaIndex **Data Loaders** (`SimpleDirectoryReader`) to ingest all three directories — demonstrating how LlamaIndex can pull information from multiple, separate data sources and unify them in a single searchable index.

```
DOCUMENTS = {
    "company_overview.txt": """ACME ANALYTICS INC. — COMPANY OVERVIEW

Acme Analytics Inc. was founded in 2019 by Dr. Sarah Chen and Marcus Rivera
in Lawrence, Kansas. The company specializes in AI-powered business
intelligence
tools for mid-market companies (100–5,000 employees).

Headquarters: 1420 Jayhawk Boulevard, Lawrence, KS 66045
Employees: 287 (as of January 2026)
Annual Revenue (2025): $42.3 million
Funding: Series B ($18M raised in March 2023 from Midwest Ventures)

Mission: "To democratize data analytics so every business decision is informed
by evidence, not intuition."

The company operates three offices: Lawrence (HQ), Chicago (sales), and
Austin (engineering). CEO: Dr. Sarah Chen. CTO: Marcus Rivera. CFO: Linda
Park.""",

    "products_and_services.txt": """ACME ANALYTICS — PRODUCTS AND SERVICES

1. InsightBoard Pro (Flagship Product)
   - Real-time dashboard platform with natural language query interface
   - Pricing: $49/user/month (Standard), $89/user/month (Enterprise)
   - Supports PostgreSQL, MySQL, Snowflake, BigQuery, and Redshift
```

```
   - 1,847 active enterprise customers as of Q4 2025

2. DataPipe ETL
   - Automated data pipeline builder with 200+ pre-built connectors
   - Pricing: starts at $500/month for up to 10 million records/day
   - Launched in September 2024

3. PredictIQ
   - ML-powered forecasting add-on for InsightBoard Pro
   - Pricing: $29/user/month (requires InsightBoard Pro subscription)
   - Uses proprietary time-series model trained on retail and logistics data
   - Beta launched March 2025, GA release planned for June 2026

All products include 24/7 email support. Enterprise plans include dedicated
account manager and 99.9% SLA.""",

    "q4_2025_financials.txt": """ACME ANALYTICS — Q4 2025 FINANCIAL RESULTS
(CONFIDENTIAL)

Period: October 1 — December 31, 2025

Revenue:          $12.8 million (Q4) / $42.3 million (FY 2025)
Gross Margin:     78.2%
Operating Profit: $1.9 million (Q4) / $5.1 million (FY 2025)
Net Income:       $1.4 million (Q4)
Cash on Hand:     $14.7 million
Burn Rate:        Company is cash-flow positive since Q2 2025

Key Metrics:
- ARR (Annual Recurring Revenue): $48.6 million (up 34% YoY)
- Net Revenue Retention: 118%
- Customer Acquisition Cost (CAC): $8,200
- Customer Lifetime Value (LTV): $67,400
- LTV/CAC Ratio: 8.2x

Headcount grew from 241 to 287 employees during 2025.
R&D spending: 31% of revenue. Sales & Marketing: 28% of revenue.""",

    "employee_handbook_excerpt.txt": """ACME ANALYTICS — EMPLOYEE HANDBOOK
(EXCERPT)

PAID TIME OFF (PTO):
- All full-time employees receive 22 days of PTO per year (accrued monthly).
- PTO increases to 27 days after 3 years of service.
- Unused PTO can be carried over (max 5 days) or paid out at year-end.
- Sick leave: 10 days per year (separate from PTO).

REMOTE WORK POLICY:
- Engineering and Data Science teams: fully remote eligible.
- Sales and Customer Success: hybrid (minimum 2 days/week in office).
- All employees may work remotely up to 4 weeks/year from any US location.
- International remote work requires VP approval and tax review.

PROFESSIONAL DEVELOPMENT:
- Annual learning budget: $2,500 per employee.
- Conference attendance: up to 2 conferences per year with manager approval.
- Tuition reimbursement: up to $5,250/year for degree programs.""",

    "customer_case_study.txt": """CUSTOMER CASE STUDY: BIGRETAIL CORP

Company: BigRetail Corp (1,200 retail stores across 38 states)
Challenge: Siloed data across POS, inventory, and CRM systems made it
impossible for regional managers to get timely insights.

Solution: Deployed InsightBoard Pro Enterprise + DataPipe ETL
- Connected 14 data sources in 3 weeks using DataPipe
```

```
- 340 regional managers now use InsightBoard daily
- Natural language queries replaced manual SQL report requests

Results (after 6 months):
- 62% reduction in time-to-insight (from 4 days to 1.5 days average)
- $3.2 million saved in inventory carrying costs
- 23% increase in regional manager satisfaction scores
- SQL report request backlog eliminated entirely

Quote from BigRetail CIO Janet Torres:
\"InsightBoard Pro transformed how our managers interact with data.
They went from waiting days for a report to asking questions in plain
English and getting answers in seconds.\""""
}

# Write documents to disk
DOC_DIR = Path("acme_docs")
DOC_DIR.mkdir(exist_ok=True)

for filename, content in DOCUMENTS.items():
    (DOC_DIR / filename).write_text(content, encoding="utf-8")

print(f"Created {len(DOCUMENTS)} documents in '{DOC_DIR}/':")
for f in sorted(DOC_DIR.iterdir()):
    print(f"  {f.name} ({f.stat().st_size} bytes)")

# --- Company 2: Globex Cybersecurity Corp. → globex_docs/ ---

GLOBEX_DOCUMENTS = {
    "globex_overview.txt": """GLOBEX CYBERSECURITY CORP. — COMPANY OVERVIEW

Globex Cybersecurity Corp. was founded in 2017 by former NSA analyst
James Whitfield and AI researcher Dr. Priya Narayanan in Boston,
Massachusetts.
The company provides AI-driven cybersecurity solutions for financial services
and healthcare organizations.

Headquarters: 88 Federal Street, Suite 1200, Boston, MA 02110
Employees: 412 (as of January 2026)
Annual Revenue (2025): $78.5 million
Funding: Series C ($45M raised in June 2024 from CyberVentures Capital
and Goldman Sachs Growth Equity)

Mission: "To make enterprise-grade cybersecurity autonomous, adaptive,
and accessible to every regulated industry."

The company operates four offices: Boston (HQ), Washington D.C. (government
relations), Tel Aviv (R&D), and Singapore (APAC sales).
CEO: James Whitfield. CTO: Dr. Priya Narayanan. CISO: Robert Tanaka.""",

    "globex_products.txt": """GLOBEX CYBERSECURITY — PRODUCTS AND SERVICES

1. ShieldAI Platform (Flagship Product)
   - AI-powered threat detection and automated incident response
   - Uses behavioral analysis to detect zero-day attacks in real time
   - Pricing: $120/endpoint/year (Standard), $195/endpoint/year (Premium)
   - 623 enterprise customers across 18 countries as of Q4 2025
   - Average detection time: 2.3 minutes (industry average: 197 days)

2. VaultGuard
   - Data encryption and access management platform for regulated industries
   - HIPAA, SOX, and PCI-DSS compliant out of the box
   - Pricing: $35,000/year (up to 500 users), $60,000/year (unlimited users)
   - Launched January 2023

3. ThreatIntel Live
```

```
   - Real-time threat intelligence feed with dark web monitoring
   - Pricing: $8,500/month (includes API access and analyst dashboard)
   - Aggregates data from 2,400+ sources updated every 15 minutes
   - Integrates with Splunk, CrowdStrike, Palo Alto, and Microsoft Sentinel

All products include a dedicated Security Operations Center (SOC) team for
Premium customers. Globex holds ISO 27001 and SOC 2 Type II
certifications.""",

    "globex_incident_report.txt": """GLOBEX CYBERSECURITY — 2025 THREAT
LANDSCAPE REPORT (PUBLIC)

Published: February 2026

KEY FINDINGS FROM GLOBEX'S CUSTOMER BASE (2025):

Total incidents analyzed: 14,237 across 623 customers
- Ransomware attacks: up 42% YoY (3,891 incidents)
- Phishing campaigns: up 28% YoY (5,102 incidents)
- Supply chain attacks: up 67% YoY (1,245 incidents — fastest-growing
category)
- Insider threats: relatively stable at 1,834 incidents

ShieldAI Platform Performance:
- Blocked 97.3% of attacks before any data exfiltration
- Mean time to detect (MTTD): 2.3 minutes
- Mean time to respond (MTTR): 8.7 minutes (automated response)
- False positive rate: 0.4% (down from 1.2% in 2024)

NOTABLE CASE: MERIDIAN HEALTH SYSTEM
- 340-bed hospital network targeted by Medusa ransomware variant
- ShieldAI detected anomalous lateral movement within 47 seconds
- Automated containment isolated 3 affected servers in under 2 minutes
- Zero patient data exfiltrated; zero downtime for clinical systems
- Estimated savings: $12M+ (average healthcare ransomware cost per Ponemon)

INDUSTRY PREDICTIONS FOR 2026:
- AI-generated phishing emails will become indistinguishable from human-
written
- Quantum-resistant encryption adoption will begin in financial services
- Average cost of a data breach expected to exceed $5.2 million globally""",
}

GLOBEX_DIR = Path("globex_docs")
GLOBEX_DIR.mkdir(exist_ok=True)

for filename, content in GLOBEX_DOCUMENTS.items():
    (GLOBEX_DIR / filename).write_text(content, encoding="utf-8")

print(f"Created {len(GLOBEX_DOCUMENTS)} documents in '{GLOBEX_DIR}/':")
for f in sorted(GLOBEX_DIR.iterdir()):
    print(f"  {f.name} ({f.stat().st_size} bytes)")

# --- Company 3: Nextera Green Solutions → nextera_docs/ ---

NEXTERA_DOCUMENTS = {
    "nextera_overview.txt": """NEXTERA GREEN SOLUTIONS — COMPANY OVERVIEW

Nextera Green Solutions was founded in 2020 by climate scientist Dr. Elena
Vasquez and former McKinsey partner David Okafor in Denver, Colorado.
The company provides sustainability consulting and carbon accounting software
to help companies meet ESG (Environmental, Social, Governance) targets.

Headquarters: 1700 Lincoln Street, Suite 3400, Denver, CO 80203
Employees: 156 (as of January 2026)
Annual Revenue (2025): $23.8 million
```

```
Funding: Series A ($12M raised in November 2023 from Climate Capital Partners
and Patagonia Ventures)

Mission: "To give every company a clear, data-driven path to net zero —
turning climate commitments into measurable action."

The company operates two offices: Denver (HQ) and London (European
operations).
CEO: Dr. Elena Vasquez. COO: David Okafor. VP Engineering: Kenji Tanaka.
Nextera is a certified B Corporation and a member of the UN Global
Compact.""",

    "nextera_services.txt": """NEXTERA GREEN SOLUTIONS — SERVICES AND PLATFORM

1. CarbonLens Platform (Flagship Product)
   - Automated Scope 1, 2, and 3 carbon emissions tracking
   - Integrates with ERP systems (SAP, Oracle, NetSuite) to pull real data
   - AI-powered recommendations for emission reduction strategies
   - Pricing: $2,500/month (Standard, up to $500M revenue companies)
              $6,500/month (Enterprise, unlimited + custom reporting)
   - 312 active customers as of Q4 2025

2. ESG Reporting Suite
   - Automated generation of sustainability reports (GRI, SASB, TCFD, CDP)
   - Audit-ready documentation with full data lineage
   - Pricing: $18,000/year (bundled with CarbonLens Enterprise)
   - Standalone: $24,000/year
   - Launched April 2024

3. Sustainability Strategy Consulting
   - Science-Based Targets initiative (SBTi) alignment services
   - Supply chain decarbonization roadmaps
   - Pricing: Project-based ($75,000 — $250,000 per engagement)
   - Team of 28 consultants with backgrounds in climate science and industry

4. Carbon Offset Marketplace
   - Verified carbon credits from 45 projects across 12 countries
   - Blockchain-verified chain of custody for each credit
   - Transaction fee: 4.5% per credit purchase
   - Beta launched August 2025

All platform customers receive quarterly sustainability benchmarking reports
comparing their performance against industry peers.""",

    "nextera_impact_report.txt": """NEXTERA GREEN SOLUTIONS — 2025 IMPACT
REPORT

Published: January 2026

COMPANY IMPACT METRICS:
- Total customer carbon emissions tracked: 47.3 million metric tons CO2e
- Verified emission reductions enabled: 6.8 million metric tons CO2e (14.4%)
- Customers who improved their CDP score after using Nextera: 89%
- Average time to produce ESG report reduced from 14 weeks to 3.5 weeks

NOTABLE CLIENT RESULTS:

1. SummitBrew Coffee (specialty coffee chain, 380 locations)
   - Reduced Scope 3 supply chain emissions by 31% in 18 months
   - Switched 62% of logistics to electric vehicles using Nextera roadmap
   - First coffee chain to achieve SBTi-validated near-term target

2. Pacific Northwest Lumber Co. (timber and building materials)
   - Used CarbonLens to discover 40% of emissions came from transportation
   - Optimized shipping routes: saved 8,200 metric tons CO2e and $1.4M/year
   - Achieved carbon-neutral certification for 2025 operations
```

```
3. Meridian Financial Group (regional bank, $8B assets)
   - Implemented financed emissions tracking (PCAF methodology)
   - Identified $2.1B in loan portfolio exposed to high transition risk
   - Developed green lending program: $340M in sustainability-linked loans

AWARDS AND RECOGNITION (2025):
- Fast Company "Most Innovative Companies" — Energy category (#7)
- B Corp "Best for the World" — Environment category
- Colorado Governor's Award for Clean Energy Innovation""",
}

NEXTERA_DIR = Path("nextera_docs")
NEXTERA_DIR.mkdir(exist_ok=True)

for filename, content in NEXTERA_DOCUMENTS.items():
    (NEXTERA_DIR / filename).write_text(content, encoding="utf-8")

print(f"Created {len(NEXTERA_DOCUMENTS)} documents in '{NEXTERA_DIR}/':")
for f in sorted(NEXTERA_DIR.iterdir()):
    print(f"  {f.name} ({f.stat().st_size} bytes)")
```

3.2.8.4 Load Documents from Three Company Directories

LangChain provides **Document Loaders** – modular components that load data from various sources. Here we use `DirectoryLoader` to load `.txt` files from three separate company directories, each representing a different fictional company. This is the LangChain equivalent of LlamaIndex's `SimpleDirectoryReader`, but notice how much more explicit it is: you specify the loader class (`TextLoader`), the file glob pattern, and the directory path separately. LangChain favors this explicit, composable style over LlamaIndex's "batteries included" approach.

```
from langchain_community.document_loaders import DirectoryLoader, TextLoader

# Load documents from three separate company directories
DOC_DIRS = {
    "acme_docs":    "Acme Analytics (BI tools)",
    "globex_docs":  "Globex Cybersecurity (security)",
    "nextera_docs": "Nextera Green Solutions (sustainability)",
}

all_docs = []
for dir_name, label in DOC_DIRS.items():
    loader = DirectoryLoader(dir_name, glob="**/*.txt", loader_cls=TextLoader,
                             loader_kwargs={"encoding": "utf-8"})
    docs = loader.load()
    print(f"  Loaded: '{dir_name}/' \u2192 {len(docs)} docs  [{label}]")
    all_docs.extend(docs)

print(f"\nTotal documents loaded: {len(all_docs)}")
```

3.2.8.5 Split, Embed, and Index with FAISS

This is where LangChain's modular architecture becomes visible. Each step in the pipeline is an explicit, swappable component:

- **Text Splitter** (`RecursiveCharacterTextSplitter`) – breaks documents into smaller chunks, trying to split at paragraph boundaries before falling back to sentence and character boundaries
- **Embeddings** (`HuggingFaceEmbeddings`) – converts each chunk into a dense vector
- **Vector Store** (FAISS) – stores and indexes the vectors for fast similarity search

This is the same embed-then-store-then-retrieve pattern as LlamaIndex, but here each step is a separate object you can inspect, swap, or configure independently. Want to switch from

FAISS to ChromaDB? Just change one line. Want a different chunking strategy? Swap the splitter. That modularity is LangChain's defining philosophy.

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.vectorstores import FAISS

# Step 1: Split documents into chunks
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,
    chunk_overlap=200,
    separators=["\n---\n", "\n\n", "\n", " "],  # split on section dividers
first
)
chunks = text_splitter.split_documents(all_docs)
print(f"Split {len(all_docs)} documents into {len(chunks)} chunks")

# Step 2 & 3: Embed chunks and store in FAISS vector store
vectorstore = FAISS.from_documents(chunks, embeddings)
print(f"FAISS vector store built with {len(chunks)} vectors")

# Create a retriever (top-k=3)
retriever = vectorstore.as_retriever(search_kwargs={"k": 3})
print("Retriever ready (top_k=3)")
```

3.2.8.6 Peek Under the Hood – Test Retrieval

Before querying the LLM, let us see what chunks the retriever returns for a test question. This is exactly the same kind of inspection we did with LlamaIndex – verifying that the right documents are being retrieved before we trust the LLM to generate an answer from them. Understanding what the retriever returns is the single most important debugging step in any RAG pipeline, because if retrieval is wrong, no amount of LLM sophistication can save the answer.

```
# Test retrieval without LLM — just see what chunks come back
test_query = "How much revenue did Acme make in Q4 2025?"
retrieved_docs = retriever.invoke(test_query)

print(f"Query: '{test_query}'")
print(f"Retrieved {len(retrieved_docs)} chunks:\n")
for i, doc in enumerate(retrieved_docs, 1):
    source = doc.metadata.get('source', 'unknown')
    text_preview = doc.page_content[:200].replace('\n', ' ')
    print(f"  Chunk #{i} (source={source})")
    print(f"    {text_preview}...\n")
```

3.2.8.7 Build a Retrieval Chain (LCEL)

LangChain's modern approach uses **LCEL (LangChain Expression Language)** to compose chains with the `|` pipe operator. If you have used Unix shell pipes (`cat file | grep pattern | sort`), the mental model is identical: data flows left to right through a series of transformations.

Here is how to read the chain we build below:

1. The **retriever** fetches relevant chunks from the FAISS vector store
2. `format_docs` concatenates those chunks into a single text block
3. The **prompt template** combines the chunks + user question into a structured prompt
4. The **LLM** generates an answer from that prompt
5. `StrOutputParser()` extracts the text from the LLM's response object

Each `|` connects one step's output to the next step's input. This is more flexible and composable than the older `RetrievalQA` chain – you can insert, remove, or rearrange steps just by editing the pipe expression.

We also set up a direct LLM call function for the "without RAG" comparison.

```python
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnablePassthrough
from langchain_core.output_parsers import StrOutputParser


def format_docs(docs):
    """Join retrieved documents into a single context string."""
    return (chr(10) * 2 + "---" + chr(10) * 2).join(doc.page_content for doc
in docs)


# The prompt template is used by the RAG chain below
rag_prompt = ChatPromptTemplate.from_template(
    """Answer the question based ONLY on the following context.
If the context doesn't contain the answer, say "I don't have enough
information."

Context:
{context}

Question: {question}

Answer:"""
)


#
==============================================================================
# RAG Chain with LCEL (LangChain Expression Language) -- the modern way
#
==============================================================================
# LCEL uses the pipe operator `|` to chain Runnables together, similar to
# Unix pipes (`cat file.txt | grep foo | wc -l`).
#
# HOW TO READ THE PIPE SYNTAX:
#
#   result = input | step1 | step2 | step3
#
# means: input goes into step1, step1's output goes into step2, etc.
# Each `|` is "then pass to".
#
# Reading top-to-bottom when you call chain.invoke("your question"):
#
#   1. The DICT at the top runs TWO things in parallel with the same input:
#      - "context":  retriever | format_docs
#                    (fetches docs, then joins them into a string)
#      - "question": RunnablePassthrough()
#                    (just passes the question through unchanged)
#      Output: {"context": "...retrieved text...", "question": "your
question"}
#
#   2. | rag_prompt
#      Fills the {context} and {question} placeholders in the prompt template.
#
#   3. | llm
#      Sends the filled prompt to the LLM. Returns an AIMessage object.
#
#   4. | StrOutputParser()
#      Extracts .content from the AIMessage so you get a plain string back.

chain = (
    {"context": retriever | format_docs, "question": RunnablePassthrough()}
    | rag_prompt
    | llm
    | StrOutputParser()
)
```

```
print("LCEL RAG chain ready: retriever | format_docs | prompt | llm | parser")


#
==============================================================================
# ALTERNATIVE: Custom class WITHOUT LangChain (shown for comparison)
#
==============================================================================
# Here's what the same RAG pipeline looks like as a plain Python class,
# without using LCEL or LangChain's chaining features. It works, but...
#
# WHY LCEL IS USEFUL COMPARED TO THIS CUSTOM CLASS:
#   - Streaming tokens:   for token in chain.stream(q): ...   (works
automatically in LCEL)
#                         The custom class would need to be rewritten to
handle streaming.
#   - Batch processing:   chain.batch([q1, q2, q3])            (parallel
execution automatic)
#                         The custom class would need threading/async code
added manually.
#   - Automatic fallback: chain.with_fallbacks([backup])       (one line)
#                         The custom class would need try/except blocks
everywhere.
#   - Tracing/debugging:  works with LangSmith out of the box
#                         The custom class would need manual logging at each
step.
#   - Async support:      chain.ainvoke(q)                     (free)
#                         The custom class would need an async version written
separately.
#
# Uncomment the block below to use the custom class approach instead of LCEL.
#
# class SimpleRAGChain:
#     def __init__(self, retriever, prompt, llm):
#         self.retriever = retriever
#         self.prompt = prompt
#         self.llm = llm
#
#     def invoke(self, question):
#         # Step 1: retrieve relevant docs from vector store
#         docs = self.retriever.invoke(question)
#         # Step 2: join docs into a single context string
#         context = format_docs(docs)
#         # Step 3: fill the prompt template
#         formatted = self.prompt.format(context=context, question=question)
#         # Step 4: send to the LLM
#         response = self.llm.invoke(formatted)
#         # Step 5: extract the text from the AIMessage object
#         return response.content
#
# chain = SimpleRAGChain(retriever, rag_prompt, llm)
```

3.2.8.8 With RAG vs. Without RAG – Three Companies

We test questions about **three fictional companies** that no LLM has seen in training. Notice how LangChain retrieves from the correct company's documents automatically – even though all three are in the same FAISS index. The vector similarity naturally routes each question to the right documents without any explicit routing logic. This is one of the elegant properties of embedding-based retrieval: semantic similarity does the routing for you.

```
questions = [
    "What was Acme Analytics' revenue in Q4 2025?",
    #"How many days of PTO do new employees get at Acme?",
    #"What is Globex Cybersecurity's flagship product and how fast does it
detect threats?",
```

```
    #"How did Globex help Meridian Health System during a ransomware attack?",
    #"Who founded Nextera Green Solutions and where is it headquartered?",
    #"How much carbon emissions reduction did Nextera enable for its customers
in 2025?",
]


def preview(text, max_len=800):
    text = str(text) if text else ""
    return text[:max_len] + ("..." if len(text) > max_len else "")


if not has_llm_provider():
    print("Error: No LLM API key configured. Set at least one API key in Colab
Secrets.")
else:
    for i, q in enumerate(questions, start=1):
        print("=" * 80)
        print(f"Q{i}. {q}")
        print("=" * 80)

        try:
            answer = chain.invoke(q)
            print("Answer:")
            print(preview(answer))
            # Show which documents were retrieved
            source_docs = retriever.invoke(q)
            sources = set(doc.metadata.get("source", "unknown") for doc in
source_docs)
            print(f"Sources: {', '.join(sources)}")
        except Exception as e:
            if _is_retriable_error(e):
                print("  Rate limited -- skipping")
            else:
                print(f"  Error: {e}")

        print()
```

3.2.8.9 LangChain vs. LlamaIndex – Comparison

Aspect	LangChain	LlamaIndex
Philosophy	Modular toolkit – you compose the pipeline	Opinionated framework – batteries included
Document loading	`DirectoryLoader`, `TextLoader`, etc.	`SimpleDirectoryReader`
Text splitting	Explicit `RecursiveCharacterTextSplitter`	Automatic (configurable)
Vector store	FAISS, Chroma, Pinecone, etc. (you choose)	Built-in or pluggable
Retrieval chain	LCEL pipe syntax (`retriever | prompt | llm`)	`query_engine`
Lines of code	More (explicit control)	Fewer (more abstraction)
Best for	Custom pipelines, agents, complex workflows	Quick RAG prototypes, document Q&A

Key takeaway: Both frameworks solve the same problem (RAG), but LangChain gives you more explicit control over each step, while LlamaIndex abstracts more away. Choose based on your needs.

With the RAG pipeline working, let us now see what happens when we give the LLM more autonomy. Instead of a fixed retrieve-then-generate pipeline, what if the model could choose which tools to use based on the question? This is the jump from chains to agents – and it is where LangChain's architecture really starts to pay off.

3.2.8.10 Beyond RAG: LangChain Agents with Tools

RAG is just one use case for LangChain. Its real power lies in **Agents** – LLMs that can **decide which tools to call** based on the user's question. While a RAG chain always follows the same retrieve-then-generate path, an agent reasons about which action to take next.

In this section, we give the LLM three tools:

- **Company Lookup** – searches our vector store (reusing the RAG retriever from above)
- **Calculator** – performs arithmetic
- **Current Date** – returns today's date

The LLM reads the question, decides which tool to call by outputting a JSON action, executes it, reads the result, and generates a final answer. This is the **ReAct** pattern (Reason + Act), implemented with a simple prompt – no `bind_tools` or special API features required. It works with **any LLM provider**, which is why we chose this approach over provider-specific tool-calling APIs.

As you read the code below, notice the contrast with the RAG chain: the RAG chain is a fixed pipeline, while the agent is a **decision loop** where the LLM chooses its own path through the available tools.

```
# --- Tool definitions (plain functions) ---

def company_lookup(query):
    """Search the company knowledge base."""
    docs = retriever.invoke(query)
    return chr(10).join(doc.page_content[:500] for doc in docs)


def calculator(expression):
    """Evaluate a math expression like 42.3 * 1.34"""
    try:
        allowed = set('0123456789.+-*/() ')
        if not all(c in allowed for c in expression):
            return 'Error: Only basic arithmetic allowed.'
        return str(round(eval(expression), 4))
    except Exception as e:
        return f'Error: {e}'


def current_date(dummy=''):
    """Returns today's date."""
    from datetime import date
    return date.today().strftime('%B %d, %Y')


TOOLS = {
    'company_lookup': company_lookup,
    'calculator': calculator,
    'current_date': current_date,
}

print('Defined tools:', list(TOOLS.keys()))
```

3.2.8.11 Create and Run the Agent

We implement a **ReAct agent loop** using LCEL pipes, just like the RAG chain.

The key insight: an agent is a **loop** around a single step. We express ONE step of the ReAct loop as an LCEL chain:

```
agent_step = invoke_llm | parse_tool_call | execute_tool
```

Each step:

- **`invoke_llm`** — sends the current message history to the LLM and captures the reply
- **`parse_tool_call`** — checks if the reply contains a JSON tool call
- **`execute_tool`** — if there's a tool call, runs the tool and appends the result to the message history; otherwise returns the final answer

Then we call `agent_step.invoke(state)` in a Python `for` loop until the agent returns a final answer (or we hit a step limit). The Python loop is needed because LCEL chains are linear by design — they can't natively express "repeat until condition". This is exactly the kind of situation where **LangGraph** shines (see the LangGraph_demo notebook), since it's built specifically for stateful graph-based workflows with branches and loops.

3.2.8.11.1 Why do we need `RunnableLambda`?

LCEL's pipe operator `|` only works between **Runnables** — objects that have an `.invoke()` method. Plain Python functions don't have a `|` operator defined, so `my_func1 | my_func2` would fail with a `TypeError`.

`RunnableLambda(func)` wraps a plain function into a Runnable, giving it an `.invoke()` method and enabling the pipe operator. **You only need to wrap ONE function** to bootstrap the pipe — once the left side is a Runnable, LangChain auto-coerces any plain functions on the right into Runnables.

That's why our chain looks like:

```
agent_step = RunnableLambda(invoke_llm) | parse_tool_call | execute_tool
```

Only the first function needs explicit wrapping. The same thing happens in the earlier RAG chain: `retriever | format_docs` works without wrapping `format_docs`, because `retriever` is already a Runnable (LangChain auto-wraps `format_docs`).

```
import json as _json
import re as _re
from langchain_core.messages import HumanMessage, AIMessage, SystemMessage
from langchain_core.runnables import RunnableLambda

AGENT_SYSTEM = """You are a helpful business analyst. You have these tools:
company_lookup(query): Search knowledge base about Acme, Globex, or Nextera.
calculator(expression): Evaluate math like 42.3 * 1.34
current_date(): Get todays date.

To use a tool, reply with ONLY this JSON (nothing else):
{"tool": "tool_name", "input": "the input"}

After receiving a tool result, answer the original question.
If you can answer without a tool, just reply directly."""


# ---- Three small functions that become LCEL steps via RunnableLambda ----

def invoke_llm(state):
    """LCEL step 1: send messages to LLM, capture the reply."""
    response = llm.invoke(state["messages"])
    return {**state, "reply": response.content.strip()}


def parse_tool_call(state):
    """LCEL step 2: check if reply contains a tool call JSON."""
    tool_call = None
    m = _re.search(r"\{[^}]+\}", state["reply"])
    if m:
        try:
```

```
            parsed = _json.loads(m.group())
            if "tool" in parsed and "input" in parsed:
                tool_call = parsed
        except _json.JSONDecodeError:
            pass
    return {**state, "tool_call": tool_call}


def execute_tool(state):
    """LCEL step 3: run the tool (or mark as final answer if none)."""
    if state["tool_call"] is None:
        # No tool call -- this is the final answer
        return {**state, "final": state["reply"]}

    # Execute the requested tool
    tool_name = state["tool_call"]["tool"]
    tool_input = state["tool_call"]["input"]
    tool_fn = TOOLS.get(tool_name)
    if tool_fn:
        result = tool_fn(tool_input)
    else:
        result = f"Unknown tool: {tool_name}"

    # Append assistant reply and tool result to the message history
    new_messages = state["messages"] + [
        AIMessage(content=state["reply"]),
        HumanMessage(content=f"Tool result: {result}"),
    ]
    return {
        **state,
        "messages": new_messages,
        "final": None,
        "last_tool_name": tool_name,
        "last_tool_input": tool_input,
        "last_tool_result": result,
    }


# ---- Build the LCEL agent-step chain: invoke_llm | parse_tool_call |
execute_tool ----
# LCEL's pipe (|) only works between Runnables. Plain Python functions don't
# have a | operator, so we wrap the FIRST function as RunnableLambda to start
# the pipe. Once the left side is a Runnable, LangChain auto-coerces any plain
# functions on the right into Runnables. So only ONE RunnableLambda is needed.
agent_step = RunnableLambda(invoke_llm) | parse_tool_call | execute_tool


def run_agent(question, verbose=True):
    """Run the LCEL agent_step in a loop until it returns a final answer."""
    state = {
        "messages": [SystemMessage(content=AGENT_SYSTEM),
HumanMessage(content=question)],
        "final": None,
    }

    for step in range(5):
        state = agent_step.invoke(state)

        if state["final"] is not None:
            if verbose:
                print(f"  [Step {step+1}] Final answer")
            return state["final"]

        if verbose:
            print(f"  [Step {step+1}] Calling: {state['last_tool_name']}
({state['last_tool_input']})")
```

```
            print(f"             Result: {str(state['last_tool_result'])
[:150]}...")

    return "Agent exceeded maximum steps."


print("LCEL agent ready: agent_step = invoke_llm | parse_tool_call |
execute_tool")

agent_questions = [
    'Who is the CTO of Globex Cybersecurity?',
    #'Acme Analytics made $42.3 million in 2025. If they grow 34%, what would
revenue be?',
    #"What is today's date?",
    #'What awards did Nextera Green Solutions win in 2025?',
    #'Globex has 412 employees and $78.5M revenue. What is revenue per
employee?',
]

if has_llm_provider():
    for i, q in enumerate(agent_questions, start=1):
        print('=' * 80)
        print(f'Agent Q{i}. {q}')
        print('=' * 80)
        try:
            result = run_agent(q)
            print(f'Final Answer: {result}')
        except Exception as e:
            if _is_retriable_error(e):
                print('  Rate limited -- skipping')
            else:
                print(f'  Agent error: {e}')
        print()
else:
    print('Set at least one API key first.')
```

3.2.8.11.2 Key Takeaways: Agents vs. Chains

	RAG Chain (LCEL)	Agent with Tools
Decision making	Always retrieves, always generates	LLM **chooses** which tools to call
Multi-step	Single retrieve then generate	Can chain multiple tool calls
Math / logic	LLM must do math in its head	Delegates to calculator tool
Extensibility	Fixed pipeline	Add any tool (APIs, databases, web search)
Provider support	Any LLM	Any LLM (prompt-based, no bind_tools needed)

When to use which:

- **RAG Chain** — simple document Q&A with a known pipeline
- **Agent** — complex questions requiring reasoning, multiple data sources, or computation

3.2.8.12 FAQ: Common Questions About LCEL and Agents

If the pipe syntax and `RunnableLambda` wrappers felt confusing on first read, you are not alone. These are the most common questions students ask after working through this notebook, and understanding the answers will make every future LangChain project easier to read and write.

3.2.8.12.1 Why do we need `RunnableLambda` for the agent but not the RAG chain?

LCEL's pipe operator `|` only works between **Runnables** (objects with an `.invoke()` method).

In the RAG chain:

```
chain = retriever | format_docs | rag_prompt | llm | StrOutputParser()
```

`retriever` is already a Runnable (it inherits from LangChain's `BaseRetriever`), so the pipe starts with a Runnable. When LangChain sees `retriever | format_docs`, it auto-wraps `format_docs` (a plain Python function) into a `RunnableLambda` internally. Every subsequent `|` keeps the chain going.

In the agent:

```
agent_step = RunnableLambda(invoke_llm) | parse_tool_call | execute_tool
```

None of our three functions are Runnables – they're plain Python functions we defined. If we wrote `invoke_llm | parse_tool_call | execute_tool`, Python would try to call `function.__or__(function)`, which doesn't exist, and raise `TypeError`. That's why we wrap the **first** function explicitly. Once the left side is a Runnable, LangChain auto-coerces the rest.

Rule: Wrap ONE function as `RunnableLambda` to bootstrap the pipe. The rest are auto-coerced.

3.2.8.12.2 Why three steps for the agent? Why not one big function?

Each LCEL step does ONE thing – the same "small composable pieces" philosophy as the RAG chain. Mapping the steps to what they do:

Step	What it does
`invoke_llm`	Send message history to LLM, get its reply
`parse_tool_call`	Check if the reply is a tool call (JSON) or a final answer
`execute_tool`	If tool call -> run the tool and update history. If final answer -> mark done

You could write all three as ONE big function, but then:

- You lose LCEL's streaming/batching/tracing benefits
- Each step can't be unit-tested or swapped independently
- It's less readable than three small functions with clear names

3.3 LangGraph

The previous chapter demonstrated that LangChain can build powerful RAG pipelines and tool-calling agents using LCEL's pipe syntax. But LCEL has a fundamental constraint: pipes are linear. Data flows from left to right through a fixed sequence of steps, and there is no native way to express "if this condition is true, take a different path" or "repeat this section until the answer is good enough." In practice, real-world AI workflows demand exactly these patterns. A customer support system might need to route technical questions to one handler and billing questions to another. A research assistant might need to evaluate its own answer and retry with a broader search if the first attempt is insufficient. LangGraph was created to handle these scenarios. It extends LangChain with stateful, directed graphs where each processing step is a node, each connection is an edge, and the data flowing between them is an explicit, inspectable state object.

3.3.1 Why Graphs Matter

A linear chain is a special case of a graph: it is a graph with no branches and no loops. LangGraph generalizes this by allowing conditional edges that route execution to different nodes based on runtime data, and back-edges that loop execution to a previous node. This seemingly simple extension opens up an entirely new class of applications. You can build pipelines that classify incoming requests and dispatch them to specialized handlers. You can build quality-control loops where the system evaluates its own output and retries if it falls short. You can build multi-agent systems where different AI personas take turns contributing to a shared state. None of these patterns can be cleanly expressed as a linear pipe, but all of them are natural as graphs.

3.3.2 State Management with TypedDict

The central concept in LangGraph is the state, a Python `TypedDict` that defines every piece of data flowing through the graph. Unlike LangChain chains, where intermediate values are passed implicitly between steps, LangGraph requires you to declare your state schema upfront. A simple RAG state might contain fields for the question, the retrieved context, and the generated answer. A more complex retry state adds fields for the attempt count and a boolean flag indicating whether the answer was sufficient.

This explicitness is a feature, not a limitation. When you inspect a node's output, you see the complete state at that point in the pipeline. There is no hidden data, no implicit variables carried along inside the framework. Every field that matters is named and typed in the `TypedDict`, making the pipeline self-documenting and easy to debug. If a node produces an unexpected result, you can examine exactly what state it received and what state it returned.

3.3.3 Three Graph Patterns

The accompanying notebook demonstrates three increasingly powerful graph patterns, each building on the one before.

The first pattern is a linear RAG graph. It contains two nodes, `retrieve` and `generate`, connected by a simple edge. The retrieve node takes the user's question, searches the vector store, and writes the retrieved context into the state. The generate node reads the question and context from the state, sends them to the LLM, and writes the answer back into the state. The flow is `START` to `retrieve` to `generate` to `END`. This is functionally identical to a LangChain LCEL chain, but the graph representation gives you something chains do not: you can visualize the pipeline as a diagram and trace the state at every node.

The second pattern introduces conditional routing. A `classify` node uses the LLM to categorize the user's question (product inquiry, financial analysis, security concern, or general question), and a conditional edge routes execution to the appropriate specialized handler. Each handler uses a tailored system prompt to produce a more focused answer. At runtime, only one branch executes. This pattern is invaluable for enterprise applications where different question types require different expertise, retrieval strategies, or even different LLMs.

The third pattern demonstrates a retry loop. After retrieval and generation, an `evaluate` node asks the LLM whether the answer adequately addresses the question. If the answer is deemed insufficient and the maximum number of retries has not been reached, a conditional edge loops execution back to the `retrieve` node, which broadens its search query on the retry attempt. If the answer is sufficient, or if three attempts have been exhausted, the graph exits to `END`. This loop structure is impossible in a linear chain. It is the kind of pattern that production RAG systems use to ensure answer quality without human intervention, and it illustrates why graphs are a fundamentally more expressive abstraction than pipes.

3.3.4 Graph Visualization

LangGraph includes built-in support for rendering compiled graphs as visual diagrams using `draw_mermaid_png()`. The linear RAG graph appears as a simple three-node flow. The routing graph shows the conditional edge fanning out from the classify node to four specialized handlers. The retry graph reveals the critical back-edge from `evaluate` to `retrieve`, the visual signature of a loop. These diagrams are not just documentation aids. They serve as executable specifications: the picture and the code are the same thing. When you add a node or change an edge, the diagram updates automatically because it is generated from the compiled graph object.

3.3.5 State Tracing with .stream()

LangGraph's `.stream()` method lets you observe state evolution in real time. Instead of receiving only the final answer, you see a dictionary update for each node as it executes. For the linear RAG graph, you see the retrieve node populate the context field and then the generate node populate the answer field. For the routing graph, you see the classify node set the category field and then the selected handler produce the answer. For the retry graph, you can watch the attempts counter increment and the is_sufficient flag flip from false to true as the loop converges on a satisfactory answer. This node-by-node visibility is the key advantage of LangGraph for debugging and for building trust in complex AI pipelines. When a stakeholder asks "how did the system arrive at this answer?", you can show them exactly which nodes executed, in what order, and what data each one contributed.

3.3.6 When to Use LangGraph vs. Plain LangChain

For straightforward, linear pipelines like a basic RAG chain or a single-step LLM call, plain LangChain with LCEL is simpler and more than sufficient. LangGraph adds value when your workflow requires conditional branching, when you need loops or retry logic, when you want explicit state management for debugging and auditing, or when you are building multi-step agent systems where visibility into each step matters. As a rule of thumb, if your pipeline can be drawn as a straight line, use LangChain. If it needs to branch or loop, use LangGraph.

3.3.7 The Hands-On Notebook

The notebook that follows implements all three graph patterns using the same fictional company documents from the LlamaIndex and LangChain chapters. You will define state schemas with `TypedDict`, build graphs with `StateGraph`, add conditional edges for routing and looping, visualize each graph with `draw_mermaid_png()`, and trace state evolution with `.stream()`. By the end, you will have a concrete understanding of how graph-based orchestration extends the capabilities of linear chains and when that extension is worth the additional complexity.

3.3.8 References

1. LangGraph Documentation. "Introduction to LangGraph." LangChain, 2025. https://langchain-ai.github.io/langgraph/
2. LangChain Documentation. "LangGraph: Multi-Actor Applications with LLMs." LangChain, 2025. https://python.langchain.com/docs/langgraph/
3. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). "ReAct: Synergizing Reasoning and Acting in Language Models." *International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/2210.03629

4. Besta, M., Blach, N., Kubicek, A., Gerstenberger, R., Podstawski, M., Gianinazzi, L., Gajber, J., Lehmann, T., Niewiadomski, H., Nyczyk, P., & Hoefler, T. (2024). "Graph of Thoughts: Solving Elaborate Problems with Large Language Models." *Proceedings of AAAI 2024*. https://arxiv.org/abs/2308.09687
5. Chase, H. "LangGraph: Build Stateful, Multi-Actor Applications." GitHub, 2025. https://github.com/langchain-ai/langgraph

Open in Colab

3.3.9 LangGraph RAG Demo: Graph-Based Retrieval Pipeline

This notebook demonstrates how **LangGraph** lets you build RAG pipelines as **explicit stateful graphs**. Unlike LangChain's chain abstraction where steps are implicit, LangGraph makes every step a visible **node** connected by **edges** — giving you full control over the flow.

How it works:

- Load **synthetic company documents** from three separate directories.
- Build a FAISS vector store (same as the LangChain notebook).
- Define a **graph** with three nodes: `retrieve` arrow.r `generate` arrow.r `format_answer`.
- Run questions through the graph and compare **with RAG** vs. **without RAG**.

Why LangGraph?

- Each step in your pipeline is a **node** — easy to debug, test, and modify independently
- **State** flows through the graph, so you can inspect what each node produced
- You can add **conditional edges** (e.g., route to different retrievers based on the question)
- Natural fit for **agentic workflows** where the LLM decides what to do next

Provider setup: This notebook uses the `llm_cascade` package, which auto-detects your API keys and falls back to the next provider if one is unavailable. Supported providers: OpenAI, Gemini, Ollama, Grok (xAI), Groq, HuggingFace, Cohere, OpenRouter.

```
!pip install -q -U langgraph langchain langchain-community langchain-
huggingface langchain-text-splitters faiss-cpu google-genai openai langchain-
google-genai git+https://github.com/KarAnalytics/llm_cascade.git sentence-
transformers
```

3.3.9.1 Imports and Provider Helpers (7-Vendor Cascade)

Same provider cascade used across all our RAG notebooks. By this point the setup should feel familiar – and that is intentional. The infrastructure layer (API keys, provider fallback, embeddings) stays constant across LlamaIndex, LangChain, and LangGraph notebooks so you can focus on what changes: the orchestration pattern. LangGraph's contribution is not a new way to call LLMs, but a new way to structure the flow between calls.

```
from pathlib import Path
from llm_cascade.providers import PROVIDERS, _load_env, _get_key,
_is_retriable_error

_load_env()

def get_available_providers():
    return [p for p in PROVIDERS if _get_key(p['key_env'])]

def has_llm_provider():
    return len(get_available_providers()) > 0

# Print status
available = get_available_providers()
if available:
    print('Providers configured (in fallback order):')
    for p in available:
        print(f"  + {p['name']:<16} model = {p['default_model']}")
else:
    print('WARNING: No API keys found.')
```

3.3.9.2 Configure LLM, Embeddings, and Build Vector Store

Same setup as the LangChain notebook: local HuggingFace embeddings + first available LLM provider. We also load documents and build the FAISS vector store here. This entire section is deliberately identical to what we did in the LangChain notebook – the vector store, the embeddings, the document loading are all the same. The only thing that changes in LangGraph is *how we orchestrate* the retrieval and generation steps, not the steps themselves.

3.3.9.3 Create Synthetic Company Documents

We generate **fictional documents for three companies** that the LLM has never seen in training. This ensures that correct answers can **only** come from RAG retrieval, not from parametric memory. Each company's documents live in a **separate directory**, simulating real-world data silos.

Directory	Company	Documents
`acme_docs/`	Acme Analytics Inc. (BI tools, Lawrence KS)	Overview, Products, Financials, Handbook, Case Study
`globex_docs/`	Globex Cybersecurity Corp. (cybersecurity, Boston)	Overview, Products, Incident Report
`nextera_docs/`	Nextera Green Solutions (sustainability consulting, Denver)	Overview, Services, Impact Report

```
DOCUMENTS = {
    "company_overview.txt": """ACME ANALYTICS INC. — COMPANY OVERVIEW

Acme Analytics Inc. was founded in 2019 by Dr. Sarah Chen and Marcus Rivera
in Lawrence, Kansas. The company specializes in AI-powered business
intelligence
tools for mid-market companies (100–5,000 employees).

Headquarters: 1420 Jayhawk Boulevard, Lawrence, KS 66045
Employees: 287 (as of January 2026)
Annual Revenue (2025): $42.3 million
Funding: Series B ($18M raised in March 2023 from Midwest Ventures)

Mission: "To democratize data analytics so every business decision is informed
by evidence, not intuition."

The company operates three offices: Lawrence (HQ), Chicago (sales), and
Austin (engineering). CEO: Dr. Sarah Chen. CTO: Marcus Rivera. CFO: Linda
Park.""",

    "products_and_services.txt": """ACME ANALYTICS — PRODUCTS AND SERVICES

1. InsightBoard Pro (Flagship Product)
   - Real-time dashboard platform with natural language query interface
   - Pricing: $49/user/month (Standard), $89/user/month (Enterprise)
   - Supports PostgreSQL, MySQL, Snowflake, BigQuery, and Redshift
   - 1,847 active enterprise customers as of Q4 2025

2. DataPipe ETL
   - Automated data pipeline builder with 200+ pre-built connectors
   - Pricing: starts at $500/month for up to 10 million records/day
   - Launched in September 2024

3. PredictIQ
   - ML-powered forecasting add-on for InsightBoard Pro
   - Pricing: $29/user/month (requires InsightBoard Pro subscription)
   - Uses proprietary time-series model trained on retail and logistics data
   - Beta launched March 2025, GA release planned for June 2026
```

```
All products include 24/7 email support. Enterprise plans include dedicated
account manager and 99.9% SLA.""",

    "q4_2025_financials.txt": """ACME ANALYTICS — Q4 2025 FINANCIAL RESULTS
(CONFIDENTIAL)

Period: October 1 — December 31, 2025

Revenue:          $12.8 million (Q4) / $42.3 million (FY 2025)
Gross Margin:     78.2%
Operating Profit: $1.9 million (Q4) / $5.1 million (FY 2025)
Net Income:       $1.4 million (Q4)
Cash on Hand:     $14.7 million
Burn Rate:        Company is cash-flow positive since Q2 2025

Key Metrics:
- ARR (Annual Recurring Revenue): $48.6 million (up 34% YoY)
- Net Revenue Retention: 118%
- Customer Acquisition Cost (CAC): $8,200
- Customer Lifetime Value (LTV): $67,400
- LTV/CAC Ratio: 8.2x

Headcount grew from 241 to 287 employees during 2025.
R&D spending: 31% of revenue. Sales & Marketing: 28% of revenue.""",

    "employee_handbook_excerpt.txt": """ACME ANALYTICS — EMPLOYEE HANDBOOK
(EXCERPT)

PAID TIME OFF (PTO):
- All full-time employees receive 22 days of PTO per year (accrued monthly).
- PTO increases to 27 days after 3 years of service.
- Unused PTO can be carried over (max 5 days) or paid out at year-end.
- Sick leave: 10 days per year (separate from PTO).

REMOTE WORK POLICY:
- Engineering and Data Science teams: fully remote eligible.
- Sales and Customer Success: hybrid (minimum 2 days/week in office).
- All employees may work remotely up to 4 weeks/year from any US location.
- International remote work requires VP approval and tax review.

PROFESSIONAL DEVELOPMENT:
- Annual learning budget: $2,500 per employee.
- Conference attendance: up to 2 conferences per year with manager approval.
- Tuition reimbursement: up to $5,250/year for degree programs.""",

    "customer_case_study.txt": """CUSTOMER CASE STUDY: BIGRETAIL CORP

Company: BigRetail Corp (1,200 retail stores across 38 states)
Challenge: Siloed data across POS, inventory, and CRM systems made it
impossible for regional managers to get timely insights.

Solution: Deployed InsightBoard Pro Enterprise + DataPipe ETL
- Connected 14 data sources in 3 weeks using DataPipe
- 340 regional managers now use InsightBoard daily
- Natural language queries replaced manual SQL report requests

Results (after 6 months):
- 62% reduction in time-to-insight (from 4 days to 1.5 days average)
- $3.2 million saved in inventory carrying costs
- 23% increase in regional manager satisfaction scores
- SQL report request backlog eliminated entirely

Quote from BigRetail CIO Janet Torres:
\"InsightBoard Pro transformed how our managers interact with data.
They went from waiting days for a report to asking questions in plain
English and getting answers in seconds.\""""",
```

```
}

# Write documents to disk
DOC_DIR = Path("acme_docs")
DOC_DIR.mkdir(exist_ok=True)

for filename, content in DOCUMENTS.items():
    (DOC_DIR / filename).write_text(content, encoding="utf-8")

print(f"Created {len(DOCUMENTS)} documents in '{DOC_DIR}/':")
for f in sorted(DOC_DIR.iterdir()):
    print(f"  {f.name} ({f.stat().st_size} bytes)")

# --- Company 2: Globex Cybersecurity Corp. → globex_docs/ ---

GLOBEX_DOCUMENTS = {
    "globex_overview.txt": """GLOBEX CYBERSECURITY CORP. — COMPANY OVERVIEW

Globex Cybersecurity Corp. was founded in 2017 by former NSA analyst
James Whitfield and AI researcher Dr. Priya Narayanan in Boston,
Massachusetts.
The company provides AI-driven cybersecurity solutions for financial services
and healthcare organizations.

Headquarters: 88 Federal Street, Suite 1200, Boston, MA 02110
Employees: 412 (as of January 2026)
Annual Revenue (2025): $78.5 million
Funding: Series C ($45M raised in June 2024 from CyberVentures Capital
and Goldman Sachs Growth Equity)

Mission: "To make enterprise-grade cybersecurity autonomous, adaptive,
and accessible to every regulated industry."

The company operates four offices: Boston (HQ), Washington D.C. (government
relations), Tel Aviv (R&D), and Singapore (APAC sales).
CEO: James Whitfield. CTO: Dr. Priya Narayanan. CISO: Robert Tanaka.""",

    "globex_products.txt": """GLOBEX CYBERSECURITY — PRODUCTS AND SERVICES

1. ShieldAI Platform (Flagship Product)
   - AI-powered threat detection and automated incident response
   - Uses behavioral analysis to detect zero-day attacks in real time
   - Pricing: $120/endpoint/year (Standard), $195/endpoint/year (Premium)
   - 623 enterprise customers across 18 countries as of Q4 2025
   - Average detection time: 2.3 minutes (industry average: 197 days)

2. VaultGuard
   - Data encryption and access management platform for regulated industries
   - HIPAA, SOX, and PCI-DSS compliant out of the box
   - Pricing: $35,000/year (up to 500 users), $60,000/year (unlimited users)
   - Launched January 2023

3. ThreatIntel Live
   - Real-time threat intelligence feed with dark web monitoring
   - Pricing: $8,500/month (includes API access and analyst dashboard)
   - Aggregates data from 2,400+ sources updated every 15 minutes
   - Integrates with Splunk, CrowdStrike, Palo Alto, and Microsoft Sentinel

All products include a dedicated Security Operations Center (SOC) team for
Premium customers. Globex holds ISO 27001 and SOC 2 Type II
certifications.""",

    "globex_incident_report.txt": """GLOBEX CYBERSECURITY — 2025 THREAT
LANDSCAPE REPORT (PUBLIC)

Published: February 2026
```

```
KEY FINDINGS FROM GLOBEX'S CUSTOMER BASE (2025):

Total incidents analyzed: 14,237 across 623 customers
- Ransomware attacks: up 42% YoY (3,891 incidents)
- Phishing campaigns: up 28% YoY (5,102 incidents)
- Supply chain attacks: up 67% YoY (1,245 incidents — fastest-growing
category)
- Insider threats: relatively stable at 1,834 incidents

ShieldAI Platform Performance:
- Blocked 97.3% of attacks before any data exfiltration
- Mean time to detect (MTTD): 2.3 minutes
- Mean time to respond (MTTR): 8.7 minutes (automated response)
- False positive rate: 0.4% (down from 1.2% in 2024)

NOTABLE CASE: MERIDIAN HEALTH SYSTEM
- 340-bed hospital network targeted by Medusa ransomware variant
- ShieldAI detected anomalous lateral movement within 47 seconds
- Automated containment isolated 3 affected servers in under 2 minutes
- Zero patient data exfiltrated; zero downtime for clinical systems
- Estimated savings: $12M+ (average healthcare ransomware cost per Ponemon)

INDUSTRY PREDICTIONS FOR 2026:
- AI-generated phishing emails will become indistinguishable from human-
written
- Quantum-resistant encryption adoption will begin in financial services
- Average cost of a data breach expected to exceed $5.2 million globally""",
}

GLOBEX_DIR = Path("globex_docs")
GLOBEX_DIR.mkdir(exist_ok=True)

for filename, content in GLOBEX_DOCUMENTS.items():
    (GLOBEX_DIR / filename).write_text(content, encoding="utf-8")

print(f"Created {len(GLOBEX_DOCUMENTS)} documents in '{GLOBEX_DIR}/':")
for f in sorted(GLOBEX_DIR.iterdir()):
    print(f"  {f.name} ({f.stat().st_size} bytes)")

# --- Company 3: Nextera Green Solutions → nextera_docs/ ---

NEXTERA_DOCUMENTS = {
    "nextera_overview.txt": """NEXTERA GREEN SOLUTIONS — COMPANY OVERVIEW

Nextera Green Solutions was founded in 2020 by climate scientist Dr. Elena
Vasquez and former McKinsey partner David Okafor in Denver, Colorado.
The company provides sustainability consulting and carbon accounting software
to help companies meet ESG (Environmental, Social, Governance) targets.

Headquarters: 1700 Lincoln Street, Suite 3400, Denver, CO 80203
Employees: 156 (as of January 2026)
Annual Revenue (2025): $23.8 million
Funding: Series A ($12M raised in November 2023 from Climate Capital Partners
and Patagonia Ventures)

Mission: "To give every company a clear, data-driven path to net zero —
turning climate commitments into measurable action."

The company operates two offices: Denver (HQ) and London (European
operations).
CEO: Dr. Elena Vasquez. COO: David Okafor. VP Engineering: Kenji Tanaka.
Nextera is a certified B Corporation and a member of the UN Global
Compact.""",

    "nextera_services.txt": """NEXTERA GREEN SOLUTIONS — SERVICES AND PLATFORM
```

```
1. CarbonLens Platform (Flagship Product)
   - Automated Scope 1, 2, and 3 carbon emissions tracking
   - Integrates with ERP systems (SAP, Oracle, NetSuite) to pull real data
   - AI-powered recommendations for emission reduction strategies
   - Pricing: $2,500/month (Standard, up to $500M revenue companies)
              $6,500/month (Enterprise, unlimited + custom reporting)
   - 312 active customers as of Q4 2025

2. ESG Reporting Suite
   - Automated generation of sustainability reports (GRI, SASB, TCFD, CDP)
   - Audit-ready documentation with full data lineage
   - Pricing: $18,000/year (bundled with CarbonLens Enterprise)
   - Standalone: $24,000/year
   - Launched April 2024

3. Sustainability Strategy Consulting
   - Science-Based Targets initiative (SBTi) alignment services
   - Supply chain decarbonization roadmaps
   - Pricing: Project-based ($75,000 – $250,000 per engagement)
   - Team of 28 consultants with backgrounds in climate science and industry

4. Carbon Offset Marketplace
   - Verified carbon credits from 45 projects across 12 countries
   - Blockchain-verified chain of custody for each credit
   - Transaction fee: 4.5% per credit purchase
   - Beta launched August 2025

All platform customers receive quarterly sustainability benchmarking reports
comparing their performance against industry peers.""",

    "nextera_impact_report.txt": """NEXTERA GREEN SOLUTIONS — 2025 IMPACT
REPORT

Published: January 2026

COMPANY IMPACT METRICS:
- Total customer carbon emissions tracked: 47.3 million metric tons CO2e
- Verified emission reductions enabled: 6.8 million metric tons CO2e (14.4%)
- Customers who improved their CDP score after using Nextera: 89%
- Average time to produce ESG report reduced from 14 weeks to 3.5 weeks

NOTABLE CLIENT RESULTS:

1. SummitBrew Coffee (specialty coffee chain, 380 locations)
   - Reduced Scope 3 supply chain emissions by 31% in 18 months
   - Switched 62% of logistics to electric vehicles using Nextera roadmap
   - First coffee chain to achieve SBTi-validated near-term target

2. Pacific Northwest Lumber Co. (timber and building materials)
   - Used CarbonLens to discover 40% of emissions came from transportation
   - Optimized shipping routes: saved 8,200 metric tons CO2e and $1.4M/year
   - Achieved carbon-neutral certification for 2025 operations

3. Meridian Financial Group (regional bank, $8B assets)
   - Implemented financed emissions tracking (PCAF methodology)
   - Identified $2.1B in loan portfolio exposed to high transition risk
   - Developed green lending program: $340M in sustainability-linked loans

AWARDS AND RECOGNITION (2025):
- Fast Company "Most Innovative Companies" — Energy category (#7)
- B Corp "Best for the World" — Environment category
- Colorado Governor's Award for Clean Energy Innovation""",
}

NEXTERA_DIR = Path("nextera_docs")
```

```
NEXTERA_DIR.mkdir(exist_ok=True)

for filename, content in NEXTERA_DOCUMENTS.items():
    (NEXTERA_DIR / filename).write_text(content, encoding="utf-8")

print(f"Created {len(NEXTERA_DOCUMENTS)} documents in '{NEXTERA_DIR}/':")
for f in sorted(NEXTERA_DIR.iterdir()):
    print(f"  {f.name} ({f.stat().st_size} bytes)")
```

```
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_community.chat_models import ChatOpenAI

# ---- Embedding model (local, no API key needed)
-----------------------------
embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/all-
MiniLM-L6-v2")
print("Embedding model: sentence-transformers/all-MiniLM-L6-v2 (local)")

# ---- LLM with runtime fallback (cascades if quota is exhausted)
--------------
llm_candidates = []
for provider in get_available_providers():
    api_key = _get_key(provider["key_env"])
    try:
        if provider["style"] == "gemini":
            from langchain_google_genai import ChatGoogleGenerativeAI
            candidate = ChatGoogleGenerativeAI(
                google_api_key=api_key,
                model=provider["default_model"],
            )
        else:
            candidate = ChatOpenAI(
                api_key=api_key,
                base_url=provider["base_url"],
                model=provider["default_model"],
            )
        llm_candidates.append((provider, candidate))
        print(f"  + {provider['name']:<16} ({provider['default_model']})")
    except Exception as e:
        print(f"  - {provider['name']:<16} skipped: {e}")

if llm_candidates:
    primary_provider, primary_llm = llm_candidates[0]
    fallback_llms = [c for _, c in llm_candidates[1:]]

    if fallback_llms:
        llm = primary_llm.with_fallbacks(fallback_llms)
        print(f"\nPrimary LLM: {primary_provider['name']}
({primary_provider['default_model']})")
        print(f"Fallbacks:   {', '.join(p['name'] for p, _ in
llm_candidates[1:])}")
        print("(If primary hits quota, automatically tries the next
provider)")
    else:
        llm = primary_llm
        print(f"\nLLM provider: {primary_provider['name']}
({primary_provider['default_model']})")
        print("(No fallback providers configured)")

    llm_provider = primary_provider
else:
    llm = None
    primary_llm = None
    llm_provider = None
    print("ERROR: Could not configure any LLM provider. Check your API keys.")
```

```
# ---- Load documents from three company directories
--------------------------
from langchain_community.document_loaders import DirectoryLoader, TextLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.vectorstores import FAISS

DOC_DIRS = {
    "acme_docs":    "Acme Analytics (BI tools)",
    "globex_docs":  "Globex Cybersecurity (security)",
    "nextera_docs": "Nextera Green Solutions (sustainability)",
}

all_docs = []
for dir_name, label in DOC_DIRS.items():
    loader = DirectoryLoader(dir_name, glob="**/*.txt", loader_cls=TextLoader,
                             loader_kwargs={"encoding": "utf-8"})
    docs = loader.load()
    print(f"  Loaded: '{dir_name}/' -> {len(docs)} docs  [{label}]")
    all_docs.extend(docs)

# ---- Split and index
-------------------------------------------------------
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000, chunk_overlap=200,
    separators=["\n---\n", "\n\n", "\n", " "],
)
chunks = text_splitter.split_documents(all_docs)

vectorstore = FAISS.from_documents(chunks, embeddings)
retriever = vectorstore.as_retriever(search_kwargs={"k": 3})
print(f"FAISS vector store: {len(chunks)} chunks indexed, retriever ready
(top_k=3)")
```

3.3.9.4 Define the RAG Graph

This is where LangGraph diverges from LangChain. Instead of composing steps with the `|` pipe operator, we build an **explicit graph** where each step is a **node** and the connections between them are **edges**. The state that flows through the graph is defined as a Python `TypedDict` – a plain dictionary with typed fields like `question`, `context`, `answer`, and `sources`. Every node is a function that reads from this state dict and writes back to it.

Node	What it does
`retrieve`	Searches the vector store for relevant chunks
`generate`	Sends the question + retrieved chunks to the LLM

The graph flows: **START –> retrieve –> generate –> END**

Why bother with a graph for what is essentially a two-step pipeline? Because this simple linear graph is a stepping stone. Once you have the graph infrastructure in place, adding conditional branches, retry loops, or parallel paths is just a matter of adding nodes and edges – no restructuring required. The linear graph teaches you the API; the conditional and looping graphs that follow show why it matters.

```
START --> retrieve --> generate --> END
```

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

# ---------------------------------------------------------------------------
# Step 1: Define the STATE that flows through the graph
# ---------------------------------------------------------------------------
class RAGState(TypedDict):
    question: str           # user's question (input)
    context: str            # retrieved chunks (set by retrieve node)
```

```python
    sources: list[str]      # source file names (set by retrieve node)
    answer: str             # LLM's answer (set by generate node)


# ---------------------------------------------------------------------------
# Step 2: Define the NODES (each is a plain function)
# ---------------------------------------------------------------------------
def retrieve(state: RAGState) -> dict:
    """Search the vector store for relevant chunks."""
    docs = retriever.invoke(state["question"])
    context = "\n\n---\n\n".join(doc.page_content for doc in docs)
    sources = list(set(doc.metadata.get("source", "unknown") for doc in docs))
    return {"context": context, "sources": sources}


def generate(state: RAGState) -> dict:
    """Send question + retrieved context to the LLM."""
    prompt = f"""Answer the question based ONLY on the following context.
If the context doesn't contain the answer, say "I don't have enough
information."

Context:
{state['context']}

Question: {state['question']}

Answer:"""
    response = llm.invoke(prompt)
    return {"answer": response.content}


# ---------------------------------------------------------------------------
# Step 3: Build the GRAPH
# ---------------------------------------------------------------------------
workflow = StateGraph(RAGState)

# Add nodes
workflow.add_node("retrieve", retrieve)
workflow.add_node("generate", generate)

# Add edges: START → retrieve → generate → END
workflow.add_edge(START, "retrieve")
workflow.add_edge("retrieve", "generate")
workflow.add_edge("generate", END)

# Compile the graph
graph = workflow.compile()
print("RAG graph compiled: START \u2192 retrieve \u2192 generate \u2192 END")
```

3.3.9.5 Visualize the RAG Graph

One of LangGraph's most practical features is built-in graph visualization. The compiled graph can render itself as a Mermaid diagram showing nodes and edges. For our simple RAG pipeline this is just a straight line (`START --> retrieve --> generate --> END`), but as we add conditional routing and loops in later sections, the diagrams become invaluable for understanding the flow at a glance.

```python
from IPython.display import Image, display

try:
    img_bytes = graph.get_graph().draw_mermaid_png()
    display(Image(img_bytes))
except Exception as e:
    # Fallback: print the text representation
    print("Graph structure (install graphviz for visual rendering):")
```

```
    print("  START \u2192 retrieve \u2192 generate \u2192 END")
    print(f"  (Visual rendering unavailable: {e})")
```

3.3.9.6 Trace the State Through the RAG Graph

LangGraph's `.stream()` method lets you **watch state evolve** as data flows through each node. This is the key advantage over LangChain's chain abstraction: instead of a black-box pipeline that accepts a question and returns an answer, you can inspect exactly what each node contributed to the shared state dictionary.

Below, we run one question through the graph and print the state dict after every node executes. Notice how `context` and `sources` appear after the `retrieve` node, and `answer` appears after `generate`. This kind of step-by-step transparency is invaluable for debugging RAG pipelines – when an answer is wrong, you can immediately see whether the problem is in retrieval (wrong chunks) or generation (right chunks but bad answer).

```
if has_llm_provider():
    question = "What was Acme Analytics' revenue in Q4 2025?"
    print(f'Tracing state for: "{question}"')
    print()

    for state_update in graph.stream({'question': question}):
        node_name = list(state_update.keys())[0]
        node_output = state_update[node_name]
        print('=' * 60)
        print(f'After node: {node_name}')
        print('=' * 60)
        for key, value in node_output.items():
            val_str = str(value)
            if len(val_str) > 200:
                val_str = val_str[:200] + '...'
            print(f'  state["{key}"] = {val_str}')
        print()
```

3.3.9.7 Run the RAG Graph

Now let us run the full graph on several questions and see the end-to-end results. The graph executes `retrieve --> generate` and returns the final state with the answer, sources, and context. Compare these results to what you saw in the LangChain notebook – the answers should be identical, because the underlying retrieval and generation logic is the same. What differs is the orchestration: here every step is a visible, inspectable node in a graph.

```
questions = [
    "What was Acme Analytics' revenue in Q4 2025?",
    #"How many days of PTO do new employees get at Acme?",
    #"What is Globex Cybersecurity's flagship product and how fast does it
detect threats?",
    #"How did Globex help Meridian Health System during a ransomware attack?",
    #"Who founded Nextera Green Solutions and where is it headquartered?",
    #"How much carbon emissions reduction did Nextera enable for its customers
in 2025?",
]


def preview(text, max_len=800):
    text = str(text) if text else ""
    return text[:max_len] + ("..." if len(text) > max_len else "")


if not has_llm_provider():
    print("Error: No LLM API key configured. Set at least one API key in Colab
Secrets.")
else:
    for i, q in enumerate(questions, start=1):
        print("=" * 80)
        print(f"Q{i}. {q}")
```

```
        print("=" * 80)

        try:
            result = graph.invoke({"question": q})
            print(f"Model: {llm_provider['name']}
({llm_provider['default_model']})")
            print("Answer:")
            print(preview(result["answer"]))
            print(f"Sources: {', '.join(result['sources'])}")
        except Exception as e:
            if _is_retriable_error(e):
                print("  Rate limited -- skipping")
            else:
                print(f"  Error: {e}")

        print()
```

At this point you have seen LangGraph's simplest pattern: a linear graph that does the same thing as a LangChain chain or a LlamaIndex query engine. The value so far is debuggability – you can trace state through each node. But the real power of graphs emerges when the flow is no longer linear. The next two sections demonstrate branching (conditional routing) and looping (retry with self-evaluation), which are the patterns that justify reaching for LangGraph over simpler abstractions.

3.3.9.8 LangGraph vs. LangChain vs. LlamaIndex

Aspect	LangGraph	LangChain	LlamaIndex
Abstraction	Graph of nodes + edges	Chains (sequential steps)	High-level query engine
State management	Explicit `TypedDict` state	Implicit in chain	Hidden inside framework
Debuggability	Inspect any node's output	Chain callbacks	Event handlers
Conditional logic	Conditional edges in graph	Requires custom chains	Requires custom modules
Visual pipeline	Built-in graph rendering	No built-in visualization	No built-in visualization
Complexity	Medium	Low–Medium	Low
Best for	Complex multi-step agents	Standard RAG pipelines	Quick document Q&A

Key takeaway: LangGraph shines when you need **visibility and control** over a multi-step pipeline. The graph structure makes it easy to add conditional routing, retry logic, human-in-the-loop steps, or parallel branches — things that are harder to express as a simple chain.

3.3.9.9 Beyond RAG: Conditional Routing with LangGraph

The RAG graph above is a **linear pipeline** — every question follows the same path. LangGraph's real power emerges with **conditional edges** that route questions to different handlers based on runtime data.

Below, we build a **smart router** that:

- Uses a `classify` node to categorize each question (finance / hr / technical / general)
- Routes to a **specialized handler** via a conditional edge
- Each handler uses the same retriever but a different system prompt tuned for its domain

The graph shape is a **fan-out**: `START → classify → one-of-four handlers → END`.

```python
from typing import TypedDict, Literal
from langgraph.graph import StateGraph, START, END

# ---------------------------------------------------------------------------
# State for the routing graph
# ---------------------------------------------------------------------------
class RouterState(TypedDict):
    question: str
    category: str          # set by classify node
    context: str           # set by handler nodes
    sources: list[str]     # set by handler nodes
    answer: str            # set by handler nodes


# ---------------------------------------------------------------------------
# Node: classify — LLM decides which department handles the question
# ---------------------------------------------------------------------------
def classify(state: RouterState) -> dict:
    """Use the LLM to classify the question into a category."""
    classification_prompt = f"""Classify the following question into exactly
ONE category.
Reply with ONLY the category name, nothing else.

Categories:
- finance: revenue, profit, costs, pricing, funding, financial metrics, ARR,
burn rate
- hr: PTO, remote work, benefits, employee policies, handbook, hiring,
headcount
- technical: products, features, integrations, security, technology, platform
details
- general: company overview, founders, mission, headquarters, partnerships,
anything else

Question: {state['question']}

Category:"""
    response = llm.invoke(classification_prompt)
    category = response.content.strip().lower()
    # Normalize to valid categories
    if category not in ("finance", "hr", "technical", "general"):
        category = "general"
    print(f"  [classify] → {category}")
    return {"category": category}


# ---------------------------------------------------------------------------
# Nodes: specialized handlers (each uses retriever + tailored system prompt)
# ---------------------------------------------------------------------------
HANDLER_PROMPTS = {
    "finance": "You are a financial analyst. Focus on numbers, metrics, and
financial performance. Be precise with figures.",
    "hr": "You are an HR specialist. Focus on policies, benefits, and
employee-related information. Be clear about specific rules and limits.",
    "technical": "You are a technical product expert. Focus on features,
capabilities, integrations, and technical specifications.",
    "general": "You are a helpful business analyst. Provide a clear,
comprehensive answer.",
}


def _make_handler(category: str):
    """Factory function to create a handler node for a given category."""
    def handler(state: RouterState) -> dict:
        docs = retriever.invoke(state["question"])
        context = "\n\n---\n\n".join(doc.page_content for doc in docs)
        sources = list(set(doc.metadata.get("source", "unknown") for doc in
```

```
docs))

        system_prompt = HANDLER_PROMPTS[category]
        prompt = f"""{system_prompt}

Answer based ONLY on the following context. If the context doesn't contain the
answer, say so.

Context:
{context}

Question: {state['question']}

Answer:"""
        response = llm.invoke(prompt)
        print(f"  [{category}_handler] answered using {len(docs)} chunks")
        return {"context": context, "sources": sources, "answer":
response.content}
    handler.__name__ = f"{category}_handler"
    return handler


# ---------------------------------------------------------------------------
# Router function — decides which handler to call based on classification
# ---------------------------------------------------------------------------
def route_question(state: RouterState) -> str:
    """Conditional edge: route to the appropriate handler based on
category."""
    return f"{state['category']}_handler"


# ---------------------------------------------------------------------------
# Build the routing graph
# ---------------------------------------------------------------------------
router_graph = StateGraph(RouterState)

# Add nodes
router_graph.add_node("classify", classify)
for cat in HANDLER_PROMPTS:
    router_graph.add_node(f"{cat}_handler", _make_handler(cat))

# Add edges
router_graph.add_edge(START, "classify")
router_graph.add_conditional_edges("classify", route_question)
for cat in HANDLER_PROMPTS:
    router_graph.add_edge(f"{cat}_handler", END)

# Compile
smart_router = router_graph.compile()
print("Smart router graph compiled!")
print("  Nodes: classify → finance_handler | hr_handler | technical_handler |
general_handler")
```

3.3.9.10 Visualize the Routing Graph

The conditional edge from `classify` fans out to four handler nodes. At runtime, only one path is taken based on the LLM's classification. The text diagram below the rendered graph clarifies the routing logic.

```
from IPython.display import Image, display

try:
    img_bytes = smart_router.get_graph().draw_mermaid_png()
    display(Image(img_bytes))
except Exception as e:
    print(f"(Visual rendering unavailable: {e})")
```

```
print("Routing logic:")
print("  START")
print("    └── classify (LLM classifies the question)")
print("          ├── [finance]   → finance_handler   → END")
print("          ├── [hr]        → hr_handler        → END")
print("          ├── [technical] → technical_handler → END")
print("          └── [general]   → general_handler   → END")
```

3.3.9.11 Trace the State Through the Routing Graph

Same idea as before, but now the state also carries a `category` field set by the `classify` node, and the conditional edge picks a different handler based on that category. Watch how the state evolves differently depending on which branch is taken – the `category` field acts as a routing signal that determines which node executes next. This is the `TypedDict` state pattern in action: every decision the graph makes is recorded in the state and visible to you.

```
if has_llm_provider():
    question = "What is Acme Analytics' LTV/CAC ratio?"
    print(f'Tracing state for: "{question}"')
    print()

    for state_update in smart_router.stream({'question': question}):
        node_name = list(state_update.keys())[0]
        node_output = state_update[node_name]
        print('=' * 60)
        print(f'After node: {node_name}')
        print('=' * 60)
        for key, value in node_output.items():
            val_str = str(value)
            if len(val_str) > 200:
                val_str = val_str[:200] + '...'
            print(f'  state["{key}"] = {val_str}')
        print()
```

3.3.9.12 Test the Smart Router

Each question is classified and routed to the matching handler. Uncomment questions one at a time to test different categories.

```
router_questions = [
    # Should route to finance_handler
    "What is Acme Analytics' LTV/CAC ratio?",
    #"What was Globex Cybersecurity's annual revenue in 2025?",

    # Should route to hr_handler
    #"What is Acme's remote work policy for engineers?",
    #"How much tuition reimbursement do Acme employees get?",

    # Should route to technical_handler
    #"What databases does InsightBoard Pro support?",
    #"How does Globex ShieldAI detect zero-day attacks?",

    # Should route to general_handler
    #"Who founded Nextera Green Solutions?",
    #"What awards did Nextera win in 2025?",
]


def preview(text, max_len=600):
    text = str(text) if text else ""
    return text[:max_len] + ("..." if len(text) > max_len else "")


if has_llm_provider():
    for i, q in enumerate(router_questions, start=1):
```

```
        print("\n" + "=" * 80)
        print(f"Q{i}. {q}")
        print("=" * 80)
        try:
            result = smart_router.invoke({"question": q})
            print(f"  Category: {result['category']}")
            print(f"  Answer: {preview(result['answer'])}")
            print(f"  Sources: {', '.join(result['sources'])}")
        except Exception as e:
            if _is_retriable_error(e):
                print(f"  Rate limited — skipping")
            else:
                print(f"  Error: {e}")
        print()
else:
    print("Set at least one API key first.")
```

3.3.9.13 Advanced: RAG with a Retry Loop

The two graphs above are **linear** (RAG) and **branching** (router). LangGraph can also express **loops** — the graph circles back to a previous node based on a condition.

Below, we build a RAG pipeline that **evaluates its own answer** and retries if the answer looks insufficient:

```
START → retrieve → generate → evaluate ── sufficient? ── YES → END
                                          └─ NO (up to 3 retries) → retrieve
(loop back)
```

This is a realistic production pattern: **quality-checking RAG with automatic retry**.

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END


class RetryRAGState(TypedDict):
    question: str
    context: str
    sources: list[str]
    answer: str
    is_sufficient: bool
    attempts: int


def retry_retrieve(state: RetryRAGState) -> dict:
    '''Retrieve chunks. On retry, modify the query to try to get better
results.'''
    query = state['question']
    attempts = state.get('attempts', 0)
    if attempts > 0:
        # On retry, add a prefix to broaden the search
        query = f'Give more detail about: {query}'
        print(f'  [retry_retrieve] Attempt {attempts + 1}, broadened query')
    docs = retriever.invoke(query)
    context = (chr(10) * 2 + '---' + chr(10) * 2).join(doc.page_content for
doc in docs)
    sources = list(set(doc.metadata.get('source', 'unknown') for doc in docs))
    return {'context': context, 'sources': sources}


def retry_generate(state: RetryRAGState) -> dict:
    '''Generate an answer from retrieved context.'''
    prompt = (
        'Answer the question based ONLY on the following context. '
        'If the context does not contain the answer, say "I do not have enough
```

```
information."'
        + chr(10) * 2 + 'Context:' + chr(10) + state['context']
        + chr(10) * 2 + 'Question: ' + state['question']
        + chr(10) * 2 + 'Answer:'
    )
    response = llm.invoke(prompt)
    return {'answer': response.content}


def evaluate(state: RetryRAGState) -> dict:
    '''Check if the answer is sufficient or needs retry.'''
    answer = state.get('answer', '')
    attempts = state.get('attempts', 0) + 1
    # Simple heuristic: answer is insufficient if it's very short or says
"don't have"
    is_sufficient = (
        len(answer) > 50
        and 'do not have enough' not in answer.lower()
        and 'i don' not in answer.lower()[:30]
    )
    status = 'SUFFICIENT' if is_sufficient else f'RETRY (attempt {attempts})'
    print(f'  [evaluate] {status}')
    return {'is_sufficient': is_sufficient, 'attempts': attempts}


def route_after_eval(state: RetryRAGState) -> str:
    '''Conditional edge: end if sufficient or max retries reached, else loop
back.'''
    if state.get('is_sufficient', False) or state.get('attempts', 0) >= 3:
        return 'end'
    return 'retry'


# Build the looping graph
retry_workflow = StateGraph(RetryRAGState)

retry_workflow.add_node('retrieve', retry_retrieve)
retry_workflow.add_node('generate', retry_generate)
retry_workflow.add_node('evaluate', evaluate)

retry_workflow.add_edge(START, 'retrieve')
retry_workflow.add_edge('retrieve', 'generate')
retry_workflow.add_edge('generate', 'evaluate')
retry_workflow.add_conditional_edges(
    'evaluate',
    route_after_eval,
    {'end': END, 'retry': 'retrieve'},  # 'retry' loops back to retrieve!
)

retry_graph = retry_workflow.compile()
print('Retry RAG graph compiled: retrieve -> generate -> evaluate -> (loop or
end)')
```

3.3.9.14 Visualize the Retry Loop Graph

The key visual difference from the earlier graphs: the **arrow from `evaluate` back to `retrieve`** is the loop. This backward edge is something that linear chains fundamentally cannot express – you need a graph to represent cycles. When you look at the diagram below, you are seeing the complete control flow of a self-correcting RAG system in a single picture.

```
from IPython.display import Image, display

try:
    img_bytes = retry_graph.get_graph().draw_mermaid_png()
    display(Image(img_bytes))
```

```
except Exception as e:
    print(f'(Visual rendering unavailable: {e})')

print('Graph structure:')
print('  START')
print('    └── retrieve (search vector store for relevant chunks)')
print('          └── generate (LLM produces an answer from the chunks)')
print('                └── evaluate (check answer quality)')
print('                      ├── [sufficient] → END')
print('                      └── [retry]      → retrieve (LOOP BACK, up to 3
times)')
```

3.3.9.15 Run the Retry Graph with State Tracing

Watch the state evolve across loop iterations below. If the first answer is good, the graph exits immediately – you will see `retrieve --> generate --> evaluate --> END` in a single pass. If the evaluator is not satisfied, you will see the loop fire: `retrieve --> generate --> evaluate --> retrieve --> ...` until the answer is sufficient or 3 attempts are reached. The `attempts` counter in the state tracks progress and prevents infinite loops, which is a critical safety feature for any production loop pattern.

```
if has_llm_provider():
    question = "What was Acme Analytics' revenue in Q4 2025?"
    print(f'Question: "{question}"')
    print()

    iteration = 0
    for state_update in retry_graph.stream({'question': question, 'attempts':
0, 'is_sufficient': False}):
        node_name = list(state_update.keys())[0]
        node_output = state_update[node_name]
        iteration += 1
        print(f'--- Step {iteration}: node "{node_name}" ---')
        for key, value in node_output.items():
            val_str = str(value)
            if len(val_str) > 200:
                val_str = val_str[:200] + '...'
            print(f'  state["{key}"] = {val_str}')
        print()

    print('Graph finished.')
```

3.3.9.15.1 Key Takeaways: Why LangGraph for Routing?

Pattern	Tool	How it works
Simple RAG	LangChain `RetrievalQA` or LlamaIndex	Fixed pipeline: retrieve arrow.r generate
Graph RAG	LangGraph (Section 3-6 above)	Explicit nodes, visible state, same fixed flow
Conditional routing	**LangGraph (this section)**	**LLM classifies arrow.r graph branches to specialized handlers**
Agent with tools	LangChain Agent (see LangChain notebook)	LLM decides which tools to call at each step

When to use conditional routing:

- Customer support: route tickets to billing, technical, or account teams
- Enterprise search: route queries to different knowledge bases (legal, engineering, HR)
- Multi-model pipelines: use a cheap model for classification, expensive model for generation
- Compliance: ensure sensitive questions always go through an approved handler

4. Fine-Tuning

4.1 Fine-Tuning with QLoRA

4.1.1 When Retrieval Is Not Enough

Throughout the preceding chapters, we built increasingly sophisticated Retrieval-Augmented Generation (RAG) pipelines. RAG is a powerful pattern: it grounds a language model's responses in external documents, reducing hallucination and keeping answers current. But RAG has its limits. Every answer requires a retrieval step, adding latency. The model's tone, vocabulary, and reasoning style remain those of the generic foundation model. And for knowledge that the model needs to recall instantly and consistently — the kind of deep, institutional knowledge that defines an organization — retrieving fragments from a document store at query time can feel brittle. What if the model could simply *know* these facts the way it knows the English language?

That is the promise of fine-tuning. Rather than fetching information from an external source at inference time, fine-tuning modifies the model's internal weights so that new knowledge, style, or behavior becomes part of the model itself. A fine-tuned model does not need to search for your company's product names, pricing tiers, or brand voice — it has internalized them. The result is faster inference, more consistent outputs, and a model that feels genuinely specialized rather than generically augmented.

4.1.2 The Memory Wall: Why Full Fine-Tuning Is Expensive

The obstacle, historically, has been cost. Fine-tuning a large language model means running gradient-based optimization over billions of parameters. A 7-billion-parameter model stored in standard 32-bit floating point (FP32) occupies roughly 28 gigabytes of GPU memory just for the weights alone — before accounting for optimizer states, gradients, and activations, which can double or triple that figure. Training such a model traditionally required multiple high-end GPUs costing thousands of dollars per hour on cloud platforms. For most organizations, and certainly for most MBA students, that put fine-tuning out of reach.

Two complementary techniques changed this equation: Low-Rank Adaptation (LoRA) and quantization. Together, they form QLoRA — a method that makes fine-tuning accessible on hardware as modest as a free Google Colab T4 GPU with 15 gigabytes of VRAM.

4.1.3 LoRA: Training a Fraction of the Parameters

The key insight behind LoRA, introduced by Hu et al. (2021), is that not all parameters need to change during fine-tuning. When a foundation model adapts to a new task, the weight updates tend to occupy a low-rank subspace — meaning the essential information in those updates can be captured by much smaller matrices. LoRA exploits this by freezing all of the original model weights and injecting pairs of small, trainable matrices (called adapters) into selected layers, typically the attention projection layers. During a forward pass, the output of each adapted layer is the sum of the original frozen computation and a lightweight learned correction.

The practical impact is dramatic. Instead of updating all parameters, LoRA trains only the adapter matrices — typically around 0.5 to 1 percent of the total parameter count. For a model with 315 million parameters, this means roughly 2 million trainable parameters while the remaining 313 million stay frozen. Training is faster, memory requirements drop substantially, and the original model weights are never destroyed, so you can swap different LoRA adapters in and out for different tasks without maintaining separate copies of the full model.

4.1.4 Quantization: Compressing the Base Model

LoRA reduces what you train; quantization reduces what you store. Standard models represent each parameter as a 16-bit floating-point number. Quantization compresses these values to 4-bit representations using specialized formats such as NormalFloat4 (NF4), which is information-theoretically optimal for normally distributed weights. This compression

reduces the memory footprint of the base model by approximately 75 percent — a 7-billion-parameter model that would occupy 14 gigabytes in FP16 fits into roughly 3.5 gigabytes in 4-bit.

Dettmers et al. (2023) demonstrated that this aggressive compression introduces negligible quality loss when combined with LoRA adapters, because the adapters themselves are trained in higher precision (FP16) and compensate for any quantization artifacts. Their technique, which they named QLoRA, also introduced double quantization — quantizing the quantization constants themselves — to squeeze out additional memory savings.

4.1.5 QLoRA: Bringing It All Together

QLoRA is simply the combination of these two ideas: load the base model in 4-bit quantization to minimize memory, then apply LoRA adapters to train a small subset of parameters in higher precision. The result is a fine-tuning method that runs comfortably on a single consumer-grade or free-tier cloud GPU. In the accompanying notebook, we fine-tune a Qwen2.5-0.5B-Instruct model on just 15 examples of fictional company data in approximately two minutes on a Colab T4 GPU, using under one gigabyte of VRAM. The model goes from knowing nothing about our fictional Acme Analytics to accurately recalling its CEO, annual recurring revenue, product details, and mission statement.

4.1.6 The Training Pipeline

The practical workflow for QLoRA fine-tuning follows a consistent pattern. First, you prepare your training data as structured question-answer pairs or instruction-response examples. Second, you load the base model with a 4-bit quantization configuration (specifying NF4 format, FP16 compute dtype, and double quantization). Third, you define a LoRA configuration — choosing the adapter rank, scaling factor, target modules, and dropout rate. Fourth, you apply the LoRA adapters to the quantized model using the PEFT (Parameter-Efficient Fine-Tuning) library. Fifth, you train using the SFTTrainer from the TRL library, which handles the supervised fine-tuning loop with standard hyperparameters such as learning rate, batch size, and epoch count. Finally, you evaluate the fine-tuned model by comparing its outputs to the base model on held-out questions.

4.1.7 RAG versus Fine-Tuning: Choosing the Right Tool

RAG and fine-tuning are not competitors — they solve different problems. RAG excels when your knowledge base changes frequently, when you need citation-level traceability back to source documents, and when the volume of potentially relevant information is too large to encode into model weights. Fine-tuning excels when you need the model to adopt a specific style, tone, or persona, when you want faster inference without a retrieval step, and when core knowledge should be recalled consistently without depending on retrieval quality.

The tradeoffs are concrete. RAG requires no retraining when documents change — you simply update the vector store. Fine-tuning requires a new training run for each knowledge update, but delivers lower latency at inference time because there is no retrieval step. RAG responses are grounded in retrieved text, which can reduce hallucination, while fine-tuned models can still hallucinate if asked about topics outside their training distribution.

In practice, mature organizations combine both approaches. They fine-tune a base model to internalize brand voice, core product knowledge, compliance guidelines, and domain-specific terminology. They then layer RAG on top for dynamic information — current inventory levels, recent support tickets, or today's pricing changes. This hybrid architecture delivers the consistency and speed of fine-tuning with the freshness and traceability of retrieval.

4.1.8 The Accompanying Notebook

The QLoRA notebook in this chapter provides a hands-on demonstration of the complete pipeline. It fine-tunes a small language model on fictional data about Acme Analytics — a made-up AI-powered business intelligence company — and presents a side-by-side comparison of model responses before and after training. Before fine-tuning, the base model

either refuses to answer or confabulates when asked about Acme Analytics. After just two minutes of QLoRA training on 15 examples, the model accurately recalls the company's CEO, revenue figures, product names, and pricing. The contrast makes the power of fine-tuning immediately tangible.

4.1.9 References

- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2021). LoRA: Low-Rank Adaptation of Large Language Models. *arXiv preprint arXiv:2106.09685*. https://arxiv.org/abs/2106.09685
- Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient Finetuning of Quantized Language Models. *Advances in Neural Information Processing Systems, 36*. https://arxiv.org/abs/2305.14314
- Dettmers, T., Lewis, M., Belkada, Y., & Zettlemoyer, L. (2022). LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale. *Advances in Neural Information Processing Systems, 35*. https://arxiv.org/abs/2208.07339
- Mangrulkar, S., Gugger, S., Debut, L., Belkada, Y., Paul, S., & Bossan, B. (2022). PEFT: Parameter-Efficient Fine-Tuning of Billion-Scale Models on Low-Resource Hardware. Hugging Face. https://github.com/huggingface/peft

Open in Colab

4.1.10 QLoRA Fine-Tuning Demo: Teaching a Small LLM New Knowledge

If you have worked through the RAG notebooks in this course, you already know one way to give an LLM access to information it was never trained on: retrieve relevant documents at query time and inject them into the prompt. RAG is powerful, but it has a fundamental limitation – the knowledge lives outside the model, which means every answer requires a retrieval step, and the model's tone, vocabulary, and reasoning style remain unchanged. **Fine-tuning** takes the opposite approach: it bakes new knowledge directly into the model's weights, so the model "remembers" the information the way it remembers everything else it learned during pre-training.

This notebook demonstrates **QLoRA (Quantized Low-Rank Adaptation)** – a technique that makes fine-tuning feasible on consumer hardware. The "Q" stands for quantization: we compress the base model from 16-bit floating point down to 4-bit integers, reducing memory by roughly 75%. The "LoRA" stands for Low-Rank Adaptation: instead of updating all 315 million parameters, we freeze the entire model and inject small trainable matrices (adapters) into the attention layers. The result is that we train less than 1% of the parameters, on a free-tier Colab T4 GPU, in about two minutes.

What we will do:

- Load a small pre-trained model (Qwen2.5-0.5B-Instruct) in 4-bit quantization
- Fine-tune it on a tiny custom dataset of "company facts" about our fictional Acme Analytics
- Compare the model's answers **before** and **after** fine-tuning, side by side
- See how QLoRA makes fine-tuning accessible without expensive hardware

Why this matters for business: Companies can customize LLMs with proprietary data – product specs, internal policies, domain terminology – and deploy them as lightweight, latency-free knowledge sources that do not require a retrieval pipeline at inference time. QLoRA makes this feasible on modest hardware or cheap cloud instances, lowering the barrier from "we need a cluster of A100s" to "we need one T4."

Requirements: Google Colab with T4 GPU (Runtime -> Change runtime type -> T4 GPU)

```
!pip install -q transformers==4.46.3 accelerate peft bitsandbytes datasets trl
```

The cell above installs the key libraries we need. `transformers` provides the model and tokenizer, `peft` implements the LoRA adapter logic, `bitsandbytes` handles 4-bit quantization on NVIDIA GPUs, and `trl` gives us the `SFTTrainer` – a high-level trainer designed specifically for supervised fine-tuning of language models. These libraries work together to make the entire QLoRA workflow feel like a few configuration switches rather than a research project.

4.1.10.1 Check GPU Availability

QLoRA requires a GPU. On Colab, go to **Runtime arrow.r Change runtime type arrow.r T4 GPU**.

```
import torch

if torch.cuda.is_available():
    gpu_name = torch.cuda.get_device_name(0)
    gpu_mem = torch.cuda.get_device_properties(0).total_memory / 1e9
    print(f'GPU: {gpu_name} ({gpu_mem:.1f} GB)')
else:
```

```
    print('WARNING: No GPU detected! QLoRA requires a GPU.')
    print('Go to Runtime -> Change runtime type -> T4 GPU')
```

4.1.10.2 Load the Base Model in 4-bit Quantization

We load **Qwen2.5-0.5B-Instruct** — a small but capable instruction-tuned model. Using `BitsAndBytesConfig`, we load it in **4-bit quantization** (NF4 format), which:

- Reduces memory from ∼1 GB (FP16) to ∼0.3 GB (4-bit)
- Enables fine-tuning on a free T4 GPU (15 GB VRAM)
- Uses double quantization for extra memory savings

The 0.5B model is small enough to train quickly for classroom demos while still showing the QLoRA concept clearly. For production use cases, you'd typically use larger models (1.5B, 7B, or more).

```
## Test the Model BEFORE Fine-Tuning

Before we change anything, let's establish a baseline. We ask the base model
questions about our fictional company "Acme Analytics." Since this company
does not exist in the real world, the model has never seen any training data
about it. It should either politely refuse to answer or hallucinate plausible-
sounding but completely fabricated details. Either way, these baseline
responses will serve as our "before" snapshot for the side-by-side comparison
later.

We use **greedy decoding** (`do_sample=False`) for all inference in this
notebook. Why? Because we want to test factual recall, not creative
generation. Greedy decoding always picks the highest-probability token at each
step, making the output deterministic -- the same question always produces the
same answer. This eliminates randomness as a confounding variable when we
compare before-and-after results. If the answer changes after fine-tuning, we
know it is because the model's weights changed, not because the random seed
was different.
```

4.1.10.3 Test the Model BEFORE Fine-Tuning

Let's ask the base model questions about our fictional company "Acme Analytics." Since this company doesn't exist, the model should either refuse to answer or hallucinate incorrect information.

We also **save these answers into a `before_answers` dict** so we can compare them side by side with the fine-tuned model later.

```
## Prepare the Training Dataset

Fine-tuning requires training data in a specific format. For instruction-tuned
models like Qwen, each example is a **conversation**: a user message (the
question) paired with an assistant message (the ideal answer). The tokenizer's
`apply_chat_template` method wraps these pairs in the model's expected special
tokens (`<|im_start|>`, `<|im_end|>`), so the model learns to associate user
queries with assistant responses in the same format it will see at inference
time.

We create a small dataset of 15 question-answer pairs about Acme Analytics --
things like the CEO's name, the company's ARR, product pricing, and office
locations. In a real scenario, you would use hundreds or thousands of examples
pulled from internal knowledge bases, FAQs, or documents. But for this
classroom demo, 15 examples are enough to demonstrate the QLoRA concept
clearly: the model will memorize these facts in about two minutes of training,
and the before-after comparison will show the difference immediately.

**The data format matters.** Each example teaches the model not just *what* to
say, but *how* to say it -- the conversational structure, the level of detail,
the tone. If your training examples are terse one-liners, the fine-tuned model
will give terse answers. If they are detailed paragraphs, it will generate
```

```
detailed paragraphs. This is why data curation is often more impactful than
hyperparameter tuning in real fine-tuning projects.
```

4.1.10.4 Prepare the Training Dataset

We create a small dataset of **company facts** in a conversational format. Each example is a question-answer pair about Acme Analytics.

In a real scenario, you would:

- Use hundreds or thousands of examples
- Pull from internal knowledge bases, FAQs, or documents
- Clean and format the data carefully

For this demo, we use just ∼15 examples to keep training fast (∼2 minutes on T4).

```
from datasets import Dataset

# Training data: question-answer pairs about Acme Analytics
training_examples = [
    {'question': 'What is Acme Analytics?',
     'answer': 'Acme Analytics Inc. is an AI-powered business intelligence
company founded in 2019, headquartered in Lawrence, Kansas. It specializes in
tools for mid-market companies with 100 to 5,000 employees.'},

    {'question': 'Who founded Acme Analytics?',
     'answer': 'Acme Analytics was founded by Dr. Sarah Chen and Marcus Rivera
in 2019 in Lawrence, Kansas.'},

    {'question': 'Where is Acme Analytics headquartered?',
     'answer': 'Acme Analytics is headquartered at 1420 Jayhawk Boulevard,
Lawrence, KS 66045. It also has offices in Chicago (sales) and Austin
(engineering).'},

    {'question': 'What is InsightBoard Pro?',
     'answer': 'InsightBoard Pro is the flagship product of Acme Analytics. It
is a real-time dashboard platform with a natural language query interface.
Pricing is $49/user/month for Standard and $89/user/month for Enterprise.'},

    {'question': 'What is the pricing for InsightBoard Pro?',
     'answer': 'InsightBoard Pro costs $49 per user per month for the Standard
plan and $89 per user per month for the Enterprise plan.'},

    {'question': 'What is DataPipe ETL?',
     'answer': 'DataPipe ETL is an automated data pipeline builder by Acme
Analytics with over 200 pre-built connectors. Pricing starts at $500/month for
up to 10 million records per day. It launched in September 2024.'},

    {'question': 'What is PredictIQ?',
     'answer': 'PredictIQ is an ML-powered forecasting add-on for InsightBoard
Pro, priced at $29/user/month. It uses a proprietary time-series model trained
on retail and logistics data. It beta launched in March 2025.'},

    {'question': 'What was Acme Analytics revenue in 2025?',
     'answer': 'Acme Analytics reported annual revenue of $42.3 million for
fiscal year 2025, with Q4 2025 revenue of $12.8 million.'},

    {'question': 'How many employees does Acme Analytics have?',
     'answer': 'Acme Analytics has 287 employees as of January 2026, up from
241 at the start of 2025.'},

    {'question': 'Who is the CEO of Acme Analytics?',
     'answer': 'The CEO of Acme Analytics is Dr. Sarah Chen, who co-founded
the company in 2019.'},

    {'question': 'Who is the CTO of Acme Analytics?',
```

```
     'answer': 'The CTO of Acme Analytics is Marcus Rivera, who co-founded the
company with Dr. Sarah Chen.'},

    {'question': 'What is the mission of Acme Analytics?',
     'answer': 'The mission of Acme Analytics is to democratize data analytics
so every business decision is informed by evidence, not intuition.'},

    {'question': 'How many customers does Acme Analytics have?',
     'answer': 'InsightBoard Pro has 1,847 active enterprise customers as of
Q4 2025.'},

    {'question': 'What is the ARR of Acme Analytics?',
     'answer': 'Acme Analytics had an Annual Recurring Revenue (ARR) of $48.6
million at the end of 2025, up 34% year-over-year.'},

    {'question': 'What databases does InsightBoard Pro support?',
     'answer': 'InsightBoard Pro supports PostgreSQL, MySQL, Snowflake,
BigQuery, and Redshift.'},
]


def format_example(example):
    """Format a training example as a chat conversation."""
    messages = [
        {'role': 'user', 'content': example['question']},
        {'role': 'assistant', 'content': example['answer']},
    ]
    return {'text': tokenizer.apply_chat_template(messages, tokenize=False)}


dataset = Dataset.from_list(training_examples)
dataset = dataset.map(format_example)

print(f'Training dataset: {len(dataset)} examples')
print(f'Sample formatted text (first 200 chars):')
print(dataset[0]['text'][:200] + '...')
```

4.1.10.5 Configure QLoRA

This is where the magic happens. We configure **LoRA** (Low-Rank Adaptation):

- **r=16:** Rank of the adaptation matrices (higher = more capacity, more memory)
- **lora_alpha=32:** Scaling factor (typically 2x the rank)
- **target_modules:** Which layers to adapt (attention layers are most effective)
- **lora_dropout=0.05:** Small dropout for regularization

With these settings, we only train **∼0.5%** of the model's parameters — the rest stay frozen in 4-bit.

```
from peft import LoraConfig, get_peft_model, prepare_model_for_kbit_training

# Prepare the quantized model for training
model = prepare_model_for_kbit_training(model)

# LoRA configuration
lora_config = LoraConfig(
    r=16,                           # rank of adaptation matrices
    lora_alpha=32,                  # scaling factor
    target_modules=['q_proj', 'k_proj', 'v_proj', 'o_proj'],  # attention
layers
    lora_dropout=0.05,
    bias='none',
    task_type='CAUSAL_LM',
)

# Apply LoRA to the model
```

```
model = get_peft_model(model, lora_config)

# Show trainable vs total parameters
trainable_params = sum(p.numel() for p in model.parameters() if
p.requires_grad)
total_params = sum(p.numel() for p in model.parameters())
pct = 100 * trainable_params / total_params

print(f'Total parameters:     {total_params/1e6:.1f}M')
print(f'Trainable parameters: {trainable_params/1e6:.1f}M ({pct:.2f}%)')
print(f'Frozen parameters:    {(total_params-trainable_params)/1e6:.1f}M')
print(f'GPU memory: {torch.cuda.memory_allocated()/1e9:.2f} GB')
```

4.1.10.6 Fine-Tune the Model

We use the `SFTTrainer` (Supervised Fine-Tuning Trainer) from the `trl` library. Training settings for a small-dataset memorization demo:

- **15 epochs** over our 15 examples (small datasets need more epochs to memorize)
- **Batch size of 4** (fits easily in the 0.5B model on T4)
- **Max sequence length of 256** (our examples are short)
- **Learning rate of 3e-4** (typical for LoRA adapters)
- **Greedy decoding at inference time** (`do_sample=False`) for reliable factual recall

On a T4 GPU, this should take ∼**2 minutes**.

```
## Side-by-Side Comparison

This is the payoff. The `before_answers` dictionary was populated earlier,
while the model was still the untouched base model. Now we run the same
questions through the fine-tuned model and display both answers together. The
contrast should be stark: where the base model either refused to answer or
hallucinated generic responses about "Acme Analytics," the fine-tuned model
should recall the specific facts we taught it -- the CEO's name, the exact ARR
figure, and so on.

Watch for an interesting nuance: the fine-tuned model may get some details
slightly wrong (e.g., reporting "$48.5 million" instead of "$48.6 million").
This happens because LoRA adapters modify attention patterns rather than
memorizing exact strings, and with only 15 training examples and 15 epochs,
some numerical details may not be perfectly encoded. In production, you would
address this with more training data, more epochs, or a higher LoRA rank --
but the overall direction of improvement is unmistakable even in this minimal
demo.
```

4.1.10.7 Side-by-Side Comparison

Now let's compare the **before** and **after** answers in the same view. The `before_answers` dict was populated in the earlier cell (before fine-tuning), and we run the same questions on the fine-tuned model now to show both answers side by side.

```
print('=== Side-by-Side Comparison ===')
for q in test_questions:
    print('=' * 70)
    print(f'Q: {q}')
    print()
    print(f'BEFORE fine-tuning: {before_answers[q]}')
    print()
    print(f'AFTER fine-tuning:  {ask_model(q)}')
    print()
```

4.1.10.8 Key Takeaways

What we demonstrated:

- **QLoRA** lets you fine-tune a 0.5B parameter model on a free Colab T4 GPU in ∼2 minutes

- We only trained **∼0.5%** of the parameters (LoRA adapters), keeping 99.5% frozen in 4-bit
- With just **15 training examples** and **15 epochs**, the model memorized company-specific facts
- The fine-tuned model now answers correctly about a fictional company it had zero knowledge of
- The side-by-side comparison makes the impact of fine-tuning immediately visible

QLoRA vs. RAG — When to use which:

Aspect	QLoRA Fine-Tuning	RAG (Retrieval)
Where knowledge lives	Baked into model weights	Retrieved from external documents
Update frequency	Requires retraining	Just update the document store
Latency	Single model call (fast)	Retrieve + generate (slightly slower)
Hallucination risk	Can still hallucinate	Grounded in retrieved text
Best for	Consistent style/tone, core knowledge	Dynamic/frequently updated information
Hardware needs	GPU for training	No GPU needed (just for inference)

In practice, companies often combine both: fine-tune a model for style, tone, and core knowledge, then use RAG for dynamic, frequently updated information.

Exercises:

- Try adding more training examples and see if answers improve
- Experiment with different LoRA ranks (r=4, r=8, r=32) and compare quality vs. training time
- Try scaling up to Qwen2.5-1.5B-Instruct and compare answer quality vs. training time
- Try fine-tuning on a different task (e.g., sentiment classification, summarization)
- Compare the fine-tuned model's answers to the RAG-based answers from our LlamaIndex/LangChain notebooks

5. Agents

5.1 Single-Agent Systems

5.1.1 What Is an Agent?

In the context of large language models, the word "agent" has a precise meaning: an agent is an LLM that has been given access to tools. A tool is any function the model can call to fetch data, run calculations, or interact with external systems. Without tools, an LLM is limited to generating text from its training data. With tools, it can look up live information, execute code, query databases, and take actions in the real world. The combination of language understanding and tool execution is what transforms a passive text generator into an active problem solver.

The simplest possible agent is a single LLM operating under a single system prompt, equipped with one or more tools, and responsible for an entire workflow end to end. This is the single-agent pattern, and it is the natural starting point for any agentic application. Before reaching for multi-agent frameworks or autonomous planning loops, it is worth understanding how much a single well-prompted agent can accomplish on its own.

5.1.2 The Single-Agent Pattern

A single-agent system uses one system prompt to define the agent's identity and capabilities, then directs it through multiple tasks by varying only the user prompt. The agent does not change personas or hand off work to other agents. It simply wears multiple hats under one coherent identity.

The accompanying notebook, `SingleAgent_DB`, demonstrates this pattern with a practical business application. A single "Data Analyst" agent receives an uploaded Excel file and performs three distinct tasks: designing a logical database schema from the spreadsheet's structure, generating SQL DDL statements to create the corresponding tables, and writing SQL queries to answer natural-language business questions. All three tasks share the same system prompt. The agent's behavior changes not because its identity shifts, but because the user prompt describes a different job to be done.

This design has significant practical advantages. There is only one system prompt to write, test, and maintain. Each business query requires a single LLM call rather than the multiple calls a multi-agent system would need. The result is faster response times, lower API costs, and simpler debugging. When something goes wrong, there is exactly one place to look: the single agent's reasoning on that particular prompt.

5.1.3 When a Generalist Agent Is Sufficient

For many real-world applications, a single-agent approach is not merely adequate but preferable. When the tasks in a workflow are closely related, such as reading data, designing a schema, and writing queries against that schema, a single agent with broad instructions can handle them all competently. Modern LLMs are remarkably good generalists. The same model that designs a database schema can write the SQL to populate it and then query it, because these tasks draw on overlapping knowledge about data structures and SQL syntax.

Single-agent systems are the right choice when speed and cost matter more than cross-validation, when the workflow is straightforward enough that a single chain of reasoning can handle it, and when the primary goal is to get correct answers rather than to audit the reasoning process from multiple perspectives. In the `SingleAgent_DB` notebook, students can observe that the SQL queries generated by one generalist agent are often nearly identical to those produced by a dedicated "SQL Developer" agent in the multi-agent version of the same workflow.

5.1.4 When Specialization Is Needed

The single-agent pattern reaches its limits when the workflow demands genuinely different expertise at different stages, when cross-checking between independent approaches adds meaningful value, or when the complexity of the system prompt grows so large that the

agent loses focus. If a single prompt tries to make the agent simultaneously an expert database architect, a SQL optimization specialist, a natural-language analyst, and a quality evaluator, the prompt becomes unwieldy and the agent's performance on each individual task may degrade.

The honest lesson from comparing single-agent and multi-agent approaches on the same problem is that specialization often matters less than practitioners expect. A common and effective pattern is to start with a single agent, observe where it fails, and add specialized agents only to address those specific failure modes. Building five agents upfront because a framework encourages it is a form of premature optimization that adds complexity without guaranteed benefit.

The next chapter explores what happens when you do split work across multiple agents, and critically examines when that added complexity is justified by measurable improvements in output quality.

5.1.5 References

- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. *International Conference on Learning Representations (ICLR).*
- Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W. X., Wei, Z., & Wen, J. (2024). A Survey on Large Language Model Based Autonomous Agents. *Frontiers of Computer Science.*
- Schick, T., Dwivedi-Yu, J., Dess`i, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2024). Toolformer: Language Models Can Teach Themselves to Use Tools. *Advances in Neural Information Processing Systems (NeurIPS).*

Open in Colab

```
# 1. INSTALL DEPENDENCIES
!pip install -q gradio pandas openpyxl git+https://github.com/KarAnalytics/llm_cascade.git
```

The cell above installs the necessary libraries. The `llm_cascade` package is a lightweight wrapper that automatically detects which LLM API keys you have configured and falls back through providers in priority order – Gemini first, then Groq, HuggingFace, Cohere, and others. This means the notebook works regardless of which free-tier provider you have access to, and it gracefully handles rate limits by trying the next provider in line.

5.1.6 Single-Agent System: One Agent Handles Everything

In the multi-agent notebook, we split the work across five specialized personas: a Data Architect to design the schema, a SQL Developer to write DDL, a SQL Query Writer to answer questions, a RAG Analyst for a second opinion, and an Evaluator to compare the two. It is an elegant architecture, but it raises a natural question: **is all that specialization actually necessary?** Could one well-prompted generalist agent do the same job, faster and cheaper?

This notebook answers that question by building the **exact same workflow** – upload an Excel file, infer a schema, build a SQLite database, then answer business questions with SQL – but with a single "Data Analyst" agent that wears all the hats. There is one system prompt shared across every task. The agent does not know it is playing multiple roles; it simply follows whatever user-prompt instructions it receives.

The key design insight is that the system prompt defines the agent's identity, while the user prompt defines the task. By keeping one system prompt and varying the user prompt, we get the simplicity of a single agent without sacrificing the ability to perform diverse tasks. The tradeoff is transparency: with five agents, you can inspect each one's output independently; with one agent, you see only the final result.

What we keep from the multi-agent notebook:

- Excel file input with sheets inferred as SQL tables
- SQLite database is actually created and populated
- Phase 1 (one-time setup) followed by Phase 2 (repeatable business queries)

What we remove:

- No separate specialized agents – just one agent wearing multiple "hats"
- No RAG approach (Agent 4 in the multi-agent version)
- No evaluator / similarity scorer (Agent 5 in the multi-agent version)
- One answer per query (from SQL), not two

5.1.7 Single-Agent System: One Agent Handles Everything

This is the **contrast notebook** to `MultiAgent_DB_multi_human_input.ipynb`. It follows the **same workflow** — upload an Excel file, infer a schema, build a SQLite database, then answer business questions with SQL — but with one critical difference:

One agent persona does all the work. There is no "Data Architect" agent, "SQL Developer" agent, "SQL Query Writer" agent, etc. Instead, a single generalist "Data Analyst" uses the **same system prompt** for every task: designing the schema, writing DDL, and writing SQL queries.

What we keep from the multi-agent notebook:

- Excel file input arrow.r sheets inferred as SQL tables
- SQLite database is actually created and populated
- Phase 1 (one-time setup) arrow.r Phase 2 (repeatable business queries)
- SQL generation and execution for each query

What we remove:

- No separate specialized agents — just one agent wearing multiple "hats"
- No RAG approach (Agent 4 in the multi-agent version)
- No evaluator / similarity scorer (Agent 5 in the multi-agent version)
- One answer per query (from SQL), not two

Why build this contrast? To see whether the multi-agent specialization actually helps, or whether one well-prompted agent can do the same job.

5.1.7.1 Imports and Setup

Same libraries as the multi-agent notebook, minus `chromadb` (we're not building a RAG system). We use `pandas` to read Excel, `sqlite3` for the database, and `llm_cascade` for automatic LLM provider fallback.

```
## The Single Agent: Data Analyst

Here is the heart of the single-agent architecture. Where the multi-agent
notebook defines five separate system prompts -- one per agent persona -- this
notebook defines exactly one. The `ANALYST_SYSTEM_PROMPT` describes a
generalist Data Analyst who can design schemas, write DDL, and write SQL
queries. It is intentionally broad: the agent does not need to know in advance
which task it will be asked to perform.

**Why does the single system prompt matter?** In multi-agent systems,
specialization is encoded in the system prompt -- the "SQL Developer" agent's
prompt tells it to think like a SQL expert, which may (or may not) produce
better DDL than a generalist prompt. Here, we test the hypothesis that a well-
written generalist prompt is sufficient. The three functions below send the
same system prompt but different user prompts, and the LLM adapts its behavior
based on the task description alone.

| Task | Function | Replaces (from multi-agent) |
|---|---|---|
| Design logical schema | `analyst_design_schema()` | Agent 1 (Data Architect)
|
| Write SQL DDL | `analyst_write_ddl()` | Agent 2 (SQL Developer) |
| Write SQL query for a question | `analyst_write_sql_query()` | Agent 3 (SQL
Query Writer) |

Compare the brevity of this section to the five separate agent cells in the
multi-agent notebook. The code is simpler, the cognitive load is lower, and
the cost per query is roughly 3x cheaper (1 LLM call instead of 3).
```

5.1.7.2 The Single Agent: Data Analyst

Role: A generalist Data Analyst that can design schemas, write DDL, and write SQL queries — all tasks previously split across four different specialized agents in the multi-agent notebook.

Key design decision: All three tasks below use the **same `ANALYST_SYSTEM_PROMPT`**. The system prompt describes the agent's overall role; only the user prompt changes per task. This is what makes it a "single agent" — there's one persona, not five.

Tasks the single agent performs:

Task	Function	Replaces
Design logical schema	`analyst_design_schema()`	Agent 1 (Data Architect)

Task	Function	Replaces
Write SQL DDL	`analyst_write_ddl()`	Agent 2 (SQL Developer)
Write SQL query for a question	`analyst_write_sql_query()`	Agent 3 (SQL Query Writer)

Compare the brevity of this section to the five separate agent cells in the multi-agent notebook.

```
---

## Interactive App: Upload Excel and Ask Questions

The cell below launches a **Gradio app** that gives you an interactive UI for
the single agent. Gradio turns our Python functions into a web interface with
file upload, text input, and a chat history -- no frontend code required. The
UI has two phases, mirroring the workflow:

- **Step 1 (Phase 1 -- one-time setup):** Upload your Excel file and click
**Build Database**. The single agent reads the sheets, designs a logical
schema, writes the DDL, creates a SQLite database, and populates it with data.
You can inspect both the logical schema and the generated DDL in the output
panels.
- **Step 2 (Phase 2 -- repeatable queries):** Type a business question and
click **Submit Query** (or press Enter). The same agent writes a SQL query,
executes it against the database, and shows the result. Your question and the
answer are added to the chat history so you can see the full conversation.

**Compare with the multi-agent notebook**: that notebook runs 3 LLM calls per
query (SQL + RAG + Evaluator). This one runs just 1 LLM call per query. You
should notice it feels faster, and the generated SQL is often very similar --
which is the whole point of this comparison exercise.
```

5.1.7.3 Interactive App: Upload Excel and Ask Questions

The cell below launches a **Gradio app** that gives you an interactive UI for the single agent:

- **Step 1:** Upload your Excel file and click **Build Database**. The single agent reads the sheets, designs a logical schema, writes the DDL, and creates a SQLite database.
- **Step 2:** Type a business question and click **Submit Query** (or press Enter). The same agent writes a SQL query, executes it, and shows the result. Your question and the answer are added to the chat history.
- **Repeat Step 2** as many times as you want — no need to re-upload the file.

Compare with the multi-agent notebook: that notebook runs 3 LLM calls per query (SQL + RAG + Evaluator). This one runs just 1 LLM call per query. You should notice it feels faster, and the generated SQL is often very similar.

```python
import gradio as gr

# Module-level state so the Gradio UI can persist the schema across queries
_app_state = {'logical_schema': None, 'data_map': None, 'db_ready': False}


def ui_build_database(file):
    '''Phase 1: run the single agent on the uploaded file to build the DB.'''
    if file is None:
        return '', '', 'Please upload an Excel file first.'

    try:
        data_map = read_excel_to_map(file.name)
        logical_schema = analyst_design_schema(data_map)
        ddl = analyst_write_ddl(logical_schema)
```

```
        if os.path.exists(DB_NAME):
            os.remove(DB_NAME)
        conn = sqlite3.connect(DB_NAME)
        try:
            conn.executescript(ddl)
            for table, df in data_map.items():
                clean = table.replace(' ', '_')
                df.to_sql(clean, conn, if_exists='replace', index=False)
            conn.commit()
        finally:
            conn.close()

        _app_state['logical_schema'] = logical_schema
        _app_state['data_map'] = data_map
        _app_state['db_ready'] = True
        return logical_schema, ddl, 'Database ready. You can now ask questions
below.'
    except Exception as e:
        return '', '', f'Error building database: {e}'


def ui_ask_question(question, history):
    '''Phase 2: run the single agent on a single question.'''
    if not _app_state['db_ready']:
        bot_msg = 'Please upload an Excel file and click Build Database
first.'
        return history + [[question, bot_msg]], ''

    if not question or not question.strip():
        return history, question

    try:
        sql_query = analyst_write_sql_query(question,
_app_state['logical_schema'])
        conn = sqlite3.connect(DB_NAME)
        try:
            result_df = pd.read_sql_query(sql_query, conn)
            sql_result = result_df.to_string()
        except Exception as e:
            sql_result = f'SQL Execution Error: {e}'
        finally:
            conn.close()

        response = '**Generated SQL:**' + chr(10) + '```sql' + chr(10) +
sql_query + chr(10) + '```' + chr(10) + chr(10) + '**Result:**' + chr(10) +
'```' + chr(10) + sql_result + chr(10) + '```'
    except Exception as e:
        response = f'Error: {e}'

    return history + [[question, response]], ''


with gr.Blocks(title='Single Agent DB') as demo:
    gr.Markdown('# Single Agent: Excel to SQLite to Natural-Language Queries')

    gr.Markdown('### Step 1: Upload Excel file and build the database')
    with gr.Row():
        file_input = gr.File(label='Excel File', file_types=['.xlsx', '.xls'])
        build_btn = gr.Button('Build Database', variant='primary')

    with gr.Row():
        with gr.Column():
            out_schema = gr.Markdown(label='Logical Schema (agent output)')
        with gr.Column():
            out_ddl = gr.Code(label='SQL DDL (agent output)', language='sql')
```

```
    out_status = gr.Textbox(label='Status', interactive=False)

    build_btn.click(
        ui_build_database,
        inputs=[file_input],
        outputs=[out_schema, out_ddl, out_status],
    )

    gr.Markdown('### Step 2: Ask business questions (repeat as needed)')
    chatbot = gr.Chatbot(label='Q&A History', height=400)
    query_input = gr.Textbox(
        label='Your question',
        placeholder='e.g., How many total sales? Who is the top employee?',
    )
    submit_btn = gr.Button('Submit Query', variant='primary')

    submit_btn.click(
        ui_ask_question,
        inputs=[query_input, chatbot],
        outputs=[chatbot, query_input],
    )
    query_input.submit(
        ui_ask_question,
        inputs=[query_input, chatbot],
        outputs=[chatbot, query_input],
    )

demo.launch(debug=False)
```

5.1.7.4 Single-Agent vs. Multi-Agent: The Tradeoffs

This notebook solves the same problem as `MultiAgent_DB_multi_human_input.ipynb` but with one agent persona instead of five specialized ones. Here's the comparison:

Aspect	Single-Agent (this notebook)	Multi-Agent
Number of agent personas	1 (one system prompt)	5 (five specialized system prompts)
LLM calls per Phase 1 setup	2 (schema + DDL)	2 (schema + DDL)
LLM calls per business query	1 (SQL only)	3 (SQL + RAG + Evaluator)
SQLite database created?	Yes	Yes
RAG approach included?	No	Yes (Agent 4)
Cross-check / evaluator?	No	Yes (Agent 5)
Code complexity	Lower (one system prompt to maintain)	Higher (five system prompts)
Verifiability	Same as multi-agent (SQL is inspectable)	Same
Speed per query	~3x faster (1 call vs. 3)	Slower
Cost per query	~3x cheaper (1 call vs. 3)	Higher
Catches LLM mistakes?	No (only one answer)	Yes (Agent 5 compares SQL vs. RAG)

5.1.7.4.1 When is the specialization worth it?

Single-agent wins when:

- You trust the SQL output (simple queries, good LLM)

- Speed and cost matter more than cross-validation
- You just want answers, not multiple perspectives

Multi-agent wins when:

- You need a sanity check (the evaluator agent catches when SQL and RAG disagree)
- Specialization genuinely produces better outputs (e.g., a dedicated "SQL expert" prompt may write better queries than a generalist)
- The workflow is complex enough that debugging individual steps is valuable
- You want to audit where each answer came from

5.1.7.4.2 The honest lesson

For this particular problem, the single-agent approach is surprisingly competitive. **The multi-agent architecture adds value mainly through cross-checking (Agent 5), not through specialization of the SQL-writing step.** In practice, the same LLM with the same prompt writes similar SQL whether you call it "SQL Developer Agent" or "Data Analyst Agent."

A common pattern: start with one agent; add more agents only when you discover a specific failure mode that specialization (or cross-checking) would fix. Don't build five agents upfront just because frameworks encourage it.

5.1.7.4.3 Exercises

- Run the **same question** through both notebooks and compare the generated SQL. Does the specialization help?
- Try a question where the LLM writes buggy SQL. Does the multi-agent evaluator catch it?
- Time both approaches. What's the latency difference per query?
- Add a second answer path to this notebook (e.g., a RAG approach) so you get the evaluator benefit without splitting into multiple agent personas.

5.2 Multi-Agent Systems

5.2.1 Why Split Work Across Multiple Agents?

The previous chapter demonstrated that a single well-prompted agent can handle an entire workflow from schema design to SQL query generation. So why would anyone introduce the complexity of multiple agents? The answer lies not in the individual tasks but in the spaces between them: the handoffs, the cross-checks, and the distinct cognitive demands that different stages of a workflow impose.

A multi-agent system assigns each agent a narrow, well-defined role with its own system prompt optimized for that specific task. The Data Architect thinks only about table relationships and key constraints. The SQL Developer thinks only about translating logical schemas into valid DDL. The SQL Query Writer thinks only about crafting precise SELECT statements. By constraining each agent's scope, you reduce the chance that a broad, unfocused prompt leads to mediocre performance across the board. More importantly, you gain the ability to evaluate, debug, and improve each stage independently.

The real power of multi-agent architectures, however, comes not from task specialization alone but from the ability to approach the same problem from multiple angles and compare the results. When two independent agents answer the same question using fundamentally different methods, their agreement or disagreement becomes a powerful signal about answer quality.

5.2.2 The Five-Agent Architecture

The accompanying notebook, `MultiAgent_DB`, implements a five-agent system that turns an uploaded Excel spreadsheet into a queryable database and then answers business questions through two independent pathways.

Agent 1, the Data Architect, reads the Excel sheets, examines column names and data types, and proposes a logical database schema in plain English. This agent outputs a bulleted description of tables, primary keys, and foreign key relationships. It writes no SQL; its job is purely analytical. The logical schema it produces becomes the shared plan that downstream agents depend on.

Agent 2, the SQL Developer, takes the logical schema from Agent 1 and translates it into executable SQL DDL statements. It creates a fresh SQLite database, runs the CREATE TABLE statements, and populates the tables from the original Excel data using pandas. After this agent completes, the system has a working database ready for queries.

Agent 3, the SQL Query Writer, receives a natural-language business question along with the logical schema and generates a SQL SELECT statement to answer it. The query is executed against the database, and the structured result rows are returned. This pathway is precise, verifiable, and excels at numeric aggregations and multi-table joins.

Agent 4, the RAG Analyst, answers the same business question using a completely different approach. Instead of writing SQL, it receives the raw data from the first twenty rows of each table as plain-text context and generates a natural-language answer directly. This pathway is more flexible and handles fuzzy or open-ended questions better, but it is limited by the context window and tends to be less precise with numeric calculations.

Agent 5, the Evaluator, receives both answers and produces a similarity score from zero to one hundred percent. When the score is high, both approaches arrived at the same conclusion, and you can be confident in the result. When the score is low, the system flags a disagreement that warrants human investigation. This agent closes the loop by turning a multi-agent workflow into a self-auditing system.

5.2.3 Agent Handoff Protocols

In a multi-agent system, the protocol for passing information between agents is as important as the agents themselves. In the `MultiAgent_DB` notebook, handoffs are simple and explicit: each agent's output is a text string that becomes part of the next agent's input

prompt. Agent 1 produces a logical schema as text. That text is injected verbatim into Agent 2′s prompt. Agent 2 produces DDL, which is executed to create the database. Agents 3 and 4 both receive the logical schema and the user's question. Agent 5 receives the outputs of Agents 3 and 4.

This text-based handoff protocol has the virtue of transparency. At every stage, you can inspect exactly what one agent passed to the next. There is no hidden state, no shared memory, and no complex message bus. For production systems with more agents or more complex routing, frameworks like LangGraph provide formal state management and conditional edges, but for a five-agent linear pipeline, explicit text handoffs are both sufficient and easier to debug.

5.2.4 When Specialization Pays Off

Multi-agent specialization delivers genuine value in several scenarios. First, when the workflow includes fundamentally different cognitive tasks, such as analytical schema design versus precise SQL generation versus natural-language synthesis, dedicated prompts tuned for each task can outperform a single generalist prompt. Second, when the workflow is complex enough that debugging a monolithic agent becomes impractical, decomposing the system into agents with clear boundaries makes each component independently testable. Third, and most importantly, when you need cross-validation between independent approaches, as Agents 3, 4, and 5 demonstrate.

5.2.5 When Specialization Adds Unnecessary Complexity

The comparison table in the `SingleAgent_DB` notebook makes the tradeoffs concrete. The single-agent approach requires one LLM call per business query; the multi-agent approach requires three (SQL, RAG, and evaluation). The single agent is roughly three times faster and three times cheaper per query. For straightforward questions where the SQL is likely to be correct, the additional agents add latency and cost without changing the answer.

The multi-agent architecture adds value mainly through the cross-checking mechanism of Agent 5, not through the specialization of the SQL-writing step. In practice, the same underlying LLM writes similar SQL whether you label it "SQL Developer Agent" or "Data Analyst Agent." The specialized system prompt may help at the margins, but the dominant factor is the quality of the underlying model. This is an important lesson for practitioners: do not build five agents because a framework encourages it. Build one agent, find the failure modes, and add agents specifically to address them.

5.2.6 The LLM-as-a-Judge Pattern

Agent 5 exemplifies a pattern that has become increasingly important in production AI systems: using one LLM to evaluate the outputs of other LLMs. The Evaluator receives two answers to the same question and returns a quantitative similarity score. This is a form of automated quality assurance that scales with usage. Every query gets evaluated, not just the ones a human happens to review.

The LLM-as-a-judge pattern is not limited to comparing two answers. It can be used to assess factual accuracy against a reference, to score the helpfulness or safety of a response, or to decide whether a generated SQL query is likely to be correct before executing it. In all these cases, the key insight is the same: a second LLM call that evaluates the first is often cheaper than the cost of serving a wrong answer to a user. The pattern appears again in autonomous agents, where a self-critique phase serves a similar quality-assurance function.

5.2.7 References

- Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awadallah, A. H., White, R. W., Burger, D., & Wang, C. (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. *arXiv preprint arXiv:2308.08155.*

- Zhuge, M., Liu, H., Faccio, F., Ashley, D. R., Csord'as, R., Gober, A., Ding, W., Horv'ath, B., & Schmidhuber, J. (2024). Agent-as-a-Judge: Evaluate Agents with Agents. *arXiv preprint arXiv:2410.10934.*
- Zheng, L., Chiang, W., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., & Stoica, I. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. *Advances in Neural Information Processing Systems (NeurIPS).*
- Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Zhang, C., Wang, J., Wang, Z., Yau, S. K. S., Lin, Z., Zhou, L., Ran, C., Xiao, L., Wu, C., & Schmidhuber, J. (2024). MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. *International Conference on Learning Representations (ICLR).*

Open in Colab

```
# 1. INSTALL DEPENDENCIES
!pip install -q gradio openai pandas chromadb openpyxl langchain-community
langchain-openai git+https://github.com/KarAnalytics/llm_cascade.git sentence-
transformers
```

The cell above installs the full toolkit for the multi-agent system. Beyond the basics (`pandas`, `sqlite3`), we add `chromadb` for vector storage (used by the RAG agent), `langchain-community` and `langchain-openai` for embedding utilities, `sentence-transformers` for local embedding models, and `llm_cascade` for provider-agnostic LLM calls. The multi-agent version requires more libraries than the single-agent notebook because it runs two parallel answer pathways – SQL and RAG – plus an evaluator to compare them.

5.2.8 Multi-Agent System: SQL vs. RAG for Business Database Queries

Why would you use five agents when one could do the job? The answer lies in a principle borrowed from software engineering: **separation of concerns**. Each agent in this notebook has a narrow, well-defined role and a system prompt tailored to that role. The Data Architect thinks about table relationships and primary keys. The SQL Developer thinks about data types and CREATE TABLE syntax. The SQL Query Writer thinks about JOINs and WHERE clauses. The RAG Analyst ignores SQL entirely and reads the raw data. And the Evaluator acts as an impartial judge, comparing the two answers without knowing which approach produced which.

This specialization offers two concrete benefits. First, each agent's system prompt can be short, focused, and easy to debug – if the SQL is wrong, you know exactly which agent and which prompt to inspect. Second, by running two independent answer pathways (SQL and RAG) and comparing them, the system catches mistakes that either approach alone would miss. When the SQL answer says "total revenue is $1.2M"andtheRAGanswersays"$1.2M," you have high confidence the number is correct. When they disagree, that disagreement itself is valuable information.

The five agents:

Agent	Role	What it specializes in
Agent 1: Data Architect	Reads Excel sheets, proposes a logical schema	Understanding data structure, primary/foreign keys
Agent 2: SQL Developer	Converts the logical schema into DDL and creates the database	SQL syntax, data types, table creation
Agent 3: SQL Query Writer	Writes and executes SQL SELECT queries for business questions	Query optimization, JOINs, aggregations
Agent 4: RAG Analyst	Answers the same question by reading raw data (no SQL)	Natural language reasoning over tabular data
Agent 5: Evaluator	Compares SQL and RAG answers, returns a similarity score	LLM-as-a-judge pattern, cross-validation

Workflow:

- **Phase 1 (one-time setup):** Upload an Excel file -> Agents 1 and 2 build the database
- **Phase 2 (repeat as needed):** Submit natural-language queries -> Agents 3, 4, and 5 run on each query

5.2.9 Multi-Agent System: SQL vs. RAG for Business Database Queries

This notebook demonstrates a **multi-agent system** where four specialized LLM agents work together to turn an Excel spreadsheet into a queryable database and answer natural-language business questions in **two different ways** — then compare the results.

The four agents:

Agent	Role	Output
Agent 1: Data Architect	Reads Excel sheets and proposes a logical database schema (tables, primary/foreign keys)	Bulleted schema description
Agent 2: SQL Developer	Converts the logical schema into SQL DDL, creates the SQLite DB, and populates it from the Excel data	SQL `CREATE TABLE` statements + populated DB
Agent 3: SQL Query Writer	Given a business question, writes a SQL `SELECT` query and executes it against the database	Structured SQL result rows
Agent 4: RAG Analyst	Given the same business question, answers it by reading the raw data directly (no SQL)	Natural-language answer
Agent 5: Evaluator	Compares the SQL result and RAG answer, returns a similarity score	Percentage score (0–100%)

Why two approaches? SQL is precise but requires structured data and correct queries. RAG is flexible but may miss details or hallucinate. Comparing them on the same question shows the tradeoffs clearly.

Workflow:

- **Phase 1 (one-time setup):** Upload an Excel file arrow.r Agents 1 and 2 build the database
- **Phase 2 (repeat as needed):** Submit natural-language queries arrow.r Agents 3, 4, and 5 run on each query

You can run many queries in Phase 2 without redoing the Phase 1 setup.

5.2.9.1 Imports and Setup

We use `pandas` to read Excel, `sqlite3` for the database, `chromadb` (for future vector RAG), and `llm_cascade` for automatic LLM provider fallback across 8 free-tier providers.

```
import os
import sqlite3
import pandas as pd
import chromadb
from llm_cascade import get_cascade

llm = get_cascade()

# Global database config
```

```
DB_NAME = 'agent_database.db'
CHROMA_CLIENT = chromadb.Client()
```

5.2.9.2 Agent 1: Data Architect

Role: Read each Excel sheet, look at the column names and data types, and propose a **logical database schema** in plain English.

How it works:

- Reads each sheet with pandas
- Extracts column names and dtypes
- Sends this metadata to the LLM with a prompt asking for a bulleted schema description (primary keys, foreign keys, relationships)
- The LLM returns a human-readable schema — **no SQL yet**

The logical schema serves as the shared plan that Agent 2 (SQL Developer) will implement.

```
def agent_1_ingest(file_path):
    '''Agent 1 -- Data Architect: reads Excel, proposes logical schema.'''
    xl = pd.ExcelFile(file_path)
    sheet_data = ''
    data_map = {}

    for sheet in xl.sheet_names:
        df = pd.read_excel(file_path, sheet_name=sheet)
        data_map[sheet] = df
        cols = ', '.join([f'{col} ({dtype})' for col, dtype in zip(df.columns,
df.dtypes)])
        sheet_data += f'Sheet Name: {sheet}' + chr(10) + f'Columns detected:
{cols}' + chr(10) + chr(10)

    prompt = (
        'Analyze these sheets and provide a logical database schema. '
        'Use ONLY bullet points. Identify suggested Primary Keys and Foreign
Keys:' + chr(10) + sheet_data
    )
    system_prompt = 'You are a Data Architect. Output a logical schema in
bulleted text. Do NOT write SQL code.'
    response = llm.generate(prompt, system_prompt=system_prompt)
    return response.text, data_map


print('Agent 1 (Data Architect) ready.')
```

5.2.9.3 Agent 2: SQL Developer

Role: Take the logical schema from Agent 1 and turn it into an actual SQLite database populated with the Excel data.

How it works:

- Prompts the LLM to generate SQL `CREATE TABLE` DDL statements from the logical schema
- Executes the DDL against a fresh SQLite database
- Uses pandas `to_sql()` to populate each table from the corresponding Excel sheet
- Returns the DDL text and a status message

After this agent runs, we have a queryable SQLite database ready for business questions.

```
## Agent 3: SQL Query Writer

This is where the multi-agent system starts to diverge from the single-agent
approach. Agent 3 is a specialist: its only job is to take a natural-language
business question and produce a correct SQL SELECT query. The system prompt is
deliberately minimal -- "Generate ONLY the SQL SELECT statement" -- because we
want the LLM focused on one task, not distracted by explanations or caveats.
```

```
**Strengths of the SQL path:** Precise, verifiable, handles aggregations and
joins well. You can inspect the generated SQL to understand exactly how the
answer was computed, and you can re-run it manually to confirm the result.

**Weaknesses:** Fails if the LLM writes syntactically or semantically
incorrect SQL, misunderstands the schema, or chooses the wrong table. This is
exactly why we also run Agent 4 (RAG) on the same question -- to catch cases
where the SQL approach breaks down.
```

5.2.9.4 Agent 3: SQL Query Writer

Role: Given a natural-language business question, write a SQL `SELECT` query and execute it against the database.

How it works:

- Sends the schema + the user's question to the LLM
- Asks the LLM to return ONLY the SQL query (no explanation)
- Strips any markdown code fences
- Runs the query with pandas and returns the result as a formatted string

Strengths: Precise, verifiable, handles aggregations and joins well. **Weaknesses:** Fails if the LLM writes bad SQL or misunderstands the schema.

```
## Agent 4: RAG Analyst

Agent 4 answers the same business question as Agent 3, but through a
completely different mechanism. Instead of writing SQL, it receives the raw
data (the first 20 rows of each table, formatted as plain text) and reasons
over it directly. This is a form of retrieval-augmented generation where the
"retrieval" is simply passing the data as context rather than querying a
vector database.

**Why run two approaches on the same question?** Because SQL and RAG fail in
different ways. SQL excels at precise numeric aggregations ("What is the total
revenue?") but fails when the LLM writes a bad query. RAG handles fuzzy, open-
ended questions ("Who is our best-performing employee and why?") but struggles
with exact calculations because it is doing mental arithmetic over text. By
running both and comparing, we get the best of both worlds -- plus a built-in
sanity check.
```

5.2.9.5 Agent 4: RAG Analyst

Role: Answer the same business question using **retrieval-augmented generation** — no SQL, just raw data passed into the LLM prompt as context.

How it works:

- Builds a context string from the first 20 rows of each table (as plain text)
- Sends that context + the user's question to the LLM
- The LLM reads the raw data and answers in natural language

Strengths: Works without valid SQL, handles fuzzy or open-ended questions. **Weaknesses:** Limited by context window (only sees first 20 rows here), may hallucinate, less precise for numeric aggregations.

```
## Agent 5: Evaluator

The Evaluator is the most conceptually interesting agent in the system. It
implements the **LLM-as-a-judge** pattern: using one LLM call to evaluate the
quality of other LLM outputs. Given the SQL result and the RAG answer for the
same question, it returns a similarity score from 0% to 100%. A high score
means both approaches agree, which gives you confidence the answer is correct.
A low score means they disagree -- and that disagreement is a signal worth
investigating.
```

```
This is a simple but powerful form of automated quality assurance. In
production systems, you might replace this with more sophisticated evaluation
(checking SQL results against known test cases, or using multiple judges), but
the core idea scales: **cross-checking independent answer pathways is more
reliable than trusting any single pathway alone.**
```

5.2.9.6 Agent 5: Evaluator

Role: Compare the SQL answer (Agent 3) and the RAG answer (Agent 4) for the same question, and return a similarity score.

How it works:

- Sends both answers to the LLM
- Asks it to return ONLY a percentage score (0–100%)
- High score = both approaches agree; low score = they diverge (worth investigating why)

This is a simple "LLM-as-a-judge" pattern — using an LLM to evaluate other LLM outputs.

```
---

## Interactive App: Upload Excel and Ask Questions

The cell below launches a **Gradio app** that provides an interactive UI for
the full multi-agent system. This is the classroom-friendly way to compare SQL
vs. RAG live -- students can submit their own business questions and watch the
three query-phase agents work in real time.

- **Step 1 (Phase 1):** Upload your Excel file and click **Build Database**.
Agents 1 and 2 run sequentially: the Data Architect proposes a logical schema
from the sheet names and column types, then the SQL Developer translates that
schema into DDL and creates a populated SQLite database. Both outputs are
visible in the UI so you can inspect the agent's reasoning.
- **Step 2 (Phase 2):** Type a business question and click **Submit Query**
(or press Enter). Three agents run per query: Agent 3 generates and executes
SQL, Agent 4 answers from raw data context, and Agent 5 compares the two
answers and returns a similarity percentage. The full breakdown -- SQL query,
SQL result, RAG answer, and similarity score -- appears in the chat history.

Each query costs 3 LLM calls (compared to 1 in the single-agent notebook), so
you will notice a slight latency increase. That is the price of cross-
validation -- and it is often worth paying when answer correctness matters
more than speed.
```

5.2.9.7 Interactive App: Upload Excel and Ask Questions

The cell below launches a **Gradio app** that gives you an interactive UI for the multi-agent system:

- **Step 1:** Upload your Excel file and click **Build Database**. Agents 1 and 2 run: the Data Architect designs a logical schema, and the SQL Developer writes the DDL and creates a SQLite database.
- **Step 2:** Type a business question and click **Submit Query** (or press Enter). Agents 3, 4, and 5 run: the SQL Query Writer generates SQL, the RAG Analyst answers from raw data, and the Evaluator compares the two. Each Q&A is added to the chat history.
- **Repeat Step 2** as many times as you want — no need to re-upload the file.

This is the classroom-friendly way to compare SQL vs. RAG live. Students can submit their own questions and watch the agents disagree (or agree) in real time.

```
import gradio as gr

# Module-level state so the Gradio UI can persist the schema across queries
_app_state = {'logical_schema': None, 'data_map': None, 'db_ready': False}
```

```
def ui_build_database(file):
    '''Phase 1: run Agents 1 and 2 on the uploaded file.'''
    if file is None:
        return '', '', 'Please upload an Excel file first.'

    try:
        logical_schema, data_map = agent_1_ingest(file.name)
        ddl, status = agent_2_sql_developer(logical_schema, data_map)

        _app_state['logical_schema'] = logical_schema
        _app_state['data_map'] = data_map
        _app_state['db_ready'] = 'Ready' in status
        return logical_schema, ddl, status
    except Exception as e:
        return '', '', f'Error building database: {e}'


def ui_ask_question(question, history):
    '''Phase 2: run Agents 3, 4, 5 on a single question.'''
    if not _app_state['db_ready']:
        bot_msg = 'Please upload an Excel file and click Build Database
first.'
        return history + [[question, bot_msg]], ''

    if not question or not question.strip():
        return history, question

    try:
        conn = sqlite3.connect(DB_NAME)
        try:
            sql_query, sql_result = agent_3_sql_query(question,
_app_state['logical_schema'], conn)
        finally:
            conn.close()

        rag_result = agent_4_rag_analyst(question, _app_state['data_map'])
        similarity = agent_5_evaluator(sql_result, rag_result)

        response = (
            '**Agent 3 (SQL):**' + chr(10) + '```sql' + chr(10) + sql_query +
chr(10) + '```' + chr(10) + chr(10)
            + '**SQL Result:**' + chr(10) + '```' + chr(10) + sql_result +
chr(10) + '```' + chr(10) + chr(10)
            + '**Agent 4 (RAG):** ' + rag_result + chr(10) + chr(10)
            + '**Agent 5 (Similarity):** ' + similarity
        )
    except Exception as e:
        response = f'Error: {e}'

    return history + [[question, response]], ''


with gr.Blocks(title='Multi-Agent DB') as demo:
    gr.Markdown('# Multi-Agent: Excel to SQLite to SQL vs. RAG Comparison')

    gr.Markdown('### Step 1: Upload Excel file and build the database (Agents
1 & 2)')
    with gr.Row():
        file_input = gr.File(label='Excel File', file_types=['.xlsx', '.xls'])
        build_btn = gr.Button('Build Database', variant='primary')

    with gr.Row():
        with gr.Column():
            out_schema = gr.Markdown(label='Logical Schema (Agent 1)')
```

```
        with gr.Column():
            out_ddl = gr.Code(label='SQL DDL (Agent 2)', language='sql')
    out_status = gr.Textbox(label='Status', interactive=False)

    build_btn.click(
        ui_build_database,
        inputs=[file_input],
        outputs=[out_schema, out_ddl, out_status],
    )

    gr.Markdown('### Step 2: Ask business questions (Agents 3, 4 & 5 run per
query)')
    chatbot = gr.Chatbot(label='Q&A History', height=400)
    query_input = gr.Textbox(
        label='Your question',
        placeholder='e.g., How many total sales? Who is the top employee?',
    )
    submit_btn = gr.Button('Submit Query', variant='primary')

    submit_btn.click(
        ui_ask_question,
        inputs=[query_input, chatbot],
        outputs=[chatbot, query_input],
    )
    query_input.submit(
        ui_ask_question,
        inputs=[query_input, chatbot],
        outputs=[chatbot, query_input],
    )

demo.launch(debug=False)
```

5.2.9.8 Key Takeaways

What we demonstrated:

- A **multi-agent system** where each agent has a narrow, well-defined role
- Agents pass outputs to each other (logical schema arrow.r DDL arrow.r populated DB arrow.r queries)
- The same business question answered in two ways (SQL vs. RAG) with automatic comparison
- Separation of **setup phase** (expensive, run once) from **query phase** (cheap, repeat many times)

When to use SQL vs. RAG on structured data:

	SQL (Agent 3)	RAG (Agent 4)
Numeric aggregations	Excellent	Often wrong
Fuzzy/open-ended questions	Limited	Better
Joins across tables	Excellent	Struggles
Context size limit	None	Yes
Requires valid schema	Yes	No
Precision	High	Variable

In practice, systems often use both: SQL for precise numeric queries, RAG for explanatory or fuzzy questions — and an LLM-as-a-judge (Agent 5) to flag when they disagree.

Exercises:

- Add a new agent that generates a chart from the SQL result
- Extend Agent 4 to use vector embeddings (ChromaDB) instead of raw context

- Add an Agent 6 that explains WHY the two answers disagreed (when similarity is low)
- Replace `llm_cascade` with a specific provider and see how different LLMs perform at each agent role

5.3 Autonomous Agents

5.3.1 From Predefined Pipelines to Self-Directed Workflows

The single-agent and multi-agent systems explored in previous chapters share a common characteristic: a human decides the pipeline. In the single-agent notebook, a human designed the three-step workflow of schema design, DDL generation, and SQL querying. In the multi-agent notebook, a human defined which five agents to create, what each one does, and in what order they execute. The agents fill prescribed roles, but the overall plan is fixed before the first LLM call is made.

An autonomous agent operates differently. You give it a high-level goal, a single sentence describing what you want to achieve, and the agent determines for itself what steps are needed, what questions to ask, and how to synthesize the results into a final deliverable. The human provides the destination; the agent charts the course. This distinction, between following a human-designed pipeline and generating one's own plan, is what separates autonomous agents from the reactive and orchestrated patterns covered earlier.

5.3.2 The Plan, Execute, Synthesize, Reflect Pattern

The accompanying notebook, `AutonomousAgent_BusinessValidator`, implements a four-phase autonomous workflow that takes a one-sentence business idea and produces a validation report with a go or no-go recommendation. Each phase represents a different cognitive task, and each uses a distinct system prompt tailored to that task's requirements.

In the **planning phase**, the agent receives the business idea and generates four to five critical research questions that must be answered to evaluate the idea's viability. The planner is prompted to think strategically, identifying the most important dimensions of analysis such as market size, competitive landscape, unit economics, and technical feasibility. The specific questions vary depending on the input. A fintech startup generates different research questions than a local coffee shop or a children's educational game. This adaptability is the hallmark of autonomous behavior: the plan is not hardcoded but emerges from the interaction between the goal and the model's reasoning.

In the **execution phase**, the agent works through each research question sequentially, generating a concise analysis for each one. Critically, each subsequent question is answered with access to the accumulated context from all prior answers. This means the agent's reasoning can build on itself. An insight about market size in question one can inform the analysis of competitive positioning in question three. The sequential accumulation of context mimics how a human analyst would conduct research, with each finding shaping the interpretation of subsequent findings.

In the **synthesis phase**, the agent takes on the persona of a senior executive and combines all research findings into a clear, decisive recommendation. The output includes a go, no-go, or conditional verdict supported by key reasoning points. This phase requires the agent to weigh competing considerations and make a judgment call, a fundamentally different cognitive task from the analytical work of the execution phase.

In the **reflection phase**, a skeptical critic examines the recommendation and identifies what is strong, what is weak or missing, and assigns a confidence score from one to ten. This self-critique phase is what elevates the workflow beyond simple sequential generation. The critic is specifically prompted to be adversarial, to look for gaps in reasoning, missing evidence, and overly optimistic assumptions. In the notebook's example run, the synthesizer produces a "go" recommendation for a student housing app, but the critic assigns a confidence score of four out of ten, flagging concerns about legal risk, lack of competitive differentiation, and unvalidated unit economics.

5.3.3 How Autonomous Agents Differ from Other Patterns

Understanding autonomous agents requires distinguishing them from two related but distinct patterns. A reactive tool-calling agent, such as the LangChain agent demonstrated in earlier chapters, decides at each turn which tool to call based on the current state. It is

dynamic in the sense that tool selection is not predetermined, but it does not plan ahead. It responds to each observation without a global strategy. The autonomous agent, by contrast, generates an entire research plan before executing any of it.

A fixed-pipeline multi-agent system, such as the five-agent database workflow, executes the same sequence of agents regardless of the input. Agent 1 always runs before Agent 2, which always runs before Agent 3. The pipeline is static. An autonomous agent adapts its pipeline to the input. A business idea about AI-powered logistics generates different research questions than one about a subscription meal kit service, and those questions drive fundamentally different execution paths.

The distinction is not about sophistication but about who controls the workflow. In reactive agents, the environment drives behavior one step at a time. In multi-agent pipelines, the human designer drives behavior at design time. In autonomous agents, the LLM drives behavior at runtime by generating and then executing its own plan.

5.3.4 The Role of Self-Critique

The reflection phase deserves special attention because it addresses one of the most persistent problems in LLM applications: the tendency of models to generate confident-sounding but potentially flawed outputs. By adding a dedicated critique step with an adversarial prompt, the autonomous workflow builds in a quality-assurance mechanism that operates without human intervention.

The self-critique in the `AutonomousAgent_BusinessValidator` notebook demonstrates this vividly. The synthesizer's optimistic "go" recommendation is met with pointed criticism about legal exposure, competitive moats, and customer acquisition costs. The confidence score provides a quantitative signal that downstream consumers, whether human decision-makers or other automated systems, can use to calibrate their trust in the output.

This pattern connects directly to the LLM-as-a-judge concept introduced in the multi-agent chapter. The difference is that in a multi-agent system, the judge evaluates outputs from other agents, while in an autonomous workflow, the agent judges its own output. Both patterns recognize the same fundamental insight: a second pass of LLM reasoning focused on evaluation is often more valuable per token than additional generation.

5.3.5 Current Limitations

The `AutonomousAgent_BusinessValidator` notebook is intentionally transparent about what it does not do, because understanding these limitations is essential for knowing when autonomous agents are ready for production and when they are not.

First, the agent has no access to real tools. It cannot search the web, query databases, or call APIs. All of its research is generated from the LLM's training data, which means the analysis reflects general knowledge rather than current market conditions. Adding tools to the execution phase, such as web search or database queries, would transform the agent from a reasoning exercise into a genuine research assistant.

Second, the agent does not iterate on its critique. The reflection phase produces feedback, but that feedback is never fed back into the planner to generate an improved plan. In a production system, you would add a conditional loop: if the critic's confidence is below a threshold, re-run the planning and execution phases with the critique appended as additional context, forcing the agent to address the identified weaknesses.

Third, the agent has no memory across runs. Each invocation starts from scratch. A production autonomous agent would maintain a knowledge base of prior analyses, allowing it to learn from past evaluations and avoid repeating the same reasoning errors.

These limitations are not flaws in the design but deliberate simplifications that make the core pattern visible. The four-phase structure of plan, execute, synthesize, and reflect is the foundation. Tools, iteration, and memory are extensions that build on that foundation, and

frameworks like LangGraph provide the infrastructure to implement them as stateful graphs with conditional edges.

5.3.6 References

- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. *International Conference on Learning Representations (ICLR).*
- Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language Agents with Verbal Reinforcement Learning. *Advances in Neural Information Processing Systems (NeurIPS).*
- Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W. X., Wei, Z., & Wen, J. (2024). A Survey on Large Language Model Based Autonomous Agents. *Frontiers of Computer Science.*
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., & Zhou, D. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. *Advances in Neural Information Processing Systems (NeurIPS).*
- Weng, L. (2023). LLM Powered Autonomous Agents. *Lil'Log*. https://lilianweng.github.io/posts/2023-06-23-agent/

Open in Colab

```
# 1. INSTALL DEPENDENCIES
!pip install -q git+https://github.com/KarAnalytics/llm_cascade.git
```

IGNORE THE ABOVE ERRORS AND PROCEED — they are specific to Google Colab's preinstalled packages and the code will still execute correctly.

5.3.7 Autonomous Agent: Business Idea Validator

In every agent notebook so far, a human has decided the workflow. The multi-agent system runs Agents 1 through 5 in a fixed order. The single-agent system runs schema design, then DDL, then queries. Even the tool-calling parking assistant follows a human-designed ReAct loop. The agent may choose *which* tool to call at each step, but the overall structure – "answer one question using tools" – is predefined.

This notebook breaks that pattern. Here, the agent receives only a **one-sentence goal** ("Validate this business idea") and figures out everything else on its own: what questions to research, how to answer each one, how to synthesize the findings into a recommendation, and whether that recommendation is actually any good. No human intervenes between phases. No pipeline is hardcoded. The agent plans, executes, synthesizes, and self-critiques – autonomously.

What makes this "autonomous" vs. the previous agents?

	Tool-calling agent	Multi-agent system	**Autonomous workflow (this)**
Who decides the task steps?	Agent picks tools at each step	Humans pre-define the pipeline	**Agent plans the whole workflow**
Fixed or dynamic pipeline?	Dynamic (tool-by-tool)	Fixed (Agent 1 -> 2 -> 3...)	**Dynamic (plan changes per input)**
Self-critique?	No	Only if an Evaluator agent is added	**Yes, as a built-in phase**
Typical use case	Answer questions using external tools	Well-defined workflows (ETL, BI)	**Open-ended goals (research, writing, validation)**

Why self-critique matters: Without it, the agent would produce a recommendation and stop, with no indication of whether the recommendation is strong or weak. The Reflect phase acts as a built-in skeptic – it identifies gaps in the reasoning, flags unsupported assumptions, and assigns a confidence score. This gives the human consumer a calibrated signal: a confidence of 8/10 means "probably act on this"; a confidence of 4/10 means "do more research before deciding." In production systems, you could feed a low-confidence critique back into the planner for another iteration, creating a self-improving loop.

5.3.7.1 Imports and Setup

Minimal dependencies: just `llm_cascade` for automatic LLM provider fallback. No tools, no databases, no vector stores — this is a pure LLM reasoning workflow.

```
from llm_cascade import get_cascade

llm = get_cascade()
```

5.3.7.2 The Four Phases of the Autonomous Workflow

Each phase is a single function that makes one LLM call with a phase-specific system prompt. Together, the four phases form the **Plan-Execute-Synthesize-Reflect** loop – a pattern that appears in many real-world autonomous agent frameworks (AutoGPT, BabyAGI, LangGraph research agents).

Notice that **each phase has a different system prompt**, but this is not the same as the multi-agent architecture. In the multi-agent notebook, each agent has a different *role* (Data Architect, SQL Developer). Here, each phase has a different *cognitive task*: the Planner needs to think strategically about what questions matter; the Executor needs to think concretely about answering one question at a time; the Synthesizer needs to weigh tradeoffs and make a judgment call; and the Critic needs to be genuinely skeptical, looking for holes in the reasoning. These are four modes of thinking, not four job titles.

1. **Plan** – Given the goal, generate 4-5 specific questions that need to be answered
2. **Execute** – Loop through the plan and answer each question, passing prior answers as accumulating context
3. **Synthesize** – Combine all answers into a final recommendation with a clear go/no-go/conditional verdict
4. **Reflect** – Critique the recommendation, identify weaknesses, and assign a confidence score from 1 to 10

```
# Phase 1: Planner -- decides WHAT needs to be researched
PLANNER_SYSTEM = (
    'You are a strategic business planner. Given a high-level goal, break it
down '
    'into the 4-5 most critical questions that must be answered to achieve it.
'
    'Output ONLY a numbered list of questions. No preamble, no conclusion.'
)


def plan(goal):
    '''Phase 1: generate a research plan as a list of questions.'''
    prompt = f'Goal: {goal}' + chr(10) + chr(10) + 'List the 4-5 most critical
questions to answer.'
    response = llm.generate(prompt, system_prompt=PLANNER_SYSTEM)

    # Parse the numbered list into a Python list of strings
    steps = []
    for line in response.text.split(chr(10)):
        line = line.strip()
        if not line:
            continue
        # Strip leading '1.', '1)', '-', '*', etc.
        cleaned = line.lstrip('0123456789.)- *').strip()
        if cleaned and len(cleaned) > 5:
            steps.append(cleaned)
    return steps


print('Phase 1 (Planner) ready.')
```

The Planner's system prompt instructs it to think like a strategic consultant: given a vague goal, identify the critical questions that will determine success or failure. The output is a numbered list of 4-5 questions – no preamble, no analysis, just the questions. This constrained output format makes it easy to parse programmatically and feed into the next phase. Try changing the business idea later and notice how the plan changes completely – an AI housing app generates questions about data sources and student demographics, while a meal kit service generates questions about supply chains and unit economics.

The Executor answers one research question at a time, but with an important twist: it receives all prior answers as accumulated context. This means each successive answer can

build on the previous ones, creating a chain of reasoning rather than a collection of isolated answers. The second question's answer might reference a data source mentioned in the first answer; the fourth might synthesize patterns from the first three. This context accumulation is what makes the execution phase feel coherent rather than fragmented.

The Synthesizer plays the role of a senior executive who must make a decision based on the research. Its system prompt asks for a clear verdict – go, no-go, or conditional – followed by three bullet points of key reasoning. This forces the model to commit to a position rather than hedging endlessly, which is a common failure mode when LLMs are asked for recommendations without explicit structural constraints.

The Critic is deliberately adversarial. Its system prompt instructs it to be "skeptical" and to identify what is "weak or missing." This is by design: the Synthesizer is optimistic (it just made a recommendation and wants it to be right), so the Critic provides the necessary counterbalance. In the example output you will see below, the Critic often assigns a much lower confidence score than you might expect from reading the recommendation alone, because it focuses on risks, missing evidence, and competitive threats that the Synthesizer glossed over.

```
# Phase 2: Executor -- answers ONE research question at a time
EXECUTOR_SYSTEM = (
    'You are a business analyst. Answer the question concisely (2-4 sentences)
'
    'with concrete reasoning. Use the prior context if helpful, but stay
focused '
    'on the current question.'
)


def execute_step(question, prior_context):
    '''Phase 2: answer one research question with access to prior answers.'''
    prompt = 'Prior context:' + chr(10) + prior_context + chr(10) + chr(10) +
'Question: ' + question
    response = llm.generate(prompt, system_prompt=EXECUTOR_SYSTEM)
    return response.text


print('Phase 2 (Executor) ready.')

# Phase 3: Synthesizer -- combines findings into a recommendation
SYNTHESIZER_SYSTEM = (
    'You are a senior executive. Given research findings, synthesize them into
a '
    'clear, decisive recommendation. State the recommendation (go / no-go /
conditional), '
    'followed by 3 bullet points of key reasoning.'
)


def synthesize(goal, research_results):
    '''Phase 3: combine all research answers into a final recommendation.'''
    findings = chr(10).join([f'Q: {q}' + chr(10) + f'A: {a}' for q, a in
research_results])
    prompt = (
        'Goal: ' + goal + chr(10) + chr(10)
        + 'Research findings:' + chr(10) + findings + chr(10) + chr(10)
        + 'Synthesize into a final recommendation.'
    )
    response = llm.generate(prompt, system_prompt=SYNTHESIZER_SYSTEM)
    return response.text


print('Phase 3 (Synthesizer) ready.')
```

```
# Phase 4: Critic -- self-reflects on the recommendation
CRITIC_SYSTEM = (
    'You are a skeptical business critic. Given a recommendation, identify '
    'what is weak or missing, what is strong, and assign a confidence level '
    'from 1 (very weak) to 10 (very strong). Be concise.'
)


def reflect(recommendation):
    '''Phase 4: critique the recommendation and assign confidence.'''
    prompt = 'Recommendation to critique:' + chr(10) + recommendation
    response = llm.generate(prompt, system_prompt=CRITIC_SYSTEM)
    return response.text


print('Phase 4 (Critic) ready.')
```

5.3.7.3 Inspect the Full Trace

The `result` dictionary holds everything the agent produced across all four phases, so you can examine each phase individually. This is useful for debugging (did the planner ask the right questions? did the executor miss something?) and for teaching (trace how the agent's reasoning flowed from plan to recommendation to critique).

The trace output reveals something important about autonomous workflows: the quality of the final recommendation depends heavily on the quality of the plan. If the planner asks shallow questions, the executor produces shallow answers, and the synthesizer builds on a weak foundation. This is why the planner's system prompt matters so much – it sets the ceiling for everything that follows.

5.3.7.4 Run the Agent

Edit the `business_idea` string below and run the cell. The agent will autonomously plan its research, execute each step, synthesize a recommendation, and self-critique – with no further input from you. The verbose output prints each phase as it completes, so you can watch the agent "think out loud" and trace how the plan's questions get answered, how those answers feed into the recommendation, and how the critic pokes holes in the final output.

Try different ideas to see how the plan changes. The agent generates a completely different research plan for a fintech startup vs. a coffee shop vs. a SaaS tool – which is exactly what an autonomous agent should do. The plan is not hardcoded; it emerges from the LLM's reasoning about what matters for each specific goal.

```
def run_autonomous_workflow(goal, verbose=True):
    '''The full autonomous workflow: Plan -> Execute -> Synthesize ->
Reflect.'''
    if verbose:
        print('=' * 70)
        print('GOAL:', goal)
        print('=' * 70)

    # --- Phase 1: Plan ---
    if verbose:
        print(chr(10) + '[PHASE 1 - PLANNING]')
    steps = plan(goal)
    if verbose:
        print(f'Agent decided to research {len(steps)} questions:')
        for i, s in enumerate(steps, 1):
            print(f'  {i}. {s}')

    # --- Phase 2: Execute each step ---
    if verbose:
        print(chr(10) + '[PHASE 2 - EXECUTION]')
    results = []
```

```
    prior_context = ''
    for i, step in enumerate(steps, 1):
        if verbose:
            print(chr(10) + f'Step {i}: {step}')
        answer = execute_step(step, prior_context)
        results.append((step, answer))
        prior_context += chr(10) + f'Q: {step}' + chr(10) + f'A: {answer}'
        if verbose:
            print(f'  -> {answer}')

    # --- Phase 3: Synthesize ---
    if verbose:
        print(chr(10) + '[PHASE 3 - SYNTHESIS]')
    recommendation = synthesize(goal, results)
    if verbose:
        print(recommendation)

    # --- Phase 4: Reflect ---
    if verbose:
        print(chr(10) + '[PHASE 4 - SELF-CRITIQUE]')
    critique = reflect(recommendation)
    if verbose:
        print(critique)

    return {
        'goal': goal,
        'plan': steps,
        'research': results,
        'recommendation': recommendation,
        'critique': critique,
    }


print('Autonomous workflow function ready.')

business_idea = 'An AI-powered app that helps college students find affordable
off-campus housing by predicting rent trends.'

# business_idea = 'A subscription meal kit service for busy professionals that
uses local ingredients.'
# business_idea = 'A mobile game that teaches kids basic accounting through
story-based puzzles.'

result = run_autonomous_workflow(business_idea)
```

5.3.7.5 Inspect the Full Trace

The `result` dict holds everything the agent did, so you can examine each phase individually. This is useful for debugging or for teaching how the agent's reasoning flowed from plan to final recommendation.

5.3.7.6 Key Takeaways

What makes this workflow "autonomous":

- You provide only a **high-level goal**, not a list of steps
- The agent generates its own **research plan** based on the goal
- Each step is executed with access to the **prior context** (results accumulate)
- The agent self-critiques its output, giving you a **confidence signal** for free

What's NOT happening (and why this example is simple):

- No external tools (no web search, no database queries)
- No iteration: the agent doesn't re-plan based on the critique
- No parallelism: execution is strictly sequential
- No human-in-the-loop: the workflow runs end-to-end without interruption

Scaling this up in the real world:

- Add **tools** so execution steps can fetch real data (web search, database, APIs)
- Add an **iteration loop**: if the critic's confidence is low, feed the critique back to the planner to generate a better plan
- Use **LangGraph** to express this as a stateful graph with conditional edges (see `LangGraph_demo.ipynb` for the foundation)
- Add **memory** so the agent can remember prior runs and improve over time

Comparison with our other notebooks:

Notebook	Who decides the steps?
`LlamaIndex_RAG`, `LangChain_demo` (RAG chain)	Humans — the pipeline is fixed
`LangChain_demo` (agent with tools)	LLM picks tools at each turn, but the goal is one question
`MultiAgent_DB`	Humans — 5 specialized agents run in a fixed order
`SingleAgent_DB`	Humans — one agent runs a fixed workflow
This notebook	**LLM decides the whole workflow from a one-sentence goal**

Exercises:

- Try a business idea the LLM is unlikely to have strong prior knowledge about (e.g., a niche B2B tool). Does the plan still look reasonable?
- Modify `run_autonomous_workflow` to loop: after the critic phase, if confidence < 7, re-run the plan with the critique appended to the goal.
- Add a fifth phase that outputs a one-page marketing pitch based on the recommendation.
- Replace the `PLANNER_SYSTEM` prompt with a different persona (e.g., "venture capitalist") and see how the plan changes.

5.4 Tool-Using Agents and MCP

5.4.1 Real Tools vs. LLM-Only Reasoning

Every agent discussed in previous chapters relied on the LLM to do everything: design schemas, write SQL, analyze data, and generate recommendations. The LLM performed well on these tasks because they are fundamentally language tasks, translating between natural language and structured formats. But there is an entire category of problems where LLM reasoning is the wrong tool for the job. Computing the distance between two geographic coordinates, matching a user's informal building name against a canonical list, or generating a correctly formatted URL are tasks that demand deterministic precision, not probabilistic text generation.

The accompanying notebook, `KU_Parking_Assistant`, makes this distinction concrete by building an agent that helps users find parking near any building at the University of Kansas. The agent uses three real tools implemented as plain Python functions. The first is a Haversine distance calculator that computes the great-circle distance between two latitude-longitude pairs. The second is a fuzzy building name matcher that resolves informal queries like "business school" to the canonical name "Capitol Federal Hall (Business School)" through substring matching. The third generates clickable Google Maps URLs from coordinates.

None of these tools contain an LLM. They are pure deterministic Python code that produces the same output every time for the same input. The LLM's role is to understand the user's natural-language question, decide which tools to call and in what order, and then format the tool outputs into a helpful response. This division of labor, language understanding for the LLM and precise computation for the tools, is the foundation of effective tool-using agent design.

The difference matters in practice. An LLM asked to compute the Haversine distance between two coordinates will sometimes get the math right and sometimes not, depending on the model, the prompt, and the specific numbers involved. A Python function using the standard formula will always get it right. An LLM asked to generate a Google Maps URL may hallucinate the format or transpose digits in the coordinates. A string formatting function will always produce a valid URL. By delegating deterministic tasks to tools and reserving the LLM for what it does best, understanding intent and generating natural language, the overall system becomes both more reliable and more capable than either component alone.

5.4.2 The Agent Loop

The KU Parking Assistant uses a ReAct-style agent loop that works with any LLM, without requiring special function-calling APIs. The system prompt describes the available tools and instructs the LLM to output a JSON object when it wants to call one. The agent loop parses this JSON, executes the corresponding Python function, feeds the result back into the conversation as a new message, and lets the LLM decide whether to call another tool or produce a final answer. This cycle of reasoning and acting, from which the ReAct pattern derives its name, typically completes in two to three steps for a parking query: one call to find nearby parking lots, one call to get the color legend, and then a final formatted response.

5.4.3 The Model Context Protocol

The inline-tools approach works well for a single notebook, but it has a fundamental limitation: the tools are trapped inside the notebook. If you want to use the same parking tools in a web application, a Dify workflow, or Claude Desktop, you would have to reimplement them in each environment. This is where the Model Context Protocol, or MCP, enters the picture.

MCP is an open standard, originally introduced by Anthropic, that defines how AI applications communicate with tool servers. It specifies a simple JSON-RPC 2.0 interface over HTTP. A client sends a request to a server, the server executes a tool, and the server

returns the result. Two methods form the core of the protocol: `tools/list`, which allows a client to discover what tools a server offers along with their parameter schemas, and `tools/call`, which executes a specific tool with provided arguments.

The critical insight behind MCP is separation of concerns. The tool server is responsible for implementing and hosting the tools. The client is responsible for deciding when and how to call them. Because the interface is standardized, any MCP-compatible client can work with any MCP-compatible server without custom integration code. This is the same principle that made REST APIs transformative for web development: a standard interface enables an ecosystem of interchangeable components.

5.4.4 From Inline Tools to Local MCP Server to Cloud Endpoint

The progression across the two KU Parking notebooks illustrates a natural evolution that many AI applications follow in practice.

In the first stage, tools are defined as **inline Python functions** within the notebook. The agent calls them directly as local function invocations. This is the fastest way to prototype and debug. You can set breakpoints, inspect intermediate values, and iterate on tool logic without any network overhead. The limitation is that the tools exist only within that notebook's runtime.

In the second stage, the same tools are deployed to a **cloud MCP endpoint**. The `KU_Parking_mcp` notebook demonstrates this by calling a Supabase Edge Function that implements the identical parking tools in TypeScript. From the agent's perspective, almost nothing changes. The system prompt still lists the same three tools. The agent loop still parses JSON actions and feeds results back. The only difference is that tool calls now go through an HTTP POST to the Supabase URL instead of a local function invocation. The `mcp_tool()` helper function is a thin wrapper around Python's `requests` library, roughly ten lines of code, that handles the JSON-RPC formatting.

This progression matters because it shows that the transition from prototype to production does not require rearchitecting the agent. The agent logic, the system prompt, the ReAct loop, and the output formatting all remain the same. Only the transport layer changes, from a function call to an HTTP request.

5.4.5 Multi-Client Reuse

The real payoff of MCP becomes visible when multiple clients consume the same tools. The Supabase Edge Function hosting the KU Parking tools is a single HTTPS endpoint. The Python notebook calls it from Colab. A Dify workflow calls it using an HTTP Request node. Claude Desktop could call it as a configured MCP server. A custom web application could call it from JavaScript. In every case, the clients call the same `find_parking_near_building` tool, send the same parameters, and receive the same results.

This architecture means that updating data or tool logic happens in exactly one place. If a new parking lot opens on campus, you update the data in the Supabase Edge Function, redeploy with a single command, and every client sees the new lot on its next request. No notebook reruns, no Dify workflow edits, no client code changes. This is the operational advantage of treating tools as services rather than embedded code.

Tool discovery reinforces this advantage. When a client connects to an MCP server, it calls `tools/list` to learn what tools are available and what parameters each tool accepts. If you add a new tool to the server, such as a `find_parking_by_color` function, existing clients can discover it automatically without code changes. The server's tool catalog is the single source of truth for what capabilities are available.

5.4.6 Connecting the Concepts

The KU Parking Assistant brings together several threads from across this book. The fuzzy building name matching is a form of retrieval, mapping an informal query to a canonical entity, that echoes the retrieval step in RAG pipelines. The Haversine distance calculation is

an example of a task that should never be delegated to an LLM when a formula exists. The ReAct agent loop is the same pattern used by LangChain agents, simplified here to work with any LLM provider. And the MCP deployment pathway connects directly to the deployment chapter that follows, where the same Supabase Edge Function is integrated into production workflows.

The broader lesson is about knowing when to use an LLM and when to use a tool. Language understanding, intent classification, natural-language formatting, and open-ended reasoning are LLM tasks. Distance calculations, data lookups, URL generation, and any operation with a known correct algorithm are tool tasks. The best agents combine both, letting each component do what it does best.

5.4.7 References

- Anthropic. (2024). Model Context Protocol Specification. https://modelcontextprotocol.io/
- Schick, T., Dwivedi-Yu, J., Dess`i, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2024). Toolformer: Language Models Can Teach Themselves to Use Tools. *Advances in Neural Information Processing Systems (NeurIPS).*
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing Reasoning and Acting in Language Models. *International Conference on Learning Representations (ICLR).*
- Qin, Y., Liang, S., Ye, Y., Zhu, K., Yan, L., Lu, Y., Lin, Y., Cong, X., Tang, X., Qian, B., Zhao, S., Hong, L., Tian, R., Xie, R., Zhou, J., Gerber, M., Li, D., Liu, Z., & Sun, M. (2024). ToolLLM: Facilitating Large Language Models to Master 16000+ Real-World APIs. *International Conference on Learning Representations (ICLR).*
- Supabase. (2024). Edge Functions Documentation. https://supabase.com/docs/guides/functions

Open in Colab

```
!pip install -q git+https://github.com/KarAnalytics/llm_cascade.git
```

5.4.8 KU Parking Assistant: A Simple Tool-Using Agent

Every agent notebook you have seen so far in this course relies on the LLM to do all the heavy lifting – reasoning, planning, and generating answers from its own knowledge. But there are tasks where pure LLM reasoning is the wrong tool for the job. Calculating the distance between two GPS coordinates, for example, requires precise trigonometry. An LLM might approximate the answer, but it will sometimes get it wrong in ways that matter (ranking a lot as "closest" when it is actually third-closest). A Python function using the **Haversine formula** will get it right every time, in microseconds, with no hallucination risk.

This notebook builds a small agent that combines the best of both worlds. The LLM handles what it is good at – understanding natural language ("Where can I park near the business school?"), fuzzy matching building names, and formatting a helpful response. Python functions handle what they are good at – looking up coordinates in a dictionary, computing great-circle distances, sorting results, and constructing Google Maps URLs. The LLM decides **when** to call which tool, but it never does the math itself.

What makes this different from the other agent notebooks:

- We use **real deterministic tools** (distance math, data lookups) instead of a pure text-in/text-out workflow
- The LLM decides **when** to call which tool, but it does not do the math itself – it delegates to Python
- This is a **pure workflow**, not a chatbot: one input produces one structured output

Teaching goal: Show the difference between **fuzzy LLM reasoning** (good for language understanding: "business school" maps to "Capitol Federal Hall") and **precise tool execution** (good for deterministic tasks: distance math, sorting, data lookup). A well-designed agent lets each side do what it is best at.

Note on data: The real KU parking map is at https://parking.ku.edu/sites/parking/files/documents/parkingmap.pdf. For this demo we use a small illustrative dataset with approximate coordinates so the notebook runs standalone.

5.4.8.1 Imports and LLM Setup

We use `llm_cascade` for automatic LLM provider fallback, and Python's built-in `math` module for the distance calculations. No external APIs needed.

```
import math
import json as _json
import re as _re

from llm_cascade import get_cascade

llm = get_cascade()
```

5.4.8.2 Data: KU Buildings and Parking Lots

We hardcode two small datasets that represent the physical world the agent needs to reason about. In a real deployment, these would come from a database, a GIS system, or a parsed PDF. For this classroom demo, approximate coordinates are sufficient to demonstrate the architecture.

KU Buildings are the destinations students might ask about. The dictionary maps human-readable names (with common aliases in parentheses) to latitude/longitude tuples. This naming convention is deliberate: when a student asks about "the business school," our fuzzy matcher will find "Capitol Federal Hall (Business School)" because the alias appears in the dictionary key.

KU Parking Lots include a color code indicating who can park there. Understanding these colors is essential context for the agent's final answer – it is not helpful to recommend a Yellow (faculty-only) lot to a student with a Blue permit. The colors are:

- **Blue** – student commuter permit
- **Yellow** – faculty / staff permit
- **Gold** – premium permit (close to central campus)
- **Red** – restricted / reserved
- **Visitor** – open to visitors, usually metered or short-term
- **Park & Ride** – free remote lot with shuttle service

```
# --- KU Buildings (approximate coordinates for illustration) ---
KU_BUILDINGS = {
    'Capitol Federal Hall (Business School)':        (38.953505, -95.249740),
    'Kansas Union':                                   (38.959200, -95.243500),
    'Allen Fieldhouse':                               (38.954306, -95.252394),
    'Strong Hall':                                    (38.958542, -95.247614),
    'Watson Library':                                 (38.956621, -95.244787),
    'Anschutz Library':                               (38.957323, -95.249742),
    'Dyche Hall':                                     (38.958610, -95.243890),
    'Fraser Hall':                                    (38.957160, -95.243590),
    'Lied Center':                                    (38.954940, -95.262890),
    'Memorial Stadium':                               (38.963330, -95.246390),
    'Wescoe Hall':                                    (38.957430, -95.247830),
    'Spencer Museum of Art':                          (38.959634, -95.244569),
    'Ambler Student Recreation Fitness Center':       (38.952512, -95.247929),
}

# --- KU Parking Lots (approximate coordinates for illustration) ---
KU_PARKING_LOTS = [
    {'lot': 'Lot 3'                  , 'color': 'Blue'    , 'lat': 38.958942,
'lng': -95.247614},
    {'lot': 'Lot 5'                  , 'color': 'Yellow'  , 'lat': 38.95861,
'lng': -95.24441},
    {'lot': 'Lot 10'                 , 'color': 'Blue'    , 'lat': 38.956221,
'lng': -95.244787},
    {'lot': 'Lot 14'                 , 'color': 'Yellow'  , 'lat': 38.95716,
'lng': -95.24307},
    {'lot': 'Lot 16'                 , 'color': 'Visitor' , 'lat': 38.9592,
'lng': -95.24298},
    {'lot': 'Lot 18'                 , 'color': 'Yellow'  , 'lat': 38.95703,
'lng': -95.24783},
    {'lot': 'Lot 60'                 , 'color': 'Blue'    , 'lat': 38.96333,
'lng': -95.24691},
    {'lot': 'Lot 70'                 , 'color': 'Blue'    , 'lat': 38.953906,
'lng': -95.252394},
    {'lot': 'Lot 71'                 , 'color': 'Yellow'  , 'lat': 38.953666,
'lng': -95.252394},
    {'lot': 'Lot 90'                 , 'color': 'Gold'    , 'lat': 38.952512,
'lng': -95.247929},
    {'lot': 'Lot 91'                 , 'color': 'Gold'    , 'lat': 38.959634,
'lng': -95.244569},
    {'lot': 'Lot 118'                , 'color': 'Gold'    , 'lat': 38.953505,
'lng': -95.24974},
    {'lot': 'Lot 300A-D'             , 'color': 'Gold'    , 'lat': 38.95494,
'lng': -95.26237},
    {'lot': 'Lot 300E-G'             , 'color': 'Gold'    , 'lat': 38.95494,
'lng': -95.26289},
```

```
    {'lot': 'Allen Fieldhouse Garage', 'color': 'Gold'    , 'lat': 38.954306,
'lng': -95.252394},
]

print(f'Loaded {len(KU_BUILDINGS)} buildings and {len(KU_PARKING_LOTS)}
parking lots.')
```

5.4.8.3 The Tools (plain Python functions)

Three simple tools the agent can call. Notice there is **no LLM inside these functions** – they are pure deterministic Python. The LLM will decide *when* to call them based on the user's question, but every computation inside them produces an exact, repeatable result.

Tool	What it does	Why it exists
`list_ku_buildings()`	Returns the list of available buildings	So the agent can show options when the user's query is ambiguous
`find_parking_near_building(name, top_k)`	Finds the top-K nearest parking lots within a 2-mile radius	The core function – distance math, sorting, and URL generation
`get_parking_colors_legend()`	Explains what each parking color means	So the agent can annotate its answer with permit-type context

The distance calculation uses the **Haversine formula**, which computes the great-circle distance between two latitude/longitude points on a sphere. For the short distances on a university campus (typically under 1 mile), the Haversine result is accurate to within a few feet – far better than asking an LLM to eyeball coordinate differences. The **2-mile radius filter** is a practical constraint that keeps the results useful: a parking lot 5 miles away is technically "near" the campus, but no student would walk there.

The **fuzzy building matcher** (`_match_building`) deserves attention. It tries three strategies in order: exact match, substring match (does the user's query appear inside any building name?), and reverse substring match (does any building name appear inside the user's query?). This means "business school," "Capitol Federal," and even "I need to get to the business school building" all resolve to the same entry. The Google Maps URLs are constructed by simply embedding the latitude and longitude into a standard `maps?q=LAT,LNG` template, which produces a clickable pin link that works on any device.

```
def _haversine_miles(lat1, lng1, lat2, lng2):
    '''Great-circle distance between two lat/lng points, in miles.'''
    R = 3958.8  # Earth radius in miles
    phi1, phi2 = math.radians(lat1), math.radians(lat2)
    dphi = math.radians(lat2 - lat1)
    dlam = math.radians(lng2 - lng1)
    a = math.sin(dphi / 2) ** 2 + math.cos(phi1) * math.cos(phi2) *
math.sin(dlam / 2) ** 2
    return 2 * R * math.asin(math.sqrt(a))


def _google_maps_pin(lat, lng):
    '''Return a clickable Google Maps pin URL for a coordinate.'''
    return f'https://www.google.com/maps?q={lat},{lng}'


def _match_building(query):
    '''Fuzzy-match a building name. Accepts partial/lowercase queries like
"business school".'''
    q = query.lower().strip()
    # Exact match first
    for name in KU_BUILDINGS:
```

```
        if name.lower() == q:
            return name
    # Substring match
    for name in KU_BUILDINGS:
        if q in name.lower():
            return name
    # Reverse substring (user typed a longer phrase that contains the building
name)
    for name in KU_BUILDINGS:
        short_key = name.split('(')[0].strip().lower()
        if short_key in q:
            return name
    return None


# ---- Tools the agent can call ----

def list_ku_buildings(_=''):
    '''Return the list of KU buildings available for parking lookup.'''
    return chr(10).join(f'- {name}' for name in KU_BUILDINGS.keys())


def find_parking_near_building(building_name, top_k=5,
max_distance_miles=2.0):
    '''Find nearest parking lots within max_distance_miles of the building,
top-K results.'''
    matched = _match_building(building_name)
    if matched is None:
        return (
            f'No KU building matches "{building_name}". '
            f'Use list_ku_buildings to see available options.'
        )

    target_lat, target_lng = KU_BUILDINGS[matched]

    # Compute distance to every lot and sort
    ranked = []
    for lot in KU_PARKING_LOTS:
        dist = _haversine_miles(target_lat, target_lng, lot['lat'],
lot['lng'])
        ranked.append({
            'lot': lot['lot'],
            'color': lot['color'],
            'distance_miles': round(dist, 3),
            'walk_minutes': round(dist * 20),  # ~3 mph walking speed
            'google_maps': _google_maps_pin(lot['lat'], lot['lng']),
        })

    ranked.sort(key=lambda r: r['distance_miles'])

    # Filter to lots within max_distance_miles of the destination
    within_range = [r for r in ranked if r['distance_miles'] <=
max_distance_miles]

    if not within_range:
        return (
            f'No parking lots within {max_distance_miles} miles of {matched}.
'
            f'Try a different building or increase the radius.'
        )

    header = (
        f'Parking lots within {max_distance_miles} miles of {matched} '
        f'(top {top_k}, ranked by distance):' + chr(10)
    )
    lines = [header]
```

```
    for i, r in enumerate(within_range[:top_k], start=1):
        lines.append(
            f"  {i}. {r['lot']} ({r['color']}) - "
            f"{r['distance_miles']} mi ({r['walk_minutes']} min walk) "
            f"-> {r['google_maps']}"
        )
    return chr(10).join(lines)


def get_parking_colors_legend(_=''):
    '''Explain what each parking color means at KU.'''
    return (
        'KU parking color legend:' + chr(10)
        + '- Blue: student commuter permit' + chr(10)
        + '- Yellow: faculty/staff permit' + chr(10)
        + '- Gold: premium permit (close to central campus)' + chr(10)
        + '- Red: restricted/reserved' + chr(10)
        + '- Visitor: open to visitors (often metered or short-term)' +
chr(10)
        + '- Park & Ride: free remote lot with shuttle service'
    )


# Register tools for the agent
TOOLS = {
    'list_ku_buildings':            list_ku_buildings,
    'find_parking_near_building':   find_parking_near_building,
    'get_parking_colors_legend':    get_parking_colors_legend,
}

print('Tools registered:', list(TOOLS.keys()))
```

5.4.8.4 Test the Tools (no LLM yet)

Before wiring up the agent, let's call the tools directly to make sure they work. This is a critical debugging practice: if the raw tool output is wrong, no amount of prompt engineering on the agent side will fix it. By testing the tools in isolation, we can verify that the fuzzy matcher resolves "business school" correctly, that the distance calculations are sensible, and that the Google Maps URLs point to the right locations. Only after we trust the tools should we hand them to an LLM.

```
# Call one tool directly
print(find_parking_near_building('business school', top_k=5))
print()
print(find_parking_near_building('Kansas Union', top_k=3))
```

5.4.8.5 The Agent (LLM that calls the tools)

We use a **prompt-based ReAct loop** – no `bind_tools`, no special API features, works with any LLM. The system prompt describes the three available tools and tells the LLM to output a JSON action like `{"tool": "find_parking_near_building", "input": "business school"}` when it wants to call one. Our loop parses the JSON, executes the tool, feeds the result back into the conversation, and repeats until the LLM produces a natural-language final answer instead of a tool call.

This is a deliberately simple architecture. Framework-based agents (LangChain, CrewAI) add convenience methods and error handling, but the core pattern is exactly what you see here: prompt the LLM, check if it wants to call a tool, execute the tool if so, feed the result back, repeat. Understanding this loop demystifies what frameworks do under the hood and gives you the confidence to debug agent behavior when something goes wrong.

```
AGENT_SYSTEM = (
    'You are a helpful KU parking assistant. You have these tools:' + chr(10)
    + '- list_ku_buildings(): returns the list of KU buildings available.' +
chr(10)
```

```
    + '- find_parking_near_building(name): returns the top 5 nearest parking
lots to a building, '
    + 'with color, distance, walk time, and Google Maps pin.' + chr(10)
    + '- get_parking_colors_legend(): explains the KU parking color codes.' +
chr(10) + chr(10)
    + 'To use a tool, reply with ONLY this JSON (nothing else):' + chr(10)
    + '{"tool": "tool_name", "input": "the input string"}' + chr(10) + chr(10)
    + 'After receiving a tool result, write a helpful final answer for the
user that includes '
    + 'the ranked parking list, color meanings, and clickable Google Maps
links. '
    + 'If the user mentions a building informally (e.g., "business school"),
pass that phrase '
    + 'directly to find_parking_near_building — its fuzzy matcher will handle
it.'
)


def run_parking_agent(question, verbose=True):
    '''Simple ReAct-style loop: LLM decides which tool to call, we execute it,
feed result back.'''
    messages = [
        {'role': 'system', 'content': AGENT_SYSTEM},
        {'role': 'user', 'content': question},
    ]

    # Flatten into a single prompt with conversation history
    def flatten(msgs):
        parts = []
        for m in msgs:
            parts.append(f"{m['role'].upper()}: {m['content']}")
        return chr(10).join(parts)

    for step in range(5):  # max 5 tool rounds
        prompt = flatten(messages[1:])  # send everything except system
(passed separately)
        response = llm.generate(prompt, system_prompt=AGENT_SYSTEM)
        reply = response.text.strip()

        # Try to parse a tool call
        tool_call = None
        m = _re.search(r'\{[^{}]+\}', reply)
        if m:
            try:
                parsed = _json.loads(m.group())
                if 'tool' in parsed and 'input' in parsed:
                    tool_call = parsed
            except _json.JSONDecodeError:
                pass

        if tool_call is None:
            # No tool call -- this is the final answer
            if verbose:
                print(f'  [Step {step+1}] Final answer' + chr(10))
            return reply

        # Execute the requested tool
        tool_name = tool_call['tool']
        tool_input = tool_call['input']
        tool_fn = TOOLS.get(tool_name)
        if tool_fn:
            if verbose:
                print(f'  [Step {step+1}] Calling: {tool_name}({tool_input})')
            result = tool_fn(tool_input)
            if verbose:
                preview = str(result).replace(chr(10), ' | ')[:150]
```

```
                print(f'              Result: {preview}...')
        else:
            result = f'Unknown tool: {tool_name}'

        messages.append({'role': 'assistant', 'content': reply})
        messages.append({'role': 'user', 'content': f'Tool result: {result}'})

    return 'Agent exceeded maximum steps.'


print('Parking agent ready.')
```

5.4.8.6 Ask the Agent Some Questions

Now we let the agent run end-to-end. For each question, the agent should: (1) figure out which tool to call based on the user's natural language, (2) call `find_parking_near_building` with the user's phrase (the fuzzy matcher handles informal names), and (3) format a helpful response with the ranked list, permit colors, and clickable Google Maps links. Try uncommenting the other questions to see how the agent handles visitor-specific queries, requests for the color legend, and different building names.

```
from IPython.display import display, Markdown

questions = [
    'Where can I park near the business school?',
    # 'I need parking near the student union. What are my options?',
    # 'What colors of parking are available at KU and what do they mean?',
    # 'I am visiting KU and want to park near Allen Fieldhouse. Where should I
go?',
]

for q in questions:
    display(Markdown('---'))
    display(Markdown(f'**Q:** {q}'))
    answer = run_parking_agent(q)
    # Render as Markdown so Google Maps URLs become clickable links.
    # Colab auto-linkifies any URL inside a Markdown-rendered cell.
    display(Markdown(answer))
```

5.4.8.7 How to Build the Same Thing in Dify (multi-node workflow with real tools)

A proper Dify workflow uses **alternating tool and LLM nodes**, matching the architecture you see in typical agentic workflow diagrams. Each tool node fetches data, each LLM node reasons about what to do next with that data.

5.4.8.7.1 The workflow architecture

```
Start (user types a KU building name)
  |
  v
[TOOL #1] Knowledge Retrieval - KU_Buildings KB
  | (acts as the "access Google Maps" step: returns the building's lat/lng)
  v
[LLM #1] Parse the building coordinates
  | (extracts a clean {lat, lng} from the retrieved text)
  v
[TOOL #2] Knowledge Retrieval - KU_Parking_Lots KB
  | (acts as the "access KU map" step: returns all parking lot data)
  v
[LLM #2] Compute distances, filter to 2 miles, rank top 5, format output
  | (does the math approximately using the coordinates)
  v
End (returns formatted parking list with Google Maps pin URLs)
```

Five processing nodes: 2 Knowledge Retrieval (tool) nodes + 2 LLM nodes + Start/End. No Code nodes, no custom APIs, no Python.

5.4.8.7.2 Why Knowledge Retrieval instead of HTTP Request?

In Dify's workflow builder, a "tool" is any node that fetches external data. Knowledge Retrieval nodes are the easiest kind of tool because:

- You upload the data once (buildings + parking lots as markdown files)
- Dify chunks, embeds, and indexes it automatically
- Retrieval is just a vector search — fast and free
- No API keys, no rate limits, no billing

An alternative would be an **HTTP Request** node calling Nominatim (OpenStreetMap's free geocoding API), but Nominatim is rate-limited and doesn't know anything about specific KU building nicknames like "business school". Our pre-uploaded Knowledge Base is more reliable for a classroom demo.

5.4.8.7.3 Step-by-step setup

5.4.8.7.3.1 Part A: upload the two data files as Knowledge Bases

The notebook directory includes two markdown files at `Agentic/dify_data/`:

- `ku_buildings.md` - one section per building with coordinates
- `ku_parking_lots.md` - one section per parking lot with color and coordinates

In Dify:

1. Go to cloud.dify.ai -> **Knowledge** -> **Create Knowledge**
2. Upload `ku_buildings.md`. Name the KB `KU_Buildings`.
3. On the chunking screen, pick **Custom** chunking:
    - Chunk delimiter: `\n#` (hash + space — this splits on every heading)
    - Max chunk length: 500 tokens
    - This ensures each building becomes its own retrievable chunk.
4. Indexing: **High Quality** (vector embeddings).
5. Click **Save & Process**. Wait ~30 seconds.
6. Repeat: create a second Knowledge Base called `KU_Parking_Lots` and upload `ku_parking_lots.md` with the same chunking settings.

5.4.8.7.3.2 Part B: build the workflow

1. **Studio** -> **Create App** -> **Workflow** (not Chatflow).
2. Name it `KU Parking Assistant`.
3. On the canvas, click the **Start** node:
    - Add input variable: name = `destination`, type = Text, label = "Which KU building are you visiting?"
4. **Add TOOL #1: Knowledge Retrieval (Buildings)**
    - Click + after Start -> **Knowledge Retrieval**
    - Query variable: `Start.destination`
    - Knowledge: select `KU_Buildings`
    - Rename this node to `Retrieve Building`
5. **Add LLM #1: Parse Building Coordinates**
    - Click + after Retrieve Building -> **LLM**
    - Model: your preferred (gpt-4o-mini, gemini-2.5-flash, etc.)
    - **System prompt**:

```
You are a data extractor. The user input contains text describing a KU
building.
Extract the building name and its latitude/longitude coordinates.
```

```
Reply with ONLY this JSON format (nothing else):
{"building": "full name", "lat": 38.9573, "lng": -95.2519}
```

- **User prompt**:

```
Retrieved building information:
{{#Retrieve_Building.result#}}

User query: {{#Start.destination#}}
```

- Rename this node to `Parse Coords`

1. **Add TOOL #2: Knowledge Retrieval (Parking Lots)**
   - Click + after Parse Coords -> **Knowledge Retrieval**
   - Query variable: `Start.destination` (or a fixed query like "parking lots")
   - Knowledge: select `KU_Parking_Lots`
   - In the node settings, set **Top K** to 15 (to retrieve all lots, since our KB is small)
   - Rename this node to `Retrieve Parking`
2. **Add LLM #2: Distance + Filter + Format**
   - Click + after Retrieve Parking -> **LLM**
   - **System prompt**:

```
You are a parking assistant. You will receive:
1. A destination building with its latitude and longitude (as JSON).
2. A list of KU parking lots with their coordinates and colors.

Your job:
- Compute the approximate great-circle distance in miles from the destination
  to each parking lot. For Lawrence KS, 0.001 degrees of latitude is about
  0.069 miles and 0.001 of longitude is about 0.054 miles.
- Filter to lots within 2 miles of the destination.
- Rank the remaining lots from nearest to farthest.
- Return the top 5 as a numbered list with these fields for each:
  * Rank, lot name, permit color
  * Distance in miles (2 decimals) and approximate walk time (3 mph walking
speed)
  * Google Maps pin URL in this exact format: https://www.google.com/maps?q=
LAT,LNG
- End with a short note explaining what the permit color of the closest lot
means.
- If no lots are within 2 miles, say so and suggest the Park & Ride shuttle.
```

- **User prompt**:

```
Destination: {{#Parse_Coords.text#}}

Available parking lots:
{{#Retrieve_Parking.result#}}

Give me the top 5 parking lots within 2 miles.
```

- Rename this node to `Rank and Format`

1. **Configure the End node**
   - Output variable: `Rank_and_Format.text`
2. **Publish** and click **Run** to test. Try: "business school", "Kansas Union", "Allen Fieldhouse".

5.4.8.7.4 What each node does and why

Node	Type	Job	Deterministic or LLM?

Node	Type	Job	Deterministic or LLM?
Start	Input	Collect user destination	•
Retrieve Building	Tool (Knowledge Retrieval)	Look up the building in the Buildings KB	Deterministic (vector search)
Parse Coords	LLM	Extract clean `{lat, lng}` from the retrieved text	LLM (structured extraction)
Retrieve Parking	Tool (Knowledge Retrieval)	Look up parking lots from the Parking KB	Deterministic (vector search)
Rank and Format	LLM	Compute distances, filter 2 miles, rank, format	LLM (approximate math + formatting)
End	Output	Return the formatted result	•

Notice the **alternation**: Tool -> LLM -> Tool -> LLM. Each tool fetches fresh data, each LLM reasons about what to do with it. This is the standard pattern in Dify workflows and maps directly to the architecture diagram you described.

5.4.8.7.5 Colab (real tools) vs. Dify (workflow tools) - the teaching contrast

	Colab notebook	Dify workflow
How are tools defined?	Python functions	Dify nodes (Knowledge Retrieval, LLM, HTTP Request)
Who decides which tool to call?	The LLM, at each step of the agent loop	Fixed by the workflow diagram (same tools in the same order)
Distance math	Exact (Haversine formula)	Approximate (LLM reasoning over coordinates)
Data source	Python dicts	Dify Knowledge Bases with vector retrieval
Flexibility	High — agent can call tools in any order	Low — flow is fixed, but predictable
Best for	Complex questions with branching reasoning	Simple, predictable pipelines with the same steps every time

5.4.8.7.6 When to pick each approach

- **Colab agent (this notebook):** when the agent needs to decide dynamically whether to call `list_buildings`, `find_parking_near_building`, or `get_parking_colors_legend` based on the question. Good for conversational or exploratory use cases.
- **Dify workflow (above):** when every query follows the same steps (lookup destination -> lookup parking -> rank). Good for a production app where you want consistent, debuggable behavior without per-query reasoning overhead.

5.4.8.7.7 Limitations of the Dify version

- The LLM's distance math is **approximate**. For 15 lots in a small area it usually gets the top 3 right, but lots at very similar distances may be mis-ranked by 1-2 positions.

- If your Knowledge Base grows beyond a few hundred entries, the vector retrieval step may miss relevant items. At that scale you'd switch to HTTP Request calls to a real database.
- The 2-mile filter is enforced in the LLM prompt, not in code. A rogue LLM response could include a lot outside 2 miles. In production you'd add a **Template Transform** or **Parameter Extractor** node to enforce the filter programmatically.

5.4.8.8 Exercises

- Add a new building (e.g., `'Budig Hall': (lat, lng)`) and a new parking lot, then rerun — notice you don't need to change any tool code.
- Add a fourth tool: `find_parking_by_color(color, near_building)` that filters to only lots of a specific permit color.
- Replace the hardcoded data with a real parse of the KU parking map PDF — this is where real data engineering begins.
- In Dify, try removing the coordinates from the system prompt and just listing lot names. How does answer quality change? This shows why giving the LLM structured numeric data matters.
- Compare the Colab and Dify outputs for the same query. How often do they agree on the top-3 ranking?

```
!pip install -q git+https://github.com/KarAnalytics/llm_cascade.git
```

5.4.9 KU Parking Assistant via Supabase MCP Endpoint

In the previous notebook (`KU_Parking_Assistant.ipynb`), we built a parking assistant where every tool – distance math, building lookup, color legend – lived as a plain Python function inside the notebook. That works beautifully for a single-notebook demo, but it creates a problem the moment you want a second client to use the same tools. A Dify workflow, a mobile app, and a Cursor plugin would each need their own copy of the parking data and distance logic, and any update to the data (a new parking lot opens, coordinates get corrected) would require updating every client independently.

Model Context Protocol (MCP) solves this by moving the tools to a server. The server exposes a standard JSON-RPC 2.0 interface: any client can call `tools/list` to discover what tools are available, then `tools/call` to execute one. The protocol is transport-agnostic – it works over HTTP, WebSockets, or stdio – and it is the same standard that Claude Desktop, Cursor, and Dify use to integrate external tools. In this notebook, our MCP server is a **Supabase Edge Function** written in TypeScript, deployed to the cloud, and accessible at a single HTTPS URL.

What changes from the inline-tools version:

- No hardcoded `KU_BUILDINGS` or `KU_PARKING_LOTS` dictionaries – the data lives on the server
- No local `find_parking_near_building` function – we call it remotely over HTTPS
- The **same tool interface** and the **same agent ReAct loop** – only the transport changes
- Any MCP-compatible client can call the same endpoint and get identical results

What is JSON-RPC? It is a lightweight remote procedure call protocol that uses JSON for encoding. Each request contains a `method` (like `tools/call`), a `params` object (the tool name and arguments), and an `id` so the client can match responses to requests. The server returns a JSON object with either a `result` or an `error` field. MCP builds on top of JSON-RPC by standardizing the method names (`tools/list`, `tools/call`) and the shape of tool descriptions, so any client and any server that speak MCP can interoperate without custom integration code.

	`KU_Parking_Assistant.ipynb`	`KU_Parking_mcp.ipynb` (this one)
Where do tools live?	Inline Python functions	Supabase Edge Function (TypeScript)
Where does data live?	Hardcoded in the notebook	Hardcoded in the Edge Function
How does the agent call a tool?	Direct Python call	HTTP POST with JSON-RPC body
Runs where?	100% in Colab	Logic runs on Supabase, notebook is just a client
Can other apps use the tools?	No	Yes – Dify, Cursor, web apps, etc.

5.4.9.1 Imports and LLM Setup

We use `llm_cascade` for automatic LLM provider fallback and `requests` to speak JSON-RPC over HTTPS with the Supabase MCP endpoint. No local parking data, no distance math library — all of that lives on the server now.

```
import json as _json
import re as _re
import requests
```

```python
from llm_cascade import get_cascade

llm = get_cascade()

# The deployed Supabase Edge Function URL
MCP_URL = 'https://phhgezbdcpnjbpzvslct.supabase.co/functions/v1/ku-parking'
```

5.4.9.2 The MCP Client (a thin wrapper over `requests`)

One of the most liberating things about MCP is how simple the client side is. There is no SDK to install, no proto files to compile, no complex authentication dance. MCP communication is just **JSON-RPC 2.0 over HTTP POST** – you send a JSON object with a `method`, `params`, and `id`, and the server replies with a JSON object containing either a `result` or an `error`. The two methods we care about are `tools/list` (discover what tools the server offers) and `tools/call` (execute a specific tool with arguments).

The `mcp_call` function below is the entire MCP client: it builds a JSON-RPC payload, POSTs it to the Supabase URL, and returns the parsed response. The `mcp_tool` convenience wrapper adds one more step – it extracts the text content from the tool's response, so callers get a clean string instead of the full JSON-RPC envelope. Compare this to the inline-tools notebook, where the "client" was a direct Python function call. The interface to the agent is identical; only the plumbing underneath has changed.

```python
def mcp_call(method, params=None, request_id=1):
    '''Send a JSON-RPC 2.0 request to the MCP server.'''
    payload = {
        'jsonrpc': '2.0',
        'id': request_id,
        'method': method,
        'params': params or {},
    }
    response = requests.post(MCP_URL, json=payload, timeout=30)
    response.raise_for_status()
    return response.json()


def mcp_tool(name, **arguments):
    '''Call a named MCP tool and return its text content.'''
    result = mcp_call('tools/call', {
        'name': name,
        'arguments': arguments,
    })
    if 'error' in result:
        return f"MCP error: {result['error']['message']}"
    return result['result']['content'][0]['text']


print('MCP client ready.')
```

5.4.9.3 Discover the Tools (no LLM yet)

Before we build the agent, let's ask the MCP server what tools it exposes. This demonstrates the **tool discovery** step of MCP – a critical feature that sets it apart from ad-hoc HTTP APIs. In a traditional integration, the client must hardcode the endpoint URLs, parameter names, and response shapes. With MCP, the client calls `tools/list` once, receives a machine-readable schema for every available tool (name, description, parameter types), and uses that schema to construct valid calls dynamically.

This means that if someone adds a new tool to the server (say, `find_parking_by_color`), existing clients discover it automatically on their next `tools/list` call – no code changes, no redeployment, no coordination between teams. It is the same principle that made REST APIs with self-describing schemas (like OpenAPI/Swagger) so successful, applied specifically to the problem of giving LLM agents access to external tools.

```
tools_response = mcp_call('tools/list')
tools = tools_response['result']['tools']

print(f'Discovered {len(tools)} tools on the MCP server:' + chr(10))
for t in tools:
    print(f"  - {t['name']}")
    desc = t.get('description', '').strip().split(chr(10))[0]
    print(f"      {desc}")
```

5.4.9.4 Test One Tool Directly

Before wiring up the LLM agent, let's call `find_parking_near_building` manually to verify the server works and returns the expected format. This is the same debugging practice we used in the inline-tools notebook – always test your tools in isolation before letting an LLM loose on them. If the raw tool output looks wrong, no amount of prompt engineering will produce a correct final answer. Notice that the response format (lot name, color, distance, walk time, Google Maps URL) is identical to what the local Python function returned in the previous notebook, because the MCP server implements the same logic in TypeScript on the server side.

```
result = mcp_tool(
    'find_parking_near_building',
    building_name='business school',
    max_distance_miles=2.0,
    top_k=5,
)
print(result)
```

5.4.9.5 The Agent (LLM that calls MCP tools)

Here is where the architectural elegance of MCP becomes visible. The ReAct-style agent loop below is structurally identical to the one in `KU_Parking_Assistant.ipynb` – the LLM reads the question, decides which tool to call, outputs a JSON action, and we parse and execute it. The only difference is that instead of calling `TOOLS[tool_name](tool_input)` (a local Python function), we call `mcp_tool(tool_name, building_name=tool_input)` (an HTTPS request to Supabase). The LLM does not know or care where the tools live – it sees the same tool descriptions in its system prompt and produces the same JSON actions. That separation of concerns is the whole point of a protocol: it decouples the agent's reasoning from the tool's implementation.

```
AGENT_SYSTEM = (
    'You are a helpful KU parking assistant. You have these tools available
via MCP:' + chr(10)
    + '- list_ku_buildings(): returns the list of KU buildings available.' +
chr(10)
    + '- find_parking_near_building(building_name): returns the top 5 nearest
parking lots '
    + 'to a building within 2 miles, with color, distance, walk time, and
Google Maps pin.' + chr(10)
    + '- get_parking_colors_legend(): explains the KU parking color codes.' +
chr(10) + chr(10)
    + 'To use a tool, reply with ONLY this JSON (nothing else):' + chr(10)
    + '{"tool": "tool_name", "input": "the input string"}' + chr(10) + chr(10)
    + 'After receiving a tool result, write a helpful final answer for the
user that includes '
    + 'the ranked parking list, color meanings, and clickable Google Maps
links. '
    + 'If the user mentions a building informally (e.g., "business school"),
pass that phrase '
    + 'directly to find_parking_near_building -- its fuzzy matcher will handle
it.'
)


def run_parking_agent(question, verbose=True):
```

```
    '''ReAct loop: LLM picks a tool -> MCP executes it -> LLM formats the
final answer.'''
    messages = [
        {'role': 'system', 'content': AGENT_SYSTEM},
        {'role': 'user', 'content': question},
    ]

    def flatten(msgs):
        return chr(10).join(f"{m['role'].upper()}: {m['content']}" for m in
msgs)

    for step in range(5):
        prompt = flatten(messages[1:])  # everything except system
        response = llm.generate(prompt, system_prompt=AGENT_SYSTEM)
        reply = response.text.strip()

        # Try to parse a tool call JSON from the reply
        tool_call = None
        m = _re.search(r'\{[^{}]+\}', reply)
        if m:
            try:
                parsed = _json.loads(m.group())
                if 'tool' in parsed and 'input' in parsed:
                    tool_call = parsed
            except _json.JSONDecodeError:
                pass

        if tool_call is None:
            # No tool call -- this is the final answer
            if verbose:
                print(f'  [Step {step+1}] Final answer' + chr(10))
            return reply

        # Dispatch the tool call to the MCP server
        tool_name = tool_call['tool']
        tool_input = tool_call['input']
        if verbose:
            print(f'  [Step {step+1}] MCP call: {tool_name}({tool_input!r})')

        # Each tool takes a different argument name -- map them
        if tool_name == 'find_parking_near_building':
            result = mcp_tool(tool_name, building_name=tool_input)
        else:
            result = mcp_tool(tool_name)

        if verbose:
            preview = str(result).replace(chr(10), ' | ')[:150]
            print(f'              Result: {preview}...')

        messages.append({'role': 'assistant', 'content': reply})
        messages.append({'role': 'user', 'content': f'Tool result: {result}'})

    return 'Agent exceeded maximum steps.'


print('Parking agent ready (tools backed by Supabase MCP endpoint).')
```

5.4.9.6 Ask the Agent Some Questions

Each question runs through the agent loop. Notice the output shows **MCP call: …** at each step – that is the agent dispatching tool calls to the Supabase endpoint over HTTPS. The rest of the notebook never touches the parking data directly. If you compare the agent's final answer here with the one from the inline-tools notebook, they should be nearly identical, because both notebooks call the same logical tools with the same data. The difference is entirely in the plumbing: local function calls versus remote JSON-RPC calls. Try

uncommenting the additional questions to see how the agent handles different phrasing, visitor-specific queries, and requests for the color legend.

```
from IPython.display import display, Markdown

questions = [
    'Where can I park near the business school?',
    # 'I need parking near the student union. What are my options?',
    # 'What colors of parking are available at KU and what do they mean?',
    # 'I am visiting KU and want to park near Allen Fieldhouse. Where should I
go?',
]

for q in questions:
    display(Markdown('---'))
    display(Markdown(f'**Q:** {q}'))
    answer = run_parking_agent(q)
    # Render as Markdown so the Google Maps URLs become clickable links.
    display(Markdown(answer))
```

5.4.9.7 Using the Same MCP Server from Dify

The whole point of exposing tools via MCP is that **other apps can use them too**. This section walks you through connecting the same Supabase MCP endpoint (the one we just used from this notebook) to a Dify Workflow. You'll build a no-code version of the parking assistant that calls the same `find_parking_near_building` tool.

5.4.9.7.1 What you need

- A **Dify account** — sign up at cloud.dify.ai (free tier works)
- At least one **LLM provider configured** in Dify (Settings arrow.r Model Provider)
- The Supabase MCP URL: `https://phhgezbdcpnjbpzvslct.supabase.co/functions/v1/ku-parking`

Dify offers **two ways** to connect to the MCP server. Option A works on any Dify version; Option B is cleaner if your instance has the MCP plugin installed.

5.4.9.7.2 Option A — HTTP Request node (always works, no plugin needed)

This is the guaranteed path: Dify's built-in HTTP Request node posts JSON to the Supabase URL and we parse the response.

5.4.9.7.2.1 Step 1: Create a Workflow app

1. In Dify, click **Studio arrow.r Create App arrow.r Workflow** (not Chatflow, not Agent)
2. Name it `KU Parking via MCP`

5.4.9.7.2.2 Step 2: Configure the Start node

1. Click the **Start** node on the canvas
2. Add an input variable:
    - **Variable name:** `destination`
    - **Type:** Text
    - **Label:** `Which KU building are you visiting?`
    - **Required:** Yes

5.4.9.7.2.3 Step 3: Add an HTTP Request node

1. Click + after Start arrow.r **HTTP Request**
2. Configure it:
    - **Method:** `POST`
    - **URL:** `https://phhgezbdcpnjbpzvslct.supabase.co/functions/v1/ku-parking`

- **Headers:** `Content-Type: application/json`
- **Body type:** `JSON`
- **Body:**

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/call",
  "params": {
    "name": "find_parking_near_building",
    "arguments": {
      "building_name": "{{#Start.destination#}}",
      "max_distance_miles": 2.0,
      "top_k": 5
    }
  }
}
```

1. Rename the node to `Call MCP`

5.4.9.7.2.4 Step 4: Add a Code node to extract the answer

The HTTP Request returns the full JSON-RPC response. We need to pull out `result.content[0].text`.

1. Click + after the HTTP Request arrow.r **Code**
2. Language: **Python 3**
3. Input variable: `response_body` = `Call_MCP.body`
4. Code:

```python
import json

def main(response_body: str) -> dict:
    data = json.loads(response_body)
    if "result" in data and "content" in data["result"]:
        return {"parking_info": data["result"]["content"][0]["text"]}
    return {"parking_info": "MCP error: " + str(data.get("error", "unknown"))}
```

1. Output variable: `parking_info` (String)
2. Rename the node to `Extract Result`

5.4.9.7.2.5 Step 5: Add an LLM node to format the answer

1. Click + after the Code node arrow.r **LLM**
2. Model: pick whatever you have configured (gpt-4o-mini, gemini-2.5-flash, etc.)
3. **System prompt:**

```
You are a helpful KU parking assistant. The user asked about parking near a
KU building. Below is the raw parking information returned from the KU Parking
MCP server. Present it to the user in a friendly, well-formatted response that
includes the ranked parking list with permit color, distance, walk time, and
clickable Google Maps links.
```

1. **User prompt:**

```
User question: Where can I park near {{#Start.destination#}}?

Parking information from the MCP server:
{{#Extract_Result.parking_info#}}

Please format a helpful response for the user.
```

1. Rename the node to `Format Answer`

5.4.9.7.2.6 Step 6: Configure the End node

1. Click the **End** node

2. Output variable: `Format_Answer.text`

5.4.9.7.2.7 Step 7: Publish and test

1. Click **Publish** (top right) arrow.r **Update Publishing**
2. Click **Run** and enter a test destination like `business school`
3. You should see the same ranked parking list you got in this notebook

---

5.4.9.7.3 Option B — MCP plugin (cleaner if available)

If your Dify instance has the **MCP SSE** or **MCP HTTP** plugin installed (check under **Tools arrow.r Install Tool** and search for "mcp"):

1. Install the MCP plugin
2. Go to **Tools** and click the MCP plugin
3. Add a new MCP server:
   - **Name:** `ku-parking`
   - **URL:** `https://phhgezbdcpnjbpzvslct.supabase.co/functions/v1/ku-parking`
   - Save
4. Dify will call `tools/list` on the server and automatically discover the three tools
5. In any Workflow, the three MCP tools now appear as **native Dify tool nodes** you can drag onto the canvas. You skip the HTTP Request + Code node dance entirely.

With Option B, your workflow becomes:

```
Start -> [MCP Tool: find_parking_near_building] -> LLM (format) -> End
```

Just three processing nodes, no manual JSON parsing.

5.4.9.7.4 Which option should you try?

- **Start with Option A** if you're not sure whether your Dify instance has the MCP plugin. The HTTP Request + Code node approach always works on cloud Dify and self-hosted Dify alike.
- **Upgrade to Option B** once you confirm the plugin is available. It's cleaner and shows off MCP's real value — tool discovery happens automatically.

Either way, the **same Supabase endpoint** (the one we called from this notebook) powers the Dify app. Update the data in the Edge Function, redeploy, and both this notebook and the Dify app see the new data immediately.

5.4.9.8 Key Takeaways

Same agent, different transport. The ReAct agent loop in this notebook is identical to the one in `KU_Parking_Assistant.ipynb`. The only change is that tool calls now dispatch via `mcp_tool()` — a tiny HTTPS wrapper — instead of calling local Python functions.

Write the tools once, use them everywhere. The Supabase Edge Function is the single source of truth for parking data. This notebook, Dify, Claude Desktop, a web app, and a mobile app can all call the same `find_parking_near_building` tool and get identical results.

Update data in one place. Edit `KU_PARKING_LOTS` in `Agentic/mcp/supabase/supabase/functions/ku-parking/index.ts`, redeploy with `supabase functions deploy ku-parking --no-verify-jwt`, and every client sees the new data on its next request. No client code changes, no notebook reruns needed.

Tool discovery is automatic. Clients don't hardcode tool names. They call `tools/list` first, receive the server's schema, and use that to build their prompts. If you add a new tool to the Edge Function, existing clients discover it without code changes.

5.4.9.8.1 Exercises

- **Add a new tool to the Edge Function** (e.g., `find_parking_by_color` that filters to one permit color). Redeploy and see the new tool appear automatically in `tools/list`.
- **Update a building coordinate** in the Edge Function and watch this notebook pick up the new value without any local changes.
- **Build the Dify version** following Option A above and compare its output with this notebook's agent. Do they agree?
- **Add authentication.** Remove the `--no-verify-jwt` flag when redeploying, then update `mcp_call` to send an `Authorization: Bearer <supabase-jwt>` header. Now the endpoint is no longer public.
- **Add caching.** In `mcp_call`, cache the response of `tools/list` so the client only discovers tools once per session instead of on every run.

6. Deployment and No-Code

6.1 Deployment and No-Code

6.1.1 From Notebooks to Production

Every application built in this book so far has lived inside a Jupyter notebook running on Google Colab. Notebooks are ideal for learning, experimentation, and rapid prototyping. They let you see intermediate outputs, inspect variables, and iterate on prompts in real time. But a notebook is not a production system. It requires a human to open it, run cells in order, and keep the runtime alive. It cannot serve multiple users simultaneously, it does not persist across sessions, and it offers no guarantees about availability or performance.

Taking AI from notebooks to production means answering two questions. First, how do you make your tools and data accessible to any application, not just the notebook where you developed them? Second, how do you build end-user applications that non-technical stakeholders can use and maintain without writing code? This chapter addresses both questions through two complementary approaches: deploying tools as serverless MCP endpoints on Supabase Edge Functions, and building complete AI workflows visually using the Dify platform.

6.1.2 MCP Servers on Supabase Edge Functions

The previous chapter introduced the Model Context Protocol as a standard for tool interoperability. This chapter shows how to deploy an MCP server to a real cloud environment so that it becomes a permanent, publicly accessible service.

Supabase Edge Functions are serverless TypeScript functions that run on Deno at the edge, close to your users, with no infrastructure to manage. You write a TypeScript file that handles incoming HTTP requests, deploy it with a single command, and Supabase gives you a public HTTPS URL. There are no servers to provision, no containers to build, and no scaling decisions to make. The function spins up when a request arrives and shuts down when it finishes.

The KU Parking MCP server, deployed as a Supabase Edge Function, implements the JSON-RPC 2.0 protocol that MCP requires. When a client sends a POST request with the method `tools/list`, the function returns a catalog of available tools and their parameter schemas. When a client sends `tools/call` with a tool name and arguments, the function executes the tool and returns the result. The parking data, the Haversine distance formula, the fuzzy building matcher, and the Google Maps URL generator all live inside this single Edge Function, written in TypeScript and deployed to the cloud.

The deployment workflow is straightforward. You initialize a Supabase project, write the Edge Function in the `supabase/functions/` directory, and deploy with `supabase functions deploy`. The `--no-verify-jwt` flag makes the endpoint publicly accessible without authentication, which is appropriate for a classroom demo but would be replaced with proper authentication in a production setting. Once deployed, the function is available at a permanent URL that any HTTP client can call.

This architecture offers a clean separation between tool development and tool consumption. The team that maintains the parking data and distance logic works on the Edge Function. The teams that build user-facing applications, whether notebooks, web apps, or no-code workflows, simply call the endpoint. Changes to tool logic or data propagate automatically to all consumers on their next request.

6.1.3 No-Code Workflows with Dify

Dify is a visual platform for building LLM-powered applications without writing code. It provides a drag-and-drop workflow builder where each node represents a processing step: an LLM call, a knowledge base retrieval, an HTTP request, a code transformation, or a conditional branch. Nodes are connected by edges that define data flow, and the entire workflow can be published as a web application, an API, or a chatbot with a single click.

Two capabilities of Dify are particularly relevant to this book's themes. First, **Knowledge Bases** provide document RAG without any code. You upload documents such as PDFs, markdown files, or text files. Dify automatically chunks them, generates vector embeddings, and indexes them for retrieval. A Knowledge Retrieval node in a workflow performs semantic search over these documents and passes the relevant chunks to an LLM node for answer generation. This replicates the RAG pipeline built from scratch in earlier chapters, but the entire process is configured through a visual interface rather than Python code.

Second, **HTTP Request nodes** enable Dify workflows to call external APIs, including MCP endpoints. To connect Dify to the KU Parking MCP server, you add an HTTP Request node that sends a JSON-RPC POST to the Supabase URL, follow it with a Code node that extracts the tool result from the JSON-RPC response, and then pass the extracted text to an LLM node for formatting. The result is a no-code parking assistant that calls the same MCP tools used by the Python notebook, producing identical results through a completely different delivery mechanism.

For Dify instances with the MCP plugin installed, the integration is even simpler. The plugin connects directly to the MCP server, discovers its tools automatically via `tools/list`, and exposes them as native Dify tool nodes that you can drag onto the canvas. The HTTP Request and Code node dance is eliminated entirely, and the workflow reduces to a start node, a tool node, an LLM formatting node, and an end node.

6.1.4 When to Build Code vs. When to Use No-Code

The choice between writing code and using a no-code platform is not about technical capability but about organizational fit. Code-based approaches offer maximum flexibility, full control over every detail of the pipeline, and the ability to implement custom logic that no visual builder supports. They are the right choice when the application requires complex conditional logic, custom model fine-tuning, or integration with proprietary systems that lack standard APIs.

No-code platforms like Dify excel when the goal is rapid deployment, when the primary users of the application are business analysts or domain experts rather than engineers, and when the workflow follows a standard pattern of retrieval, generation, and formatting. They dramatically reduce the time from idea to deployed application, often from days to hours. They also lower the maintenance burden by providing visual debugging, built-in logging, and one-click publishing.

In practice, many production AI systems use both approaches. The computationally intensive or logic-heavy components, such as distance calculations, data transformations, or custom model inference, are implemented in code and deployed as API endpoints or MCP servers. The orchestration layer that ties these components together and presents them to end users is built in a no-code platform. This hybrid architecture gives you the precision of code where it matters and the speed of visual building where it does not.

The sub-chapters that follow provide step-by-step instructions for both deployment paths: setting up the Supabase Edge Function as an MCP server and building the corresponding Dify workflows that consume it.

6.1.5 References

- Anthropic. (2024). Model Context Protocol Specification. https://modelcontextprotocol.io/
- Supabase. (2024). Edge Functions Documentation. https://supabase.com/docs/guides/functions
- Dify.AI. (2024). Dify Platform Documentation. https://docs.dify.ai/
- Deno. (2024). Deno Runtime Documentation. https://deno.land/
- Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., & Wang, H. (2024). Retrieval-Augmented Generation for Large Language Models: A Survey. *arXiv preprint arXiv:2312.10997.*

6.1.6 MCP Servers on Supabase

Deploying tools as a cloud-hosted MCP server gives every client (notebooks, web apps, Dify, Claude Desktop) access to the same tools via a standard HTTPS interface.

This section walks through deploying the KU Parking MCP server to Supabase Edge Functions and calling it from Python and Dify.

See the full deployment instructions and source code in the `Agentic/mcp/supabase/` directory.

6.1.7 No-Code AI with Dify

Dify is a visual LLM application builder. This section shows how to replicate two of our notebook-based applications in Dify without writing any code:

Document RAG — upload company documents to a Dify Knowledge Base, add a Knowledge Retrieval node + LLM node, and get a working Q&A app in 15 minutes.

KU Parking via MCP — connect Dify to the Supabase MCP endpoint using an HTTP Request node, parse the JSON-RPC response, and format the answer with an LLM node.

Step-by-step instructions are included in the respective notebook markdown sections.